

Tessent[®] TestKompress[®] User's Manual

Software Version 2019.2

Document Revision 13

© 2009-2019 Mentor Graphics Corporation
All rights reserved.

This document contains information that is proprietary to Mentor Graphics Corporation. The original recipient of this document may duplicate this document in whole or in part for internal business purposes only, provided that this entire notice appears in all copies. In duplicating any part of this document, the recipient agrees to make every reasonable effort to prevent the unauthorized use and distribution of the proprietary information.

This document is for information and instruction purposes. Mentor Graphics reserves the right to make changes in specifications and other information contained in this publication without prior notice, and the reader should, in all cases, consult Mentor Graphics to determine whether any changes have been made.

The terms and conditions governing the sale and licensing of Mentor Graphics products are set forth in written agreements between Mentor Graphics and its customers. No representation or other affirmation of fact contained in this publication shall be deemed to be a warranty or give rise to any liability of Mentor Graphics whatsoever.

MENTOR GRAPHICS MAKES NO WARRANTY OF ANY KIND WITH REGARD TO THIS MATERIAL INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

MENTOR GRAPHICS SHALL NOT BE LIABLE FOR ANY INCIDENTAL, INDIRECT, SPECIAL, OR CONSEQUENTIAL DAMAGES WHATSOEVER (INCLUDING BUT NOT LIMITED TO LOST PROFITS) ARISING OUT OF OR RELATED TO THIS PUBLICATION OR THE INFORMATION CONTAINED IN IT, EVEN IF MENTOR GRAPHICS HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

U.S. GOVERNMENT LICENSE RIGHTS: The software and documentation were developed entirely at private expense and are commercial computer software and commercial computer software documentation within the meaning of the applicable acquisition regulations. Accordingly, pursuant to FAR 48 CFR 12.212 and DFARS 48 CFR 227.7202, use, duplication and disclosure by or for the U.S. Government or a U.S. Government subcontractor is subject solely to the terms and conditions set forth in the license agreement provided with the software, except for provisions which are contrary to applicable mandatory federal laws.

TRADEMARKS: The trademarks, logos and service marks ("Marks") used herein are the property of Mentor Graphics Corporation or other parties. No one is permitted to use these Marks without the prior written consent of Mentor Graphics or the owner of the Mark, as applicable. The use herein of a third-party Mark is not an attempt to indicate Mentor Graphics as a source of a product, but is intended to indicate a product from, or associated with, a particular third party. A current list of Mentor Graphics' trademarks may be viewed at: mentor.com/trademarks.

The registered trademark Linux[®] is used pursuant to a sublicense from LMI, the exclusive licensee of Linus Torvalds, owner of the mark on a world-wide basis.

End-User License Agreement: You can print a copy of the End-User License Agreement from: mentor.com/eula.

Mentor Graphics Corporation
8005 S.W. Boeckman Road, Wilsonville, Oregon 97070-7777
Telephone: 503.685.7000
Toll-Free Telephone: 800.592.2210
Website: mentor.com
Support Center: support.mentor.com

Send Feedback on Documentation: support.mentor.com/doc_feedback_form

Revision History ISO-26262

Revision	Changes	Status/ Date
13	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released May 2019
12	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Mar 2019
11	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Dec 2018
10	Modifications to improve the readability and comprehension of the content. Approved by Lucille Woo. All technical enhancements, changes, and fixes listed in the <i>Tessent Release Notes</i> for this product are reflected in this document. Approved by Ron Press.	Released Aug 2018

Author: In-house procedures and working practices require multiple authors for documents. All associated authors for each topic within this document are tracked within the Mentor Graphics Technical Publication's source. For specific topic authors, contact Mentor Graphics Technical Publication department.

Revision History: Released documents include a revision history of up to four revisions. For earlier revision history, refer to earlier releases of documentation on Support Center.

Table of Contents

Revision History ISO-26262

Chapter 1

Getting Started	17
Tessent TestKompress	17
EDT Technology	20
Scan Channels	20
Structure and Function	21
Test Patterns	22
TestKompress Compression Logic	23
TestKompress Usage Flow Overview	29
EDT IP Creation and Pattern Generation Flow	29
Pre-Synthesis Flow	30
Tessent Core Description (TCD)	30
EDT IP Generation	31
EDT Logic Synthesis	32
EDT Pattern Generation	32
Using TCD-Based Flow with Flattened EDT Hierarchy	34
Tessent Shell User Interface	35

Chapter 2

The Compressed Pattern Flows	39
Top-Down Design Flows	42
The Compressed Pattern Flows	44
Design Requirements for a Compressed Pattern Flow	44
Compressed Pattern External Flow	45
Compressed Pattern Internal Flow	48

Chapter 3

Scan Chain Synthesis	51
Design Preparation	51
Scan Chain Insertion	53
OCC Sub-Chain Stitching	58
ATPG Baseline Generation	61

Chapter 4

Creation of the EDT Logic	63
Compression Analysis	64
Analyzing Compression	64
Preparation for EDT Logic Creation	68
Parameter Specification for the EDT Logic	72
Dual Compression Configurations	73

Defining Dual Compression Configurations	75
Asymmetric Input and Output Channels	78
Bypass Scan Chains	78
Latch-Based EDT Logic	78
Compactor Type	78
Pipeline Stages in the Compactor	78
Pipeline Stages Added to the Channel	79
Longest Scan Chain Range	79
EDT Logic Reset	79
EDT Architecture Version	79
Specifying Hard Macros	80
Pulse EDT Clock Before Scan Shift Clocks	81
Reporting of the EDT Logic Configuration	81
EDT Control and Channel Pins	83
EDT Control and Channel Pin Configuration	83
Functional/EDT Pin Sharing	85
Shared Pin Configuration	87
Connections for EDT Pins (Internal Flow only)	90
Internally Driven EDT Pins	91
Structure of the Bypass Chains	93
Decompressor and Compactor Connections	93
IJTAG and the EDT IP TCD Flow	94
Design Rule Checks	94
Creation of EDT Logic Files	96
The EDT Logic Files	99
IJTAG and EDT Logic	100
Specification of Module/Instance Names	100
EDT Logic Description	100
Inserting EDT Logic During Synthesis	110
Creation of a Reduced Netlist for Synthesis	117
Chapter 5	
Synthesizing the EDT Logic	121
The EDT Logic Synthesis Script	121
Synthesis and External EDT Logic	122
Synthesis and Internal EDT Logic	124
SDC Timing File Generation	125
SDC Timing File Generation Using extract_sdc	125
SDC Timing File Generation Using write_edt_files	126
EDT Logic/Core Interface Timing Files	126
Scan Chain and ATPG Timing Files	131
Chapter 6	
Generating and Verifying Test Patterns	133
Preparation for Test Pattern Generation	133
EDT Pattern Generation Overview	136
IJTAG Mapping	136
Scan Chain Handling	137

Table of Contents

Core Instance Parameters	138
Used Input Channels	142
Pattern Generation with Internal Chain Masking Hardware	145
Updating Scan Pins for Test Pattern Generation	145
Verification of the EDT Logic	149
Design Rules Checking (DRC)	149
EDT Logic and Chain Testing	149
Reducing Serial EDT Chain Test Simulation Runtime	152
Test Pattern Generation	154
Generating Patterns	154
Compression Optimization	155
Saving of the Patterns	156
Post-Processing of EDT Patterns	157
Simulation of the Generated Test Patterns	157
 Chapter 7	
Modular Compressed ATPG	159
The Modular Flow	159
Understanding Modular Compressed ATPG	161
Development of a Block-Level Compression Strategy	163
Balancing Scan Chains Between Blocks	164
Sharing Input Scan Channels on Identical EDT Blocks	165
Channel Sharing for Non-Identical EDT Blocks	167
Overview of Channel Sharing Functionality	167
Compression Analysis	169
EDT IP Creation With Separate Control and Data Input Channels	170
Rules for Connecting Input Channels from Cores to Top	173
Channel Sharing Reporting	174
Limitations	174
Mixing Channel Sharing for Non-Identical EDT Blocks and Channel Broadcasting for Identical EDT Blocks	175
Generating Modular EDT Logic for a Fully Integrated Design	177
Estimating Test Coverage/Pattern Count for EDT Blocks	178
Legacy ATPG Flow	179
 Chapter 8	
Compressed ATPG Advanced Features	185
Low-Power Test	187
Low-Power Shift	187
Setting Up Low-Power Test	192
Low Pin Count Test Controller	195
LPCT Controller Decision Tree	196
Test Mode Clock Multiplexer Requirement	197
Sharing of the LPCT Clock and a Top-Level Scan Clock	197
Shift Clock Control for LPCT Controllers	198
Type 1 LPCT Controller	202
Type 2 LPCT Controller	204
Type 3 LPCT Controller	206

Tessent OCC and LPCT Usage	208
LPCT Limitations	208
LPCT Controller Types	210
Type 1 - LPCT Controller with Top-level Scan Enable	210
Type 2 - LPCT Controller with a TAP	213
Type 3 - LPCT Controller-Generated Scan Enable	215
LPCT Configuration Examples	222
Type 1 Controller Generation Example	222
Type 2 Controller Generation Example	223
Type 3 Controller Example	225
Type 1 Controller LPCT Clock Example	227
Type 2 Controller Scan Shift Clock Example	228
Compression Bypass Logic	230
Structure of the Bypass Logic	230
Generating EDT Logic When Bypass Logic is Defined in the Netlist	231
Dual Bypass Configurations	233
Generation of Identical EDT and Bypass Test Patterns	234
Use of Bypass Patterns in Uncompressed ATPG	235
Creating Bypass Test Patterns in Uncompressed ATPG	238
Uncompressed ATPG (External Flow) and Boundary Scan	240
Boundary Scan Coexisting with EDT Logic	240
Drive Compressed ATPG with the TAP Controller	245
Use of Pipeline Stages in the Compactor	245
Use of Pipeline Stages Between Pads and Channel Inputs or Outputs	247
Channel Output Pipelining	247
Channel Input Pipelining	248
Clocks for Channel Input Pipeline Stages	249
Clocks for Channel Output Pipeline Stages	249
Input Channel Pipelines Must Hold Their Value During Capture	250
DRC for Channel Input Pipelining	251
DRC for Channel Output Pipelining	251
Input/Output Pipeline Examples	251
Change Edge Behavior in Bypass and EDT Modes	252
Understanding Lockup Cells	254
Lockup Cell Insertion	254
Lockup Cell Analysis For Bypass Lockup Cells Not Included as Part of the EDT Chains	256
Lockups Between Decompressor and Scan Chain Inputs	256
Lockups Between Scan Chain Outputs and Compactor	258
Lockups in the Bypass Circuitry	259
Lockup Cell Analysis For Bypass Lockup Cells Included as Part of the EDT Chains	264
EDT Lockup and Scan Chain Boundary Lockup Cells	264
Differences Based on Inclusion/Exclusion of Bypass Lockup Cells in EDT Chains	266
Lockup Cell Functionality Limitations	269
Comparison of Bypass Lockup Cell Insertion Results	270
Lockups Between Channel Outputs and Output Pipeline Stages	272
Compression Performance Evaluation	274
Establishing a Point of Reference	275
Performance Measurement	276
Performance Improvement	277

Table of Contents

Variance in the Number of Scan Chains	277
Variance in the Number of Scan Channels	278
Determining the Limits of Compression	279
Speed up the Process	279
Understanding Compactor Options	279
Understanding Scan Chain Masking in the Compactor	282
Fault Aliasing	285
About Reordering Patterns	287
Handling of Last Patterns	287
EDT Aborted Fault Analysis	288
 Chapter 9	
Integrating Compression at the RTL Stage	291
IP Generation and Insertion Using EDT Specification	292
Basic Flow	292
Pipeline Stage Insertion	293
Bused EDT Channel Input and Output Connections	294
Lockup Cells on the Input Side of the EDT Controller	295
Lockup Cells on the Output Side of the EDT Controller	295
Lockup Cells Clock Connections	296
EDT Specification Wrapper Creation	296
Validating the EDT Specification and Creating the EDT IP	298
Legacy Skeleton RTL Flow	301
Skeleton Flow Overview	301
Skeleton Design Input and Interface Files	304
Skeleton Design Input File	305
Skeleton Design Interface File	308
Creation of the EDT Logic for a Skeleton Design	309
Longest Scan Chain Range Estimate	309
Integration of the EDT Logic into the Design	310
Skeleton Flow Example	312
Input File	313
 Appendix A	
Getting Help	321
The Tessent Documentation System	321
Mentor Support Services	322
 Appendix B	
EDT Logic Specifications	323
EDT Logic with Basic Compactor and Bypass Module	323
EDT Logic with Xpress Compactor and Bypass Module	324
Decompressor Module with Basic Compactor	325
Decompressor Module with Xpress Compactor	325
Input Bypass Logic	326
Compactor Module	327
Output Bypass Logic	328
Single Chain Bypass Logic	329

Basic Compactor Masking Logic	330
Xpress Compactor Controller Masking Logic	331
Dual Compression Configuration Input Logic	332
Dual Compression Configuration Output Logic	334
EDT Logic with Power Controller	334
Appendix C	
Troubleshooting	337
Debugging Simulation Mismatches	337
Resolving DRC Issues	339
K19 through K22 DRC Violations	339
Debugging Best Practices	341
Understanding K19 Rule Violations	342
Incorrect Control Signals	345
Inverted Signals	348
Incorrect EDT Channel Signal Order	349
Incorrect Scan Chain Order	350
X Generated by EDT Decompressor	352
Using set_gate_report K19	353
Understanding K22 Rule Violations	354
Inverted Signals	357
Incorrect Scan Chain Order	359
Masking Problems	361
Using set_gate_report K22	363
Miscellaneous	364
Incorrect References in Synthesized Netlist	364
Limiting Observable Xs for a Compact Pattern Set	365
Applying Uncompressable Patterns Thru Bypass Mode	365
If Compression is Less Than Expected	366
If Test Coverage is Less Than Expected	366
If there are EDT aborted faults	367
Internal Scan Chain Pins Incorrectly Shared with Functional Pins	367
Masking Broken Scan Chains in the EDT Logic	367
Appendix D	
Dofile-Based Legacy IP Creation and Pattern Generation Flow	369
EDT IP Generation Dofiles	370
Test Pattern Generation Files	370
EDT Bypass Files	374
EDT Pattern Generation Dofiles	376
Generated Bypass Dofile and Procedure File	376
Creation of Test Patterns	377
Low Pin Count Test Controller Dofiles	379
Type 1 Controller Example	379
Type 2 Controller Example	383
Type 3 Controller Example	388

Table of Contents

Index

Third-Party Information

End-User License Agreement

List of Figures

Figure 1-1. EDT as Seen from the Tester	20
Figure 1-2. Tester Connected to a Design with EDT	21
Figure 1-3. EDT logic Located Outside the Core (External Flow)	24
Figure 1-4. EDT logic Located Within the Core (Internal Flow)	24
Figure 1-5. Post-Synthesis EDT IP Creation and EDT Pattern Generation Flow	29
Figure 1-6. Pre-Synthesis EDT IP Creation and EDT Pattern Generation TCD Flow	30
Figure 2-1. Top-Down Design Flow - External	40
Figure 2-2. Top-Down Design Flow - Internal	41
Figure 2-3. Compressed Pattern External Flow	47
Figure 2-4. Compressed Pattern Internal Flow	49
Figure 3-1. Bad Specified Bit Alignment	58
Figure 3-2. Better Specified Bit Alignment	59
Figure 3-3. Best Specified Bit Alignment (Few Cells)	60
Figure 3-4. Best Specified Bit Alignment (Many Cells)	60
Figure 4-1. Default EDT Logic Pin Configuration with Two Channels	84
Figure 4-2. Example of a Basic EDT Pin Configuration (Internal EDT Logic)	85
Figure 4-3. Example with Pin Sharing Shown in (External EDT Logic)	90
Figure 4-4. Internally Driven edt_update Control Pin	92
Figure 4-5. Contents of the Top-Level Wrapper	101
Figure 4-6. Contents of the EDT Logic	102
Figure 4-7. Design Netlist with Internal Connection Nodes	118
Figure 5-1. Contents of Boundary Scan Top-Level Wrapper	123
Figure 6-1. Sample EDT Test Procedure Waveforms	134
Figure 6-2. Used Input Channels Example	144
Figure 6-3. Example Decoder Circuitry for Six Scan Chains and One Channel	150
Figure 7-1. Modular Design with Five EDT blocks	162
Figure 7-2. Non-Separated Control Data Input Channels	168
Figure 7-3. Separated Control Data Input Channels	168
Figure 7-4. Channel Sharing Example	170
Figure 7-5. Non-Channel Sharing	171
Figure 7-6. Channel Sharing Scenario 1	171
Figure 7-7. Channel Sharing Scenario 2	172
Figure 7-8. Case 1	175
Figure 7-9. Case 2	176
Figure 7-10. Case 3	177
Figure 7-11. Netlist with Two Cores Sharing EDT Control Signals	180
Figure 8-1. Low Power Controller Logic	191
Figure 8-2. LPCT Design Process	195
Figure 8-3. LPCT Controller Decision Tree	196
Figure 8-4. Clock Gater for Sharing LPCT Clock with Top-Level Scan Clock	198

Figure 8-5. -shift_control Option - clock.	199
Figure 8-6. -shift_control Option - enable.	200
Figure 8-7. shift_control Option -enable With Mentor Graphics OCC.	201
Figure 8-8. Shift Clock Option - none.	202
Figure 8-9. Type 1 LPCT Controller Configuration.	203
Figure 8-10. Type 2 LPCT Controller Configuration.	205
Figure 8-11. Type 3 LPCT Controller Configuration.	207
Figure 8-12. Type 1 LPCT Controller Operation.	211
Figure 8-13. Signal Waveforms for Type 1 LPCT Controller.	212
Figure 8-14. LPCT Controller with TAP.	214
Figure 8-15. Signal Waveforms for TAP-based LPCT Controller.	215
Figure 8-16. Before and After EDT and LPCT Controller Logic.	216
Figure 8-17. Scan Test Pattern Timing.	219
Figure 8-18. Chain Test Pattern Timing.	221
Figure 8-19. Type 2 LPCT Design Example.	229
Figure 8-20. Bypass Mode Circuitry.	231
Figure 8-21. Channel Outputs and Pipelining.	248
Figure 8-22. Scan Chain and Bypass Lockup Cells Not in the EDT Scan Chain.	267
Figure 8-23. Scan Chain and Bypass Lockup Cells in the EDT Scan Chain.	268
Figure 8-24. TE CLK to TE CLK.	270
Figure 8-25. LE Clk to TE Clk.	271
Figure 8-26. LE Clk1 to LE Clk2 Overlapping.	271
Figure 8-27. LE ClkS to TE ClkD.	272
Figure 8-28. ClkS to ClkD, Both Clocks Later Than EDT Clock.	272
Figure 8-29. Evaluation Flow.	274
Figure 8-30. Basic Compactor.	280
Figure 8-31. Xpress Compactor.	281
Figure 8-32. X-Blocking in the Compactor.	282
Figure 8-33. X Substitution for Unmeasurable Values.	283
Figure 8-34. Example of Scan Chain Masking.	284
Figure 8-35. Handling of Scan Chain Masking.	284
Figure 8-36. Example of Fault Aliasing.	286
Figure 8-37. Using Masked Patterns to Detect Aliased Faults.	286
Figure 8-38. Handling Scan Chains of Different Length.	287
Figure 9-1. Lockup Cell EDT Controller Input Side.	295
Figure 9-2. Lockup Cells on EDT Controller Output Side.	296
Figure 9-3. EDT IP Creation RTL Stage Flow.	302
Figure 9-4. Create_skeleton_design Inputs and Outputs.	304
Figure 9-5. Skeleton Design Input File Format.	305
Figure 9-6. Skeleton Design Input File Example.	308
Figure C-1. Flow for Debugging Simulation Mismatches.	338
Figure C-2. Order of Diagnostic Checks by the K19 DRC.	343
Figure C-3. Order of Diagnostic Checks by the K22 DRC.	355

List of Tables

Table 1-1. Supported Scan Architecture Combinations	18
Table 4-1. Example Pin Sharing	87
Table 4-2. Default EDT Pin Names	88
Table 5-1. Timing File Variables	126
Table 6-1. Core Instance Parameters and Values by Instrument	140
Table 7-1. Modular Flow Stage Descriptions	161
Table 7-2. Modular Compressed ATPG Command Summary	182
Table 8-1. LPCT Controller Type 1 Commands and Switches	203
Table 8-2. LPCT Controller Type 2 Commands and Switches	205
Table 8-3. LPCT Controller Type 3 Commands and Switches	207
Table 8-4. Lockup Cells Between Decompressor and Scan Chain Inputs	257
Table 8-5. Lockup Cells Between Scan Chain Outputs and Compactor	258
Table 8-6. Bypass Lockup Cells	260
Table 8-7. EDT Lockup and Scan Chain Boundary Lockup Cells	264
Table 8-8. Lockup Insertion Between Channel Outputs and Output Pipeline	273
Table 8-9. Summary of Performance Issues	277

Chapter 1

Getting Started

This manual describes how to integrate Tessent[®] TestKompress[®] into your design process.

More information can be found in the following manuals:

- [Tessent Shell Reference Manual](#) — Contains information on Tessent TestKompress commands and information for all DRCs including the Tessent TestKompress-specific EDT Rules.
- [Tessent Shell User's Manual](#) — Contains information about the Tessent Shell environment in which you use Tessent TestKompress.

For a complete list of Mentor Graphics Tessent-specific terms, including Tessent TestKompress-specific terms, refer to the [Tessent Glossary](#).

Tessent TestKompress	17
EDT Technology	20
Scan Channels.....	20
Structure and Function	21
Test Patterns	22
TestKompress Compression Logic	23
TestKompress Usage Flow Overview	29
EDT IP Creation and Pattern Generation Flow	29
Pre-Synthesis Flow	30
Tessent Core Description (TCD)	30
EDT IP Generation	31
EDT Logic Synthesis	32
EDT Pattern Generation	32
Using TCD-Based Flow with Flattened EDT Hierarchy	34
Tessent Shell User Interface	35

Tessent TestKompress

Tessent TestKompress is a Design-for-Test (DFT) product that creates test patterns and implements compression for the testing of manufactured ICs. Advanced compression reduces ATE memory and channel requirements and reduced data volume results in shorter test application times and higher tester throughput than with traditional ATPG. TestKompress also supports traditional ATPG.

Tessent TestKompress creates and embeds compression logic (EDT logic) and generates compressed test patterns as follows:

- **Test patterns** — Compressed test patterns are generated and loaded onto the Automatic Test Equipment (ATE).
- **Embedded logic** — EDT logic is generated and embedded in the IC to:
 - a. Receive the compressed test patterns from the ATE and decompress them.
 - b. Deliver the uncompressed test patterns to the core design for testing.
 - c. Receive and compress the test results and return them to the ATE.

Tessent TestKompress is command-line driven from Tessent Shell:

- The IP Creation phase of Tessent TestKompress is executed in the Tessent Shell “dft -edt” context.
- The Pattern Generation phase of Tessent TestKompress is executed in the Tessent Shell “patterns -scan” context.

Supported Test Patterns

Tessent TestKompress supports most types of test patterns except the following:

- Random pattern generation.
- Tessent FastScan™ MacroTest. You can only apply MacroTest patterns to a design with Tessent TestKompress by accessing the scan chains directly, bypassing the EDT logic.

Supported Scan Architectures

Tessent TestKompress logic supports mux-DFF and LSSD or a mixture of the scan architectures as listed in [Table 1-1](#).

Table 1-1. Supported Scan Architecture Combinations

EDT Logic	Supported Scan Architectures
DFF-based	LSSD, Mux-DFF, and mixed
Latch-based	LSSD

Tessent TestKompress Inputs

You need the following components to use Tessent TestKompress:

- Scan-inserted gate-level Verilog netlist.
- Synthesis tool.

- Compatible Tessent cell library of the models used for your design scan circuitry. If necessary, you can convert Verilog libraries to a compatible Tessent cell library format with the LibComp utility. For more information, see [“Create Tessent Simulation Models Using LibComp”](#) in the *Tessent Cell Library Manual*.
- Timing simulator such as ModelSim.

Potential Affects of Tessent TestKompress on the Design

Depending on the configuration and placement of the EDT logic, your design may be affected as follows:

- **Extra Level of Hierarchy** — If you place the EDT logic outside the core design, you must add a boundary scan wrapper which adds a level of hierarchy.
- **Minimal Physical Space** — The size of the EDT logic is roughly about 25 gates per internal scan chain. The following examples can be used as guidelines to roughly estimate the size of the EDT logic for a design:
 - For a one million gate design with 200 scan chains, the logic BIST controller including PRPG, MISR and the BIST controller, is 1.25 times the size of the EDT logic for 16 channels.
 - For a one million gate design configured into 200 internal scan chains, the EDT logic including decompressor, compactor, and bypass circuitry with lockup cells requires less than 20 gates per chain. The logic occupies an estimated 0.35% of the area. The size of the EDT logic does not vary significantly based on the size of the design.
 - For 8 scan channels and 100 internal scan chains, the EDT logic was found to be twice as large as a TAP controller, and 19% larger than the MBIST controller for a 1k x 8-bit memory.

EDT Technology

Embedded Deterministic Testing (EDT) is the technology used by Tessent TestKompress. EDT technology is based on traditional, deterministic ATPG and uses the same fault models to obtain similar test coverage using a familiar flow. EDT extends ATPG with improved compression of scan test data and a reduction in test time.

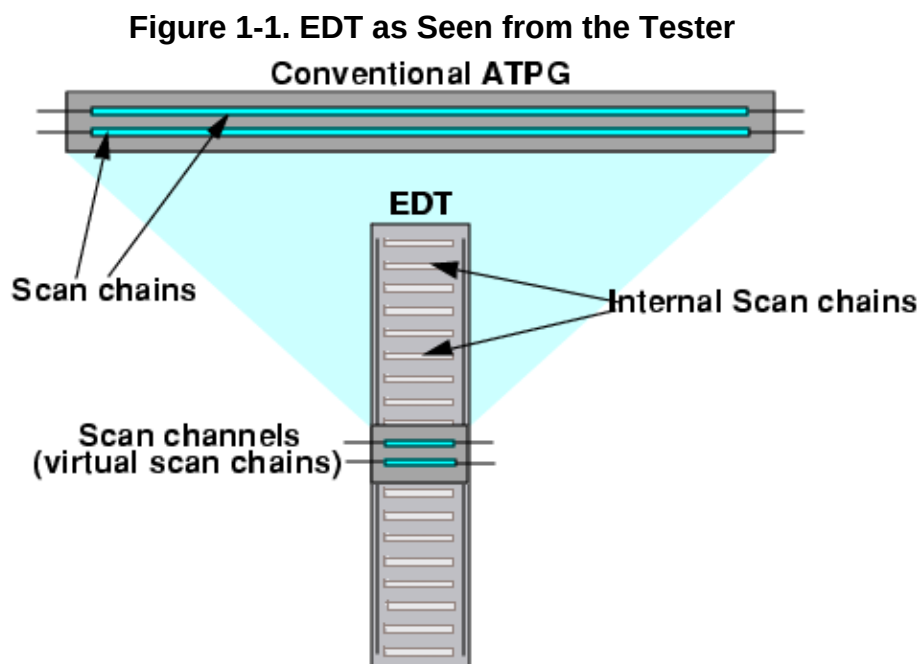
Tessent TestKompress achieves compression of scan test data by controlling a large number of internal scan chains using a small number of scan channels. Scan channels can be thought of as *virtual* scan chains because, from the point of view of the tester, they operate exactly the same as traditional scan chains. Therefore, any tester that can apply traditional scan patterns can apply compressed patterns as described in the following topics:

Scan Channels	20
Structure and Function	21
Test Patterns	22

Scan Channels

With Tessent TestKompress, the number of internal scan chains is significantly larger than the number of external *virtual* scan chains the EDT logic presents to the tester.

Figure 1-1 illustrates conceptually how a design tested with EDT technology is seen from the tester compared to the same design tested using conventional scan and ATPG.



Under EDT methodology, the *virtual* scan chains are called *scan channels* to distinguish them from the scan chains inside the core. Their number is significantly less than the number of internal scan chains. The amount of compression is determined by two parameters:

- Number of scan chains in the design core
- Number of scan channels presented to the tester

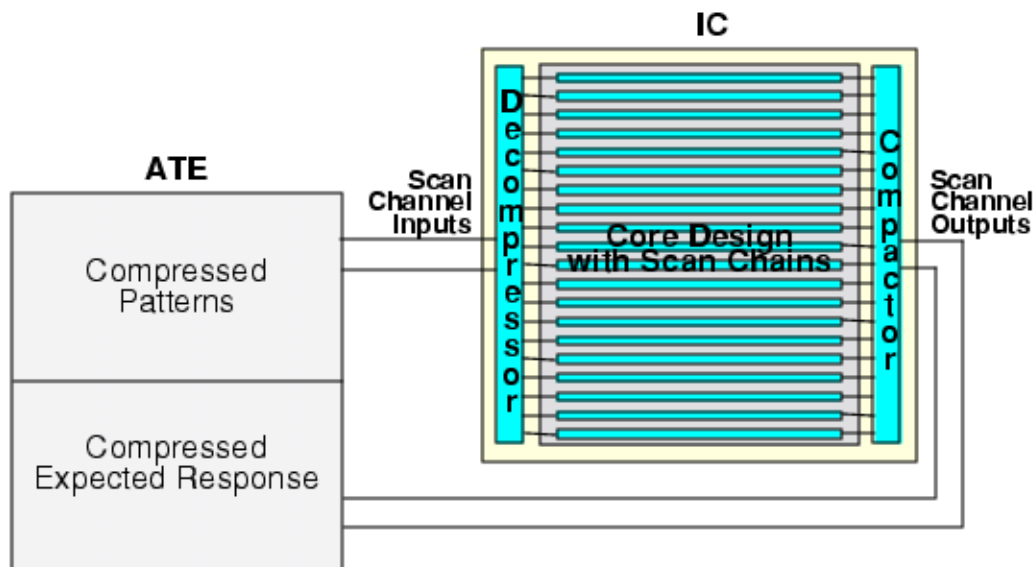
For more information on establishing a compression target for your application, see “[Effective Compression](#)” on page 27 and “[Compression Analysis](#)” on page 64.

Structure and Function

EDT technology consists of logic embedded on-chip, EDT-specific DRCs, and a deterministic pattern generation technique.

The embedded logic includes a decompressor located between the external scan channel inputs and the internal scan chain inputs, and a compactor located between the internal scan chain outputs and the external scan channel outputs. See [Figure 1-2](#).

Figure 1-2. Tester Connected to a Design with EDT



You have the option of including bypass circuitry for which a third block (not shown) is added. No additional logic (test points or X-bounding logic) is inserted into the core of the design. Therefore, EDT logic affects only scan channel inputs and outputs, and thus has no effect on functional paths.

[Figure 1-2](#) shows an example design with two scan channels and 20 short internal scan chains. From the point of view of the ATE, the design appears to have two scan chains, each as long as the internal scan chains. Each compressed test pattern has a small number of *additional shift*

cycles, so the total number of shifts per pattern would be slightly more than the number of scan cells in each chain.

Note

 The term *additional shift cycles* refers to the sum of the initialization cycles, masking bits (when using Xpress), low-power bits (when using a low-power decompressor), and user-defined pipeline bits.

You can use the following equation to predict the number of initialization cycles the tool adds to each pattern load. In this equation, *ceil* indicates the ceiling function that rounds a fraction to the next highest integer. This equation applies except when you have very few channels in which case there are four extra cycles per scan load. Note, this equation does not factor in additional shift cycles added to support masking and low-power.

$$\text{Number of initialization cycles} = \text{ceil}\left(\frac{\text{decompressor size}}{\text{number of channels}}\right)$$

For example, if a design has 16 scan channels, 1250 scan cells per chain, and a 50-bit decompressor, we can calculate the number of initialization cycles as 4 by using the above formula. Since each chain has 1,250 scan cells and each compressed pattern requires four initialization cycles, the tester sees a design with 16 chains requiring 1,254 shifts per pattern.

Note

 The EDT IP creation phase and ATPG generation phase may report a different number of initialization cycles depending on whether low power is enabled. Enabling low power increases the number of initialization cycles in the EDT IP creation phase.

Test Patterns

Tessent Shell generates compressed test patterns specifically for on-chip processing by the EDT logic. For a given testable fault, a compressed test pattern satisfies ATPG constraints and avoids bus contention, similar to conventional ATPG.

A set of compressed test patterns is stored on the ATE and each test pattern applies data to the inputs of the decompressor and holds the responses observed on the outputs of the compactor. The ATE applies the compressed test patterns to the circuit through the decompressor, which lies between the scan channel pins and the internal scan chains. From the perspective of the tester, there are relatively few scan chains present in the design.

The compressed test patterns, after passing through the decompressor, create the necessary values in the scan chains to guarantee fault detection. The functional input and output pins are directly controlled (forced) and observed (measured) by the tester, same as in a conventional test. On the output side of the internal scan chains, hardware compactors reduce the number of internal scan chains to feed the smaller number of external channels. The response captured in

the scan cells is compressed by the compactor and the compressed response is compared on the tester. The compactor ensures faults are not masked and X-states do not corrupt the response.

You define parameters, such as the number of scan channels and the insertion of lockup cells, which are also part of the RTL code. The tool automatically determines the internal structure of the EDT hardware based on the parameters you specify, the number of internal scan chains, the length of the longest scan chain, and the clocking of the first and last scan cell in each chain. Test patterns include parallel and serial test benches for Verilog as well as parallel and serial WGL, and most other formats supported formats.

TestKompress Compression Logic

Tessent TestKompress generates hardware in blocks in VHDL or Verilog RTL. You integrate the compression logic (EDT logic) into your design by using Tessent Shell with the core level of the design. The tool then generates the following three components:

- **Decompressor** — Feeds a large number of scan chains in your core design from a small number of scan channels, and decompresses EDT scan patterns as they are shifted in.

The decompressor resides between the channel inputs (connected to the tester) and the scan chain inputs of the core. Its main parts are a Linear Feedback Shift Machine (LFSM) and a phase shifter.

- **Compactor** — Compacts the test responses from the scan chains in your core design into a small number of scan output channels as they are shifted out.

The compactor resides between the core scan chain outputs and the channel outputs connected to the tester. It primarily consists of spatial compactor(s) and gating logic.

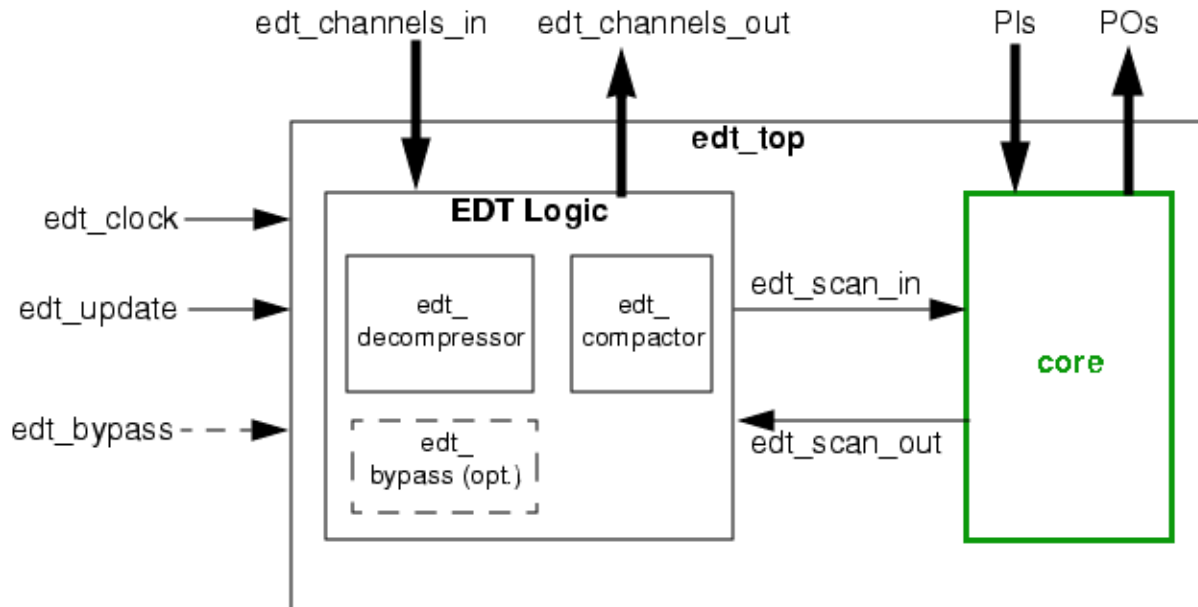
- **Bypass Module (Optional)** — Bypasses the EDT logic by using multiplexers (and lockup cells if necessary) to concatenate the internal scan chains into fewer, longer chains. Enables you to access the internal scan chains directly through the channel pins. Generated by default.

If you choose to implement bypass circuitry, the tool includes bypass multiplexers in the EDT logic. See [“Compression Bypass Logic”](#) on page 230 for a discussion of bypass mode. You can also insert the bypass logic in the netlist at scan insertion time to facilitate design routing. For more information, see [“Insertion of Bypass Chains in the Netlist”](#) on page 54.

These three components are all contained within the EDT logic block that, by default, is instantiated in a top-level “wrapper” module. The design core is also instantiated in the top-level wrapper. [Figure 1-3](#) illustrates this conceptually.

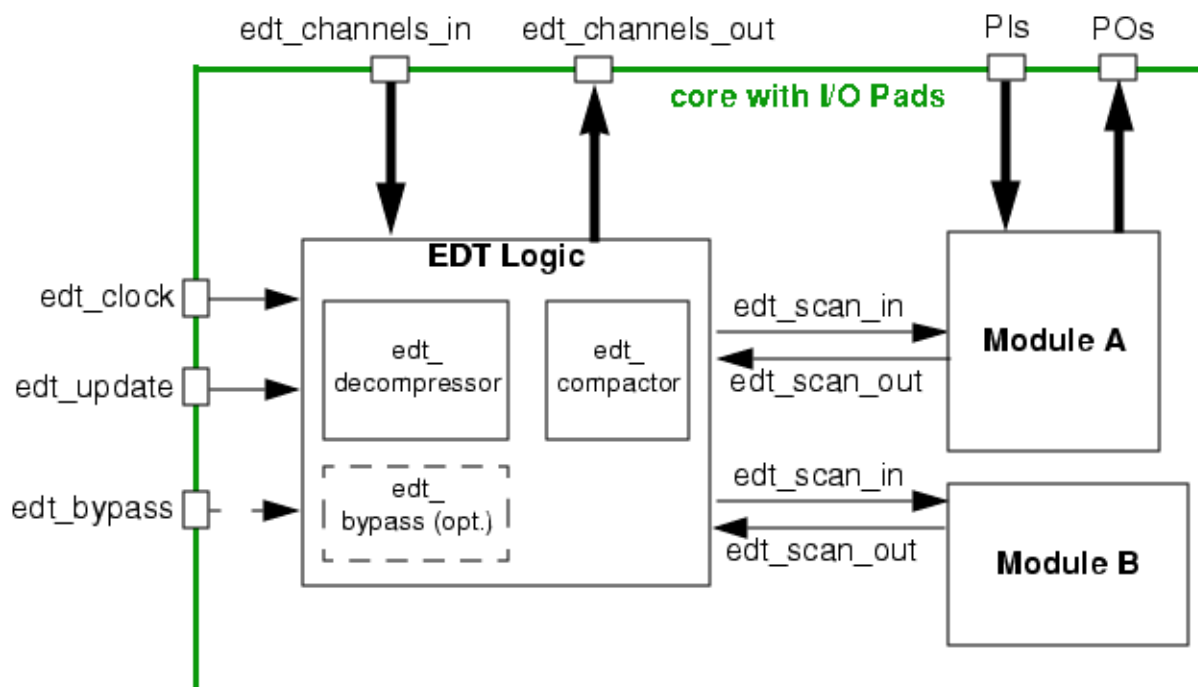
You insert pads and I/O cells on this new top level. Because the EDT logic is outside the core design (that is, outside the netlist used in Tessent Shell), the tool flow you use to implement this configuration is referred to as the external EDT logic location flow, or simply the “external flow.”

Figure 1-3. EDT logic Located Outside the Core (External Flow)



Alternatively, you can invoke Tessent Shell and use a design that already contains I/O pads. For these designs, the tool enables you to insert the EDT logic block in the existing top level within the original design. Figure 1-4 shows this conceptually. Because the EDT logic is instantiated within the netlist used in Tessent Shell, this configuration is referred to as the internal EDT logic location flow or simply the “internal flow.”

Figure 1-4. EDT logic Located Within the Core (Internal Flow)



By default, the tool automatically inserts lockup cells as needed in the EDT logic. They are placed within the EDT logic, between the EDT logic and the design core, and in the bypass circuitry that concatenates the scan chains. “[Understanding Lockup Cells](#)” on page 254 describes in detail how the tool determines where to insert lockup cells.

DRC Rules

Tessent TestKompress performs the same ATPG design rules checking (DRC) after design flattening that Tessent FastScan performs. A detailed discussion of DRC is included in “[ATPG Design Rules Checking](#)” in the *Tessent Scan and ATPG User’s Manual*.

In addition, Tessent TestKompress also runs a set of DRCs specifically for EDT. For more information, see “[Design Rule Checks](#)” on page 94.”

Internal Control

In many cases, it is preferable to use internal controllers (JTAG or test registers) to control EDT signals, such as `edt_bypass`, `edt_update`, `scan_en`, and to disable the `edt_clock` in functional mode. For detailed information about how to do this with boundary scan, refer to “[Uncompressed ATPG \(External Flow\) and Boundary Scan](#)” on page 240.

Logic Clocking

The default EDT logic contains combinational logic and flip-flops. All the flip-flops, except lockup cells, are positive edge-triggered and clocked by a dedicated clock signal that is different from the scan clock. There is no clock gating within the EDT logic, so it does not interfere with the system clock(s) in any way.

You can set up the clock to be a dedicated pin (named `edt_clock` by default) or you can share the clock with a functional non-clock pin. Such sharing may cause a decrease in test coverage because the tool constrains the clock pin during test pattern generation. You must not share the `edt_clock` with another clock or RAM control pin for several reasons:

- If shared with a scan clock, the scan cells may be disturbed when the `edt_clk` is pulsed in the load_unload procedure during pattern generation.
- If shared with RAM control signals, RAM sequential patterns and multiple load patterns may not be applicable.
- If shared with a non-scan clock, test coverage may decline because the `edt_clk` is constrained to its off-state during the capture cycle.

Because the clock used in the EDT logic is different than the scan clock, lockup cells can be inserted automatically between the EDT logic and the scan chains as needed. The tool inserts lockup cells as part of the EDT logic and never modifies the design core.

Note

 You can set the EDT clock to pulse before the scan chain shift clocks and avoid having lockup cells inserted. For more information, see “[Pulse EDT Clock Before Scan Shift Clocks](#)” on page 81.

Latch-based EDT logic uses two clocks (a master and a slave clock) to drive the logic. For reasons similar to those listed above for DFF-based logic, you must not share the master EDT clock with the system master clock. You can, however, share the slave EDT clock with the system slave clock.

Note

 During the capture cycle, the system slave clock, which is shared with the slave EDT clock, is pulsed. This does not affect the EDT logic because the values in the master latches do not change. Similarly, in the load_unload cycle, although the slave EDT clock is pulsed, the value at the outputs of the system slave latches is unchanged because the slave latches capture old values.

In a skew load procedure, when a master clock is only pulsed at the end of the shift cycle (so different values can be loaded in the master and slave latches), the EDT logic is unaffected because the master EDT clock is not shared.

ASCII and Binary Patterns

Compressed ATPG test patterns can be written out in ASCII and binary formats, and can also be read back into the tool. As with uncompressed patterns, you use these formats primarily for debugging simulation mismatches and archiving. However, there are some differences with compressed and uncompressed patterns as follows:

- Compressed and uncompressed ASCII patterns are different in several ways. When you create patterns with compression, the captured data is stored with respect to the internal scan chains and the load data is stored with respect to the external scan channels. The load data in the pattern file is in compressed format—the same form it is fed to the decompressor.
- With the simulation of compressed patterns, Xs may not be due to capture; they may result from the emulation of the compactor. For a detailed discussion of this effect and how masking is done with compressed patterns, refer to “[Understanding Scan Chain Masking in the Compactor](#)” on page 282.

Fault Models and Test Patterns

For compression, the tool uses fault-model independent and pattern-type independent compression algorithms. The compression technology supports all fault models (stuck-at, transition, Iddq, and path delay) and deterministic pattern types (combinational, RAM sequential, clock-sequential, and multiple loads) supported and/or generated by uncompressed ATPG.

To summarize, the compression technology:

- Accepts the same fault models as uncompressed ATPG.
- Accepts the same deterministic pattern types as uncompressed ATPG with the exception of MacroTest which is not supported.
- Produces the same test coverage as uncompressed ATPG.

Effective Compression

Effective compression is the actual compression achieved for a specific test application. The effective compression is determined by balancing the EDT compression characteristics with the test environment/design needs.

The effective compression is limited by many parameters, including the following:

- Number of scan chains in your design core
- Number of scan channels presented to the tester

Use the following ratio to determine the chain to channel ratio for an application:

$$\text{Chain to channel ratio} \cong \frac{\# \text{ of Scan Chains}}{\# \text{ of Scan Channels}}$$

The effective compression achieved for a design is always less than the chain to channel ratio because the EDT technology generates more test patterns than traditional ATPG. With EDT technology, compression is achieved by reducing the amount of data per test pattern and not by reducing the number of test patterns generated. Consequently, additional test patterns require additional shift cycles that reduce the overall compression.

Note



The term *additional shift cycles* refers to the sum of the initialization cycles, masking bits (when using Xpress), and low-power bits (when using a low-power decompressor).

It is also important to balance the compression target with the testing resources and design needs. Using an unnecessarily large compression target may have an adverse affect on compression, testing quality, and design layout as follows:

- **Lower Test Coverage** — Higher compression ratios increase the compression per test pattern but also increase the possibility of generating test patterns that cannot be compressed and can lead to lower test coverage.
- **Decrease in Overall Compression** — Higher compression ratios also decrease the number of faults that dynamic compaction can fit into a test pattern. This can increase the total number of test patterns and, therefore, decrease overall compression.

- **Routing Congestion** — There is no limit to the number of internal scan chains, however, routing constraints may limit the compression ratio. Most practical configurations will not exceed the compression capacity.

For more information on determining the right compression for your design, see “[Compression Analysis](#)” on page 64.

TestKompress Usage Flow Overview

This section describes the default Tessent TestKompress flow by briefly introducing the steps required to incorporate EDT into a gate-level Verilog netlist.

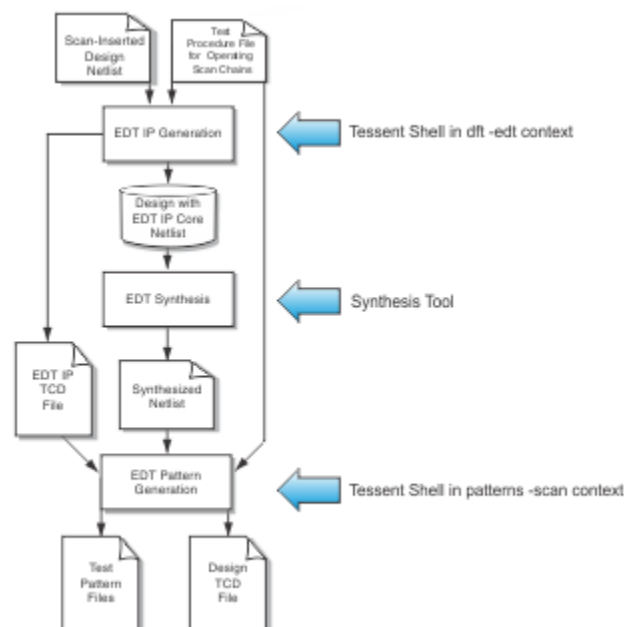
EDT IP Creation and Pattern Generation Flow	29
Pre-Synthesis Flow	30
Tessent Core Description (TCD)	30
EDT IP Generation	31
EDT Logic Synthesis	32
EDT Pattern Generation	32
Using TCD-Based Flow with Flattened EDT Hierarchy	34

EDT IP Creation and Pattern Generation Flow

The post-synthesis EDT IP creation and pattern generation flow allows you to generate the EDT logic and subsequently create patterns for the logic.

Figure 1-5 illustrates this flow.

Figure 1-5. Post-Synthesis EDT IP Creation and EDT Pattern Generation Flow



This flow consists of the following stages:

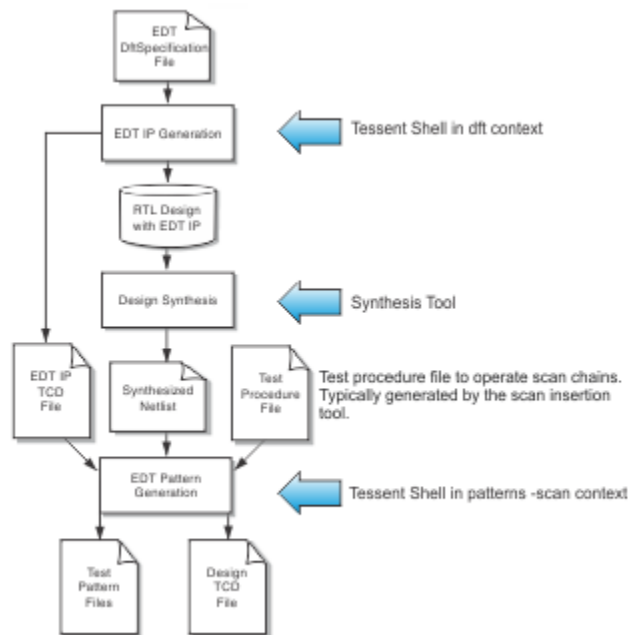
- **EDT IP Creation** — Use Tessent Shell in EDT IP generation and insertion context (dft -edt) to create the EDT IP and write the EDT logic and the TCD file. Refer to “[Creation of EDT Logic Files](#)” on page 96.

- **EDT Pattern Generation** — Use a design with inserted EDT IP and Tessent Shell in test pattern generation context (patterns -scan) to generate patterns. See “[EDT Pattern Generation Overview](#)” on page 136.

Pre-Synthesis Flow

When using the pre-synthesis flow, the tool extracts the EDT IP during the pattern generation phase and configures the tool to use the extracted IP.

Figure 1-6. Pre-Synthesis EDT IP Creation and EDT Pattern Generation TCD Flow



Tessent Core Description (TCD)

The Tessent Core Description (TCD) is a single file that contains the EDT IP core description and eliminates the use of multiple dofiles and test procedure files for pattern generation. Use of a TCD file supersedes the EDT mapping functionality for automating ATPG setup of the EDT IP.

The TCD file is generated with the `write_edt_files` command along with the other EDT logic files during EDT IP generation. The TCD file contains the description of the generated EDT IP.

With a TCD file, Tessent Shell can automatically extract the connectivity between the EDT IP and the chip, apply any needed adjustment to test procedures, and allow pattern generation. Refer to “[Creation of EDT Logic Files](#)” on page 96 for more information.

Note

The EDT IP TCD file describes the configuration of the EDT IP. You should never modify the TCD file.

If your EDT IP can operate in multiple configurations (for example, low power, bypass, and so on), then a single TCD file contains all the configurations in contrast to the multiple EDT IP dofile usage. During pattern generation, you will be able to specify how you want those parameters of the EDT IP configured for that ATPG mode.

If you are using a Low Pin Count Test (LPCT) controller, the tool also creates a LPCT-specific TCD file that you use for pattern generation—see “[Low Pin Count Test Controller](#)” on page 195.

EDT IP Generation

The following steps demonstrate the basic EDT post-synthesis IP creation flow.

Procedure

1. Invoke Tessent Shell.

```
<Tessent_Tree_Path>/bin/tessent -shell -dofile edt_ip_creation.do \  
-logfile ../transcripts/edt_ip_creation.log -replace
```

2. Provide Tessent Shell commands. For example:

Tip

The following commands can be located in the dofile used for invocation in step 1.

```
// Set context, read library,
read and set current design
SETUP> set_context dft -edt
SETUP> read_verilog gatelevel_netlist.v
SETUP> read_cell_library atpg.lib
SETUP> set_current_design top
// Setup Scan Chains and Clocks
SETUP> add_scan_groups grp1 ../generated/atpg.testproc
SETUP> add_scan_chains chain1 grp1 edt_si1 edt_so1
SETUP> add_scan_chains chain2 grp1 edt_si2 edt_so2
...
SETUP> add_scan_chains chain5 grp1 edt_si5 edt_so5
SETUP> analyze_control_signals -auto_fix
// Specify the number of scan channels.
SETUP> set_edt_options -channels 1
// Flatten the design, run DRCs.
SETUP> set_system_mode analysis
// Verify the EDT configuration is as expected.
ANALYSIS> report_edt_configurations -verbose
// Generate the RTL EDT logic and save it.
ANALYSIS> write_edt_files created -verilog -replace
// The write_edt_files command also creates a
// Tessent Core Description file
// At this point, you can optionally create patterns
//(without saving them)
// to get an estimate of the potential test coverage.
ANALYSIS> create_patterns
// Create reports
ANALYSIS> report_statistics
ANALYSIS> report_scan_volume
// Close the session and exit.
ANALYSIS> exit
```

EDT Logic Synthesis

You must synthesize the design before you generate EDT patterns.

Procedure

Run Design Compiler.

Note



The Design Compiler synthesis script referenced in the following invocation line is output from the “write_edt_files” command in preceding step 2.

```
dc_shell -f ../created_dc_script.scr |& tee ../transcripts/
dc_edt.log
```

EDT Pattern Generation

The following steps demonstrate the basic EDT pattern generation flow.

Procedure

1. Invoke Tessent Shell.

Note

 The netlist `created_edt_top_gate.v` referenced in the following invocation line is output from Design Compiler—see [EDT Logic Synthesis](#).

```
<Tessent_Tree_Path>/bin/tessent -shell -logfile  
../transcripts/edt_pattern_gen.log -replace
```

2. Provide Tessent Shell commands. For example:

```
// Set context, read library, read and set current design  
SETUP> set_context patterns -scan  
SETUP> read_verilog created_edt_top_gate.v  
SETUP> read_cell_library atpg.lib  
SETUP> set_current_design top  
  
// Read the TCD file for EDT IP using the read_core_description command.  
// For example:  
SETUP> read_core_description created_cpu_edt.tcd  
  
// Define parameter values to automatically configure the EDT logic using the  
// add_core_instances command. For example:  
SETUP> add_core_instances -core cpu_edt -modules cpu_edt -parameter_values \  
    {edt_bypass off}  
  
// Add top-level clocks driving the scan changes using the add_clocks command.  
// Provide the top-level test procedure file using the set_procfile_name command.  
// For example:  
SETUP> set_procfile_name created_cpu_edt.testproc  
  
// Change the system mode to Analysis using the set_system_mode  
// command as follows:  
SETUP> set_system_mode analysis  
  
// Verify the EDT configuration.  
ANALYSIS> report_edt_configurations  
  
// Generate patterns.  
ANALYSIS> create_patterns  
  
// Create reports.  
ANALYSIS> report_statistics  
ANALYSIS> report_scan_volume
```

```
// Save the patterns in ASCII format.  
ANALYSIS> write_patterns ../generated/patterns_edt.ascii -ascii -replace  
  
// Save the patterns in parallel and serial Verilog format.  
ANALYSIS> write_patterns ../generated/patterns_edt_p.v -verilog -replace -parallel  
ANALYSIS> write_patterns ../generated/patterns_edt_s.v -verilog -replace -serial \  
-sample 2  
  
// Save the patterns in tester format; WGL for example.  
ANALYSIS> write_patterns ../generated/test_patterns.wgl -wgl -replace  
  
// Optionally write out the core description corresponding to  
// the current chip level using the write_core_description command.  
  
// For example:  
ANALYSIS> write_core_description cpu_core_final.tcd -replace  
  
// Close the session and exit.  
ANALYSIS> exit
```

Using TCD-Based Flow with Flattened EDT Hierarchy

This procedure describes using the TCD-based flow with a flattened EDT hierarchy.

In certain flows and design styles, the top-level hierarchy of the EDT IP is flattened during synthesis or layout. In this case, there may not be an instance that can be associated with the core description in the TCD file of the EDT IP mapping flow. This is needed for ATPG and any mode when the EDT IP needs to be used.

In this situation, you need to create module-level TCD files for various modes of operation prior to flattening the hierarchy as follows:

Procedure

1. Setup the core with each mode, for example ATPG for a particular fault model.
2. Run DRC using this command:
set_system_mode analysis
3. Use the [write_core_description](#) command to save the TCD file to be used with the design that does not have hierarchy for the EDT IP.

Results

With the completion of these steps, the information for the core instance is ready for you to use in other procedures, such as ATPG for a particular mode.

You can use the [add_core_instances](#) command before or after flattening to define the core instance for other uses:

```
add_core_instances ... -current_design
```

Note

 If you store the flat model after core instances have been added, you do not need to provide TCD files externally before reading the flat model back; the core instance data is stored in the flat model.

Tessent Shell User Interface

Tessent Shell is a command line driven tool that provides access to Tessent FastScan for uncompressed ATPG and to Tessent TestKompress for compressed ATPG.

Invocation

You can invoke Tessent Shell from the command line by entering the `tessent` command. For example:

```
prompt> <Tessent_Tree_Path>/bin/tessent -shell -dofile edt_ip_creation.do \  
-logfile ../transcripts/edt_ip_creation.log -replace
```

To exit Tessent Shell and return to the operating system, type “exit” at the command line:

```
prompt> exit
```

For more information on invoking Tessent Shell, see the [tessent](#) command in the *Tessent Shell Reference Manual*.

Uncompressed and Compressed ATPG

For uncompressed ATPG, you use Tessent Shell in the “patterns -scan” context.

For compressed ATPG, you use Tessent Shell in the “dft -edt” context to create the EDT logic, and in the “patterns -scan” context to generate compressed test patterns.

EDT must be on whenever you are creating test patterns or EDT logic. You can use the [report_environment](#) command to check the tool status. You can use the [set_edt_options](#) command to enable compression.

For more information about Tessent Shell and contexts, see “[Tessent Shell Introduction](#)” in the *Tessent Shell User’s Manual*.

Supported Design Format

For pattern generation, you can read in a scan-inserted gate-level Verilog netlist and a compatible Tessent cell library of the models used for the scan circuitry.

For more information on the Tessent cell library, see “[Create Tessent Simulation Models Using LibComp](#)” in the *Tessent Cell Library Manual*.

Batch Mode

You can run Tessent Shell in batch mode by using a *dofile* to pipe commands into the application. Dofiles let you automatically control the operations of the tool. The dofile is a text file you create that contains a list of application commands that you want to run, but without entering them individually. If you have a large number of commands, or a common set of commands you use frequently, you can save time by placing these commands in a dofile.

If you place all commands, including the exit command, in a dofile, you can run the entire session as a batch process from the command line. Once you generate a dofile, you can run it at invocation.

For example, to run a dofile as a batch process using the commands contained in the dofile *my_dofile.do*, enter:

```
<Tessent_Tree_Path>/bin/tessent -shell -dofile my_dofile.do
```

The following shows an example Tessent Shell dofile:

```
// my_dofile.do
//
// Dofile for EDT logic Creation Phase.

// Execute setup script from Tessent Scan.
dofile edt_ip_creation.do

// Set up EDT.
set_edt_options -channels 2

// Run DRC.
set_system_mode analysis

// Report and write EDT logic.
report_edt_configurations
report_edt_pins
write_edt_files created -verilog -replace

// Exit.
exit
```

By default, if the tool encounters an error when running one of the commands in the dofile, it stops dofile execution. However, you can turn this setting off or specify to exit to the shell prompt by using the [set_dofile_abort](#) command.

Log Files

Log files provide a useful way to examine the operation of the tool, especially when you run the tool in batch mode using a dofile. If errors occur, you can examine the log file to see exactly

what happened. The log file contains all DFT application operations and any notes, warnings, or error messages that occur during the session.

You can generate log files by using the `-Logfile` switch when you invoke the tool. When setting up a log file, you can instruct Tessent Shell to generate a new log file, replace an existing log file, or append information to a log file that already exists.

You can also use the [set_logfile_handling](#) command to generate a log file during a tool session.

Note



A log file created during a tool session only contains notes, warnings, and error messages that occur after you issue the `set_logfile_handling` command. Therefore, you should enter it as one of the first commands in the session.

System Commands

You can run operating system commands within Tessent Shell by using the “`system`” command. For example, the following command executes the operating system command **date** within a Tessent Shell session:

```
prompt> system date
```


Chapter 2

The Compressed Pattern Flows

The compressed internal and external pattern flows compare the basic steps and tools used for an uncompressed ATPG top-down design flow with the steps and tools used to incorporate compressed patterns in both an external and an internal flow. These flows primarily show the typical top-down design process flow using a structured compression strategy.

Figures [2-1](#) and [2-2](#) illustrate the steps in the APTG flow (shown in grey); it also mentions certain aspects of other design steps, where applicable. For more information on the ATPG flow, see the *Tessent Scan and ATPG User's Manual*.

Figure 2-1. Top-Down Design Flow - External

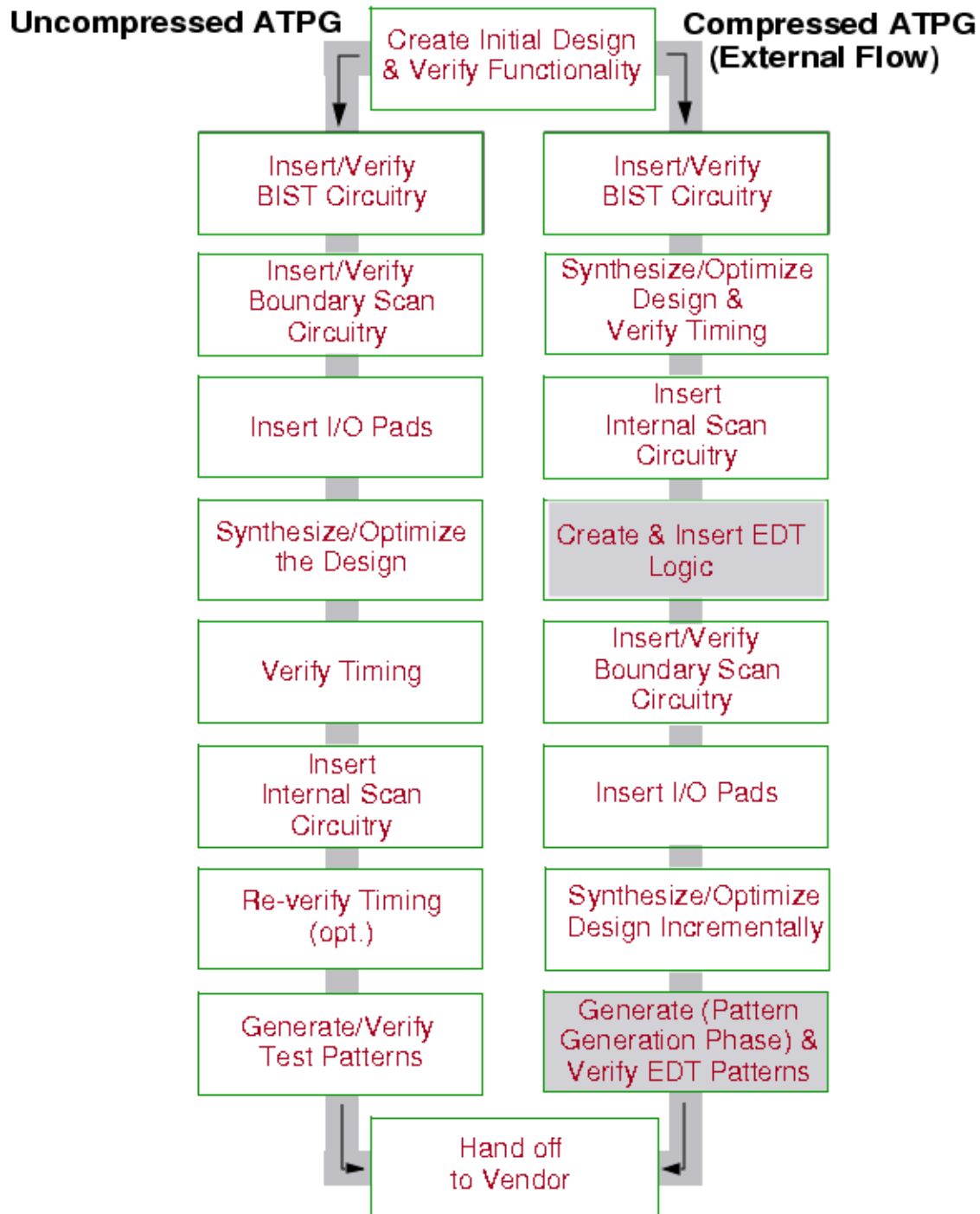
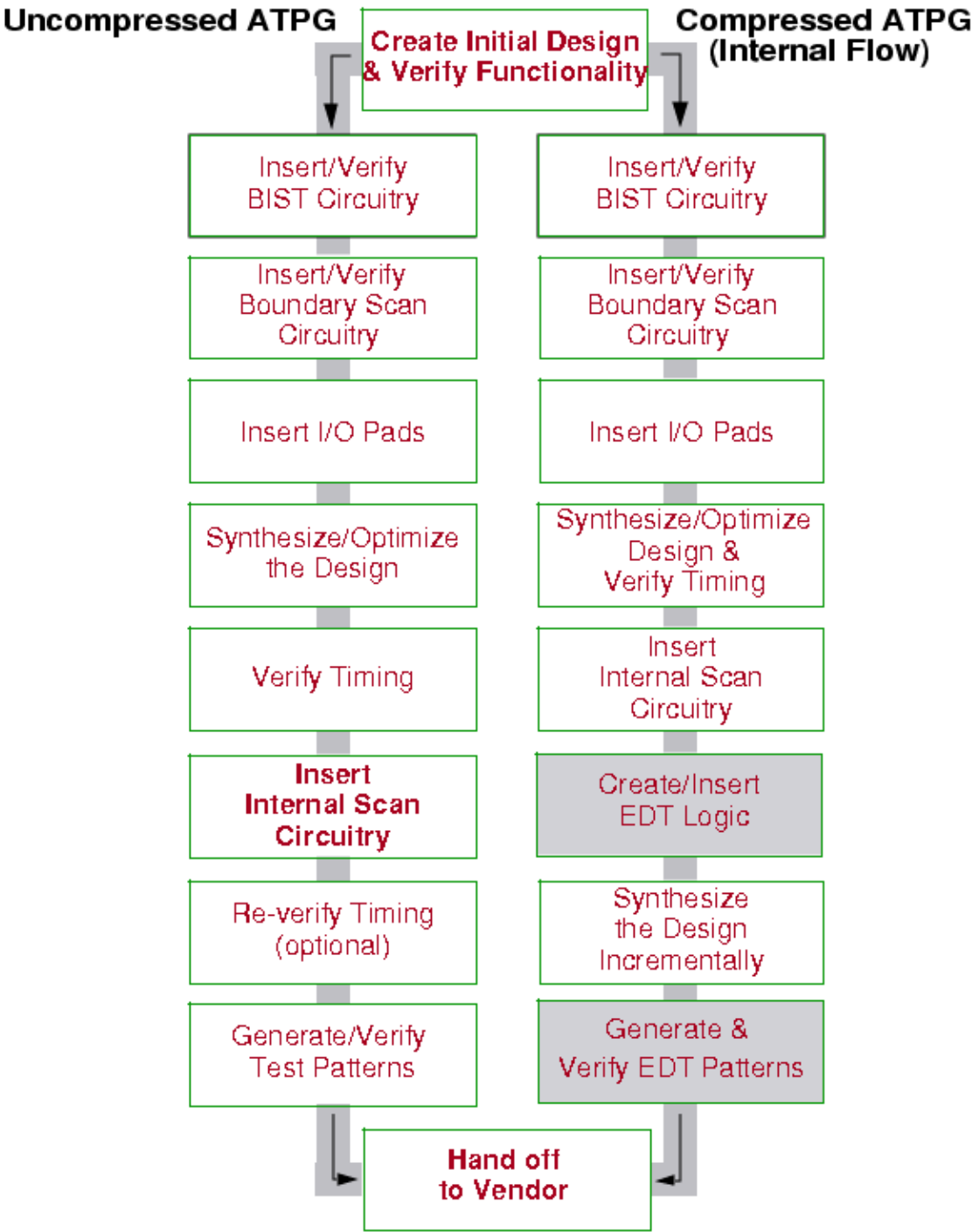


Figure 2-2. Top-Down Design Flow - Internal



Top-Down Design Flows	42
The Compressed Pattern Flows	44
Design Requirements for a Compressed Pattern Flow	44

Compressed Pattern External Flow.	45
Compressed Pattern Internal Flow	48

Top-Down Design Flows

The first task in any design flow is to create the initial register transfer level (RTL) design, using whatever means you choose. If your design is in Verilog format and contains memory models, you can add built-in self-test (BIST) circuitry to your RTL design. You then choose to use either an uncompressed or a compressed pattern flow.

Uncompressed ATPG Flow

Commonly, in an ATPG flow that does not use compression, you would next insert and verify I/O pads and boundary scan circuitry. Then, you would synthesize and optimize the design using the Synopsys Design Compiler tool or another synthesis tool, followed by a timing verification with a static timing analyzer such as PrimeTime.

After synthesis, you are ready to insert internal scan circuitry into your design using Tessent Scan. In the uncompressed ATPG flow, after you insert scan, you could optionally re-verify the timing because you added scan circuitry. Once you were sure the design is functioning as desired, you would generate test patterns using Tessent FastScan and generate a test pattern set in the appropriate format.

Compressed Pattern Flows

Compared to an uncompressed ATPG flow, a compressed pattern flow can take one of two paths:

- **External Flow (External Logic Location Flow)** — Differs from the uncompressed ATPG flow in that you do not insert I/O pads and boundary scan until *after* you run Tessent Shell with the scan-inserted core to insert the EDT logic. The EDT logic is located external to the design netlist.
- **Internal Flow (Internal Logic Location Flow)** — Similar to an uncompressed ATPG flow, you may insert and verify I/O pads and boundary scan circuitry before you synthesize and optimize the design. The EDT logic is instantiated in the top level of the design netlist, permitting the logic to be connected to internal nodes (I/O pad cells or an internal test controller block, for example) or to the top level of the design. Typically, the EDT logic is connected to the internal nodes of the pad cells used for channel and control signals. You run Tessent Shell with the scan-inserted core that includes I/O pads and boundary scan.

Choosing a Compressed Pattern Flow

You should choose between the external and internal flows based on whether the EDT logic signals need to be connected to nodes internal to the design netlist read into the tool (internal nodes of I/O pads, for example), or whether the EDT logic can be connected to the design using a wrapper.

In the external flow, after you insert scan circuitry the next step is to insert the EDT logic. Following that, you insert and verify boundary scan circuitry if needed. Only then do you add I/O pads. Then, you incrementally synthesize and optimize the design using either Design Compiler or another synthesis tool.

In the internal flow, you can integrate I/O pads and boundary scan into the design before the scan insertion step. Then, after you create and insert the EDT logic, use Design Compiler with the script created by Tessent Shell to synthesize the EDT logic.

In either flow, once you are sure the design is functioning as desired, you generate compressed test patterns. In this step, the tool performs extensive DRC that, among other things, verifies the synthesized EDT logic.

You should also verify that the design and patterns still function correctly with the proper timing information applied. You can use ModelSim or another simulator to achieve this goal. You may then have to perform a few additional steps required by your ASIC vendor before handing off the design for manufacture and testing.

Note



It is important to check with your vendor early in your design process for requirements and restrictions that may affect your compression strategy. Specifically, you should determine the limitations of the vendor's test equipment. To plan effectively for using EDT, you must know the number of channels available on the tester and its memory limits.

The Compressed Pattern Flows

The compressed pattern flow requires the design file format is Verilog, that the design allows access to all clock pins through primary inputs, and special I/O pad considerations based upon internal and external patterns.

Note

 Tessent Shell supports mux-DFF and LSSD scan architectures, or a mixture of the two, within the same design. The tool creates DFF-based EDT logic by default. However, you can direct the tool to create latch-based IP for pure LSSD designs. [Table 1-1](#) on page 18 summarizes the supported scan architecture combinations.

Design Requirements for a Compressed Pattern Flow	44
Compressed Pattern External Flow	45
Compressed Pattern Internal Flow	48

Design Requirements for a Compressed Pattern Flow

Before you begin a compressed pattern flow, you must ensure that your design satisfies a set of prerequisites.

The prerequisites are

- **Format** — Your design input must be in gate-level Verilog. The logic created by Tessent TestKompress is in Verilog or VHDL RTL.
- **Pin Access** — The design needs to allow access to all clock pins through primary input pins. There is no restriction on the number of clocks.
- **I/O Pads**— I/O pad requirements for the external and internal flows are quite different.
 - **External Flow** — The tool creates the EDT logic as a collar around the circuit (see [Figure 1-3](#)). Therefore, the core design ready for logic insertion must consist of only the core *without* I/O pads. In this flow, the tool cannot insert the logic between scan chains and I/O pads already in the design.

Note

 Add the I/O pads around the collar after it is created, but before logic synthesis. The same applies to boundary scan cells: add them after the EDT logic is included in the design.

The design may or may not have I/O pads when you generate test patterns. To determine the expected test coverage, you can perform a test pattern generation trial run on the core when the EDT logic is created before inserting I/O pads.

Note

 You should not save the test patterns generated when the EDT logic is created; these patterns do not account for how I/O pads are integrated into the final synthesized design.

When producing the final patterns for a whole chip, run Tessent Shell on the synthesized design after inserting the I/O pads. For more information, refer to the procedure for managing pre-existing I/O pads in “[Preparation for the External Flow](#)” on page 51.

- **Internal Flow** — The core design, ready for EDT logic insertion, may include I/O pad cells for all the I/Os you inserted before or during initial synthesis. The I/O pads, when included, can be present at any level of the design hierarchy and do not necessarily have to be at the top level. If the netlist includes I/O pads, there should also be some pad cells reserved for EDT control and channel pins that are not going to be shared with functional pins. See “[Functional/EDT Pin Sharing](#)” on page 85 for more information about pin sharing.

Note

 The design may have I/O pads; it is not a requirement. When EDT logic is inserted in the netlist, you can connect it to any internal design nodes or to the top level of the design netlist.

Compressed Pattern External Flow

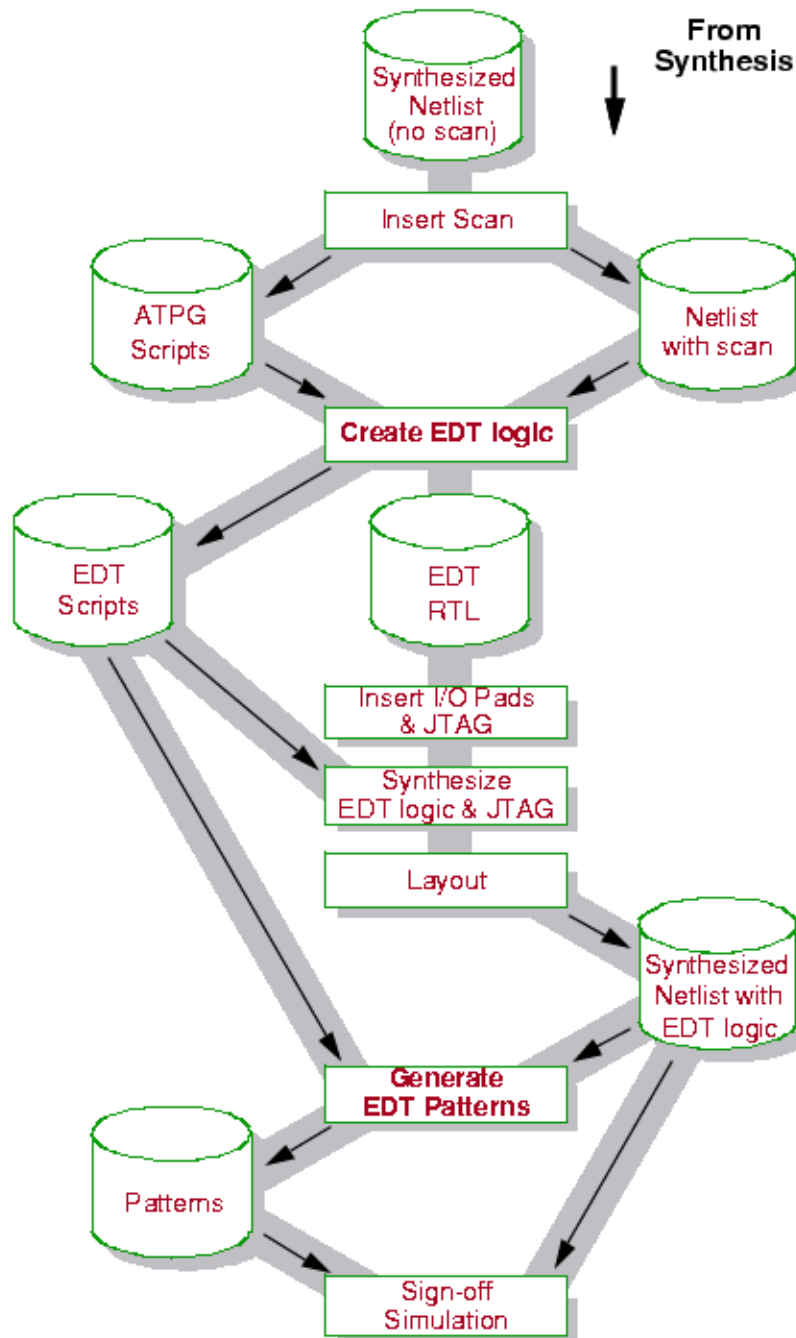
The compressed pattern external flow is focused on EDT logic creation and EDT pattern generation.

[Figure 2-3](#) expands the steps shown in grey in [Figure 2-1](#), and shows the files used in the tool’s external flow. The basic steps in the flow are summarized in the following list.

1. Prepare and synthesize the RTL design.
2. Insert an appropriately large number of scan chains using Tessent Scan or a third-party tool. For information on how to do this using Tessent Scan, refer to “[Internal Scan and Test Circuitry](#),” in the Tessent Scan and ATPG User’s Manual.
3. Optionally, perform an ATPG run on the scan-inserted design without EDT. Use this run to ensure there are no basic issues such as simulation mismatches caused by an incorrect library. If you want, you can run Tessent Shell in “patterns -scan” context to perform this step.
4. Optionally, simulate the patterns created in step 3.
5. EDT Logic Creation Phase: Invoke Tessent Shell with the scan-inserted gate-level description of the core without I/O pads or boundary scan. Create the RTL description of the EDT logic.

6. Insert I/O pads and boundary scan (optional).
7. Incrementally synthesize the I/O pads, boundary scan, and EDT logic.
8. EDT Pattern Generation Phase: After you insert I/O pads and boundary scan, and synthesize all the added circuitry (including the EDT logic), invoke Tessent Shell with the synthesized top-level Verilog netlist and generate the EDT test patterns. You can write test patterns in a variety of formats including Verilog and WGL.
9. Simulate the compressed test patterns that you created in the preceding step 8. As for regular ATPG, the typical practice is to simulate all parallel patterns and a sample of serial patterns.

Figure 2-3. Compressed Pattern External Flow



Compressed Pattern Internal Flow

The compressed pattern internal flow is focused on EDT logic creation and EDT pattern generation.

[Figure 2-4](#) details the steps shown in gray in [Figure 2-2](#), and shows the files used in the tool's internal flow. The basic steps in the flow are summarized in the following list.

1. Prepare and synthesize the RTL design, including boundary scan and I/O pads cells for all I/Os. Provide I/O pad cells for any EDT control and channel pins that will not be shared with functional pins.

Note



In this step, you must know how many EDT control and channel pins are needed, so you can provide the necessary I/O pads.

2. Insert an appropriately large number of scan chains using Tessent Scan or a third-party tool. Be sure to add new primary input and output pins for the scan chains to the top level of the design. These new pins are only temporary; the signals to which they connect will become internal nodes and the pins removed when the EDT logic is inserted into the design and connected to the scan chains. For information on how to insert scan chains using Tessent Scan, refer to “[Internal Scan and Test Circuitry](#),” in the *Tessent Scan and ATPG User's Manual*.

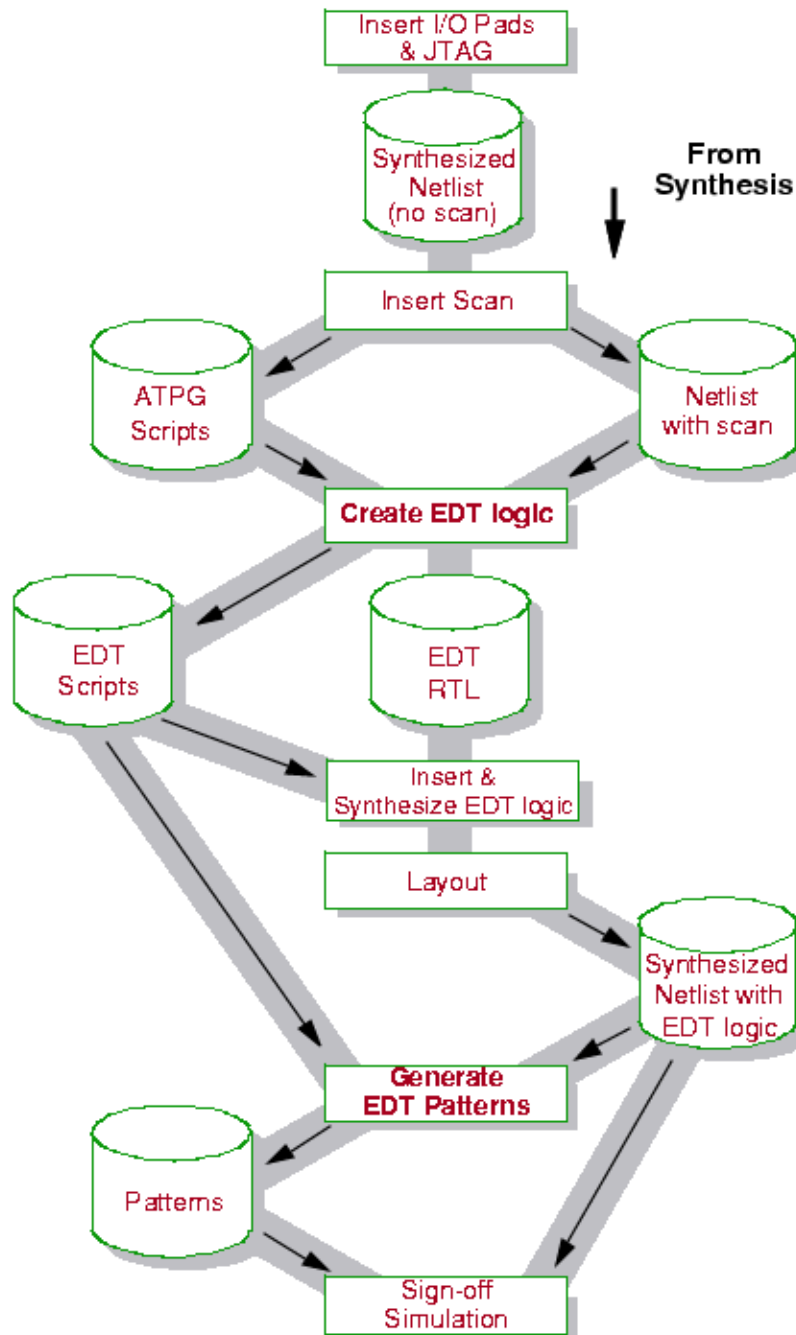
Note



As the new scan I/Os at the top level are only temporary, take care not to insert I/O pads on them.

3. Perform an ATPG run on the scan-inserted design without EDT (optional). Use this run to ensure there are no basic issues such as simulation mismatches caused by an incorrect library.
4. Simulate the patterns created in step 3. (optional).
5. EDT logic Creation Phase: Invoke Tessent Shell with the scan-inserted gate-level description of the core. Create the RTL description of the EDT logic. The tool creates the EDT logic, inserts it into the design, and generates a Design Compiler script to synthesize the EDT logic inside the design.
6. Run the Design Compiler script to incrementally synthesize the EDT logic.
7. EDT Pattern Generation Phase: After you insert the EDT logic, invoke Tessent Shell with the synthesized top-level Verilog netlist and generate the EDT test patterns. You can write test patterns in a variety of formats including Verilog and WGL.
8. Simulate the compressed test patterns that you created in the preceding step. As for regular ATPG, the typical practice is to simulate all parallel patterns and a sample of serial patterns.

Figure 2-4. Compressed Pattern Internal Flow



Chapter 3

Scan Chain Synthesis

For ATEs with scan options, the number of channels is usually fixed and the only variable parameter is the number of scan chains. In some cases, the chip package rather than the tester may limit the number of channels. Therefore, scan insertion and synthesis is an important part of the compressed ATPG flow.

You can use Tessent Scan or another scan insertion product to insert scan chain circuitry in your design before generating EDT logic. You can also generate the EDT logic before scan chain insertion. For more information, see “[Integrating Compression at the RTL Stage](#)” on page 291.

Design Preparation	51
Scan Chain Insertion	53
OCC Sub-Chain Stitching.....	58
ATPG Baseline Generation.....	61

Design Preparation

Before you insert test structures into your design there are EDT-specific issues you need to consider.

It is recommended that before in insert test structures into your design, you understand the information in “[Internal Scan and Test Circuitry Insertion](#)” in the *Tessent Scan and ATPG User’s Manual*.

Preparation for the External Flow

- **Managing Pre-existing I/O Pads**

Because the synthesized hardware is added as a collar around the core design, the core should not have I/O pads when you create the EDT logic. If the design has I/O pads, you need to extract the core or remove the I/O pads.

Note

 If you must insert I/O pads prior to or during initial synthesis, consider using the internal flow, which does not require you to perform the steps a through e.

If the core and the I/O pads are in separate blocks, removing the I/O pads is simple to do as described here:

- a. Invoke Tessent Shell and read in the design.

- b. Set the current design to the core module using the [set_current_design](#) command.
- c. Write out the core using the `write_design` command.
- d. Insert scan into the core and synthesize the EDT logic around it.
- e. Reinsert the EDT logic/core combination into the original circuit in place of the core you extracted, such that it is connected to the I/O pads.

If your design flow dictates that the I/O pads be inserted prior to scan insertion, you can create a blackbox as a place holder that corresponds to the EDT block. You can then stitch the I/O pads and, subsequently, the scan chains to this block. Once the RTL model of the block is created, you use the RTL model as the new architecture or definition of the blackbox placeholder. The port names of the EDT block must match those of the blackbox already in the design, so only the architectures need to be swapped.

- **Managing Pre-existing Boundary Scan**

If your design requires boundary scan, you must add the boundary scan circuitry outside the top-level wrapper created by Tessent Shell. The EDT logic is typically controlled by primary input pins and not by the boundary scan circuitry. In test mode, the boundary scan circuitry just needs to be reset.

Note

 If you must insert boundary scan prior to or during initial synthesis, consider using the internal flow, which is intended for pre-existing boundary scan or I/O pads.

If the design already includes boundary scan, you need to extract the core or remove the boundary scan. This is the same requirement, described in [Managing Pre-existing I/O Pads](#). Use the procedure for managing pre-existing I/O pads in “[Preparation for the External Flow](#)” on page 51.

Note

 Boundary scan adds a level of hierarchy outside the EDT wrapper and requires you to make certain modifications to the generated dofile and test procedure file that you use for the test pattern generation.

For more complete information about including boundary scan, refer to “[Boundary Scan](#)” on page 122.

- **Synthesizing a Gate-level Version of the Design** — As a prerequisite to starting the compressed ATPG flow, you need a synthesized gate-level netlist of the core design without scan. The described in the Compressed Pattern Flows section in [Uncompressed ATPG Flow](#), the design must not have boundary scan or I/O pads. You can synthesize the netlist using any synthesis tool and any technology.

Preparation For the Internal Flow

The EDT logic is connected between the I/O pads and the core so the core should have I/O pad cells in place for all the design I/Os. You must also add I/O pads for any EDT control and channel pins that you do not want to share with the design's functional pins.

There are three mandatory EDT control pins: `edt_clock`, `edt_update`, and `edt_bypass` unless you disable bypass circuitry during setup. There are $2n$ channel I/Os where n is the number of external channels for the netlist. See “[EDT Control and Channel Pins](#)” on page 83 for detailed information about EDT control and channel pins.

Scan Chain Insertion

You should insert an appropriately large number of scan chains. For testers with the scan option, the number of channels is usually fixed, and the variable is the number of chains.

Note



Scan configuration is an important part of the compressed ATPG flow. Refer to “[Determining How Many Scan Chains to Use](#)” on page 54 for more information.

The scan chains can be connected to dedicated top-level scan pins. In designs that implement hierarchical scan insertion, the scan chains can be defined at internal pins on the block instances. In such a case, there is no need to bring these block scan chains to dedicated scan pins at the top level. For more information, see “[Scan Chain Pins](#)” on page 55.

The following limitations exist for the insertion of scan chains:

- Only scan using the mux-DFF or LSSD scan cell type (or a mixture of the two) is supported. The tool creates DFF-based EDT logic by default; however, you can direct it to create latch-based logic for pure LSSD designs. [Table 1-1](#) on page 18 summarizes the EDT logic/scan architecture combinations the tool supports. For information about specific scan cell types, refer to “[Scan Architectures](#)” in the *Tessent Scan and ATPG User's Manual*.
- Both prefixed and bused scan input and output pins are allowed; however, the buses for bused pins must be in either ascending or descending order (not in random order).
- Unlike uncompressed ATPG, “dummy” scan chains are not supported in compressed ATPG. This is because EDT logic is dependent on the scan configuration, particularly the number of scan chains. Uncompressed ATPG performance is independent of the scan configuration and you can assume that all scan cells are configured into a single scan chain when dummy scan chains are used.

Insertion of Bypass Chains in the Netlist

Tessent Shell can generate EDT logic for netlists that contain two sets of pre-defined scan chains. This enables you to insert both the bypass chains for bypass mode and the core scan chains for compression mode into the netlist with a scan-insertion tool before the EDT logic is generated.

You can use any scan insertion tool, but you must adhere to the following rules when defining the scan chains:

- Scan chains and bypass chains must use the same I/O pins.
- If the control pin used to select bypass or compression mode is shared with the `edt_bypass` pin, the bypass chains must be active when the `edt_bypass` pin is at 1, and the scan chains must be active when the `edt_bypass` pin is at 0.
- Test procedure file for the EDT logic must set up the mux select, so the shortened internal scan chains can be traced.

Inserting bypass chains with a scan insertion tool ensures that lockup cells and multiplexers used for bypass mode operation are fully integrated into the design netlist to allow more effective design routing.

For more information, see “[Compression Bypass Logic](#)” on page 230.

Inclusion of Uncompressed Scan Chains

Uncompressed scan chains (scan chains not driven by or observed through EDT logic) are permitted in a design that also uses EDT logic. You can insert and synthesize them like any other scan chains, but you do not define them when creating the EDT logic.

You must define the uncompressed scan chain during test pattern generation using the `add_scan_chains` command without the `-Internal` switch.

You can set up uncompressed scan chains to share top-level pins by defining existing top-level pins as equivalent or physically defining multiple scan chains with the same top-level pin. For more information, see the [add_scan_chains](#) command in the *Tessent Shell Reference Manual*.

Determining How Many Scan Chains to Use

Although you generally determine the number of scan chains based on the number of scan channels and the desired compression, routing congestion can create a practical limitation on the number of scan chains a design can have. With a very large number of scan chains (usually more than a thousand), you can run into problems similar to those for RAMs, where routing can be a problem if several hundred scan chains start at the decompressor and end at the compactor.

Other reasons to decrease the number of scan chains might be to limit the number of incompressible patterns and/or to reduce the pattern count. For more information, see “[Effective Compression](#)” on page 27.

For testers with a scan option, the number of channels is usually fixed and the variable you modify will be the number of chains. Because the effective compression will be slightly less than the ratio between the two numbers (the chain-to-channel ratio), in most cases it is sufficient to do an approximate configuration by using slightly more chains than indicated by the chain-to-channel ratio. How many more depends on the specific design and on your experience with the tool. For example, if the number of scan channels is 16 and you need five times (5X) effective compression, you can configure the design with 100 chains (20 more than indicated by the chain-to-channel ratio). This typically results in 4.5X to 6X compression.

Scan Groups

EDT supports the use of exactly one scan group. A scan group is a grouping of scan chains based on operation. For more information, see “[Scan Groups](#)” in the *Tessent Scan and ATPG User’s Manual*.

Scan Chain Pins

When you perform scan insertion, you must not share any scan chain pins with functional pins. You can connect the inserted scan chains to dedicated pins you create for them at the top level.

If you use the external flow, these dedicated pins become internal nodes when the tool creates the additional wrapper. If you use the internal flow, the dedicated pins are removed when the EDT logic is instantiated in the design and connected. Therefore, using dedicated pins does not increase the number of pins needed for the chip package.

You can also leave the scan chains anchored to internal scan pins instead of connecting them to the top level.

Note



You can share functional pins with the external decompressor scan channel pins. Remember, these channels become the new “virtual” scan chains seen by the tester. You specify the number of channels, as well as any pin sharing, in a later step when you set up Tessent Shell for inserting the EDT logic. See “[EDT Control and Channel Pins](#)” on page 83 for more information.

Note



If a scan cell drives a functional output, avoid using that output as the scan pin. If that scan cell is the last cell in the chain, you must add a dedicated scan output.

About Reordered Scan Chains

The EDT logic (including bypass circuitry) depends on the clocking of the design. When necessary to prevent clock skew problems, the tool automatically includes lockup cells in the EDT logic. If, after you create the EDT logic, you reorder the scan chains incorrectly, the automatically inserted lockup cells will no longer behave correctly. The following are potential problem areas:

- Between the decompressor and the scan chains (between the EDT clock and the scan clock(s))
- Between the scan chain output and the compactor when there are pipeline stages (between the scan clock(s) and the EDT clock)
- In the bypass circuitry where the internal scan chains are concatenated (between different scan clocks)

You can avoid regenerating the EDT logic by ensuring the following are true after you reorder the scan chains:

- The first and last scan cell of each chain have the same clock and phase.

To satisfy this condition, you should reorder within each chain and within each clock domain. If both leading edge (LE) triggered and trailing edge (TE) triggered cells exist in the same chain, do not move these two domains relative to each other. After reordering, the first and last cell in a chain do not have to be precisely the same cells that occupied those positions before reordering, but you do need to have the same clock domains (clock pin and clock phase) at the beginning and end of the scan chain, that you had during IP creation.

- If you use a lockup cell at the end of each scan chain and if all scan cells are LE triggered, you do not have to preserve the clock domains at the beginning and end of each scan chain.

When all scan cells in the design are LE triggered, the lockup cell at the end of each chain enables you to reorder however you want. You can move clock domains and you can reorder across chains. But if there are both LE and TE triggered flip-flops, you must maintain the clock and edge at the beginning and end of each chain. Therefore, the effectiveness and need of the lockup cell at the end of each chain depends on the reordering flow, and whether you are using both edges of the clock.

For flows where re-creating the EDT logic is unnecessary, you still must regenerate patterns (just as for a regular ATPG flow). You should also perform serial simulation of the chain test and a few patterns to ensure there are no problems. If you include bypass circuitry in the EDT logic (the default), you should also create and serially simulate the bypass mode chain test and a few patterns.

Scan Insertion Dofile Example

The scan chains must have dedicated pins. The following is an example dofile for inserting scan chains with Tessent Scan.

```
// tscan.do
//
// Tessent Scan dofile to insert scan chains for EDT.

// Set context, read library, read and set current design, etc.
...

// Set up control signals.
add_clocks 0 clk1 clk2 clk3 clk4 ramclk

// Define test logic for lockup cells.
add_cell_models inv02 -type inv
add_cell_models latch -type dlat CLK D -active high
set_scan_insertion_options -enable_retiming on

// Set up Test Control Pins.
set_scan_signals -sen scan_en
set_scan_signals -ten test_en

// Set up scan chain naming.
Add_scan_mode -si_port_format edt_si%s%d -so_port_format \
edt_so%s%d -port_index_start_value 1 -port_scalar_index_modifier 1

// Flatten design, run DRCs, and identify scan cells.
set_system_mode analysis
report_statistics
run

// Insert scan chains and test logic.
Add_scan_mode unwrapped -type unwrapped -chain_count 16 \
-single_clock_domain off -single_clock_edge off
// Report information.
report_scan_chains
report_test_logic

// Write output files.
write_design my_gate_scan.v -verilog -replace
write_atpg_setup my_atpg -replace

exit
```

You should obtain the following outputs from Tessent Scan:

- Scan-inserted gate-level netlist of the design
- Test procedure file that describes how to operate the scan chains
- Dofile that contains the circuit setup and test structure information

Related Topics

[Determining the Limits of Compression](#)

OCC Sub-Chain Stitching

OCCs have a short sub-chain that you need to incorporate into the overall scan chain configuration to allow ATPG to program the OCC.

For EDT compression, any time you must specify a particular value to any of the sequential elements, encoding capacity is reduced. For example, when you use a clock control definition to force bits to a particular value or when you constrain a scan cell to a certain value. Adding such controls or constraints requires encoding impacts the number of patterns or pattern generation. When the required encoding exceeds the encoding capacity of the decompressor, you will encounter clustering issues.

The impact of specified bits increases dramatically if they are aligned in the same shift cycle.

A design may have a large number of clocks, each having an OCC containing four bits. Since all four OCC bits need to be specified to generate the required capture sequence, you must be careful when stitching them into the scan chains in order to avoid clustering issues. The basic recommendation is to add the OCC scan chain segments as part of the compressed scan chains. [Figure 3-1](#) shows, highlighted in red, groups of four bits representing OCC bits in different scan segments.

This is a poor alignment for the added OCC scan chain segments, because they are aligned so that most of these bits are in the same shift position in the scan chain. Such a positioning of these scan segments creates a potential clustering issue, because these bits must be encoded to specific values on the decompressor input.

Figure 3-1. Bad Specified Bit Alignment

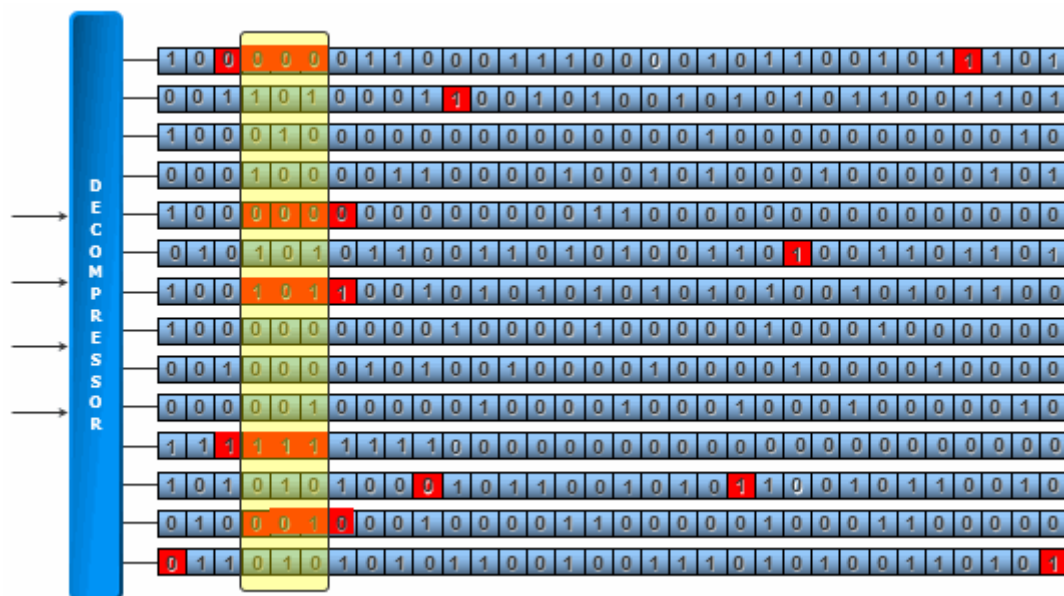


Figure 3-2 shows that the segments are spread throughout the configuration, avoiding the alignment of bits that may lead to clustering. This is a better alignment of the bits, however the scan insertion tool cannot guarantee such a configuration.

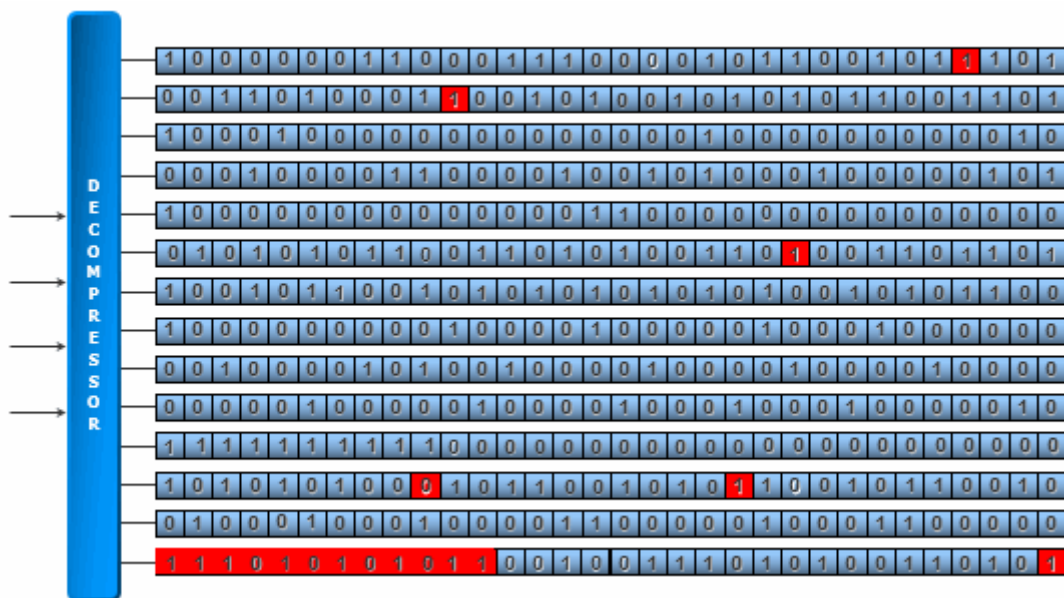
Figure 3-2. Better Specified Bit Alignment



There are two OCC sub-chain stitching recommendations:

- If there are few OCC bits (roughly 25% or less of the longest chain). Stitch OCC bits into a single compressed chain. Place them in one chain, as shown highlighted in red in Figure 3-3, to avoid the alignment of bits in the same shift cycle. This is automated in Tessent Scan. The impact to encoding capacity should be small.

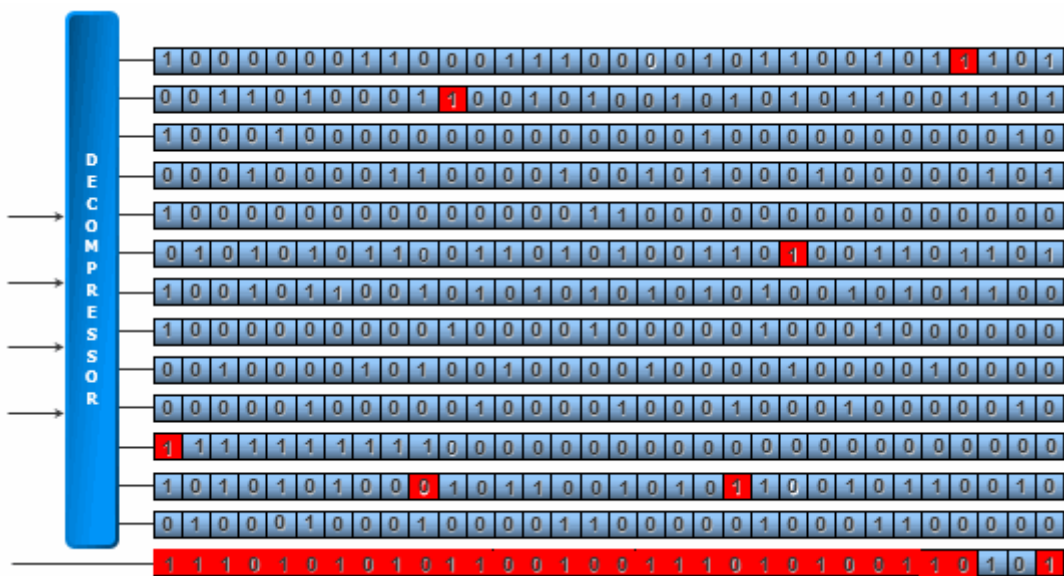
Figure 3-3. Best Specified Bit Alignment (Few Cells)



- If the OCC bits add up to more than one chain or could fill approximately 75% of a chain. Use an uncompressed chain for OCC bits as shown in [Figure 3-4](#). Before doing this, however, first run ATPG to determine if you have a clustering issue, because using compressed chains is preferred over uncompressed chains. Note that for every core that needs an uncompressed chain you need two additional pins at the chip level.

Add any remaining bits into a compressed chain, similar to the case where there are few OCC bits.

Figure 3-4. Best Specified Bit Alignment (Many Cells)



OCC sub-chains may be part of wrapper chains to enable access in internal and external modes, or they can be part of internal chains if only used in internal mode. Tessent Scan understands this and decides whether or not they should be part of wrapper chains.

For wrapper chains, Tessent Scan will use the global scan-enable for OCC cells and use a gate's scan-enable for non-OCC cells.

ATPG Baseline Generation

You can generate an ATPG baseline after scan chain insertion.

An ATPG baseline can be used to

- Estimate the final test coverage early in the flow, before you insert the EDT logic.
- Obtain the scan data volume for the test patterns pre-compression. You can then compare the scan data volume for test patterns before and after compression to evaluate the effects of compression.

Note



Directly comparing pattern counts is not meaningful because EDT patterns are much smaller than ATPG patterns. This is because the relatively short scan chains used in EDT require many fewer shift cycles per scan pattern.

- Provide additional help for debugging. You can simulate the patterns you generate in this step to verify that the non-EDT patterns simulate without problems.
- Find other problems, such as library errors or timing issues in the core, before you create the EDT logic.

Note



If you include bypass circuitry, you also can run regular ATPG after you insert the EDT logic.

This run is like any ATPG run and does not have any special settings; the key is using the same settings (pattern types, constraints, and so on) used to create the compressed test patterns.

The test procedure file used for this ATPG run can be identical to the one generated by scan insertion. However, it should be modified to include the same timing, specified by the tester, that is used to generate the compressed test patterns. By using the same timing information, you ensure simulation comparisons are realistic. To avoid DRC violations when you save test patterns, update the test procedure file with information for RAM clocks and for non-scan-related procedures.

Use the [report_scan_volume](#) command to report test data before and after compression and compare the data to evaluate the effect of compression.

Save the patterns if you want to simulate them. You can use any Verilog timing simulator.

Note



This ATPG run is intended to provide test coverage and pattern volume information for traditional ATPG. Save the patterns if you want to simulate them, but be aware that they have no other purpose. The final compressed test patterns are generated and saved after the EDT logic is inserted and synthesized.

Chapter 4

Creation of the EDT Logic

You can create and insert EDT logic into a scan-inserted design.

For more information on specific commands, see the [Tessent Shell Reference Manual](#).

Compression Analysis	64
Analyzing Compression	64
Preparation for EDT Logic Creation	68
Parameter Specification for the EDT Logic	72
Dual Compression Configurations	73
Defining Dual Compression Configurations	75
Asymmetric Input and Output Channels	78
Bypass Scan Chains	78
Latch-Based EDT Logic	78
Compactor Type	78
Pipeline Stages in the Compactor	78
Pipeline Stages Added to the Channel	79
Longest Scan Chain Range	79
EDT Logic Reset	79
EDT Architecture Version	79
Specifying Hard Macros	80
Pulse EDT Clock Before Scan Shift Clocks	81
Reporting of the EDT Logic Configuration	81
EDT Control and Channel Pins	83
EDT Control and Channel Pin Configuration	83
Functional/EDT Pin Sharing	85
Shared Pin Configuration	87
Connections for EDT Pins (Internal Flow only)	90
Internally Driven EDT Pins	91
Structure of the Bypass Chains	93
Decompressor and Compactor Connections	93
JTAG and the EDT IP TCD Flow	94
Design Rule Checks	94
Creation of EDT Logic Files	96
The EDT Logic Files	99
JTAG and EDT Logic	100
Specification of Module/Instance Names	100
EDT Logic Description	100

Inserting EDT Logic During Synthesis	110
Creation of a Reduced Netlist for Synthesis	117

Compression Analysis

You need to determine a scan chain to scan channel ratio (chain:channel ratio) for your application before you create the EDT logic. The chain:channel ratio determines the compression for an application.

Usually the number of scan channels are dictated by hardware resources such as test channels on the ATE and the top-level design pins available for test. However, you can usually vary the number of scan chains to optimize the compression for an application.

You can determine the optimal chain:channel ratio for an application by varying the number of scan channels or scan chains and then generating test patterns and evaluating the following elements:

- **Test Coverage** — Determine if the test effectiveness is adequate for the application.
- **Data Volume** — Determine how much test pattern data is generated after compression and whether it is within the test hardware limitations.
- **ATPG Baseline (optional)** — Compare the test data statistics for the ATPG baseline with the compressed test pattern statistics. See “[ATPG Baseline Generation](#)” on page 61.

You can use the [analyze_compression](#) command to explore the effects of different chain:channel ratios on test data without making modifications to your design. For more information, see “[Effective Compression](#)” on page 27 and “[Analyzing Compression](#)” on page 64.

Related Topics

[TestKompress Compression Logic](#)

[Analyzing Compression](#)

[Determining the Limits of Compression](#)

Analyzing Compression

Use compression analysis to explore chain:channel ratios, test coverage, and test data volume for an EDT application. You can perform this procedure before or after the EDT logic is created and on block-level or chip-level architecture designs.

Note



This procedure is used for analysis only and does not permanently alter design configurations or produce any test patterns.

Prerequisites

- Scan-inserted gate-level netlist. Can be any scan chain configuration. The tool disregards the configuration if other settings are specified for the analysis. For more information, see the [analyze_compression](#) command.
- It is recommended to use the [reset_state](#) command to discard existing test patterns and restore the fault population before analyzing the design.

Procedure

1. Invoke Tessent Shell on your design. The tool invokes in setup mode. For more information, see “[Supported Design Format](#)” on page 35.

```
<Tessent_Tree_Path>/bin/tessent -shell
```

2. Provide Tessent Shell commands. For example:

```
set_context patterns -scan
read_verilog my_gate_scan.v
read_cell_library my_lib.aptg
set_current_design top
```

3. Define scan chains and add clocks using the `add_scan_chains` and `add_clocks` commands.
4. Analyze the design to determine the maximum chain:channel configurations that can be used for your design. Use this step to analyze both chip-level and block-level designs. For example:

```
set_fault_type stuck
set_fault_sampling 5
analyze_compression
```

The tool analyzes the design and returns a range of chain:channel ratio values beginning with the ratio where a negligible drop in fault coverage occurs and ending with the ratio where a 1% drop in fault coverage occurs as follows:

```
// For stuck-at_faults
//
// Chain:Channel Ratio PredictedFault CoverageDrop
// -----
// 153 negligible fault coverage drop
// 154 0.01 % - 0.05 % drop
// 160 0.10 %
// 168 0.15 %
// 171 0.20 %
...
// CPU time is 155 seconds.
```

The design is analyzed for the fault type specified by the [set_fault_type](#) command before the `analyze_compression` command is executed. The `analyze_compression` command uses the current fault population. If no faults are added, the tool operates on all faults or a subset of sampled faults that are determined by the fault sampling rate specified by the “[set_fault_sampling <rate>](#)” command.

For example, if you want to analyze transition faults with a 10% fault sampling rate, you would use the following commands:

```
set_fault_type transition  
set_fault_sampling 10  
analyze_compression
```

For more information, see the [analyze_compression](#) command.

5. Select a chain:channel ratio from the list and calculate how many scan chains and scan channels to use for your first trial run. For more information, see “[Compression Analysis](#)” on page 64.
6. Depending on the chip architecture, specify the chain:channel ratios, emulate the EDT logic, and generate test patterns as follows:
 - Emulating a virtual single block EDT configuration

```
set_fault_type stuck  
analyze_compression -chains 270 -channels 9
```

- Emulating a virtual modular EDT configuration

If you are analyzing compression for a block-level design, you may need to manually determine how to allocate chains and channels across blocks to achieve the selected chain:channel ratio before you perform this step. For example:

```
set_fault_sampling 80  
analyze_compression -Edt_block BLK1 \  
-CHAINS 400 - CHANNELs 8 -Edt_block BLK2 -CHAINS 200 \  
-CHANNELs 4 -ENable_edt_power_controller  
-MIN_Switching_threshold_percentage 20
```

The tool emulates the EDT logic with the specified sampling rate, fault type, and chain:channel ratio, generates temporary test patterns, and displays a statistics report similar to the following:

```

                        Statistics Report
                        Stuck-at Faults
-----
Fault Classes                                #faults
                                           (total)
-----
FU (full)                                2173901
-----
UC (uncontrolled)                        729 ( 0.03%)
UO (unobserved)                          17523 ( 0.81%)
DS (det_simulation)                      1696097 (78.02%)
DI (det_implication)                     342047 (15.73%)
PU (posdet_untestable)                   1099 ( 0.05%)
PT (posdet_testable)                     633 ( 0.03%)
UU (unused)                              12547 ( 0.58%)
TI (tied)                                25920 ( 1.19%)
BL (blocked)                             18120 ( 0.83%)
RE (redundant)                           29870 ( 1.37%)
AU (atpg_untestable)                     29316 ( 1.35%)
-----
Untested Faults
-----
AU (atpg_untestable)
  PC (pin_constraints)                    186 ( 0.01%)
  Unclassified                           29130 ( 1.34%)
UC+UO
  AAB (atpg_abort)                       6619 ( 0.30%)
  UNS (unsuccess)                        11633 ( 0.54%)
-----
Coverage
-----
test_coverage                            97.68%
fault_coverage                           93.79%
atpg_effectiveness                        99.15%
-----
#test_patterns                           2285
#basic_patterns                           2108
#clock_po_patterns                        3
#clock_sequential_patterns                174
#simulated_patterns                       4544
CPU_time (secs)                           4755.1
-----

```

Note: The reported statistics are based on a 80% fault sample.

```

// CPU time to analyze_compression is 4751 seconds.
//
// -----
// Scan volume report.
// -----
//      channels      : 12
//      shift cycles  : 145
// -----

```

```
// pattern          # test # scan          volume
// type            patterns loads (cell loads or unloads)
// -----
// setup_pattern      2      2              3480
// chain_test         71     71             123540
// basic              2108   2108           3667920
// clock_po           3      3              5220
// clock_sequential   174    174            302760
// -----
// total              2358   2358           4102920 (4.1M)
//
Power Metrics          Min.    Average    Max.
-----
WSA                    0.08%   27.39%   46.28%
State Element Transitions 0.00%   30.48%   50.67%
-----
Peak Cycle
-----
WSA                    0.08%   28.26%   46.28%
State Element Transitions 0.00%   31.55%   50.67%
-----
Load Shift Transitions  7.32%   15.97%   19.91%
Response Shift Transitions 9.85%   33.69%   50.51%
```

7. Review the statistics report to determine whether the chain:channel ratio is adequate as follows:
 - If the chain:channel ratio yields adequate results, insert the scan chains and create the EDT logic. See “[Scan Chain Synthesis](#)” on page 51 and “[Preparation for EDT Logic Creation](#)” on page 68.
 - If the data volume and/or test coverage is unacceptable, repeat steps 3, 4, and 5 until you determine the optimal chain:channel ratio to use for your application.

Related Topics

[Compression Analysis](#)

[If Compression is Less Than Expected](#)

[If Test Coverage is Less Than Expected](#)

[Speed up the Process](#)

Preparation for EDT Logic Creation

Depending on your application, there are certain tasks you must perform to prepare for creating/inserting EDT logic into your design. The tasks include setting up EDT context and defining clocks and scan chains.

You can create the EDT logic immediately after you insert scan chains, or you can run traditional ATPG and simulate the resulting patterns first, as described in the “[ATPG Baseline Generation](#)” on page 61. EDT must be on whenever you are creating test patterns or EDT logic.

Note

 You can use the [report_environment](#) command to check the tool status. You can use the [set_edt_options](#) command to enable compression.

Scan Chain Definition

You must define the clocks and scan chain information. You can include these commands in a dofile or invoke the dofile that Tessent Scan generates to define clocks and scan chains. For example:

dofile my_atpg.dofile

The following shows an example setup dofile generated by Tessent Scan:

```
add_scan_groups grp1 my_atpg_setup.testproc
add_scan_chains chain1 grp1 edt_si1 edt_so1
add_scan_chains chain2 grp1 edt_si2 edt_so2
add_scan_chains chain3 grp1 edt_si3 edt_so3
...
add_scan_chains chain98 grp1 edt_si98 edt_so14
add_scan_chains chain99 grp1 edt_si99 edt_so15
add_scan_chains chain100 grp1 edt_si100 edt_so16
add_write_controls 0 ramclk
add_read_controls 0 ramclk
add_clocks 0 clk
```

These commands are explained in “[Scan Data Definition](#)” in the *Tessent Scan and ATPG User’s Manual*.

Internal Scan Chains in Tessent Shell IP Creation

You can add internal scan chains in the EDT IP creation phase (dft -edt context). Internal scan chains are scan chains where the scan input and output signals are not brought to the top level of the design and connected to top-level pins. This supports the hierarchical scan insertion flow and removes the requirement to bring core-level scan pins to the top level.

You use the [add_scan_chains -internal](#) command to define internal scan chains during IP creation as shown in the following example. Note, the K4, K9, and K10 IP creation DRCs do not apply to internal scan pins and are skipped. These DRCs will still be run for top-level scan pins.

Note

 TestKompress not invoked from Tessent Shell still requires top-level scan pins during IP creation.

This example shows IP creation in a design with three EDT blocks: cpu and alu have internal scan chains, whereas TOP has top-level scan chains. Note, the scan pins for the cpu and alu blocks are defined at the respective instance pins and not brought to the top level. The scan pins for the TOP block are defined at the top level.

```
set_context dft -edt
add_clock 0 clk
add_scan_group grp1 scan_setup.testproc
set_edt_options -location internal
//
// EDT block: cpu
add_edt_block cpu
add_scan_chains -internal cpu_chain1 grp1 /cpu/scan_in1 /cpu/scan_out1
...
add_scan_chains -internal cpu_chain100 grp1 /cpu/scan_in100 \
/cpu/scan_out100
set_edt_options -channel 5
//
// EDT block: alu
add_edt_block alu
add_scan_chains -internal alu_chain1 grp1 /alu/scan_in1 /alu/scan_out1
...
add_scan_chains -internal alu_chain60 grp1 /alu/scan_in60 /alu/scan_out60
set_edt_options -channel 3
//
// EDT block: TOP
add_edt_block TOP
add_scan_chains TOP_chain1 grp1 scan_in1 scan_out1
...
add_scan_chains TOP_chain20 grp1 scan_in20 scan_out20
set_edt_options -channel 1
//
//System mode transition - perform DRC
set_system_mode analysis
write_edt_files created -verilog -replace
```

Tessent Shell (dft -edt) supports internal scan chains during IP creation. However, non-Tessent Shell TestKompress does not. If you were to define internal scan chains during IP Creation using the following dofile commands in non-Tessent Shell TestKompress:

```
set_edt_options -location internal
add_scan_chains -internal chain1 grp1 /u1/scan_in1 /u1/scan_out1
add_scan_chains -internal chain2 grp1 /u1/scan_in2 /u1/scan_out2
...
set_system_mode atpg
```

The tool would infer the Pattern Generation phase and would possibly fail with pattern generation DRCs like those shown here:

```
// -----/
// Begin EDT setup and rules checking.
// -----/
// Running EDT Pattern Generation Phase.
// Error: Defined pin "edt_clock" for EDT clock signal is not in design.
// Violation safe to ignore, correct operation verified by subsequent
// DRCs. (K5-1)
// Error: Defined pin "edt_update" for EDT update signal is not in design.
// Violation safe to ignore, correct operation verified by subsequent
// DRCs. (K5-2)
// Error: Defined pin "edt_channels_in1" for channel input 1 signal is not
// in design. (K5-3)
// Error: Defined pin "edt_channels_out1" for channel output 1 signal is
// not in design. (K5-4)
// Error: Defined pin "edt_channels_in2" for channel input 2 signal is not
// in design. (K5-5)
// Error: Defined pin "edt_channels_out2" for channel output 2 signal is
// not in design. (K5-6)
// Error: 6 defined EDT pin(s) not in design. (K5)
// EDT setup and rules checking aborted, CPU time=0.00 sec.
// Error: Rules checking unsuccessful, cannot exit SETUP mode.
```

Parameter Specification for the EDT Logic

You use the `set_edt_options` command to set parameters for the EDT logic. The two most important parameters are the position of the EDT logic, internal or external to the design core, and the number of scan channels.

For a basic run to create external EDT logic (the default), you only need to specify the number of channels. For example, the following command sets up external EDT logic with two input channels and two output channels:

`set_edt_options -channels 2`

There are other parameters for the “`set_edt_options`” command to specify whether to create DFF-based or latch-based EDT logic and whether to include bypass circuitry in the EDT logic, lockup cells in the decompressor, and/or pipeline stages in the compactor.

By default, Tessent Shell generates

- EDT logic external to the design core
- DFF-based EDT logic
- Lockup cells in the decompressor, compactor, and bypass logic
- An Xpress compactor without pipeline stages
- Bypass logic

For more information, see the [set_edt_options](#) command in the *Tessent Shell Reference Manual*.

Dual Compression Configurations	73
Defining Dual Compression Configurations	75
Asymmetric Input and Output Channels	78
Bypass Scan Chains	78
Latch-Based EDT Logic	78
Compactor Type	78
Pipeline Stages in the Compactor	78
Pipeline Stages Added to the Channel	79
Longest Scan Chain Range	79
EDT Logic Reset	79
EDT Architecture Version	79
Specifying Hard Macros	80
Pulse EDT Clock Before Scan Shift Clocks	81

Dual Compression Configurations

Using two compression configurations when setting up the EDT logic allows you to easily set up and reuse the EDT logic for two different test phases. For example, wafer test versus package test.

When two distinct configurations are defined, an additional EDT pin is generated to select the active configuration: `edt_configuration`. For more information on EDT pins, see “[EDT Control and Channel Pins](#)” on page 83.

Separate ATPG dofiles and procedure files are created for each configuration. A single dofile and test procedure file is generated for the bypass mode. These ATPG files are then used to generate test patterns for each configuration separately as you would with a single compression configuration.

In the modular flow, you should coordinate compression configuration usage between design groups to ensure the compression configurations are defined and set up properly for each block as follows:

- A maximum of two compression configurations can be defined for the entire design, across all EDT blocks, although the configuration parameters can be different for different EDT blocks belonging to that design.
- Channel parameters for each of the two configurations can vary from block to block.

In the following example, blocks *b1* and *b2* have the same *config_high* configuration name but have different parameters: in *b1*, *config_high* has two input channels and four output channels parameters and, in *b2*, *config_high* has one input and one output channel:

```
set_current_edt_block b1
set_current_edt_configuration config_high
set_edt_options -input 2 -output 4
set_current_edt_configuration config_low
set_edt_options -input 4 -output 5
```

```
set_current_edt_block b2
set_current_edt_configuration config_high
set_edt_options -input 1 -output 1
set_current_edt_configuration config_low
set_edt_options -input 3 -output 3
```

- The configuration with the highest compression ratio must always have the highest compression ratio for each of the EDT blocks.
- To create a single compression configuration for a block, only define parameters for one of the compression configurations.
- The control and data input channels can be separated using the “[set_edt_options -separate_control_data_channels on](#)” command. For more information see the “[Separate Control and Data Channels and Dual Compression Configuration](#)” section.

Limitations

- A configuration with a higher number of input channels than the other configuration must also have an equal or higher number of output channels than the other configuration. For example:

The following configurations are valid because in each case the configuration with a higher input channel count also has an equal or higher number of output channels than the other configuration:

```
Config1 = 4 input channels and 2 output channels  
Config2 = 2 input channels and 1 output channel
```

```
Config1 = 2 input channels and 2 output channels  
Config2 = 4 input channels and 2 output channels
```

The following configurations are *not* valid because in each case the configuration with a higher input channel count has a lower output channel count than the other configuration:

```
Config1 = 4 input channels and 1 output channel  
Config2 = 2 input channels and 2 output channels
```

```
Config1 = 2 input channels and 2 output channels  
Config2 = 4 input channels and 1 output channel.
```

- The channels for the high compression configuration cannot be explicitly specified. By default, the high-compression configuration uses the first channels defined for the low-compression configuration. This applies to both input and output channels.
- Bypass mode is supported for the lowest-compression configuration only. You can define the number of bypass chains in either of the configurations as long as the specified number does not exceed the number of input/output channels of the lowest-compression configuration. For example,

```
Configuration 1 = 2 input channels and 2 output channels  
Configuration 2 = 4 input channels and 4 output channels  
The maximum number of bypass chains = 4
```

For more information on bypass mode, see “[Compression Bypass Logic](#)” on page 230.

- You cannot generate test patterns during EDT logic creation to determine the test coverage. The [analyze_compression](#) command does not support dual compression configurations.
- The Basic compactor does not support more than one configuration. By default the tool generates logic that contains the Xpress compactor. For more information on compactors, see “[Understanding Compactor Options](#)” on page 279.
- There are no DRCs specific to dual compression configurations, so you must run DRC on each configuration in the test pattern generation phase. For more information, see “[Test Pattern Generation](#)” on page 154.

Defining Dual Compression Configurations

You can create EDT logic with two compression configurations for a single design block.

Prerequisites

- Scan chains must be defined. For more information, see “[Scan Chain Definition](#)” on page 69.

Procedure

1. Invoke Tessent Shell. For example:

```
<Tessent_Tree_Path>/bin/tessent -shell
```

Tessent Shell invokes in setup mode.

2. Provide Tessent Shell commands. For example:

```
set_context dft -edt
read_verilog my_gate_scan.v
read_cell_library my_lib.aptg
set_current_design top
```

3. Define the first compression configuration. For example:

```
add_edt_configurations config1
set_edt_options -input_channels 6 -output_channels 5
```

4. Define the second configuration. For example:

```
add_edt_configurations config2
set_edt_options -input_channels 3 -output_channels 3
```

To create a single compression configuration for a block, only define parameters for one of the compression configurations.

5. Define the remaining parameters for the EDT logic. See “[Parameter Specification for the EDT Logic](#)” on page 72.
6. Run DRC and fix any violations. See “[Design Rule Checks](#)” on page 94. You must run DRC on each configuration.
7. Generate the EDT logic. For more information, see “[Creation of EDT Logic Files](#)” on page 96. A separate dofile and procedure file is created for each configuration. The configuration name is appended to the prefix specified with the [write_edt_files](#) command:

```
<filename_prefix>_<configuration_name>_edt.dofile
<filename_prefix>_<configuration_name>_edt.testproc
```

Examples

The following example uses a dofile to create dual compression configurations for a single block.

```
set_context dft -edt
read_verilog my_gate_scan.v
read_cell_library my_lib.aptg
set_current_design top

// edt_ip_creation.do
//
// Dofile for EDT logic Creation Phase
// Execute setup script from Tessent Scan
dofile scan_chain_setup.dofile

// Set up EDT configurations
add_edt_configurations my_pkg_test_config
set_edt_options -channels 16
add_edt_configurations my_wafer_test_config
set_edt_options -channels 2

// Set bypass pin
set_edt_pins bypass my_bypass_pin

//set_edt_options configuration pin
set_edt_pins configuration my_configuration_pin
set_system_mode analysis

// Report and write EDT logic.
report_edt_configurations -all //reports configurations for all blocks.
report_edt_pins //reports all pins including compression configuration
                // specific pins.
write_edt_files created -verilog -replace //Create dofiles and
                                          //testproc files for both the
                                          //configs and bypass mode
```

The following example shows a dofile that sets up modular EDT blocks with dual compression configurations at the top-level.

```
// Set up dual compression configurations
add_edt_configuration manufacturing_test
add_edt_blocks B1
set_edt_options -pipe 2 -channels 4
add_edt_blocks B2
set_edt_options -channels 1
add_edt_blocks B3
set_edt_options -channels 2

add_edt_configuration system_test
set_current_edt_block B1
set_edt_options -channels 2
set_current_edt_block B2
set_edt_options -channels 1
set_current_edt_block B3
set_edt_options -channels 1
// Set up top-level clocks and channel pins for each block
set_current_edt_block B1
add_clocks 0 clk
add_clocks 0 reset

dofile scan/atpg1.dofile_top
set_edt_pins in 1 coreA_channel_in1
set_edt_pins out 1 coreA_channel_out1
set_edt_pins in 2 coreA_channel_in2
set_edt_pins out 2 coreA_channel_out2
set_edt_pins in 3 coreA_channel_in3
set_edt_pins out 3 coreA_channel_out3
set_edt_pins in 4 coreA_channel_in4
set_edt_pins out 4 coreA_channel_out4

set_current_edt_block B2
dofile scan/atpg2.dofile2
set_edt_pins in 1 coreB_channel_in1
set_edt_pins out 1 coreB_channel_out1

set_current_edt_block B3
dofile scan/atpg3.dofile3
set_edt_pins in 1 coreC_channel_in1
set_edt_pins out 1 coreC_channel_out1
set_edt_pins in 2 coreC_channel_in2
set_edt_pins out 2 coreC_channel_out

//Run DRC
set_system_mode analysis

//Report EDT configuration and generate EDT logic
report_edt_configurations -all -verbose

write_edt_files ./edt_ip/created1_core_top -verilog -synth dc_shell \
               -replace -rtl_prefix chip_level

exit -force
```

Asymmetric Input and Output Channels

You can specify a different number of input versus output channels for the EDT logic with the -Input_channels and -Output_channels switches of the set_edt_options command.

Bypass Scan Chains

You can use the set_edt_options -bypass_chains integer to specify how many bypass chains the EDT logic is configured to support. By default, the number of bypass chains created equals the number of input/output channels. If the number of input and output channels differ, the smaller number is used.

You can only specify a number of bypass chains equal to or less than the number of bypass chains created by default. For dual configuration applications, you can only specify the bypass chains after both configurations are defined.

For more information on bypass mode, see [“Compression Bypass Logic”](#) on page 230.

Latch-Based EDT Logic

Tessent Shell supports mux-DFF and LSSD scan architectures, or a mixture of the two, within the same design.

The tool creates DFF-based EDT logic by default. If you have a pure LSSD design and prefer the logic to be latch-based, you can use the -Clocking switch to get the tool to create latch-based EDT logic.

Compactor Type

Use the -COMpactor_type switch to specify which compactor is used in the generated EDT logic.

By default, the Xpress compactor is used. For more information, see [“Understanding Compactor Options”](#) on page 279.

Pipeline Stages in the Compactor

The EDT logic can be set up to include pipeline stages between logic levels within the compactor.

The “set_edt_options -Pipeline_logic_levels_in_compactor” command allows you to specify a maximum number of logic levels (XOR gates) in a compactor before pipeline stages are inserted. By default, no pipeline stages are inserted. For more information on inserting pipeline stages, see [“Use of Pipeline Stages in the Compactor”](#) on page 245.

Pipeline Stages Added to the Channel

When generating the EDT IP, if output channel pipeline stages will be added later, you must specify “set_edt_pins -change_edge_at_compactor_output trailing_edge” to ensure that the compactor output changes consistently on the trailing edge of the EDT clock. Output channel pipeline stages should then start with leading-edge sequential elements.

Longest Scan Chain Range

Sometimes, you may need to change the length of the scan chains in your design after generating the EDT logic. Ordinarily, you must regenerate the EDT logic when such a change alters the length of the longest scan chain.

During setup, before you generate the EDT logic, you can optionally specify a length range for the longest scan chain using the -Longest_chain_range switch. As long as any subsequent scan chain modifications do not result in the longest scan chain exceeding the boundaries of this range, you will not have to regenerate the EDT logic because of a shortening or lengthening of the longest chain.

Note

 This applies only to scan chain length. Other scan chain changes, such as reordering the scan chains may require EDT logic regeneration. For more information, see “[About Reordered Scan Chains](#)” on page 56.

EDT Logic Reset

While in most case it is not needed, if you have a design requirement that all the sequential elements in a design are resettable, you can provide an asynchronous reset signal (edt_reset) for the EDT logic.

Use “-reset_signal asynchronous” with the [set_edt_options](#) command if you want the EDT logic to include this signal. If you choose to include the reset, the hardware will also include a dedicated control pin for it (named “edt_reset” by default).

EDT Architecture Version

To ensure backward compatibility between older EDT logic architectures (created with older versions of the tool) and pattern generation in the current version of the tool, use the -Ip_version switch which enables you to specify the version of the EDT architecture the tool should expect in the design.

In the EDT logic creation phase, the tool writes a dofile containing EDT-specific commands used for ATPG. Any [set_edt_options](#) commands included in this dofile will also use this switch

to specify the EDT architecture version; therefore, you usually do not need to explicitly specify this switch.

Note

 The logic version is incremented only when the hardware architecture changes. If the software is updated, but the logic generated is still functionally the same, only the software version changes.

You can generate test patterns for the older EDT logic architectures, but by default, the EDT logic version is assumed to be the currently supported version.

Specifying Hard Macros

You can specify the hard macros in a design so the tool recognizes and avoids modifying them while tracing clock paths for EDT logic bypass mode.

When one of the specified hard macros are encountered, the tool uses tap points identified from the boundary of the macro cells to drive the bypass lockup cell clocks.

In cases where localized clock gaters are used, a tap point identified for one scan cell may not be appropriate for another scan cell even when they use the same top-level clocks. So, in cases where localized clock gaters are involved, the tool routes the clock pin of each scan cell involved with bypass lockup cells to the EDT logic to avoid clock skew.

For more information on EDT logic bypass mode, see “[Compression Bypass Logic](#)” on page 230.

Note

 This functionality does not effect the type or quantity of lockup cells inserted for bypass mode.

Note

 Compression must be used to insert the EDT logic in the design core before synthesis.

Prerequisites

- Tessent Shell is invoked with a design netlist containing hard macros.

Procedure

1. Set up the EDT logic to be inserted internal to the design core. For example:

```
add_clocks 0 pll/clk1
add_clocks 0 pll/clk2
set_edt_options -location internal
add_scan_chains chain1 grp1 scan_in1 scan_out1
add_scan_chains chain2 grp1 scan_in2 scan_out2
```


2. Set up any additional EDT logic requirements for your test application.
3. Identify each hard macro inside the design. For example:

```
set_attribute_value SCBcg1 SCBcg2 -name is_hard_module -value true
```

4. Run DRC and fix any errors. For example:

```
set_system_mode analysis
```

5. Create the EDT logic RTL and insert it in the design core netlist. For example:

```
write_edt_files created -replace
```

Related Topics

[Compressed Pattern Internal Flow](#)

Pulse EDT Clock Before Scan Shift Clocks

You can set up the EDT clock to pulse before the scan chain shift clocks with the `-pulse_edt_before_shift_clocks` switch of the `set_edt_options` command.

By default, the EDT and scan chain shift clocks are pulsed simultaneously. Setting the EDT logic to pulse before the scan shift clocks makes it independent of the scan chain clocking and provides the following benefits:

- Makes creating EDT logic for a design in the RTL stage easier because scan chain clocking information is not required. For more information on creating EDT logic at the RTL stage, see [“Integrating Compression at the RTL Stage”](#) on page 291.
- Removes the need for lockup cells between scan chains and the EDT logic because correct timing is ensured by the clock sequence. Only a single lockup cell between pairs of bypass scan chains is necessary. For more information, see [“Understanding Lockup Cells”](#) on page 254.
- Simplifies clock routing because the lockup cells used for bypass scan chains are driven by the EDT clock instead of a system clock. This eliminates the need to route system clocks to the EDT logic.

To use this functionality, the shift speed must be able to support two independent clock pulses in one shift cycle, which may increase test time.

Reporting of the EDT Logic Configuration

You can report the current EDT logic configuration with the `report_edt_configurations` command. This command lists configuration details including the number of scan channels and logic version.

For example:

report_edt_configurations

```
// IP version:2  
// External scan channels:2  
// Longest chain range:600 - 700  
// Bypass logic:On  
// Lockup cells:On  
// Clocking:edge-sensitive
```

Note



Because the [report_edt_configurations](#) command needs a flat model and DRC results to produce the most useful information, you usually use this command in analysis or insertion mode. For an example of the command's output when issued after DRC, see "[DRC when EDT Pins are Shared with Functional Pins](#)" on page 95.

EDT Control and Channel Pins

EDT logic includes both control and channel pins. The control pins, such as the `edt_clock`, `edt_update`, and `edt_bypass`, control the functionality of the EDT. The channel pins, such as `edt_channels_in` and `edt_channels_out` are the scan channels.

EDT Control and Channel Pin Configuration	83
Functional/EDT Pin Sharing	85
Shared Pin Configuration	87
Connections for EDT Pins (Internal Flow only)	90
Internally Driven EDT Pins	91
Structure of the Bypass Chains	93
Decompressor and Compactor Connections	93
IJTAG and the EDT IP TCD Flow	94

EDT Control and Channel Pin Configuration

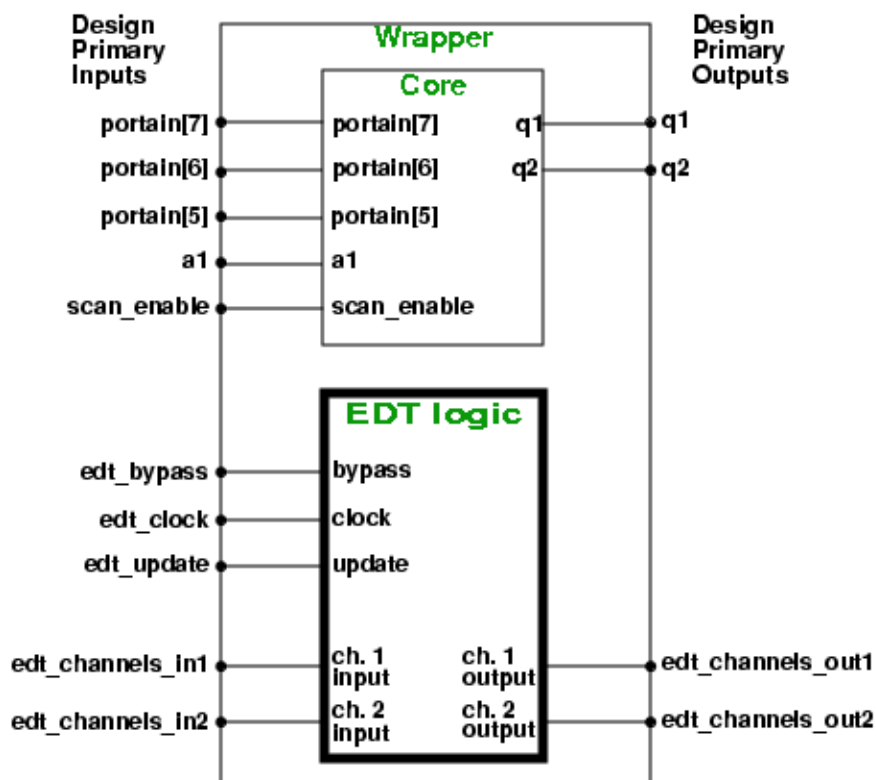
The configuration of the EDT control and channel pins varies on its use.

EDT logic includes the following pins:

- Scan channel input pins
- Scan channel output pins
- EDT clock
- EDT update
- Scan-enable (optional—included when any scan channel output pins are shared with functional pins)
- Bypass mode control
- Reset control (optional—included when you specify an asynchronous reset for the EDT logic)
- `EDT_configuration` (optional—included when you specify multiple configurations)

Figure 4-1 shows the basic configuration of these pins for an example design when the EDT logic is instantiated externally and configured with bypass circuitry and two scan channels. External EDT logic is always instantiated in a top-level EDT wrapper.

Figure 4-1. Default EDT Logic Pin Configuration with Two Channels



The default configuration consists of pins for the EDT clock, update, and bypass inputs. There are also two additional pins (one input and one output) for each scan channel. If you do not rename an EDT pin or share it with a functional pin, as described in [“Functional/EDT Pin Sharing”](#) on page 85, the tool assigns the default EDT pin names shown.

To see the names of the EDT pins, issue the [report_edt_pins](#) command:

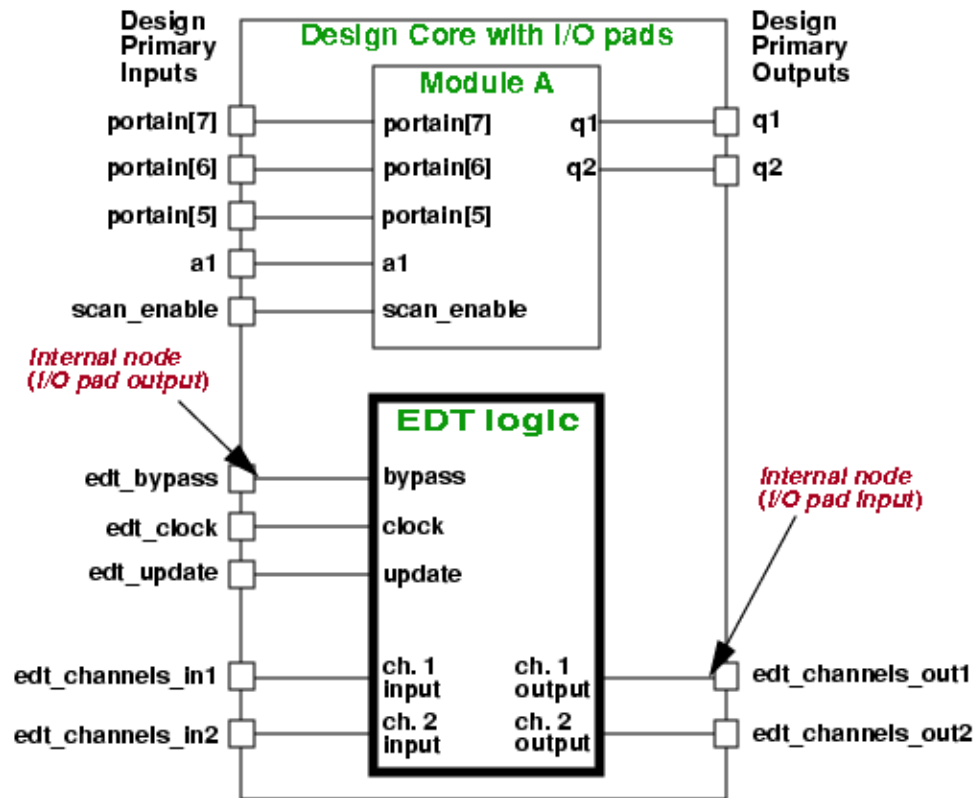
report_edt_pins

Pin description	Pin name	Inversion
Clock	edt_clock	-
Update	edt_update	-
Bypass mode	edt_bypass	-
Scan channel 1 input	edt_channels_in1	-
" " " output	edt_channels_out1	-
Scan channel 2 input	edt_channels_in2	-
" " " output	edt_channels_out2	-

[Figure 4-2](#) shows how the preceding pin configuration looks if the EDT logic is inserted into a design netlist that includes I/O pads (internal EDT logic location). Notice that the EDT control and channel I/O pins are now connected to internal nodes of I/O pads that are part of the core design. You set up these connections by specifying an internal node for each EDT control and

channel I/O pin. For more information, see “Connections for EDT Pins (Internal Flow only)” on page 90.

Figure 4-2. Example of a Basic EDT Pin Configuration (Internal EDT Logic)



Functional/EDT Pin Sharing

EDT pins can be shared with functional pins, with a few restrictions. You use the `set_edt_pins` command to specify sharing of an EDT pin with a functional pin and to specify whether a signal is inverted in the I/O pad for the pin. For more information, see the `set_edt_pins` command.

When you share a channel output pin with a functional pin, the tool inserts a multiplexer before the output pin. This multiplexer is controlled by the `scan_enable` signal, and you must define the `scan_enable` signal with the `set_edt_pins` command. If you do not define the `scan_enable` signal, the tool defaults to “scan_en”, and adds this pin if it does not exist. During DRC, all added pins are reported with K13 DRC messages. You can report the exact names of added pins using the `report_drc_rules` command.

For channel input pins and control pins, you use the `-Inv` switch to specify (on a per pin basis) if a signal inversion occurs between the chip input pin and the input to the EDT logic. For example, if an I/O pad you intend to use for a channel pin inverts the signal, you must specify the inversion when creating the EDT logic. The tool requires the pin inversion information, so

the generated test procedure file operates correctly with the full netlist for test pattern generation.

If bypass circuitry is implemented, you need to force the bypass control signal to enable or disable bypass mode. When you generate compressed EDT patterns, you disable bypass mode by setting the control signal to the off state. When you generate regular ATPG patterns for example, you must enable the bypass mode by setting the bypass control signal to the on state. The logic level associated with the on or off state depends on whether you specify to invert the signal. The bypass control pin is forced in the automatically generated test procedure.

In all cases, EDT pins shared with bidirectional pins must have the output enable signal configured so that the pin has the correct direction during scan. The following list describes the circumstances under which the EDT pins can be shared.

- **Scan channel input pin** — No restrictions.
- **Scan channel output pin** — Cannot be shared with a pin that is bidirectional or tri-state at the core level. This is because the tool includes a multiplexer between the compactor and the output pad when a channel output pin is shared, and tri-state values cannot pass through the multiplexer. A scan channel output pin that later will be connected to a pad and is bidirectional at the top level is allowed.

Note



Scan channel output pins that are bidirectional need to be forced to Z at the beginning of the load_unload procedure. Otherwise, the tool is likely to issue a K20 or K22 rule violation during DRC, without indicating the reason.

- **EDT clock** — Must be defined as a clock and constrained to its defined off state. If shared with a bit of a bus, problems can occur during synthesis. For example, Design Compiler (DC) does not accept a bit of a bus being a clock. The EDT clock pin must only be shared with a non-clock pin that does not disturb scan cells; otherwise, the scan cells will be disturbed during the load_unload procedure when the EDT clock is pulsed. This restriction might cause some reduced coverage. You should use a dedicated pin for the EDT clock or share the EDT clock pin only with a functional pin that controls a small amount of logic. If any loss of coverage is not acceptable, then you must use a dedicated pin.
- **EDT reset** — Should be defined as a clock and constrained to its defined off state. If shared with a bit of a bus, problems can occur during synthesis. For example, DC does not accept a bit of a bus being a clock. The EDT reset pin must only be shared with a non-clock pin that does not disturb scan cells. This restriction might cause some reduced coverage. You should use a dedicated pin for the EDT reset, or share the EDT reset pin only with a functional pin that controls a small amount of logic. If any loss of coverage is not acceptable, then you must use a dedicated pin.
- **EDT update** — Can be shared with any non-clock pin. Because the EDT update pin is not constrained, sharing it has no impact on test coverage.

- **Scan enable** — As for regular ATPG, this pin must be dedicated in test mode; otherwise, there are no additional limitations. EDT only uses it when you share channel output pins. Because it is not constrained, sharing it has no impact on test coverage.
- **Bypass (optional)** — Must be forced during scan (forced on in the bypass test procedures and forced off in the EDT test procedures). It is not constrained, so sharing it has no impact on test coverage. For more information on bypass mode, see [“Compression Bypass Logic”](#) on page 230.
- **Edt_configuration (optional)** — The value corresponding with the selected configuration must be forced on during scan chain shifting.

Note

 RTL generation allows sharing of control pins. The restrictions for EDT pin sharing ensure the EDT logic operates correctly and with only negligible loss, if any, of test coverage.

Shared Pin Configuration

The synthesis methodology does not change when you specify pin sharing. You do, however, need to add a step to the EDT logic creation phase. In this extra step, you define how pins are shared.

For example, you are using the external flow with two scan channels and you want to share three of the channel pins, as well as the EDT update and EDT clock pins, with functional pins. Assume the functional pins have the names shown in [Table 4-1](#).

Table 4-1. Example Pin Sharing

EDT Pin Description	Functional Pin Name
Input 1 (Channel 1 input)	portain[7]
Output 1 (Channel 1 output)	edt_channels_out1 (new pin, default name)
Input 2 (Channel 2 input)	portain[6]
Output 2 (Channel 2 output)	q2
Update	portain[5]
Clock	a1
Bypass	my_bypass (new pin, non-default name)

You can see the names of the EDT pins, prior to setting up the shared pins, by issuing the [report_edt_pins](#) command:

```
report_edt_pins
```

//	Pin description	Pin name	Inversion
//	-----	-----	-----
//	Clock	edt_clock	-
//	Update	edt_update	-
//	Bypass mode	edt_bypass	-
//	Scan channel 1 input	edt_channels_in1	-
//	" " " output	edt_channels_out1	-
//	Scan channel 2 input	edt_channels_in2	-
//	" " " output	edt_channels_out2	-

You can use the [set_edt_pins](#) command to specify the functional pin to share with each EDT pin. With this command, you can specify to tap an EDT pin from an existing core pin. You can also use the command to change the name of the new pin the tool creates for each dedicated EDT pin. [Figure 4-3](#) on page 90 illustrates both of these cases conceptually.

Note

 In the external flow, the specified pin sharing is implemented in the wrapper generated when the EDT logic is created. The “[Top-level Wrapper](#)” section contains additional information about this wrapper. In the internal flow, the pin sharing is implemented when you create and insert the EDT logic into the design before synthesis.

If a specified pin already exists in the core, the tool shares the EDT signal with that pin. [Figure 4-3](#) shows an example of this for the EDT clock signal. The command “set_edt_options clock a1”, will cause the tool to share the EDT clock with the a1 pin instead of creating a dedicated pin for the EDT clock. If you specify a pin name that does not exist in the core, a dedicated EDT pin with the specified name is created. For example, “set_edt_pins bypass my_bypass” will cause the tool to create the new pin my_bypass and connect it to the EDT bypass pin.

For each EDT pin you do not share or rename using the [set_edt_pins](#) command, if its default name is unique, the tool creates a dedicated pin with the default name. If the default name is the same as a core pin name, the tool automatically shares the EDT pin with that core pin. [Table 4-2](#) lists the default EDT pin names.

Table 4-2. Default EDT Pin Names

EDT Pin Description	Default Name
Clock	edt_clock If “edt_clock” DFT signal is defined then its value is used as the default name.
Reset	edt_reset
Update	edt_update If “edt_update” DFT signal is defined then its value is used as the default name.

Table 4-2. Default EDT Pin Names (cont.)

EDT Pin Description	Default Name
Scan Enable	scan_en If “scan_en” DFT signal is defined then its value is used as the default name.
Bypass mode	edt_bypass
Scan Channel Input	“edt_channels_in” followed by the index number of the channel
Scan Channel Output	“edt_channels_out” followed by the index number of the channel
EDT configuration select	edt_configuration

When you share a pin between an EDT channel output and a core output, the tool includes a multiplexer in the circuit together with the EDT logic, but in a separate module at the top level. An example is shown in red in [Figure 4-3](#) for the shared EDT channel output 2 signal, and the core output signal q2. As previously mentioned, the multiplexer is controlled by the defined scan enable pin. If a scan enable pin is not defined, the tool adds one with the EDT default name, “scan_en.” Here are the commands that would establish the example pin sharing shown in [Table 4-1](#):

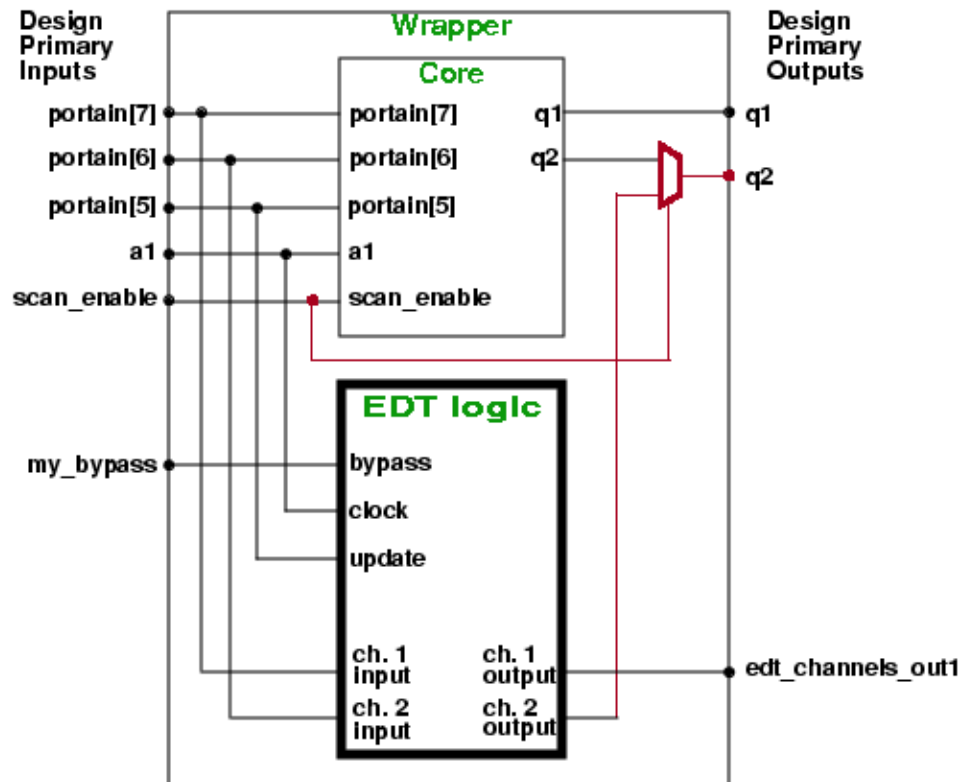
```
set_edt_pins input 1 portain[7]
set_edt_pins input 2 portain[6]
set_edt_pins output 2 q2
set_edt_pins update portain[5]
set_edt_pins clock a1
set_edt_pins bypass my_bypass
```

If you report the EDT pins using the “report_edt_pins” command after issuing the preceding commands, you will see that the shared EDT pins have the same name as the functional core pins. You will also see, for each pin, whether the pin’s signal was specified as inverted. Notice that the listing now includes the scan enable pin because of the shared EDT output pin:

report_edt_pins

```
//      Pin description      Pin name      Inversion
//      -----
//      Clock                a1                -
//      Update                portain[5]         -
//      Scan enable           scan_enable        -
//      Bypass mode           my_bypass           -
//      Scan channel 1 input   portain[7]         -
//      "      "      " output   edt_channels_out1      -
//      Scan channel 2 input   portain[6]         -
//      "      "      " output   q2                -
```

Figure 4-3. Example with Pin Sharing Shown in (External EDT Logic)



After DRC, you can use the “`report_drc_rules k13`” command to report the pins added to the top level of the design to implement the EDT logic.

report_drc_rules k13

```
// Pin my_bypass will be added to the EDT wrapper. (K13-2)
// Pin edt_channels_out1 will be added to the EDT wrapper.
// (K13-3)
```

Connections for EDT Pins (Internal Flow only)

For the internal flow, you must specify the name of each internal node (instance pin name) to connect each EDT control and channel pin.

Note



Before specifying internal nodes, you must specify internal logic placement with the `set_edt_options -location` internal command.

For every EDT pin, you should provide the name of a design pin and the corresponding instance pin name for the internal node that corresponds to it. The latter is the input (or output) of an I/O pad cell where you want the tool to connect the output (or input) of the EDT logic. For example:

```
set_edt_pins clock pi_edt_clock edt_clock_pad/po_edt_clock
```

The first argument “clock” is the description of the EDT pin; in this case the EDT clock pin. The second argument “pi_edt_clock” is the name of the top-level design pin on the I/O pad instance. The last argument is the instance pin name of the internal node of the pad. The pad instance is “edt_clock_pad” and the internal pin on that instance is “po_edt_clock.”

If you specify only one of the pin names, the tool treats it as the I/O pad pin name. If you specify an I/O pad pin name, but not a corresponding internal node name, the EDT logic is connected directly to the top-level pin, ignoring the pad. This may result in undesirable behavior.

If you do not specify either pin name, and the tool does not find a pin at the top level by the default name, it adds a new port for the EDT pin at the top level of the design. You will need to add a pad later that corresponds to that port.

For the internal flow, the [report_edt_pins](#) command lists the names of the internal nodes to which the EDT pins are connected. For example (note that the pin inversion column is omitted for clarity):

```
report_edt_pins
```

```
//
//      Pin description           Pin name           Internal connection
//      -----
//      Clock                     edt_clock         edt_clock_pad/Z
//      Update                    edt_update       edt_update_pad/Z
//      Bypass mode               edt_bypass       edt_bypass_pad/Z
//      Scan ch... 1 input        edt_ch..._in1    channels_in1_pad/Z
//      "      "      " output    edt_ch..._out1   channels_out1_pad/Z
//      Scan ch... 2 input        edt_ch..._in2    channels_in2_pad/Z
//      "      "      " output    edt_ch..._out2   channels_out2_pad/Z
//
```

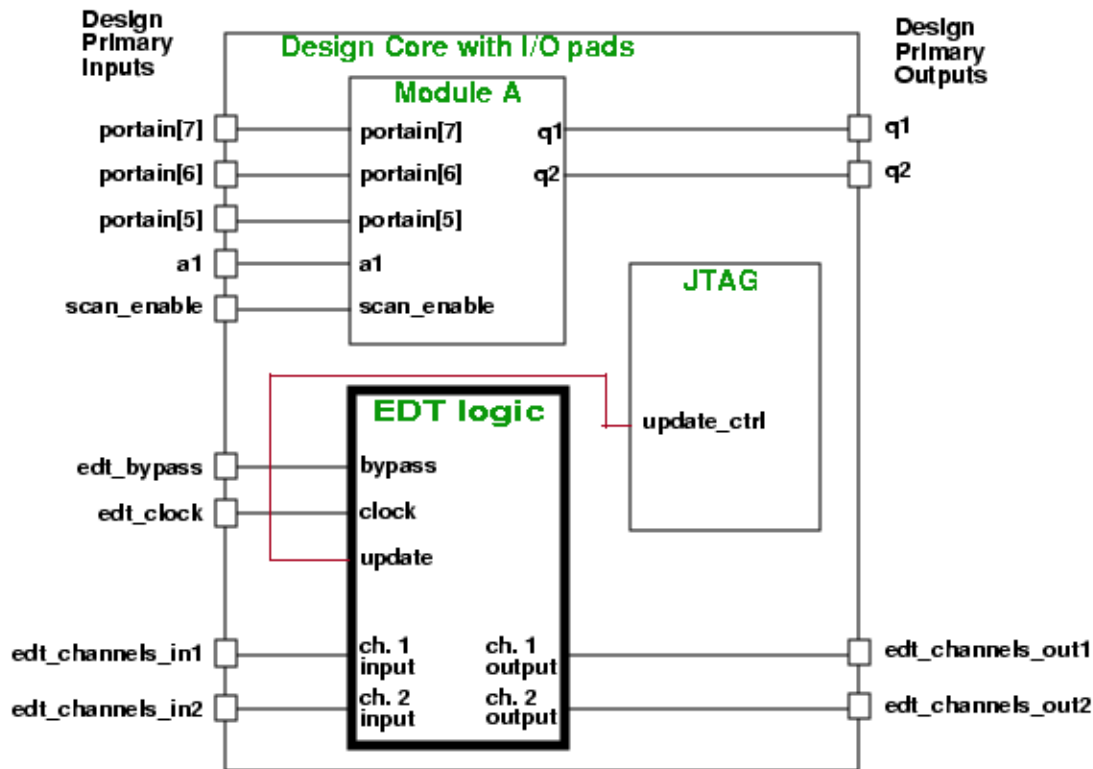
Related Topics

[set_edt_pins \[Tessent Shell Reference Manual\]](#)

Internally Driven EDT Pins

When an EDT control pin is driven internally by JTAG or other control registers (as shown in the following figure), you should use the `set_edt_pins` command to specify that no top-level pin exists for the control pin.

Figure 4-4. Internally Driven edt_update Control Pin



Specifying these types of pins prevents false K5 DRC violations. You should specify internally driven pins in one of the following ways:

- **EDT logic creation**

- Specify the internal node that drives the control pin during logic creation. For example:

```
set_edt_options -location internal
set_edt_pins update - JTAG/update_ctrl
set_system_mode analysis
write_edt_files my_design -verilog -replace
```

Where *JTAG/update_ctrl* is the internal node driving the *update* control pin.

- Edit the test procedure file to include any procedures or pin constraints needed to drive the specified internal node (*JTAG/update_ctrl*) to the correct value.

- **Pattern generation**

- a. Specify the internally driven control pin has no top-level pin during test pattern generation. For example:

```
set_edt_pins update -  
set_system_mode analysis  
add_faults /my_design  
create_patterns
```

Note



All input and output channels must have a corresponding top-level pin.

Structure of the Bypass Chains

When bypass logic is generated, the connections for each bypass chain are automatically configured. These interconnections are fine for most designs.

However, you can specify custom chain connections with the [set_bypass_chains](#) command. For more information, see “[Compression Bypass Logic](#)” on page 230.

Decompressor and Compactor Connections

After you specify the number of scan channels in the EDT logic, the tool automatically determines which scan chain outputs to compact into each channel output.

For more information on specifying the number of scan channels, see “[Parameter Specification for the EDT Logic](#)” on page 72.

You can modify the tool’s default connections using one of the following methods:

Note



Redefining compactor connections for a channel that has already been defined overwrites the previous settings for that channel.

- **Reorder the add_scan_chains Commands** — When generating the EDT IP, the tool uses the sequence of [add_scan_chains](#) commands to connect the EDT hardware to the scan chains. You can change the order of the [add_scan_chains](#) commands in your dofile to change how they are connected to the decompressor and compactor. Note, this method changes both the decompressor and compactor connections for a particular chain.
- **Specify New Connections Using the set_compactor_connections Command** — You can use the [set_compactor_connections](#) command to override the tool’s default connections and explicitly define the connections between scan chains and compactor. This method allows you to change the compactor connections without changing the default decompressor connections to those chains.

If you have dual configurations, you can still define the compactor connections using the `set_compactor_connections` command but only for the configuration that uses all scan channels as shown in the following example. (`set_compactor_connections` is not tied to a specific configuration since you only need to define connections once for each channel.)

```
set_current_edt_configuration config_high
set_edt_options -input 1 -output 1

set_current_edt_configuration config_low
set_edt_options -input 3 -output 3

set_compactor_connections -channel 1 -chains ...
set_compactor_connections -channel 2 -chains ...
set_compactor_connections -channel 3 -chains ...
```

IJTAG and the EDT IP TCD Flow

To fully benefit from the use of the EDT IP TCD flow, you can use IJTAG. With the use of IJTAG, you only need to provide the EDT IP parameters (low power, bypass, and so on); the setup of the configuration is fully automated.

Using IJTAG to configure the EDT IP's static control signals allows the tool to automatically generate the required test_setup sequence through more complex connections such as a TAP and TDRs. Also, when you use IJTAG as part of test_setup for a core, that configuration is automatically carried up the hierarchy as the core is used in a higher level of the design. It is recommended that you extract an ICL description for the design such that IJTAG can be used to configure the EDT IP.

Without IJTAG, you must provide the complete test_setup at most levels of the hierarchy. When IJTAG is not used, you must provide a complete test_setup to configure the EDT static control signals unless those signals are connected directly to the boundary of the design. In this case, the tool will automatically map them. Additionally, IJTAG usage must be explicitly disabled using the "`set_procedure_retargeting_options -ijtag off`" command, otherwise the tool expects to find an ICL description for the design.

Note



The use of IJTAG does not require changing the access mechanism to the EDT IP. Direct connections and any 1149.1 network are IJTAG-compatible.

For the EDT IP TCD flow, IJTAG is the default. Refer to "[IJTAG Mapping](#)" on page 136.

Design Rule Checks

A design rule check (DRC) a rule that checks whether or not a design or design element meets a tool criteria. They indicate that there may a design issue that prevents the tool from creating a

usable test pattern. DRC runs automatically when you leave setup mode by issuing the “set_system_mode analysis” command.

Tessent Shell provides a class of EDT-specific “K” rules. See “[EDT Rules \(K Rules\)](#)” in the Tessent Shell Reference Manual for reference information on each EDT-specific rule.

Notice the DRC message describing the EDT rules in the following example transcript. This transcript is for the design with two scan channels shown in [Figure 4-1](#) on page 84, in which none of the EDT pins are shared with functional pins:

```
// -----  
// Begin EDT rules checking.  
// -----  
// Running EDT logic Creation Phase.  
// 7 pin(s) will be added to the EDT wrapper. (K13)  
// EDT rules checking completed, CPU time=0.01 sec.  
// All scan input pins were forced to TIE-X.  
// All scan output pins were masked.  
// -----
```

These messages indicate the tool will add seven pins, which include scan channel pins, to the top level of the design. The last two messages refer to pins at both ends of the core-level scan chains. Because these pins are not connected to the top-level wrapper (external flow) or the top level of the design (internal flow), the tool does not directly control or observe them in the capture cycle when generating test patterns.

To ensure values are not assigned to the internal scan input pins during the capture cycle, the tool automatically constrains all internal scan chain inputs to X (hence, the “TIE-X” message). Similarly, the tool masks faults that propagate to the scan chain output nodes. This ensures a fault is not counted as observed until it propagates through the compactor logic. The tool only adds constraints on scan chain inputs and outputs added within the tool as PIs and POs.

Note

 To properly configure the internal scan chain inputs and outputs so that the tool can constrain them as needed, you must use the -Internal switch with the [add_scan_chains](#) command when setting up for pattern generation in the Pattern Generation phase.

DRC when EDT Pins are Shared with Functional Pins

If you specified to share any EDT pin with a functional pin, DRC will include messages for K rules affected by the sharing. Here is DRC output for the design shown in [Figure 4-1](#) on

page 84, after it is re-configured to share certain EDT pins with functional pins, as illustrated in [Figure 4-3](#):

```
// -----  
// Begin EDT rules checking.  
// -----  
// Running EDT logic Creation Phase.  
// Warning: 1 EDT clock pin(s) drive functional logic. May  
//   lower test coverage when pin(s) are constrained. (K12)  
// 2 pin(s) will be added to the EDT wrapper. (K13)  
// EDT rules checking completed, CPU time=0.00 sec.  
// All scan input pins were forced to TIE-X.  
// All scan output pins were masked.  
// -----
```

Notice only two EDT pins are added, as opposed to seven pins before pin sharing. Shared pins can create a test situation in which a pin constraint might reduce test coverage. The K12 warning about the shared EDT clock pin points this out to you. For details, refer to “[Functional/EDT Pin Sharing](#)” on page 85.

If you report the current configuration with the [report_edt_configurations](#) command after DRC, you will get more useful information. For example:

report_edt_configurations

```
// IP version:          1  
// Shift cycles:        381, 373 (internal scan length)  
//                      + 8 (additional cycles)  
// External scan channels: 2  
// Internal scan chains:  16  
// Masking registers:    1  
// Decompressor size:    32  
// Scan cells:           5970  
// Bypass logic:         On  
// Lockup Cells:         On  
// Clocking:             edge-sensitive  
// Compactor pipelining: Off
```

Notice that the number of shift cycles (381 in this example) is more than the length of the longest chain. This is because the EDT logic requires additional cycles to set up the decompressor for each EDT pattern (eight in this example). The number of extra cycles is dependent on the EDT logic and the scan configuration.

Creation of EDT Logic Files

By default, the tool writes out the RTL files in the same format as the original netlist. You can use either the EDT pre-synthesis flow or the post-synthesis flow to generate the EDT IP and create the TCD file, which you use during EDT pattern generation instead of the traditional dofiles.

Adapt and use this procedure to generate the EDT IP core and create the TCD file. The procedure illustrated in this section is the modified version of the post-synthesis flow. See also “[Tessent Core Description \(TCD\)](#)” on page 30.

Prerequisites

- You must satisfy all requirements for EDT logic generation.

Procedure

1. Invoke Tessent Shell from a shell using the following syntax:

```
% tessent -shell
```

The tool’s system mode defaults to Setup mode after invocation.

2. With the [set_context](#) command, change the context to EDT IP generation and insertion (dft -edt) as follows:

```
SETUP> set_context dft -edt
```

3. Read the design with scan cells using the [read_verilog](#) command. For example:

```
SETUP> read_verilog cpu_scan.v
```

4. Read the library using the [read_cell_library](#) command. For example:

```
SETUP> read_cell_library adk.tcelllib
```

5. Designate the current design using the [set_current_design](#) command. For example:

```
SETUP> set_current_design
```

6. Depending on your design, you must specify additional parameters such as setting up scan chains and defining clocks and constraints—see “[Parameter Specification for the EDT Logic](#)” on page 72.

7. Define the EDT logic configuration using the [set_edt_options](#) command. For example:

```
SETUP> set_edt_options -input_channels 2 -output_channels 2 -location internal
```

8. Change the system mode to Analysis using the [set_system_mode](#) command as follows:

```
SETUP> set_system_mode analysis
```

The mode change runs the design rule checks and performs the analysis.

9. Use the [write_edt_files](#) command to create the files that make up the EDT logic and the TCD file. For example:

```
ANALYSIS> write_edt_files created -verilog -replace
```

10. Exit Tessent Shell.

```
ANALYSIS> exit
```

Results

In addition to the EDT logic files that are normally created, the tool writes out the TCD. For example: *created_edt_top.tcd*

Once you have specified the EDT logic parameters, you use the [write_edt_files](#) command to create the files that make up the EDT logic. For example:

write_edt_files created -replace

Where *created* is the name string prepended to the files and *-replace* is a switch that allows the tool to overwrite any existing files with the same name.

The TCD file is created during EDT IP core generation by issuing the `write_edt_files` command. The procedure in this section provides the minimal set of Tessent Shell commands needed to generate and insert the EDT IP core and create the TCD file. The tool also generates the ICL and PDL files even if you did not specify the `-ijtag` option; the TCD-based flow is designed to take advantage of the automation IJTAG provides in updating `test_setup` to configure the EDT IP.

The EDT Logic Files

The `write_edt_files` command generates all of the necessary EDT logic files and the design's TCD file.

Depending on the EDT logic placement, the following EDT logic files are created:

- **created_edt_top.tcd** — The design TCD file. You use this file as input to the tool during EDT pattern creation. See [“Generating and Verifying Test Patterns”](#) on page 133.
- **created_edt_top.v** (external EDT logic only) — Top-level wrapper that instantiates the core, EDT logic circuitry, and channel output sharing multiplexers.
- **created_edt_top_rtl.v** (internal EDT logic only) — Core netlist with an instance of the EDT logic connected between I/O pads and internal scan chains but without a gate-level description of the EDT logic.
- **created_edt.v** — EDT logic description in RTL.
- **created_edt.icl** — EDT logic ICL.
- **created_edt.pdl** — EDT logic PDL.
- **created_core_blackbox.v** (external EDT logic only) — Blackbox description of the core for synthesis.
- **created_dc_script.scr** — DC synthesis script for the EDT logic.
- **created_rtlc_script.scr** — RTL Compiler synthesis script for the EDT logic.

The tool also writes out the following dofiles for the legacy flows that do not use the TCD to pass information from IP creation to pattern generation (see [“Dofile-Based Legacy IP Creation and Pattern Generation Flow”](#) on page 369):

- **created_edt.dofile** — Dofile for test pattern generation.
- **created_edt.testproc** — Test procedure file for test pattern generation.
- **created_bypass.dofile** — Dofile for uncompressed test patterns (bypass mode)
- **created_bypass.testproc** — Test procedure file for uncompressed test patterns (bypass mode)

IJTAG and EDT Logic	100
Specification of Module/Instance Names.....	100
EDT Logic Description	100

IJTAG and EDT Logic

By default, the `write_edt_files` command creates IJTAG files that describe the static configuration inputs of the TestKompress IP.

These static configuration inputs set, enable, or disable certain features of the TestKompress IP: EDT bypass, single chain bypass, low power, and EDT configuration. For details on how to use the IJTAG files for TestKompress ATPG, see “[EDT IP Setup for IJTAG Integration](#)” in the *Tessent IJTAG User’s Manual*.

Specification of Module/Instance Names

By default, the tool prepends the name of the top module in the associated netlist to the names of modules/instances in the generated EDT logic files. This ensures that internal names are unique, as long as all module names are unique.

If necessary, you can specify the prefix used for internal modules/instance names in the EDT logic with the `write_edt_files-rtl_prefix prefix_string` command. For example:

```
write_edt_files... -rtl_prefix core1
```

All internal module/instance names are prepended with *core1* instead of the top module name.

Note



The specified string must follow the standard rules for Verilog or VHDL identifiers.

EDT Logic Description

The structure of the logic described in the tool-generated files depends on many variables.

These variables include the following:

- Location of the EDT logic (internal or external with respect to the design netlist)
- Number of external scan channels
- Number of internal scan chains and the length of the longest chain
- Clocking of the first and last scan cell in every chain (if lockup cells are inserted)
- Names of the pins

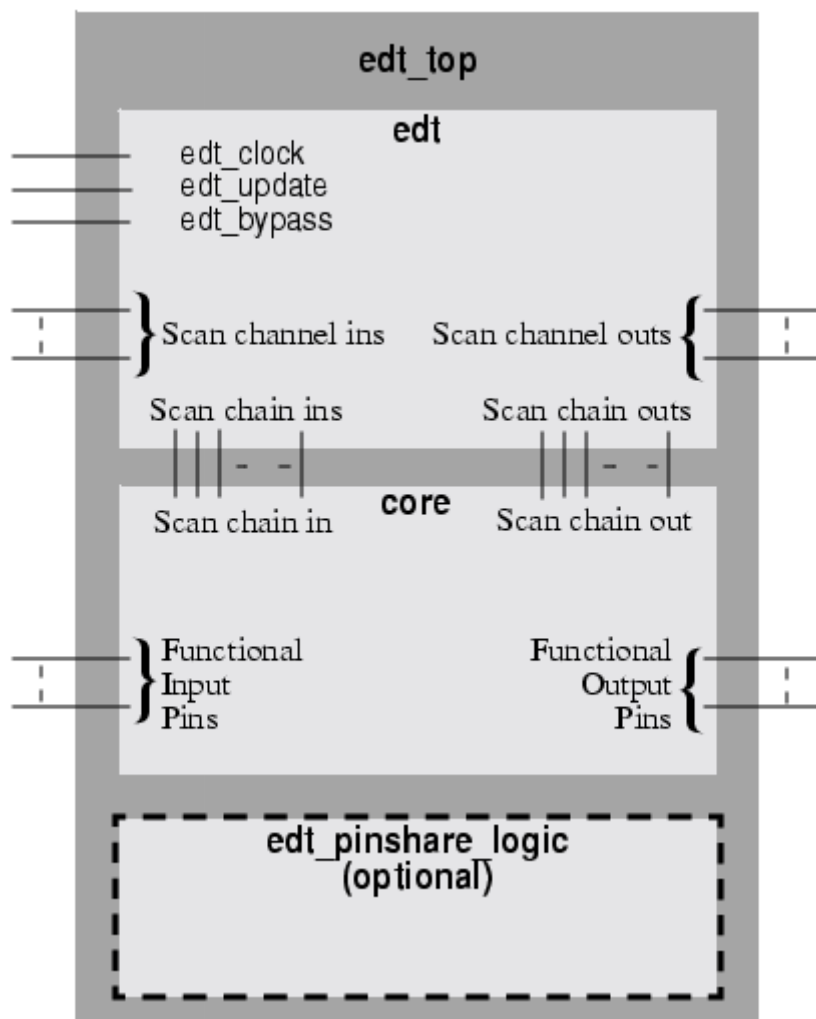
Except for the clocking of the scan chain boundaries, which affects the insertion of lockup cells, nothing in the EDT logic is dependent on the functionality of the core logic.

Note

Generally, you must regenerate the EDT logic if you reorder the scan chains and the clocking of the first and last scan cell or the scan chain length is affected. See “[About Reordered Scan Chains](#)” on page 56.

Top-level Wrapper

Figure 4-5 illustrates the contents of the new top-level netlist file, *created_edt_top.v*. The tool generates this file only if you are using the external flow.

Figure 4-5. Contents of the Top-Level Wrapper

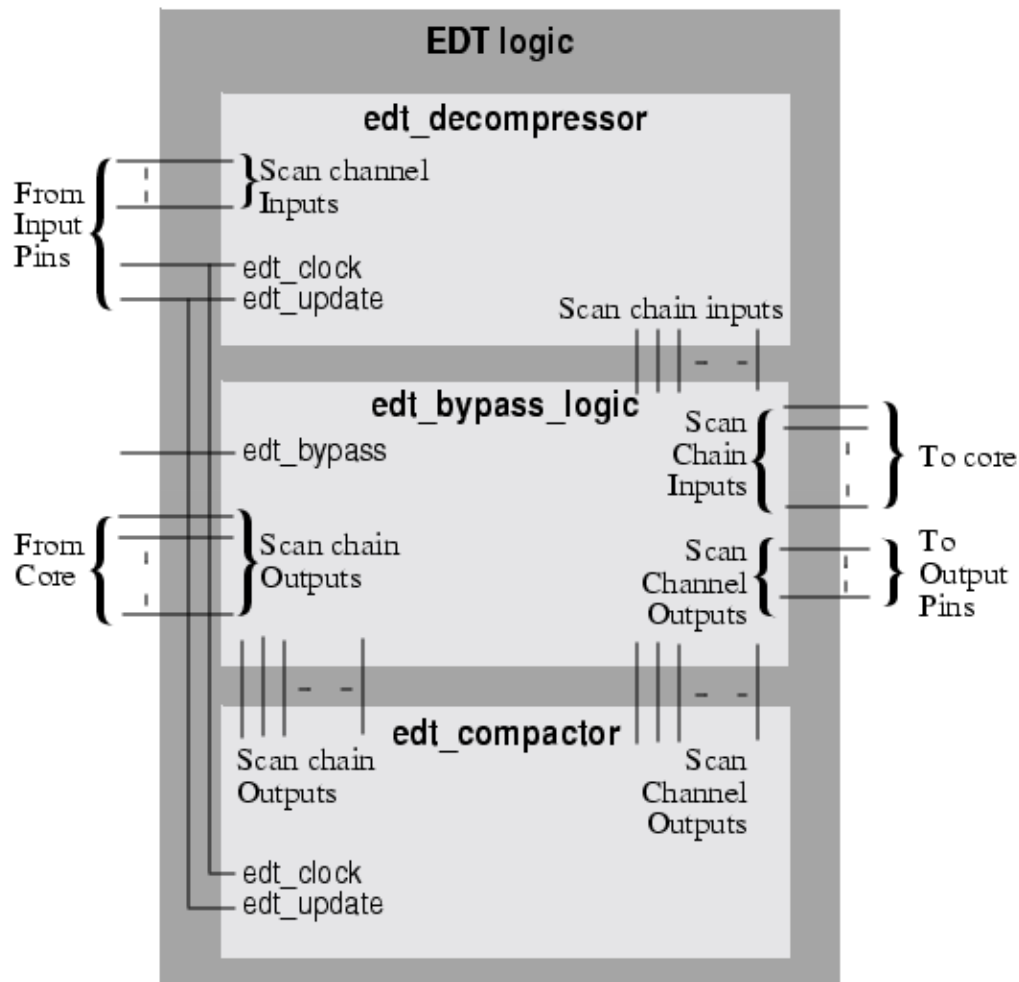
This netlist contains a module, “edt_top”, that instantiates your original core netlist and an “edt” module that instantiates the EDT logic circuitry. If any EDT pins are shared with functional pins, “edt_top” instantiates an additional module called “edt_pinshare_logic” (shown as the optional block in [Figure 4-5](#)). The EDT pins and all functional pins in the core are connected to the wrapper. Scan chain pins are not connected because they are driven and observed by the EDT block.

Because scan chain pins in the core are only connected to the “edt” block, these pins must not be shared with functional pins. For more information, refer to “[Scan Chain Pins](#)” on page 55. Scan channel pin sharing (or renaming) that you specified using the `set_edt_pins` command is implemented in the top-level wrapper. This is discussed in detail in “[Functional/EDT Pin Sharing](#)” on page 85.

EDT Logic Circuitry

Figure 4-6 shows a conceptual view of the contents of the EDT logic file, *created_edt.v*.

Figure 4-6. Contents of the EDT Logic



The EDT logic file contains the top-level module and three main blocks:

- **Decompressor** — Connected between the scan channel input pins and the internal scan chain inputs
- **Compactor** — Connected between the internal scan chain outputs and the scan channel output pins

- **Bypass Logic** — Connected between the EDT logic and the design core. Bypass logic is optional but generated by default.

Core

Generated only when the EDT logic is inserted external to the design core, the file *created_core_blackbox.v* contains a black-box description of the core netlist. This can be used when synthesizing the EDT block so the entire core netlist does not need to be loaded into the synthesis tool.

Note



Loading the entire design is advantageous in some cases as it helps optimize the timing during synthesis.

Design Compiler Synthesis Script External Flow

The tool generates a Design Compiler (DC) synthesis script, *created_dc_script.scr*. By default, the script is in Tool Command Language (TCL), but you can get the tool to write it in DC command language (dcsh) by including a `-synthesis_script dc_shell` argument with the [write_edt_files](#) command.

The following is an example script, in the default TCL format, for a core design that contains a top-level Verilog module named “cpu”:

```
#####
#   Synopsys Design Compiler synthesis script for created_edt_top.v
#
#####

# Read input design files
read_file -f verilog created_core_blackbox.v
read_file -f verilog created_edt.v
read_file -f verilog created_edt_top.v

current_design cpu_edt_top

# Check design for inconsistencies
check_design

# Timing specification
create_clock -period 10 -waveform {0 5} edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
set_clock_transition 0.0 edt_clock
set_dont_touch_network edt_clock

# Avoid assign statements in the synthesized netlist.
set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants

# Compile design
uniquify
set_dont_touch cpu
compile -map_effort medium

# Report design results for EDT logic
report_area > created_dc_script_report.out
report_constraint -all_violators -verbose >> created_dc_script_report.out
report_timing -path full -delay max >> created_dc_script_report.out
report_reference >> created_dc_script_report.out

# Remove top-level module
remove_design cpu

# Read in the original core netlist
read_file -f verilog gate_scan.v
current_design cpu_edt_top
link

# Write output netlist
write -f verilog -hierarchy -o created_edt_top_gate.v
```

Design Compiler Synthesis Script for Internal Flow

The tool generates a Design Compiler (DC) synthesis script *created_dc_script.scr* that synthesizes the EDT logic in the core netlist for the internal flow as shown in the following example.


```
#####
# Synopsys Design Compiler synthesis script for config1_edt.v
# Tessent TestKompress version: v8.2009_3.10-prerelease
# Date: Thu Aug 6 01:44:15 2009
#####

# Bus naming style for Verilog
set bus_naming_style {%s[%d]}

# Read input design files
read_file -f verilog results/config1_edt.v

# Synthesize EDT IP
current_design circle_edt

# Check design for inconsistencies
check_design

# Timing specification
create_clock -period 10 -waveform {0 5} edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
set_clock_transition 0.0 edt_clock
set_dont_touch_network edt_clock

# Avoid assign statements in the synthesized netlist.
set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants

# Compile design
uniquify
compile -map_effort medium

# Report design results for EDT IP
report_area > results/config1_dc_script_report.out
report_constraint -all_violators -verbose >> results/
config1_dc_script_report.out
report_timing -path full -delay max >> results/
config1_dc_script_report.out
report_reference >> results/config1_dc_script_report.out

write -f verilog -hierarchy -o results/config1_circle_edt_gate.v

# Write output netlist
exec cat results/config1_circle_edt_gate.v results/config1_edt_top_rtl.v
> results/config1_edt_top_gate.v
```

RTL Compiler Synthesis Script External Flow

The tool generates an RTL Compiler synthesis script *created_rtlc_script.scr* when the `-synthesis_script rtl_compiler` option is used with the [write_edt_files](#) command as shown:

write_edt_files created -synthesis_script rtl_compiler

This script synthesizes the EDT logic and the top-level wrapper that instantiates the core design and EDT logic for the external flow as shown in the following example.

```
#####
# Cadence RTL Compiler synthesis script for created_edt_top.vhd
# Tessent TestKompress version: v9.1-snapshot_2010.08.19_05.02
# Date: Thu Aug 19 14:07:25 2010
#####

# Set RTL Compiler attributes
set_attribute hdl_auto_async_set_reset true

# Read input design files
read_hdl -vhdl created_core_blackbox.vhd
read_hdl -vhdl created_edt.vhd
read_hdl -vhdl created_edt_top.vhd

# Elaborate design
set_attribute hdl_infer_unresolved_from_logic_abstract true /
elaborate
cd /designs/core_edt_top

# Check design for inconsistencies
check_design

# Timing specification
define_clock -period 10000 -rise 0 -fall 50 edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
# set_attribute ideal_network true edt_clock

# Avoid reset signal buffering during synthesis. However, remember
# to perform reset tree synthesis later for edt_reset
set_attribute ideal_network true edt_reset

# Avoid assign statements in the synthesized netlist.
set_attribute remove_assigns true core_edt_top
set_remove_assign_options -preserve_dangling_nets
-respect_boundary_optimization -verbose -design core_edt_top

# Compile design
edit_netlist uniquify core_edt_top
synthesize -to_mapped -effort medium
change_names -verilog

# Report design results for EDT IP
report_area > created_rtlc_script_report.out
report_design_rules >> created_rtlc_script_report.out
report_timing >> created_rtlc_script_report.out
report_gates >> created_rtlc_script_report.out

# Read in the original core netlist
read_hdl -v1995 m8051_scan.v
elaborate

# Write output netlist
write_hdl > created_edt_top_gate.v
```

RTL Compiler Synthesis Script for Internal Flow

The tool generates an RTL Compiler synthesis script *created_rtlc_script.scr* when the `-synthesis_script rtl_compiler` option is used with the [write_edt_files](#) command as shown:

```
write_edt_files created -synthesis_script rtl_compiler
```

This script synthesizes the EDT logic in the core netlist for the internal flow as shown in the following example.

```
*****
# Cadence RTL Compiler synthesis script for created_edt.v
# Tessent TestKompress version: v9.1-snapshot_2010.08.19_05.02
# Date: Thu Aug 19 14:10:04 2010
*****

# Bus naming style for Verilog
set_attribute bus_naming_style {%s[%d]}

# Read input design files
read_hdl -v1995 created_edt.v

# Elaborate design
set_attribute hdl_infer_unresolved_from_logic_abstract true /
elaborate

# Synthesize EDT IP
cd /designs/B1_edt

# Check design for inconsistencies
check_design

# Timing specification
define_clock -period 10000 -rise 0 -fall 50 edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
set_attribute ideal_network true edt_clock

# Avoid assign statements in the synthesized netlist.
set_attribute remove_assigns true B1_edt
set_remove_assign_options -preserve_dangling_nets
-respect_boundary_optimization -verbose -design B1_edt

# Compile design
edit_netlist uniquify B1_edt
synthesize -to_mapped -effort medium

# Report design results for EDT IP
report area > created_rtlc_script_report.out
report design_rules >> created_rtlc_script_report.out
report timing >> created_rtlc_script_report.out
report gates >> created_rtlc_script_report.out
write_hdl > created_B1_edt_gate.v

cd /designs/B2_edt

# Check design for inconsistencies
check_design

# Timing specification
define_clock -period 10000 -rise 0 -fall 50 edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
set_attribute ideal_network true edt_clock

# Avoid assign statements in the synthesized netlist.
```

```
set_attribute remove_assigns true B2_edt
set_remove_assign_options -preserve_dangling_nets
-respect_boundary_optimization -verbose -design B2_edt

# Compile design
edit_netlist uniquify B2_edt
synthesize -to_mapped -effort medium

# Report design results for EDT IP
report area >> created_rtlc_script_report.out
report design_rules >> created_rtlc_script_report.out
report timing >> created_rtlc_script_report.out
report gates >> created_rtlc_script_report.out
write_hdl > created_B2_edt_gate.v

# Synthesize EDT multiplexor
cd /designs/core_edt_mux_2_to_1

# Check design for inconsistencies
check_design

# Compile design
synthesize -to_mapped -effort medium

# Report design results for EDT mux
report area >> created_rtlc_script_report.out
report timing >> created_rtlc_script_report.out
write_hdl > created_core_edt_mux_2_to_1_gate.v

# Write output netlist
exec cat created_core_edt_mux_2_to_1_gate.v created_B2_edt_gate.v
created_B1_edt_gate.v created_edt_top_rtl.v >
created_edt_top_gate.v

# Remove all temporary files
exec rm created_core_edt_mux_2_to_1_gate.v created_B2_edt_gate.v
created_B1_edt_gate.v
```

Bypass Mode Files

By default, the EDT logic includes bypass circuitry. If your EDT IP can operate in multiple configurations (for example, low power, bypass, and so on), then a single TCD file contains all the configurations. During pattern generation, you will be able to specify how you want those parameters of the EDT IP configured for that ATPG mode. See [“Generating and Verifying Test Patterns”](#) on page 133.

To disable the generation of bypass logic, see the [set_edt_options](#) command.

For improved design routing, the bypass logic can be inserted into the netlist instead of the EDT logic. For more information, see [“Generating EDT Logic When Bypass Logic is Defined in the Netlist”](#) on page 231.

Inserting EDT Logic During Synthesis

When you place EDT logic internal to the design core, the tool writes a core netlist with an instance of the EDT logic connected between I/O pads and internal scan chains without the gate-level description of the EDT logic.

Alternatively, you can use this procedure to create a DC synthesis script that both inserts and synthesizes the EDT logic inside the core netlist with the “[write_edt_files -insertion dc](#)” option. This DC script is more complex because it includes additional commands that help instantiate the EDT logic within the design core and requires that the entire design be loaded into the synthesis tool so the necessary connections can be specified.

Note



The order of the gate-level EDT logic modules in the resulting design is different than that written when the tool inserts the EDT logic before synthesis.

Prerequisites

- Gate-level netlist.

Procedure

1. Invoke Tessent Shell and configure the EDT logic parameters. See “[Creation of the EDT Logic](#)” on page 63. You must set up the EDT logic insertion to be internal to the design core with the [set_edt_options](#) command.
2. Run DRC and correct any errors. See “[Design Rule Checks](#)” on page 94.
3. Examine test coverage and data volume estimates and adjust EDT configurations if necessary. See “[Generating and Verifying Test Patterns](#)” on page 133.
4. Create the EDT logic files. For example:

```
write_edt_files created -insertion dc
```

The following files are created:

- **created_edt.v** — EDT logic description in RTL.
 - **created_dc_script.scr** — DC synthesis script for the EDT logic.
 - **created_edt.dofile** — Dofile for test pattern generation.
 - **created_edt.testproc** — Test procedure file for test pattern generation.
 - **created_bypass.dofile** — Dofile for uncompressed test patterns (bypass mode).
 - **created_bypass.testproc** — Test procedure file for uncompressed test patterns (bypass mode).
5. Synthesize the EDT logic. See “[Synthesizing the EDT Logic](#)” on page 121.

6. Generate test patterns. See “[Generating and Verifying Test Patterns](#)” on page 133.

Examples

The following DC synthesis script is an example of the script generated when the “write_edt_files -insertion dc” option is used.

```
#####
#  Synopsys Design Compiler script for EDT logic insertion
#
#####

# Initialize DC variables
set bus_naming_style {%s[%d]}

# Read input design files
read_file -f verilog results/created_edt.v
read_file -f verilog netlists/gate_scan_sco.v

# Current design is the top-most level.
current_design retimetest

# Create an instantiation of EDT logic within the top-level of the design.
create_cell retimetest_edt_i [find design retimetest_edt]

# Create instantiation(s) of an EDT mux to support sharing of output
channel and core pins.
create_cell retimetest_edt_mux_2_to_1_i1 [find design
retimetest_edt_mux_2_to_1]

# Connect core scan inputs to EDT logic scan inputs.
set scan_inputs { "edt_si1" "edt_si2" "edt_si3" \
                  "edt_si4" "edt_si5" "edt_si6" \
                  "edt_si7" "edt_si8" "edt_si9" \
                  "edt_si10" "edt_si11" "edt_si12" \
                  "edt_si13" "edt_si14" "edt_si15" \
                  "edt_si16" }

set count 0
foreach i $scan_inputs {
    set temp_net [all_connected [find port $i]]
    disconnect_net $temp_net [find port $i]
    set temp "retimetest_edt_i/edt_scan_in[$count]"
    connect_net $temp_net [find pin $temp]
    query_objects [all_connected [find net $temp_net]]
    incr count
}

# Remove scan input ports from top-level.
set scan_inputs_to_delete { "edt_si1" "edt_si2" "edt_si3" \
                             "edt_si4" "edt_si5" "edt_si6" \
                             "edt_si7" "edt_si8" "edt_si9" \
                             "edt_si10" "edt_si11" "edt_si12" \
                             "edt_si13" "edt_si14" "edt_si15" \
                             "edt_si16" }

foreach i $scan_inputs_to_delete {
    remove_port [find port $i]
}

# Connect core scan outputs to EDT logic scan outputs.
set scan_outputs { "edt_so1" "edt_so2" "edt_so3" \
                   "edt_so4" "edt_so5" "edt_so6" \
                   "edt_so7" "edt_so8" "edt_so9" \
                   "edt_so10" "edt_so11" "edt_so12" \
                   "edt_so13" "edt_so14" "edt_so15" \
                   "edt_so16" }
```



```

set count 0
foreach i $scan_outputs {
    set temp_net [all_connected [find port $i]]
    disconnect_net $temp_net [find port $i]
    set temp "retimetest_edt_i/edt_scan_out[$count]"
    connect_net $temp_net [find pin $temp]
    query_objects [all_connected [find net $temp_net]]
    incr count
}

# Remove scan output ports from top-level.
set scan_outputs_to_delete { "edt_so1" "edt_so2" "edt_so3" \
    "edt_so4" "edt_so5" "edt_so6" \
    "edt_so7" "edt_so8" "edt_so9" \
    "edt_so10" "edt_so11" "edt_so12" \
    "edt_so13" "edt_so14" "edt_so15" \
    "edt_so16" }
foreach i $scan_outputs_to_delete {
    remove_port [find port $i]
}

# Connect EDT control pins to the top-level.

# Route pin edt_clk_buf/Z to the EDT control pin retimetest_edt_i/
edt_clock.
set temp_net [all_connected [find pin edt_clk_buf/Z]]
connect_net $temp_net [find pin retimetest_edt_i/edt_clock]
query_objects [all_connected [find net $temp_net]]

# Route pin edt_update to the EDT control pin retimetest_edt_i/edt_update.
create_port edt_update -direction in
create_net retimetest_edt_update_net
connect_net retimetest_edt_update_net [find port edt_update]
connect_net retimetest_edt_update_net [find pin retimetest_edt_i/
edt_update]
query_objects [all_connected [find net retimetest_edt_update_net]]

# Route pin edt_bypass to the EDT control pin retimetest_edt_i/edt_bypass.
create_port edt_bypass -direction in
create_net retimetest_edt_bypass_net
connect_net retimetest_edt_bypass_net [find port edt_bypass]
connect_net retimetest_edt_bypass_net [find pin retimetest_edt_i/
edt_bypass]
query_objects [all_connected [find net retimetest_edt_bypass_net]]

# Connect EDT input channel pins to the top-level.

# Route pin edt_channels_in1 to the EDT channel input pin
retimetest_edt_i/edt_channels_in[0].
create_port edt_channels_in1 -direction in
create_net retimetest_edt_channels_in0_net
connect_net retimetest_edt_channels_in0_net [find port edt_channels_in1]
connect_net retimetest_edt_channels_in0_net [find pin retimetest_edt_i/
edt_channels_in[0]]
query_objects [all_connected [find net retimetest_edt_channels_in0_net]]

# Route pin edt_channels_in2 to the EDT channel input pin
retimetest_edt_i/edt_channels_in[1].

```

```
create_port edt_channels_in2 -direction in
create_net retimetest_edt_channels_in1_net
connect_net retimetest_edt_channels_in1_net [find port edt_channels_in2]
connect_net retimetest_edt_channels_in1_net [find pin retimetest_edt_i/
edt_channels_in[1]]
query_objects [all_connected [find net retimetest_edt_channels_in1_net]]

# Connect EDT output channel pins to the top-level.

# Route EDT channel output pin retimetest_edt_i/edt_channels_out[0] to
left/rcmd_fc_1_fromCore.
set temp_net [all_connected [find pin left/rcmd_fc_1_fromCore]]
disconnect_net $temp_net [find pin left/rcmd_fc_1_fromCore]
connect_net $temp_net [find pin retimetest_edt_mux_2_to_1_i1/a_in]
create_net retimetest_edt_channels_out0_top_net
connect_net retimetest_edt_channels_out0_top_net [find pin
retimetest_edt_mux_2_to_1_i1/b_in]
connect_net retimetest_edt_channels_out0_top_net [find pin
retimetest_edt_i/edt_channels_out[0]]

create_net retimetest_edt_mux_2_to_1_i1_out_net
connect_net retimetest_edt_mux_2_to_1_i1_out_net [find pin
retimetest_edt_mux_2_to_1_i1/z_out]
connect_net retimetest_edt_mux_2_to_1_i1_out_net [find pin left/
rcmd_fc_1_fromCore]
set temp_net [all_connected [find pin scanen_buf/scan_en_out]]
connect_net $temp_net [find pin retimetest_edt_mux_2_to_1_i1/sel]

# Route EDT channel output pin retimetest_edt_i/edt_channels_out[1] to
edt_channels_out2_buf/A.
create_net retimetest_edt_channels_out1_net
connect_net retimetest_edt_channels_out1_net [find pin
edt_channels_out2_buf/A]
connect_net retimetest_edt_channels_out1_net [find pin retimetest_edt_i/
edt_channels_out[1]]
query_objects [all_connected [find net retimetest_edt_channels_out1_net]]

# Connect system clocks to the EDT logic bypass logic.

# Route clock output clk2 to bypass logic.
set temp_net [all_connected [find port clk2]]
connect_net $temp_net [find pin retimetest_edt_i/clk2]
query_objects [all_connected [find net $temp_net]]

# Synthesize EDT logic
current_design retimetest_edt

# Check design for inconsistencies
check_design

# Timing specification
create_clock -period 10 -waveform {0 5} edt_clock

# Avoid clock buffering during synthesis. However, remember
# to perform clock tree synthesis later for edt_clock
set_clock_transition 0.0 edt_clock
set_dont_touch_network edt_clock
```

```
# Avoid assign statements in the synthesized netlist.
set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants

# Compile design
uniquify
compile -map_effort medium

# Report design results for EDT logic
report_area > results/created_dc_script_report.out
report_constraint -all_violators -verbose >> results/
created_dc_script_report.out
report_timing -path full -delay max >> results/
created_dc_script_report.out
report_reference >> results/created_dc_script_report.out

# Synthesize EDT multiplexor
current_design retimetest_edt_mux_2_to_1

# Check design for inconsistencies
check_design

# Compile design
compile -map_effort medium

# Report design results for EDT mux
report_area >> results/created_dc_script_report.out
report_timing -path full -delay max >> results/
created_dc_script_report.out

# Write output netlist
current_design retimetest
write -f verilog -hierarchy -o results/created_edt_top_gate.v
```

The preceding script performs the following EDT logic insertion and synthesis steps:

1. **Fixing the Bus Naming Style** — Because bus signals can be expressed in either bus form (for example, `foo[0]` or `foo(0)`), or bit expanded form (`foo_0`), this command fixes the bus style to the bus form. This is particularly necessary during logic insertion because the script looks for the EDT logic bus signals to be connected to the scan chains.
2. **Read Input Files** — Next, the input gate-level netlist for the core and the RTL description of the EDT logic are read.
3. **Set Current Design** — The current design is set to the top-most level of the input netlist.
4. **Instantiate the EDT Logic and 2x1 Multiplexer Module** — The EDT logic is instantiated within the top-level of the design. If there is sharing between EDT channel outputs and functional pins, a 2x1 multiplexer module (the description is included in the `created_edt.v` file) is also instantiated.

5. **Connect Scan Chain Inputs** — Scan chain inputs should be connected to the top level without any I/O pads associated with them. This part of the script disconnects the nets that are connected to the scan chain input ports and connects them to the EDT logic.
6. **Remove Scan Chain Input Ports** — Remove the dangling scan chain input ports that are not connected to any logic after completing step 5.
7. **Connect Scan Chain Outputs** — Same as step 5, except that now the scan chain outputs are connected to the EDT logic.
8. **Remove Scan Chain Output Ports** — Scan chain output ports, which are left dangling after step 7, are removed from the top level.
9. **Connect EDT Control Pins** — The EDT control pins are connected to the output of pre-existing pads. This is done only if you specified an internal node pin to which to connect the EDT control signal. If not, a new port is created, and the EDT control pin is connected to the new port. The script shows the connection for only the `edt_clock` signal. Similar commands are necessary to connect each of the other EDT control pins.
10. **Connect EDT Channel Input Pins** — The next set of commands create a new port for the input channel pin and connect the EDT input channel pin to the newly created port. This has to be repeated for each input channel pin. This is done only when no internal node name was specified for the EDT pin. If an internal node name was specified, the script would be the same as in step 9.

Note



Be aware you need to add an I/O pad later for each new port that is created.

11. **Connect EDT Channel Output Pin to a Specified Internal Node (or a Top-Level Pin) that is Driven by Functional Logic** — The output channel pin in this case is shared with a functional pin. Whenever the node is shared with functional logic or is connected to TIE-X (a blackbox is assumed in such cases), a multiplexer is inserted. However, if the specified internal node is connected to a constant signal value, the net is disconnected and connected to the EDT channel output pin. This section of the script inserts a multiplexer and connects the inputs from the EDT logic and the functional net to the multiplexer inputs. The output of the multiplexer is connected to the input of the I/O pad cell.

Note



The select input of the multiplexer is connected to the existing scan enable signal, as the functional or scan chain output can be propagated through the multiplexer depending on the shift and capture modes of the scan operation.

You should specify the name of the scan enable signal. If a name is not specified and a pin with a default name (`scan_en`) does not exist at the top level, a new `scan_en` pin is created.

12. **Connect EDT Channel Output Pin to a Non-Shared Specified Internal Node (Or A Top-Level Pin)** — In this case, the specified internal node is not shared with any functional logic. The tool removes any net that is connected to the internal node. It creates a new net and connects it to the channel output pin.
13. **Connect System Clocks to EDT Logic** — For the bypass mode, scan chains are concatenated so that the EDT logic can be bypassed and normal ATPG scan patterns can be applied to the circuit. During scan chain concatenation, lockup cells are often needed (especially if the clocks are different or the clock edges are different) to guarantee proper scan shifting. These “bypass” lockup cells are driven by the clocks that drive the source or the destination scan cells. As a result, some system clocks have to be routed to the bypass module of the EDT logic. Clock lines are tapped right before they fan out to multiple modules or scan cells and are brought up to the topmost level and connected to the EDT logic.
14. **Synthesis of RTL Code** — At this point, the insertion of the EDT logic within the top-level of the original core is complete. The subsequent parts of the script are mainly for synthesizing the logic. This part of the script is almost the same as that of the external flow, with the following exceptions: the EDT logic is synthesized first followed by the EDT multiplexer(s). In both cases, the synthesis is local to the RTL blocks and does not affect the core, which is already at the gate level.
15. **Write Out Final Netlist** — Once the synthesis step is completed, DC writes out a gate-level netlist that contains the original core, the EDT logic, and any multiplexers the tool added to facilitate sharing of output pins.

Related Topics

[The EDT Logic Files](#)

[Creation of a Reduced Netlist for Synthesis](#)

[Inserting EDT Logic During Synthesis](#)

Creation of a Reduced Netlist for Synthesis

In the internal flow, the EDT logic is instantiated within an existing pad-inserted netlist and connections must be made from the pad terminals to the EDT logic pins.

When using DC to insert and synthesize the EDT logic as described in “[Inserting EDT Logic During Synthesis](#),” the entire netlist must be read into the synthesis tool, so that the proper connections can be specified.

Input netlists can be huge and it may take a lot of time or perhaps not even be possible to run the synthesis tool. If your netlist is in this category, you can use the tool to write out a reduced-size netlist especially for the synthesis run with the [write_edt_files](#) -reduced_netlist command. The reduced netlist includes only the modules required for the synthesis tool to make the necessary connections between the pad terminals and EDT logic pins.

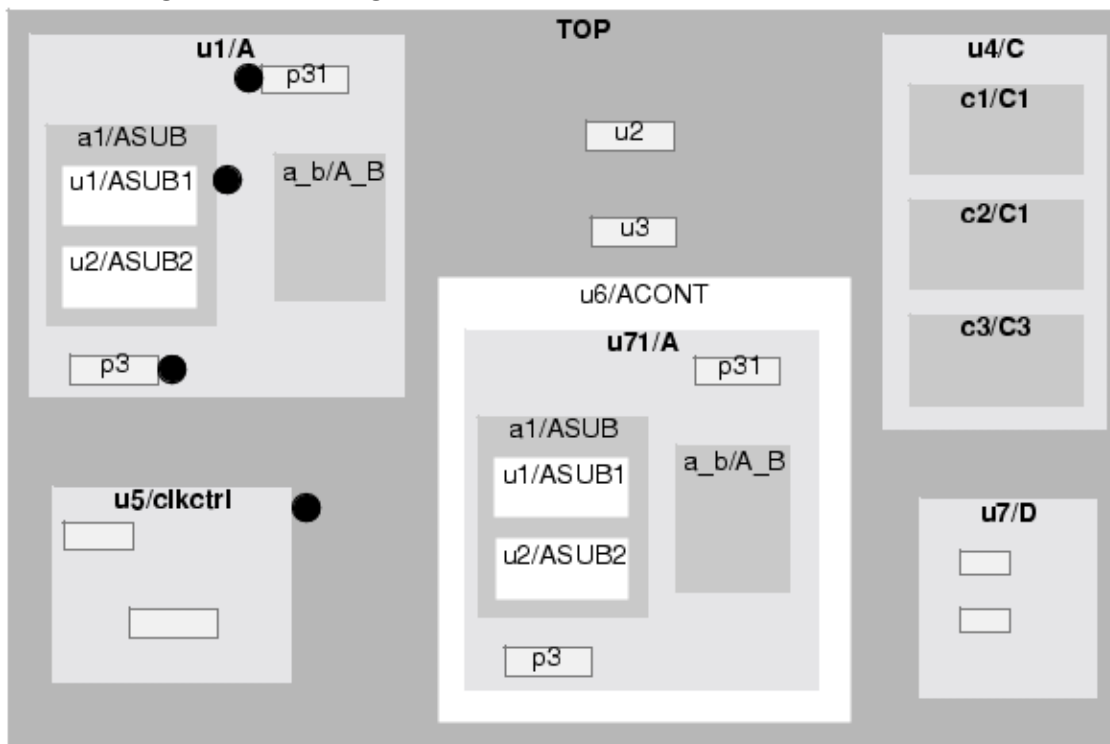
You provide this smaller file to the synthesis tool instead of providing the entire input netlist. The tool writes the rest of the input netlist into a second netlist file that excludes the modules written out in the *synthesisonly* file but includes everything else. You then use the output netlist from the synthesis run along with the second netlist as inputs for ATPG.

Note

 Another option is to use the tool to insert the EDT logic into the core netlist before synthesis. This is the default behavior for the internal flow.

Figure 4-7 is a conceptual example of an input netlist. Each box represents an instance with the instance_name/module_names shown. The small rectangles shaded with dots represent instances of technology library cells. The black circles represent four internal EDT logic connection nodes.

Figure 4-7. Design Netlist with Internal Connection Nodes



When writing out the compression files with the -Reduced_netlist switch, the modules are distributed between the two netlists as follows:

- **<prefix>_reduced_netlist.v(for synthesis)** — TOP, A, ASUB, clkctrl
- **<prefix>_rest_of_netlist.v(rest of the netlist for ATPG)** — ASUB1, ASUB2, A_B, C, C1, C3, ACONT, D

Note



The reduced netlist for synthesis does not include the technology library cells that have EDT logic connection nodes because the port list is sufficient for making these connections.

Chapter 5

Synthesizing the EDT Logic

After you create the EDT logic, as discussed in “Creation of the EDT Logic”, the next step is to synthesize it. The tool creates a basic Design Compiler (DC) synthesis script, in either dcsh or TCL format, that you can use as a starting point. Running the synthesis script is a separate step in which you exit the tool and use DC to synthesize the EDT logic. You can use any synthesis tool; the generated DC script provides a template for developing a custom script for any synthesis tool.

The EDT Logic Synthesis Script	121
Synthesis and External EDT Logic	122
Synthesis and Internal EDT Logic	124
SDC Timing File Generation	125
SDC Timing File Generation Using <code>extract_sdc</code>	125
SDC Timing File Generation Using <code>write_edt_files</code>	126

The EDT Logic Synthesis Script

If you use DC to synthesize the netlist, you should examine the `.synopsys_dc.setup` file and verify that it points to the correct libraries. Also, examine the DC synthesis script generated by the tool and make any needed modifications.

Note

 You should preserve the pin names in the EDT logic hierarchy. Preserving pin names ensures that pins resolve when test patterns are created and increases the usefulness of the debug information returned during DRC.

Note

 When using the external flow and boundary scan, you must modify this script to read in the RTL description of the boundary scan circuitry. Refer to “[Preparation for Synthesis of Boundary Scan and EDT Logic](#)” on page 240 for an example DC synthesis script with modifications for boundary scan.

The following DC commands are included in the synthesis scripts created by the tool:

- `set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants`

This command prevents DC from including *assign* statements in the Verilog gate-level netlist to prevent problems later in the design flow.

- `set_clock_transition 0.0 edt_clock set_dont_touch_network edt_clock`

These commands prevent buffering of the EDT clock during synthesis and preserves the EDT clock network. However, you must perform clock tree synthesis later for the EDT clock.

After you run DC to synthesize the netlist without any errors, verify the tri-state buffers were correctly synthesized. In some cases, DC may insert incorrect references to `\**TSGEN**`. For information on correcting these references, see [“Incorrect References in Synthesized Netlist”](#) on page 364.

For more information, see [“The EDT Logic Files”](#) on page 99.

Related Topics

[The EDT Logic Files](#)

Synthesis and External EDT Logic

Once the EDT logic is created but before you synthesize it, you should insert I/O pads and (optionally) boundary scan. For designs that require boundary scan, you should insert the boundary scan first, followed by I/O pads. Then, synthesize the I/O pads and boundary scan together with the EDT logic.

Note

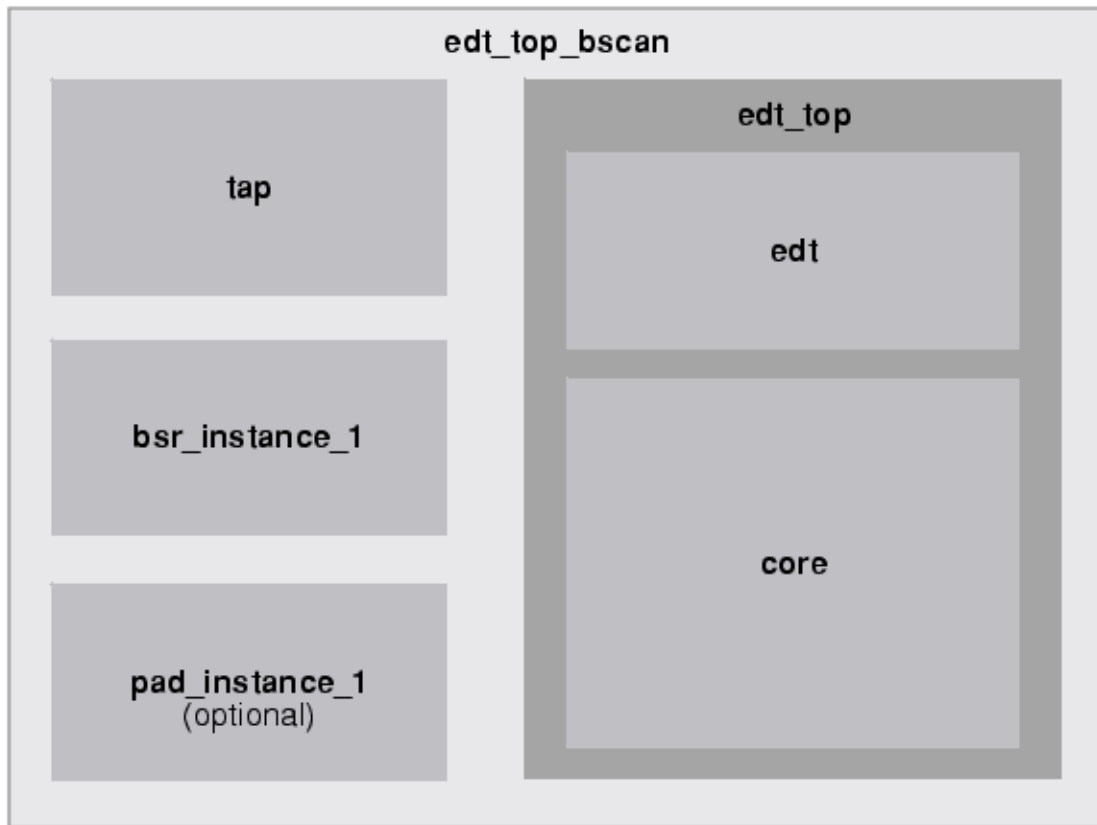


You can add boundary scan and I/O pads simultaneously with a boundary scan tool.

Boundary Scan

Boundary scan cells cannot be present in your design before the EDT logic is inserted. To include boundary scan, you perform an additional step *after* the EDT logic is created. In this step, you can use any tool to insert boundary scan. As shown in [Figure 5-1](#), the circuitry should include the boundary scan register, TAP controller, and (optionally) I/O pads.

Figure 5-1. Contents of Boundary Scan Top-Level Wrapper



I/O Pad Insertion

You can use any method to insert I/O pads after boundary scan insertion and EDT logic creation. If you need to integrate EDT logic after the I/O pads are inserted, see “[Managing Pre-existing I/O Pads](#)”.

If the core and pads are separated as described in “Managing Pre-existing I/O Pads”, you should reinsert the EDT logic-core combination into the original circuit in place of the extracted core. When you reinsert it, ensure the EDT logic-core combination is connected to the I/O pads. Add pads for any new EDT pins not shared with existing core pins.

If you need to insert I/O pads before scan insertion and you used the architecture swapping solution described in the “Managing Pre-existing I/O Pads” section, then I/O pads are already included in your scan-inserted design and you can proceed to insert boundary scan.

Related Topics

[Creation of the EDT Logic](#)

Synthesis and Internal EDT Logic

The tool inserts and connects an instance of the EDT logic into the design netlist and creates a DC script to synthesize the EDT logic.

You may be able to run the script without modification if the following are true:

- DC is the synthesis tool.
- The default clock definitions are acceptable.
- Technology library files are set up correctly in the *.synopsys_dc.setup* file.

Note



The syntax of the *.synopsys_dc.setup* file and the DC synthesis script differ depending on which format, dcsh or TCL, they support. If the *.synopsys_dc.setup* file does not exist, you must add the library file references to the synthesis script.

Optionally, you can insert the EDT logic into the core during synthesis. See “[Inserting EDT Logic During Synthesis](#)” on page 110.

SDC Timing File Generation

You can use the tool to generate Synopsys Design Constraint (SDC) timing files for the static timing analysis of the test logic. There are two methods for generating SDC timing files, using the “extract_sdc” and the “write_edt_files -timing_constraints” commands.

The SDC generated with the extract_sdc method is suitable for the DftSpecification flow with the [process_dft_specification](#) command where DFT signals for logictest clocks and signals such as “scan_enable” are defined. This method can provide logictest timing constraints right from the RTL context, which can be merged with user-defined timing constraints into a “single-mode” set of constraints that apply to the whole design, for running global pre-layout synthesis, or even some phases of layout. The [extract_sdc](#) command also generates modal STA procs for pre or post-layout signoff.

The SDC generated with the “write_edt_files -timing_constraints” method is suitable when generating EDT logic on gate level scan inserted designs, or when using the Legacy Skeleton RTL flow. This method uses port names used during EDT insertion for clocks and control signals instead of DFT signals.

SDC Timing File Generation Using extract_sdc..... 125

SDC Timing File Generation Using write_edt_files 126

SDC Timing File Generation Using extract_sdc

The Synopsys Design Constraint (SDC) timing file can be generated using the extract_sdc command.

The “write_edt_files -timing_constraints method” generates timing tool driver files and Tcl procedures for different modes to be used in static timing analysis. It targets gate level design. The extract_sdc method generates timing constraints for both RTL-to-gate synthesis and for pre or post-layout STA.

The EDT SDC can be generated on a design without IjtagNetwork where ICL extraction can still be performed on the design. An example of this usage is when the static control signals of the EDT controller are brought up to ports at the current_design level.

Both the “process_dft_specification” RTL flow and the “write_edt_files -tsdb” gate flow can use extract_sdc to generate SDC timing files. The “extract_sdc” flow is the only supported method for “write_edt_files -tsdb” flow, that is the flow where the EDT IP is generated in the gate level after scan insertion. For this method, the “extract_sdc” flow has the same requirements as the DftSpecification flow, specifically knowledge of DFT signals.

For more information, see “[Timing Constraints \(SDC\)](#)” in the *Tessent Shell User’s Manual*.

SDC Timing File Generation Using write_edt_files

The write_edt_files command can be used to generate SDC timing files.

Separate SDC files provide timing constraints for the EDT logic and the ATPG setups as described in the following topics:

Note

 The SDC files are generated from the timing specified in the test procedure file. The generated SDC files should be used as templates and employed for static timing analysis only after appropriate values are inserted to correspond with actual timing information.

The timing files are formatted in the TCL programming language with multiple sections. This allows you to select one or all sections depending on your needs.

You can also set variables before the timing files are loaded to specify values in the timing files as described in Table 5-1.

Table 5-1. Timing File Variables

Description	Variables
Parameters for system clocks	system_clock_latency_min system_clock_latency_max system_clock_uncertainty_setup system_clock_uncertainty_hold
Parameters for EDT clocks	edt_clock_latency_min edt_clock_latency_max edt_clock_uncertainty_setup edt_clock_uncertainty_hold
I/O delay for EDT pins	edt_pins_input_delay edt_pins_output_delay

EDT Logic/Core Interface Timing Files	126
Scan Chain and ATPG Timing Files	131

EDT Logic/Core Interface Timing Files

You can output timing files specific to the EDT logic and design core interface with the write_edt_files -Timing_constraints command.

Depending on the application, the tool writes out these timing files:

- **filename_prefix_edt_shift_sdc.tcl** — Specifies constraints for the EDT shift mode.

- **filename_prefix_bypass_shift_sdc.tcl** — Specifies constraints for the EDT bypass shift mode. This file is written for applications that include a bypass configuration. By default, the tool outputs an EDT bypass configuration.
- **filename_prefix_slow_capture_sdc.tcl** — Specifies constraints for slow-capture mode. This file is only written when stuck-at patterns or launch-off-shift capture patterns are used.
- **filename_prefix_fast_capture_sdc.tcl** — Specifies constraints for fast-capture mode. This file is only written when launch-off capture transition patterns are used.

Timing files can also be generated for EDT logic with dual compression configurations. When test patterns are applied, only one of the configurations is active at any time. So, the paths originating at `edt_configuration` are declared as multi-cycle paths to avoid the need to verify each of the individual configurations separately. For more information on dual configurations, see “[Dual Compression Configurations](#)” on page 73.

Note



When the EDT logic is placed inside the design and a top-level pin name is not specified for a control pin, then the specified internal connection name is used for synthesis and in the constraints. For more information, see the [set_edt_pins](#) command.

EDT Shift Mode Clock Constraints

During shift, the EDT logic is clocked along with all the scan cells as new data is loaded and the captured data is unloaded from the scan chains. Therefore, the tool declares the `edt_clock` and all the shift clocks:

- **create_clock** — Declares all clocks used for scan chain shifting at the very beginning of the file. For example:

```
create_clock -name edt_clock -period 100 -waveform {50 90}
[get_ports edt_clock]
```

- **set_clock_latency** — Describes the clock network latency for all clocks used during shift. Clock latency for both the minimum and maximum operating conditions is specified. Because the tool has no timing information, a default value of 0 is used for the latencies. These default values can be changed to reflect the actual values as necessary. For example:

```
set_clock_latency -min 0 [get_clocks edt_clock]
set_clock_latency -max 0 [get_clocks edt_clock]
```

- **set_clock_uncertainty** — Describes the uncertainty (skew) related to the setup and hold times for the flops driven by specified shift clocks. For example:

```
set_clock_uncertainty -setup <def_value> [get_clocks edt_clock]
set_clock_uncertainty -hold <def_value> [get_clocks edt_clocks]
```

EDT Shift Mode Input/Output Pin Delay Constraints

During shift mode, the input and output delays for the EDT control and channel pins are declared. For the `edt_channel` pins, the input delay is measured with respect to the *force_pi* and *measure_po* events in the test procedure. Default values can be changed to reflect the actual values as necessary. For example:

- **set_input_delay** – Specifies the arrival time of the signals relative to when the clock edge appears. For example:

```
set_input_delay <def_value> -clock force_pi [get_ports  
edt_channel_in]
```

- **set_output_delay** – Specifies the departure time of the signals relative to when the clock edge appears. For example:

```
set_output_delay <def_value> -clock measure_po [get_ports  
edt_channel_out]
```

EDT Shift Mode Static Constraints

During EDT shift mode, static values for certain EDT-specific signals are declared as follows:

- **EDT bypass mode signal** — `edt_bypass` signal is constrained to 0 (off), when EDT mode is enabled. For example:

```
set_case_analysis 0 edt_bypass
```

- **EDT reset signal** — `edt_reset` signal is constrained to 0 (off). For example:

```
set_case_analysis 0 edt_reset
```

- **Scanenable (SEN) signal** — `scan_en` signal controls the select input of the muxes when channel output pins are shared with functional pins. For example:

```
set_case_analysis <on_state> scan_en
```

- **Dual configuration signal** — `edt_configuration` signal is set to either a 1 or 0 value by the test patterns depending on the configuration being used. Instead of constraining the `edt_configuration` signal, all paths originating from the pin are declared as multi-cycle paths. For example:

```
set_multicycle_path <path_multiplier> -from edt_configuration
```

EDT Shift Mode Timing Exceptions

- **False and multi-cycle paths** — During shift, all paths in the EDT logic are exercised, so no false or multi-cycle paths are declared except as follows:
 - Hold timing exception added from `masks_hold_reg` through EDT channel output when there are no pipeline or lockup cells used uniformly for all scan chains in that channel.

- Hold timing exception from the low-power hold register on the scan chain input side. See “[Static Timing Analysis and Hold Violations From Low-Power Hold Registers](#)” for complete information.

For example:

```
global reg_suffix
set rsfx [expr {[info exists reg_suffix] ? $reg_suffix : "_reg"}]
# Relax min_delay(hold) check from low power hold registers to scan cells,
# since scan clocks are OFF when low power hold registers are updated

set_multicycle_path -hold 1 -from [get_cells piccpu_edt_i/
piccpu_edt_controller_i/low_power_shift_controller_i/
low_power_hold_reg*$rsfx*]

# Relax min_delay(hold) check from mask hold registers through edt
# channel output, since scan clocks are OFF when the mask hold registers
# are updated

set_multicycle_path -hold 1 -from [get_cells piccpu_edt_i/
piccpu_edt_controller_i/masks_hold_reg*$rsfx*] -through [get_pins
piccpu_edt_i/edt_channels_out?0?]

set_multicycle_path -hold 1 -from [get_cells piccpu_edt_i/
piccpu_edt_controller_i/masks_hold_reg*$rsfx*] -through [get_pins
piccpu_edt_i/edt_channels_out?1?]
```

In the example, `reg_suffix` defaults to “`_reg`” that matches how Design Compiler names registers from RTL net names. When using an skeleton flow, you may need to update the EDT instance path based on the final design.

Bypass Shift Mode Constraints

In bypass shift mode, the EDT decompressor, compactor, and masking logic are completely bypassed and the scan chains behave as uncompressed chains that operate with regular ATPG patterns. The timing constraints for bypass shift mode are similar to those for regular scan operation except as follows:

- **EDT bypass signal** — `edt_bypass` signal is constrained to 1 (on). For example:

```
set_case_analysis 1 edt_bypass
```

- **Scan enable (SEN) signal** — `scan_en` signal is asserted to its on state when an EDT channel output pin is shared with a functional output pin. The `set_edt_pins` command specifies the `scan_en` pin. For example:

```
set_case_analysis <on_state> scan_en
```

- **Clock constraints** — All clocks used during bypass shift mode are declared using the same commands as for EDT shift mode. See “[EDT Shift Mode Clock Constraints](#)” on page 127.

- **Input and output delays** — The input and output delays should be described for all scan chain I/Os. The input and output delay constraints for bypass shift are declared with the same commands as for EDT shift. See “[EDT Shift Mode Input/Output Pin Delay Constraints](#)” on page 128.
- **EDT logic** – In the bypass mode, the EDT logic is completely bypassed, and therefore, any paths originating and ending in the EDT logic are declared as false paths as follows:

```
set_false_path -from edt_clock
set_false_path -to edt_clock
```

Bypass/EDT Capture Mode Constraints

In the capture mode, the primary objective is to mimic the functional operation of the design, but only timing constraints related to the test logic are written. Constraints related to the functional mode of operation should be specified by the functional timing constraints file for the design. Specifically, some of the timing constraints are as follows:

- **Clock constraints** — All clocks used during capture mode are declared using the same commands as for EDT shift. See “[EDT Shift Mode Clock Constraints](#)” on page 127.
- **Input and output delays** — The input and output delays are declared for all scan chain I/Os using the same commands as for EDT shift. See “[EDT Shift Mode Input/Output Pin Delay Constraints](#)” on page 128.
- **Static constraints** — The `edt_reset` signal is constrained to its off (0) state during capture. For example:

```
set_case_analysis 0 edt_reset
```

Note

 `edt_bypass`, `edt_update`, and `edt_configuration` could potentially be shared with functional pins set by ATPG, so they are not constrained. During capture, the EDT clock is not pulsed, so the values on these pins do not interfere with the EDT logic.

- **Inactive paths** — The `edt_clock` is not pulsed during capture, so the following paths are unused and need to be declared as false paths:
 - Between the `mask_shift_reg` and `mask_hold_reg`.
 - Between the `mask_hold_reg` and the output channels, pipeline cells, or lockup cells (if they exist).
 - Between the lockup cells at the output of the decompressor and the input of the scan chains.
 - Between the pipeline stages at the compactor and the EDT channel output pins.

False paths are declared for all these cases by declaring all paths originating from state elements clocked by `edt_clock` as false paths. For example:

```
set_false_path -from edt_clock
```

- **Paths between the scan chain outputs and the compactor** — The scan chain outputs feeding the compactor are not active during capture. Therefore, all paths from the decompressor outputs to the `edt_channel_output` or the lockup cells in front of the pipeline stages inside the compactor are declared as false paths.

If no pipeline stages or lockup cells exist, then the following constraints declare all EDT channels as false paths. For example:

```
set_false_path -to <edt_channel_output>
```

If pipeline stages or lockup cells do exist, then all paths originating from state elements clocked by `edt_clock` are declared as false paths. For example:

```
set_false_path -from edt_clock
```

- **Slow and fast capture modes** — The slow capture mode corresponds to stuck-at and launch-off-shift patterns, and fast capture mode corresponds to launch-off-capture (broadside) patterns.
 - **Slow capture mode** — The `scan_enable` pin is unconstrained, so the scan path could potentially be used by ATPG. For bypass patterns, the bypass chain concatenation path through `edt_bypass_logic` is unconstrained. For the EDT capture mode this path is not used, but it is unconstrained so that both bypass and EDT capture can share the same timing constraints.
 - **Fast capture mode** — The bypass chain concatenation path through the `edt_bypass_logic` is not used and is declared a false path. For example:

```
create_clock -period 100 { edt_i/sysclk }
set_false_path -to edt_i/sysclk
set_case_analysis 0 edt_i/edt_bypass
```

Scan Chain and ATPG Timing Files

You can output timing files specific to the scan path and ATPG setup in the core with the “`write_core_timing_constraints filename_prefix`” command. Depending on the application, the following timing files are written out.

- **filename_prefix_core_shift_sdc.tcl** — Specifies shift mode constraints.
- **filename_prefix_slow_capture_sdc.tcl** — Specifies slow-capture mode constraints. This file is only written when stuck-at or launch-off-shift capture patterns are used.
- **filename_prefix_fast_capture_sdc.tcl** — Specifies fast-capture mode constraints. This file is only written when launch-off capture transition patterns are used.

Scan Chain and ATPG Core Constraints

The scan chain and ATPG constraints associated with the core are determined as follows:

- For scan shift mode, the *scan_en* signal is constrained to its active value, so paths from scan cell outputs to the functional logic are declared as false paths. This is done by forcing the values found in the shift procedure.
- For capture mode, all shift paths between successive scan cells are declared as false paths, unless launch-off-shift (LOS) transition patterns are in effect. This is done by forcing pin constraint values.
- For at-speed testing, the *hold_pi* and *mask_po* constraints are translated into timing constraints. A warning message is issued when writing out the fast capture mode timing constraints if *hold_pi* and *mask_po* are not specified.
- Cell constraints specified for an ATPG run are declared during the capture mode.

Constraints specified using a form other than *pin_pathname* are converted into structurally reachable pins at the boundary of library cells that contain the target sequential element. This includes all non-clock input pins for *set_false_path -to* and all output pins for *set_false_path -from* and *set_case_analysis*.

Constraints specified using *-clock* and *-chain* are translated into individual sequential elements. All constraints except TX are translated to *set_false_path -from* and *set_false_path -to*.

Chapter 6

Generating and Verifying Test Patterns

This chapter describes how to generate compressed test patterns. In this part of the flow, you generate and verify the final set of test patterns for the design.

After you insert I/O pads and boundary scan and synthesize the EDT logic, invoke Tessent Shell with the synthesized top-level netlist and generate compressed test patterns.

Note



You can write test patterns in a variety of formats including Verilog and WGL.

Preparation for Test Pattern Generation	133
EDT Pattern Generation Overview	136
IJTAG Mapping	136
Scan Chain Handling	137
Core Instance Parameters.....	138
Used Input Channels.....	142
Pattern Generation with Internal Chain Masking Hardware.....	145
Updating Scan Pins for Test Pattern Generation	145
Verification of the EDT Logic.....	149
Design Rules Checking (DRC).....	149
EDT Logic and Chain Testing	149
Reducing Serial EDT Chain Test Simulation Runtime	152
Test Pattern Generation	154
Generating Patterns.....	154
Compression Optimization	155
Saving of the Patterns	156
Post-Processing of EDT Patterns	157
Simulation of the Generated Test Patterns	157

Preparation for Test Pattern Generation

To prepare for EDT pattern generation, check that EDT is on, and configure the tool the same as when you created the EDT logic. For example, if you create the EDT logic with one scan channel, you must generate test patterns for circuitry with one channel.

Note

 You can reuse uncompressed ATPG dofiles, with the addition of some EDT-specific commands, to generate compressed patterns with the same test coverage as the original uncompressed patterns. You cannot directly reuse pre-computed existing ATPG patterns.

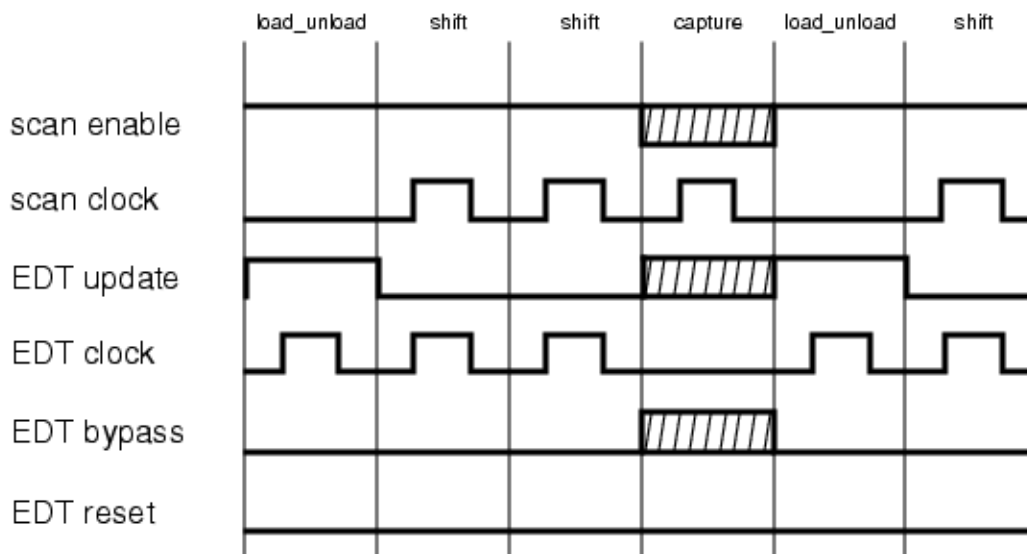
DRC violations occur if you attempt to generate patterns for a different number of scan channels than what the EDT logic is configured for.

You must also add scan chains in the same order they were added to the EDT logic—see “[Scan Chain Handling](#)” on page 137 and “[IJTAG Mapping](#)” on page 136.

The `report_scan_chains` command lists the scan chains in the same order they were added originally.

Compared to when you generated uncompressed test patterns with the scan-inserted core design (see “[ATPG Baseline Generation](#)” on page 61), there are certain differences in the tool setup. One of the differences arises because in the Pattern Generation phase you need to set up the patterns to operate the EDT logic. This is done by exercising the EDT clock, update and bypass (if present) control signals as illustrated in Figure 6-1.

Figure 6-1. Sample EDT Test Procedure Waveforms



This figure illustrates how the tool creates the EDT events automatically. Prior to each scan load, the EDT logic needs to be reset. This is done by pulsing the EDT clock once while EDT update is high.

During shifting, the EDT clock should be pulsed together with the scan clock(s). In [Figure 6-1](#), both scan enable and EDT update are shown as 0 during the capture cycle. These two signals can have any value during capture; they do not have to be constrained. On the other hand, the EDT clock must be 0 during the capture cycle.

On the command line or in a dofile, you must do the following:

- Identify the EDT clock signal as a clock and constrain it to the off-state (0) during the capture cycle. This ensures the tool does not pulse it during the capture cycle.
- Use the -Internal option with the [add_scan_chains](#) command to define the compressed scan chains as internal, as opposed to external channels. This definition is different from the definition you used to create the EDT logic because the scan chains are now connected to internal nodes of the design and not to primary inputs and outputs. Also, scan_in and scan_out are internal nodes, not primary inputs or outputs.
- Uncompressed scan chains are chains not defined with the [add_scan_chains](#) command when setting up the EDT logic and whose scan inputs and outputs are primary inputs and outputs. If your design includes uncompressed scan chains, you must define each uncompressed scan chain using the add_scan_chains command *without* the -Internal switch during test pattern generation.
- If you add levels of hierarchy (due, for example, to boundary scan or I/O pads), revise the pathnames to the internal scan pins listed in the generated dofile. An example dofile with this modification is shown “[Modification of the Dofile and Procedure File for Boundary Scan](#)” on page 242.

EDT Pattern Generation Overview

After the EDT IP core is generated and embedded in the design, you use the TCD file to perform connectivity extraction and pattern generation.

The following sections discuss best practices and usage considerations for EDT pattern generation:

IJTAG Mapping	136
Scan Chain Handling	137

IJTAG Mapping

By default when using an EDT IP TCD file to configure EDT, Tessent Shell uses IJTAG to configure the EDT IP static signals such as `edt_bypass`. IJTAG mapping uses the IJTAG infrastructure to retarget setup values through any IJTAG network.

The EDT static signals are those which only need to be configured once at the start of the session, then held constant. In contrast, EDT signals such as `edt_update` change per pattern and consequently must be controlled directly from a port.

If you provide the top-level ICL and PDL (EDT core) files, the tool maps and configures the EDT IP during test setup. Using IJTAG provides the most automation, but is not mandatory if you setup the EDT IP manually for each mode. IJTAG also allows for mapping all the way to the top through the TAP controller and TDRs.

IJTAG mapping is used under the following circumstances:

- The ICL for the current design is available. This is checked by [R10](#) DRC rule.
- The ICL includes an ICL module matching the EDT IP instrument the tools is configuring. This is checked by [R11](#) DRC rule.
- The EDT IP instrument “setup” iProc is available. This is checked by [R13](#) DRC rule.

The ICL module name must match the instrument name stored in the TCD file.

You can disable IJTAG by issuing the following command in Tessent Shell:

```
set_procedure_retargeting_options -ijtag off
```

If you disable IJTAG-based mapping of static signals, then either those static signals need to be driven directly by ports, or you must provide a `test_setup` procedure that configures the static signals to the correct values. For more information, refer to the *Tessent IJTAG User’s Manual*.

Scan Chain Handling

The EDT pattern generation flow with a TCD file automates scan chain handling.

If you do not specify any EDT instrument instance scan chains, then the tool automatically adds all the required EDT instrument instance scan chains. In other words, you can avoid explicitly adding the compressed scan chains using the `add_scan_chains` command. In this case, Tessent Shell automatically adds the scan chain definitions for the EDT IP instances when the tool transitions from Setup to Analysis system mode.

In some cases, however, scan chains may need to be added during setup mode if there are other setup-mode commands that need to reference the scan chain by name. In this case, the tool automatically matches (binds) the scan chains with the EDT IP instance(s) to which they are connected, and, by extension, no longer adds scan chain definitions for EDT IP instances for which you have defined scan chains. You can specify scan chains manually, but you must specify all of the scan chains needed and add all of the chains for the EDT controller. See “Manual Scan Chain Definition” below.

Automated Scan Chain Definition

By default, you are not required to add scan chains. After you have loaded the EDT IP core-inserted design and the TCD file, Tessent Shell automatically adds scan chains based on the EDT IP configuration.

Manual Scan Chain Definition

You can manually provide internal scan chains. The tool detects which scan chains can be used for instrument binding by assuming the first N scan chains (number defined during IP creation) are correctly assigned to the EDT block. In contrast to the dofile-based flow, the automated scan chains binding does not require the chains to be in the correct order as the tool fixes the order once the proper binding is determined.

Top-Level Test Procedure File

At a minimum, a test procedure file that contains `load_unload` and `shift` procedures for the scan chains is needed. If top-level uncompressed scan chains are used, then you must define these using the `add_scan_groups` and `add_scan_chains` commands.

If there are no top-level chains, then you must set the test procedure file using with `set_procfile_name` command since no test procedure file has been specified with the `add_scan_group` command.

Core Instance Parameters

When using a TCD file in the EDT pattern generation context in Tessent Shell, you can define core instance parameter values to automatically configure the EDT logic. These parameters are a superset of the EDT IP setup PDL procedure and can be changed for an EDT IP instance after the hardware has been created. These parameters are used in EDT, LPCT, and OCC.

To modify these parameters, use the following option to the [add_core_instances](#) command:

```
add_core_instances -core core_name ... -parameter_values {parameter_list}
```

where the *parameter_list* can include parameters from Table 6-1.

Table 6-1. Core Instance Parameters and Values by Instrument

Instrument Type	Parameter	Value	Description
EDT	edt_bypass	on <u>off</u> The default is off.	Defines whether the EDT is used in bypass mode. See Compression Bypass Logic for more information on the parameter and bypass in general.
	edt_configuration	A string that is the name of the configuration specified during IP creation. When used with the DFTSpecification flow and the HighCompressionConfiguration wrapper, there are only two valid values: <u>low_compression</u> <u>high_compression</u> The default is low_compression	Defines which compression configuration is used by the EDT. See Dual Compression Configurations for more information on compression configurations.
	edt_low_power_shift_en	on off The default value is based on the EDT IP generation time user-defined power controller status.	Defines whether the EDT is used in low power shift mode. See Power Controller Logic for more information on edt_low_power_shift_en.
	edt_single_bypass_chain	on <u>off</u> The default is off.	Defines whether the EDT is used in single bypass mode. See Dual Bypass Configurations for more information on edt_single_bypass_chain.
	used_input_channels	integer The default value is the maximum available input channels.	Defines how many input channels should be used by the EDT. See the description for used_input_channels in the set_edt_options command arguments in the <i>Tessent Shell Reference Manual</i> .

Table 6-1. Core Instance Parameters and Values by Instrument (cont.)

Instrument Type	Parameter	Value	Description
OCC	capture_window_size <i>integer</i>	integer The default value is the maximum size supported by the OCC instance.	Specifies the maximum number of clock pulses during capture cycle. This value must not exceed the registers created during IP creation. By default, the tool creates OCC able to pulse up to four times between scan loads, but you may not need to use them all. For example you are generating transition patterns and have minimal non-scan logic such that you only need to pulse clocks twice. Set this parameter to 2. This cuts the number of OCC cells in half and therefore the number of bits that need to be encoded during pattern generation. When the OCC chains are driven by EDT, you free up encoding capacity to be used for the generated tests and may reduce pattern count.
	fast_capture_mode	on <u>off</u> The default is off.	Defines whether the fast capture clock is used during capture. See Clock Control Operation Modes in the <i>Tessent Scan and ATPG User's Manual</i> for information on this parameter.
	parent_mode	on <u>off</u> The default is off.	Defines whether the OCC is used in parent mode. When the default off is used, the OCC is used in standard mode. See The Parent OCC section in the <i>Tessent Scan and ATPG User's Manual</i> for more information on this parameter.

Table 6-1. Core Instance Parameters and Values by Instrument (cont.)

Instrument Type	Parameter	Value	Description
LPCT	reset_control	on <u>off</u> The default is off.	Controls the value being loaded into the LPCT Type-3 reset control cell. See Type 3 - LPCT Controller-Generated Scan Enable .
	scan_en_control	on <u>off</u> The default is off.	Controls the value being loaded into the LPCT Type-3 scan enable control cell. See Type 3 - LPCT Controller-Generated Scan Enable .

A given parameter is only available for an EDT IP instance if that EDT IP module was created with the capability set by the parameter. For example, an EDT IP instance will only have the edt_configuration parameter available if it was created to support dual EDT configurations.

You must specify these parameters before going to Analysis mode and generating any patterns.

Used Input Channels	142
Pattern Generation with Internal Chain Masking Hardware.....	145

Used Input Channels

You can configure the EDT IP to use a subset of the input channels. For example, you might generate the EDT IP to support some number of input channels, but then when the IP or core is embedded into the design, only a subset of the channels can be driven due to scan port limitations. This is especially useful for channel sharing.

To configure the input channels, you use the used_input_channels EDT parameter. This parameter allows you to indicate to the tool that the unused channels have been tied off. The used_input_channels EDT parameter specifies the number of input channels to the EDT IP that can be used during pattern generation. The remaining channels which must be data-only must be tied off to 0 by user-created hardware. Data-only input channels are channels that do not include any control data such as Xpress masking or low-power control bits. When the EDT IP is created with the [set_edt_options](#) -separate_control_data_channels on command or with the basic compactor, some or all of the channels will be data-only.

You can specify a value for this parameter using either of the following commands:

- [add_core_instances](#) — Used to define an instance of the EDT IP that has been instantiated in the design and needs to be used in the current mode.

- `set_core_instance_parameters` — Used to change parameters of a core instance that has already been defined using the `add_core_instances` command.

The following usage conditions apply:

- To use this switch, you must set the following in the IP Creation Phase:

`set_edt_options -separate_control_data_channel ON`

The only exception is if you are using the basic compactor and no low-power controller.

- Only data channels with highest channel indexes can become unused. For example, an EDT block has one control channel (`channel1`) and three data channels (`channel2`, `channel3`, `channel4`). If you want to only use 3 input channels, you can only tie `channel4` to “0”. If you want to only use 2 input channels, you can only tie both `channel3` and `channel4` to “0”. You must have at least one data channel.
- If the unused input channels are connected to top-level primary inputs, patterns at these pins must hold at “0”s. This can be accomplished by constraining the unused input channels using the `add_input_constraints` command:

`add_input_constraints XXX -C0`

or

`add_input_constraints XXX -C1 (if there is an inversion between the tied pin and the EDT channel input)`

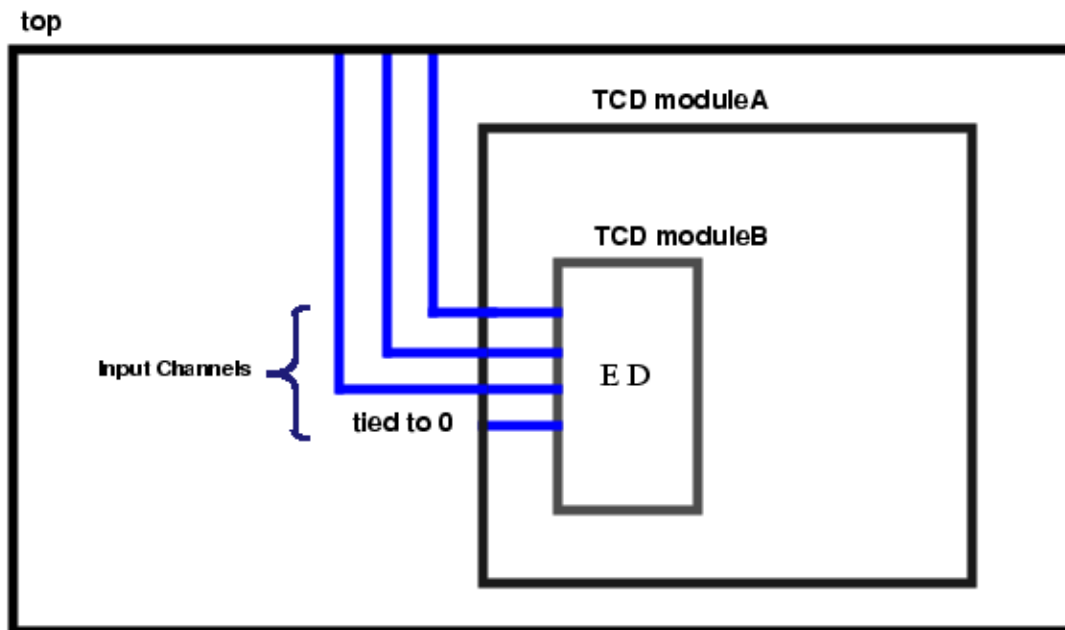
Otherwise, the tool issues DRC violations during system mode transition.

- If the unused input channels are driven by internal signals, these channels must be tied off to “0” to ensure that “0”s are injected into the decompressor from these unused channels.

Figure 6-2 shows an EDT IP logic block with four input channels nested within a higher-level block. In the EDT flow using TCD files, you can tie these used input channels to 0 (zero) and reduce the number of input channels.

Figure 6-2. Used Input Channels Example

`set_core_instance_parameters -modules moduleB -parameter_values {used_input_channels 3}`



Example 1

Report all of the EDT parameters and their values that were specified when the core instance was added:

SETUP> report_core_instance_parameters -mod piccpu_maxlen16_1_edt

Core instance 'core_1/piccpu_maxlen16_1_edt_i' (core
'piccpu_maxlen16_1_edt', instrument type 'edt')

Parameter Name	Parameter Value	Legal Override Values
edt_bypass	off	off, on
edt_configuration	low	high, low
edt_low_power_shift_en	on	off, on
used_input_channels	4	2..4

Example 2

Reports the parameters available for an EDT IP module, but not specific to how a given instance of this module was configured

SETUP> report_core_parameters -cores piccpu_maxlen16_1_edt

Core 'piccpu_maxlen16_1_edt' (instrument type 'edt')

Parameter Name	Default Value	Legal Values
edt_bypass	off	off, on
edt_configuration		high, low
edt_low_power_shift_en	on	off, on
used_input_channels	4	2..4

Pattern Generation with Internal Chain Masking Hardware

In this mode the masking hardware is provided within the Hybrid TK/LBIST controller.

The internal masking logic is capable of masking scan chains with constant unload value of “0” (zero). Consequently, there is no need for you to manually provide the scan chain definitions.

You must, however, enable the internal masking by setting the EDT IP parameter `tessent_chain_masking` to “on”. You must also specify which chains should be masked by using [add_chain_masks](#) command with `-instance` switch. Additionally, you must provide the block chain index of the chains and whether the chains should be traced by specifying `-used_chains` ON or `-used_chains` OFF to the [add_core_instances](#) command.

Regardless of the tracing mode, the specified chains are masked with a constant unload value of 0.

Example

```
add_core_instances -core piccpu_maxlen16_3 \
  -instances my_core/edt_logic_top_1 \
  -parameter_values {tessent_chain_masking on}

add_chain_masks -instances my_core/edt_logic_top_1 \
  -block_chain_index_list {1} -used_chains on
// chain is masked and tool will trace it

add_chain_masks -instances my_core/edt_logic_top_1 \
  -block_chain_index_list {2} -used_chains off
// chain is masked tool will not trace it
```

Updating Scan Pins for Test Pattern Generation

EDT Finder automatically finds EDT logic and updated scan pin information for test pattern generation. EDT Finder identifies the EDT logic contained in the gate-level netlist and updates the I/O pins associated with the scan chains.

Note

 EDT Finder is enabled by default ([set_edt_finder on](#)). If you have disabled EDT Finder, you will need to manually update the scan pin information that has changed since the EDT logic was generated.

Prerequisites

- Gate-level Verilog netlist or flat model containing EDT logic.

Note

 EDT Finder must be enabled before any internal scan chains are added and saved to the flat model. Otherwise, the flat model cannot be used with the EDT Finder in subsequent sessions.

Procedure

1. Invoke Tessent Shell. For example:

```
<Tessent_Tree_Path>/bin/tessent -shell
```

Tessent Shell invokes in setup mode.

2. Provide Tessent Shell commands. For example:

```
set_context patterns -scan read_verilog my_gate_scan.v read_cell_library \
my_lib.atpg set_current_design top
```

3. Set up for test pattern generation as needed. For more information, see “[Preparation for Test Pattern Generation](#)” on page 133.
4. Read the core-level TCD files using the [read_core_descriptions](#) command.
5. Identify the core instances using the [add_core_instances](#) command.
6. Specify compressed and uncompressed chains, if any.
7. Exit setup mode. For example:

```
set_system_mode analysis
```

The EDT logic and internal scan chain inputs are identified, scan chains are traced, and DRC is run.

8. Correct any DRC violations.

For information on DRCs related to the EDT Finder command, see [EDT Finder \(F Rules\)](#) in the *Tessent Shell Reference Manual*.

9. Report the EDT Finder results. For example:

```
report_edt_finder -decompressors
```

```
// id  #bits #inputs #chains EDT block  type
// -----
// 1    16    4      28    m1_28x16 active
// 2    16    4      4     m2_4x32 active
// 3    16    4     50    m3_50x187 active
// 4    10    1      8     m4_8x16 active
```

All active decompressors are reported. For more information on reporting EDT Finder results, see the [report_edt_finder](#) command.

Tip

 You can also use the [Test Structures Window](#) within DFTVisualizer to browse the EDT logic.

10. Generate and save test patterns. For more information, see “[Generating Patterns](#)” on page 154.

Examples

The following example demonstrates a modular design. After you have generated a TCD file for each of the cores in your design using the `write_core_description` command, you map the cores to the chip level using the core TCD files, add any additional scan logic, and finally generate patterns for the entire design.

```
# Set the proper context for core mapping and subsequent ATPG
set_context pattern -scan

# Read cell library (library file)
read_cell_library technology.tcelllib

# Read the top-level netlist and all core-level netlists
read_verilog generated_1_edt_top_gate.vg generated_2_edt_top_gate.vg \
    generated_top_edt_top_gate.vg

# Specify the top level of design for all subsequent commands
set_current_design

# Read all core description files
read_core_descriptions piccpu_1.tcd
read_core_descriptions piccpu_2.tcd
read_core_descriptions small_core.tcd

# Bind core descriptions to core instances
add_core_instances -instance core1_inst -core piccpu_1
add_core_instances -instance core2_inst -core piccpu_2
add_core_instances -instance core3_inst -core small_core

# Specify top-level compressed chains and EDT
dofile generated_top_edt.dofile

# Specify top-level uncompressed chains
add_scan_chains top_chain_1 grp1 top_scan_in_3 top_scan_out_3
add_scan_chains top_chain_2 grp1 top_scan_in_4 top_scan_out_4

# Report instance bindings
report_core_instances

# Change to analysis mode
set_system_mode analysis

# Create patterns
create_patterns

# Write patterns
write_patterns top_patts.stil -stil -replace

# Report procedures used to map the core to the top level (optional)
report_procedures
```

Verification of the EDT Logic

Two mechanisms are used to verify that the EDT logic works properly: design rules checking (DRC) and enhanced chain and EDT logic (chain+EDT logic) test.

Design Rules Checking (DRC)	149
EDT Logic and Chain Testing	149
Reducing Serial EDT Chain Test Simulation Runtime	152

Design Rules Checking (DRC)

Several K DRCs verify that the EDT logic operates correctly. F rules also verify the EDT logic (unless you have disabled EDT Finder).

The tool provides the most complete information about violations of these rules when you have preserved the EDT logic structure through synthesis. Following is a brief summary of just the K rules that verify operation of the EDT logic:

- **K19** — Simulates the decompressor netlist and performs diagnostic checks if a simulation-emulation mismatch occurs.
- **K20** — Identifies the number of pipeline stages within the compactors, based on simulation.
- **K22** — Simulates the compactor netlist and performs diagnostic checks if a simulation-emulation mismatch occurs.

For detailed descriptions of all of the EDT design rules (K and F rules) that are checked during DRC, refer to “[Design Rule Checking](#)” in the Tessent Shell Reference Manual.

EDT Logic and Chain Testing

In addition to performing DRC verification of the EDT logic, the tool saves, as part of the pattern set, an EDT logic and chain test. This test consists of several scan patterns that verify correct operation of the EDT logic and the scan chains when faults are added on the core or on the entire design. This test is necessary because the EDT logic is not the standard scan-based circuitry that traditional chain test patterns are designed for. The EDT logic and chain test helps in debugging simulation mismatches and guarantees very high test coverage of the EDT logic.

You can use the following equation to predict the number of additional chain test patterns the tool generates to test the EDT logic. (In this equation, *ceil* indicates the ceiling function that rounds a fraction to the next highest integer.) Note, this equation provides a lower bound; the actual number may be higher.

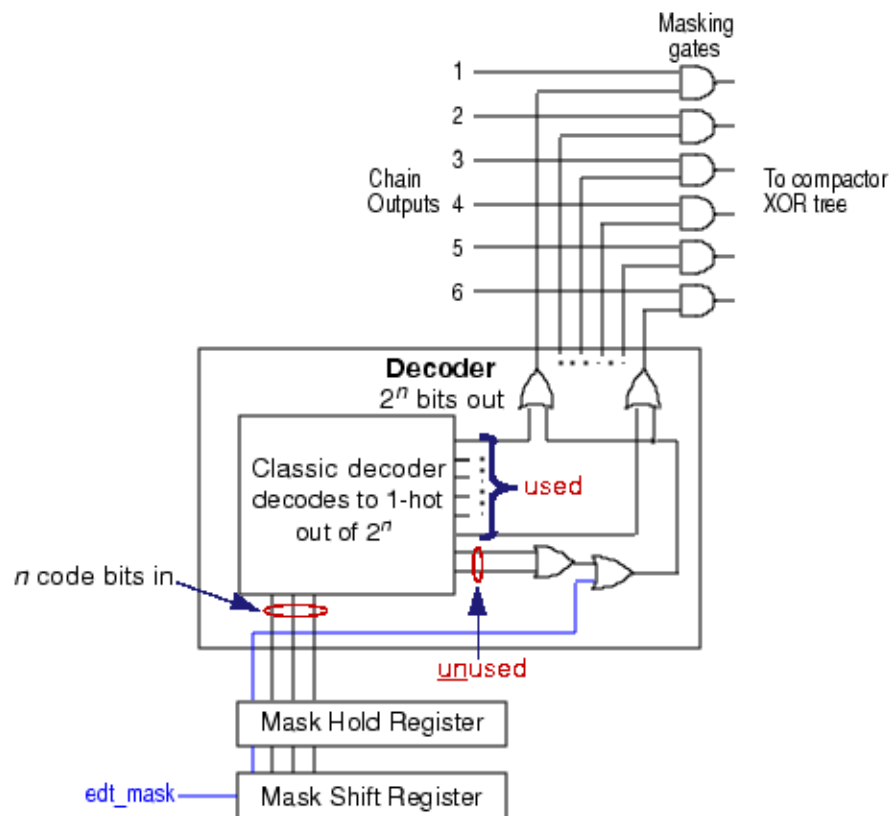
$$\text{Minimum number of chain test patterns} = 1 + 2^{\text{ceil}(\log_2(\text{number of chains}))}$$

How it Works

To better understand the enhanced chain test, you need to understand how the masking logic in the compactor works. Included in every EDT pattern are mask codes that are uncompressed and shifted into a mask shift register as the pattern data is shifted into the scan chains. Once a pattern's mask codes are in the mask shift register, they are parallel loaded into a hold register that places the bit values on the inputs to a decoder. [Figure 6-3](#) shows a conceptual view of the decoder circuitry for a six chains/one channel configuration.

The decoder basically has a classic binary decoder within it and some OR gates. The classic decoder decodes its n inputs to one-hot out of $2n$ outputs. The $2n$ outputs fall into one of two groups: the “used” group or the “unused” group. (Unless the number of scan chains exactly equals $2n$, there will always be one or more unused outputs.)

Figure 6-3. Example Decoder Circuitry for Six Scan Chains and One Channel



Each output in the used group is AND'd with one scan chain output. For a masked pattern, the decoder typically places a high on one of the used outputs, enabling one AND gate to pass its chain's output for observation.

The decoder also has a single bit control input provided by the edt_mask signal. Unused outputs of the classic decoder are OR'd together and the result is OR'd with this control bit. If any of the OR'd signals is high, the output of the OR function is high and indicates the pattern is a non-

masking pattern. This OR output is OR'd with each *used* output, so that, for a non-masking pattern, all the AND gates will pass their chain's outputs for observation.

The code scanned into the mask shift register for each channel of a multiple channel design determines the chain(s) observed for each channel. If the code scanned in for a channel is a non-masking code, that channel's chains are all observed. If a channel's code is a masking code, usually only one of the chains for that channel is observed. The chain test essentially tests for all possible codes plus the `edt_mask` control bit.

The example shows a typical EDT logic and chain test for a 10X configuration with 22 patterns. These 22 patterns can be composed of the following masking and non-masking patterns: ten masking patterns, six masking patterns that observe all chains due to unused codes, two non-masking patterns that observe all chains, and four XOR patterns that observe a set of chains as listed below:

- Patterns 0 through 9 — Masking patterns. Control bit is set to 1. Only one chain will be observed per channel, due to “used” codes for each channel.
- Patterns 10 through 15 — Masking patterns. Control bit is set to 1. All chains will be observed due to “unused” codes.
- Patterns 16 and 17 — Non-masking patterns that observe all chains. Control bit is set to 0.
- Pattern 18 through 21 — XOR masking patterns that observe a set of chains. Control bit is set to 0.

You can clearly see this in the ASCII patterns. For a masking pattern, if the scanned in code corresponding to a channel is a “used” code, only one of that channel's chains will have binary expected values. All other chains in that channel will have X expected values. To see an example of a masked ASCII pattern, refer to “[Understanding Scan Chain Masking in the Compactor](#)” on page 282.

Note

When the tool generates the EDT chain patterns, in addition to non-masking and masking (1-hot and xor) patterns, the tool also generates chain test patterns to test all unused decoder values when all chain patterns are selected. Because of this, the total sum of all generated patterns may be greater than the sum of non-masking and masking patterns.

So, depending on which chain test is failing, it is possible to deduce which chain might be causing problems. In the preceding example, if a failure occurred for any of the patterns 0 through 9, you could immediately map it back to the failing chain and, based on the cycle information, to a failing cell. If only a non-masking pattern or a masking pattern with “unused” codes failed, then mapping is a little bit tricky. But in this case, most likely masking patterns would fail as well.

Optionally, you can shift in custom chain sequences to the current chain test by specifying the “set_chain_test -sequence” command. For more information, see the [set_chain_test](#) command in the *Tessent Shell Reference Manual*.

Controlling the edt_update Signal for Load_Unload

When the tool generates chain test patterns, it adds an extra cycle to the end of the shift cycles before the load_unload procedure when neither the edt_clock or system clock is pulsed. This “dead” cycle guarantees the edt_update signal goes high during load_unload regardless of how you choose to control the edt_update signal.

For example, if you do not explicitly force edt_update high during load_unload because it has a C1 pin constraint, the STIL pattern file keeps edt_update low during load_unload unless the extra cycle is specified.

You can choose to remove this cycle from the pattern file using set_chain_test -suppress_capture_cycle on. However, you should use CAUTION when using this command option. If you remove the extra cycle and do not explicitly force edt_update high in the load_unload procedure, the pattern file will be incorrect and edt_update will be low during load_unload.

Coverage for EDT Logic and Chain Test

Experiments performed by Mentor Graphics engineers using sequential fault simulation demonstrate that test coverage for the EDT logic with the enhanced chain test is nearly 100% when the EDT logic does not include bypass logic (essentially multiplexers that bypass the decompressor and compactor). Test coverage declines to just above 94% when the EDT logic includes bypass logic. This is because the EDT chain test does not test the bypass mode input of each bypass multiplexer (edt_bypass is kept constant in EDT mode during the chain test).

Note



99+% coverage can be achieved in any event by including a bypass mode chain test (the standard chain test).

The size of the chain test pattern set depends on the configuration of the EDT logic and the specific design. Typically, about 18 chain test patterns are required when you approach 10X compression.

Reducing Serial EDT Chain Test Simulation Runtime

You can simulate a small subset of the chain test patterns serially.

If you are not using and enabling the low-power decompressor, you can simply save one non-masking pattern as shown here:

```
set_chain_test -type nomask  
write_patterns pattern_filename -pattern_sets chain -serial -end 0
```

If you are using a low-power decompressor, it is safest to run all non-masking patterns (which is still a small subset of all chain patterns) as shown here:

```
set_chain_test -type nomask  
write_patterns pattern_filename -pattern_sets chain -serial
```

For more information, see the [set_chain_test](#) command in the *Tessent Shell Reference Manual*.

Adding Faults on the Core Only is Recommended

When you generate patterns, if you add faults on the entire design, the tool tries to target faults in the EDT logic. Traditional scan patterns can probably detect most EDT logic faults. But because EDT logic fault detection cannot be serially simulated, the tool conservatively does not give credit for them. This results in a relatively high number of undetected faults in the EDT logic being included in the calculation of test coverage. You, therefore, see a lower reported test coverage than is actually the case.

The EDT logic and chain test targets faults in the EDT logic. The tool always performs the this test, so adding faults on the entire design is not necessary in order to get EDT logic test coverage. To avoid false test coverage reports, the best practice is to add faults on the core only.

Test Pattern Generation

The compression technology supports all of the pattern functionality in uncompressed ATPG, with the exception of MacroTest and random patterns. This includes combinational, clock-sequential (including patterns with multiple scan loads), and RAM sequential patterns. It also includes all the fault types.

See “[EDT Aborted Fault Analysis](#)” on page 288 for additional considerations.

Generating Patterns.....	154
Compression Optimization	155
Saving of the Patterns	156

Generating Patterns

Use this procedure to load the design information including the TCD and generate patterns.

Prerequisites

The design containing the EDT IP core must be synthesized.

Procedure

1. Invoke Tessent Shell from a Linux shell using the following syntax:

```
% tessent -shell
```

The tool’s system mode defaults to Setup mode after invocation.

2. With the [set_context](#) command, change the context to test pattern generation (patterns -scan) as follows:

```
SETUP> set_context patterns -scan
```

3. Read the design containing the EDT IP using the [read_verilog](#) command. For example:

```
SETUP> read_verilog created_cpu_edt.v
```

4. Read the library using the [read_cell_library](#) command. For example:

```
SETUP> read_cell_library adk.tcelllib
```

5. Designate the current design using the [set_current_design](#) command. For example:

```
SETUP> set_current_design
```

6. Read the TCD file for EDT IP using the [read_core_descriptions](#) command. For example:

```
SETUP> read_core_description created_cpu_edt.tcd
```

7. Define parameter values to automatically configure the EDT logic using the `add_core_instances` command. For example:

```
SETUP> add_core_instances -core cpu_edt -modules cpu_edt \
    -parameter_values {edt_bypass off}
```

8. Add top-level clocks driving the scan changes using the `add_clocks` command.
9. Provide the top-level test procedure file using the `set_procfile_name` command. For example:

```
SETUP> set_procfile_name created_cpu_edt.testproc
```

Refer to “[Scan Chain Handling](#)” on page 137 for more information.

10. Change the system mode to Analysis using the `set_system_mode` command as follows:

```
SETUP> set_system_mode analysis
```

The mode change runs the design rule checks.

11. Create the EDT patterns using the `create_patterns` command as follows:

```
ANALYSIS> create_patterns
```

12. Optionally write out the core description corresponding to the current chip level using the `write_core_description` command. For example:

```
ANALYSIS> write_core_description cpu_core_final.tcd -replace
```

13. Save the patterns using the `write_patterns` command:

```
ANALYSIS> write_patterns core_level_patterns.v -verilog
```

14. Exit Tessent Shell.

```
ANALYSIS> exit
```

Results

The tool writes the patterns to the file you specified with the `write_patterns` command.

Compression Optimization

You can do a number of things to ensure maximum compression: limit observable Xs and use dynamic compaction.

Using Dynamic Compaction

You should use dynamic compaction during ATPG if your primary objective is a compact pattern set. Dynamic compaction helps achieve a significantly more compact pattern set, which is the ultimate goal of using EDT. Because the two compression methods are largely independent of each other, you can use dynamic compaction and EDT concurrently. Try to use

[create_patterns](#) for the smallest pattern set, as it executes a good ATPG compression flow that is optimal for most situations.

Note



For circuits where dynamic compaction is very time-consuming, you may prefer to generate patterns without dynamic compaction. The test set that is generated is not the most compact, but it is typically more compact than the test set generated by traditional ATPG with dynamic compaction. And it is usually generated in much less time.

Saving of the Patterns

Save EDT test patterns in the same way you do in uncompressed ATPG.

For complete information about saving patterns, refer to the [write_patterns](#) command in the *Tessent Shell Reference Manual*.

Serial Patterns

One important restriction on EDT serial patterns is that the patterns must not be reordered after they are written. Because the padding data for the shorter scan chains is derived from the scan-in data of the next pattern, reordering the patterns may invalidate the computed scan-out data. For more detailed information on pattern reordering, refer to “[About Reordering Patterns](#)” on page 287.

Parallel Patterns

Because parallel simulation patterns force and observe the uncompressed data directly on the scan cells, they have to be written by the EDT technology which understands and emulates the EDT logic.

Some ASIC vendors write out parallel WGL patterns, and then convert them to parallel simulation patterns using their own tools. This is not possible with default EDT patterns, as they provide only scan *channel* data, not scan chain data. To convert these patterns to parallel simulation patterns, a tool must understand and emulate the EDT logic.

There is an optional switch, `-Edt_internal`, you can use with the `write_patterns` command to write parallel EDT patterns with respect to the core scan chains. You can write these patterns in tester or ASCII format and use them to produce parallel simulation patterns as described in the next section.

EDT Internal Patterns

The optional `-Edt_internal` switch to the `write_patterns` command enables you to save parallel patterns as EDT internal patterns. These are tester or ASCII formatted EDT patterns that the tool writes with respect to the core scan chains instead of with respect to the top-level scan channel PIs and POs. These patterns contain the core scan chain force and observe data with the

exception that they have X expected values for cells which would not be observed on the output of the spatial compactor due to X blocking or scan chain masking. X blocking and scan chain masking are explained in “[Understanding Scan Chain Masking in the Compactor](#)” on page 282. Also, of course, the scan chain force and observe points are internal nodes, not top-level PIs and POs. Because they provide data with respect to the core scan chains, EDT internal patterns can be converted into parallel simulation patterns.

Note

The number of scan chain inputs and outputs in EDT internal patterns corresponds to the number of scan chains in the design core, *not* the number of top-level scan channels. Also, the apparent length of the chains, as measured by the number of shifts required to load each pattern, will be shorter because the extra shift cycles that occur in normal EDT patterns for the EDT circuitry are unnecessary.

Post-Processing of EDT Patterns

Sometimes there is a need to process patterns after they are written to a file. Post-processing might be needed, for example, to control on-chip phase-locked loops (PLLs). Scan pattern post-processing requires access to the uncompressed patterns. The tool, however, writes patterns in EDT-compressed format, at which point it is too late to make any changes. Traditional post-processing, therefore, is not feasible with EDT patterns.

Note

An exception is parallel tester or ASCII patterns you write out as EDT *internal* patterns. Using your own post-processing tools, you can convert these patterns into parallel simulation patterns. See “[Parallel Patterns](#)” on page 156 for more information.

The compressed ATPG engine must set or constrain any scan cells prior to compressing the pattern. So it is essential you identify the type of post-processing you typically need and then translate it into functionality you can specify in the tool as part of your setup for pattern generation. The compressed ATPG engine can then include it when generating EDT patterns.

Simulation of the Generated Test Patterns

You can verify the test patterns using parallel and serial test benches the same way you would for normal scan and ATPG. When you simulate serial simulation patterns, you can verify the correctness of the captured data for the pattern, the chain integrity, and the EDT logic (both the decompressor and the compactor blocks). When simulation mismatches occur, you can still use the parallel test bench to debug mismatches that occur during capture. You can use the serial test bench to debug mismatches related to scan chain integrity and the EDT logic.

To verify that the test patterns and the EDT circuitry operate correctly, you need to serially simulate the test patterns with full timing. Typically, you would simulate all patterns in parallel

and a sample of the patterns serially. Only the serial patterns exercise the EDT circuitry. Because simulating patterns serially takes a long time for loading and unloading the scan chains, be sure to use the `?Sample` switch when you write patterns for serial simulation. This is true even though serial patterns simulate faster with EDT than with traditional ATPG due to the fewer number of shift cycles needed for the shorter internal scan chains. “[Design Simulation with Timing](#)” in the *Tessent Scan and ATPG User’s Manual* provides useful background information on the use of this switch. Refer to the [write_patterns](#) command description in the *Tessent Shell Reference Manual* for usage information.

Note



You must use Tessent Shell to generate parallel simulation patterns. You cannot use a third party tool to convert parallel WGL patterns to the required format, as you can for traditional ATPG. This is because parallel simulation patterns for EDT are uncompressed versions of the compressed EDT patterns applied by the tester to the scan channel inputs. They also contain EDT-specific modifications to emulate the effect of the compactor.

HDL Simulation Setup

First, set up a work directory for ModelSim.

```
../modeltech/<platform>/vlib work
```

Then, compile the simulation library, the scan-inserted netlist, and the simulation test patterns. Notice that both the parallel and serial patterns are compiled:

```
../modeltech/<platform>/vlog my_parallel_pat.v my_serial_pat.v \  
../created_edt_top_gate.v -y my_sim_lib
```

This will compile the netlist, all necessary library parts, and both the serial and parallel patterns. Later, if you need to recompile just the patterns, you can use the following command:

```
../modeltech/<platform>/vlog pat_p_edt.v pat_s_edt.v
```

Running the Simulation

After you have compiled the netlist and the patterns, you can simulate the patterns using the following commands:

```
../modeltech/<platform>/vsim edt_top_pat_p_edt_v_ctl -do "run -all" \  
-l sim_p_edt.log ?c  
../modeltech/<platform>/vsim edt_top_pat_s_edt_v_ctl -do "run -all" \  
-l sim_s_edt.log -c
```

The “-c” runs the ModelSim simulator in non-GUI mode.

Chapter 7

Modular Compressed ATPG

Modular Compressed ATPG is the process used to integrate compression into the block-level design flow. Integrating compression at the block-level is similar to integrating compression at the top-level, except you create/insert EDT logic into each design block and then, integrate the blocks into a top-level design and generate test patterns.

Note

 In this chapter, an EDT block refers to a design block that contains a full complement of EDT logic controlling all the scan chains associated with the block.

The modular flow includes one or more of the top-level compressed pattern flows. For information on these top-level flows, see, “[The Compressed Pattern Flows](#)” on page 39 of this manual.

The Modular Flow	159
Understanding Modular Compressed ATPG	161
Development of a Block-Level Compression Strategy	163
Balancing Scan Chains Between Blocks	164
Sharing Input Scan Channels on Identical EDT Blocks	165
Channel Sharing for Non-Identical EDT Blocks	167
Mixing Channel Sharing for Non-Identical EDT Blocks and Channel Broadcasting for Identical EDT Blocks	175
Generating Modular EDT Logic for a Fully Integrated Design	177
Estimating Test Coverage/Pattern Count for EDT Blocks	178
Legacy ATPG Flow	179

The Modular Flow

The modular flow has five distinct stages and requirements for them.

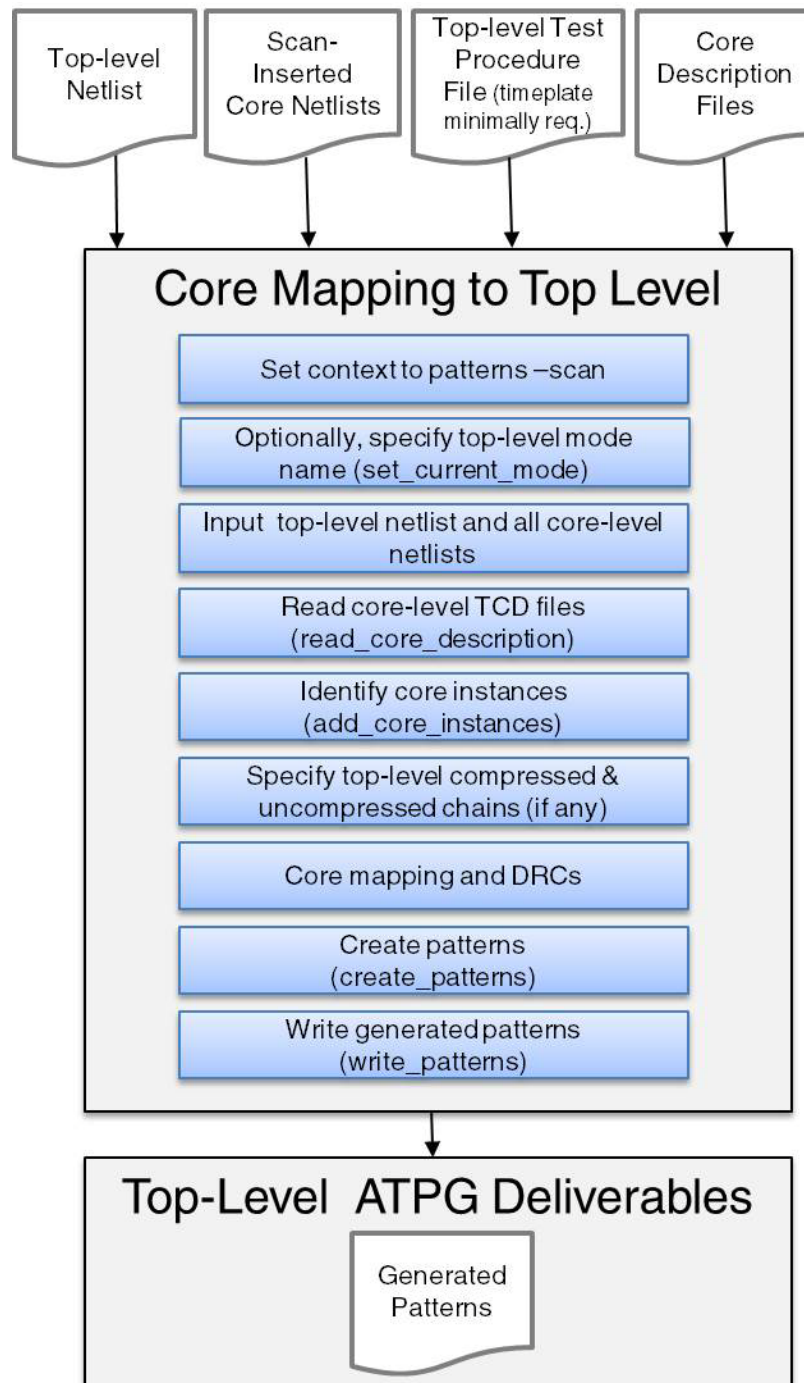
Requirements

The requirements for the modular flow are:

- Block-level compression strategy
- Gate-level or RTL netlist for each block in the design
- Tessent Scan or other scan insertion tool (optional)

- Tessent cell library
- Design Compiler or other synthesis tool
- ModelSim or other timing simulator

Modular Flow Diagram



Flow Stage Descriptions

Table 7-1. Modular Flow Stage Descriptions

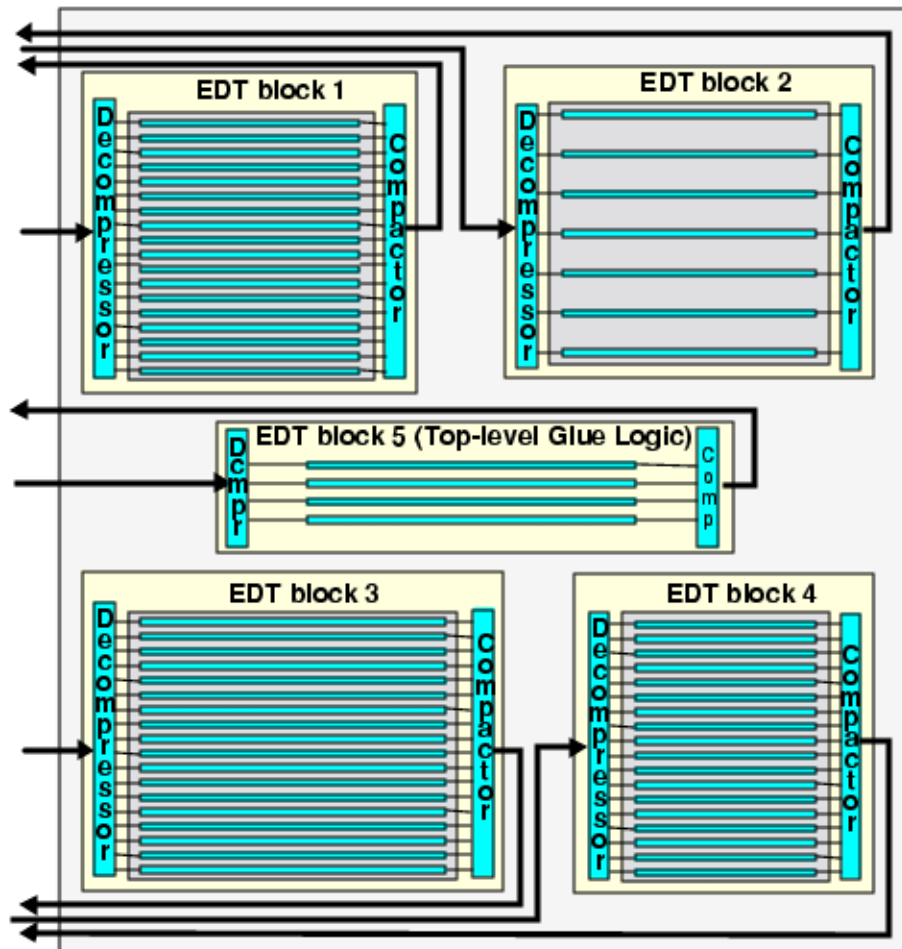
Stage	Description
Integrate EDT logic into each Design Block	EDT logic can be integrated into each design block using any of the top-level methods described in this document. For more information, see the following sections of this document: • “Integrating Compression at the RTL Stage” on page 291 • “The Compressed Pattern Flows” on page 39 The first step to using compression in your design flow is developing a compression strategy. For more information, see “Generation of Top-Level Test Patterns” on page 179 “Development of a Block-Level Compression Strategy” on page 163.
Generate Test Patterns	Test patterns are set up and generated using the top-level netlist, test procedure file, and dofile. For more information, see “Generation of Top-Level Test Patterns” on page 179. You should also create bypass test patterns for the top-level netlist at this point. For more information, see “Compression Bypass Logic” on page 230.

Understanding Modular Compressed ATPG

The EDT logic inserted in a design block controls all scan chains within the block.

[Figure 7-1](#) shows an example of a modular design with four EDT blocks. Each EDT block consists of a design block with integrated EDT logic. The design also contains a separate EDT block for the top-level glue logic. The top-level glue logic can be tested with EDT logic as shown or with bypass logic as described in [“Compression Bypass Logic”](#) on page 230.

Figure 7-1. Modular Design with Five EDT blocks



Each EDT block has a discrete netlist, dofile, and test procedure file that are integrated together to form top-level files for test pattern generation.

Development of a Block-Level Compression Strategy

You can create and insert EDT logic into design blocks with any of the methods outlined in this manual. You can also mix and match methods between blocks.

Reference the following rules and guidelines while developing your compression strategy for the modular flow:

- **Scan Chain Lengths Should Be Balanced** — Balanced scan chains yield optimal compression. Plan the lengths of scan chains inside all blocks in advance so that top-level (inter-block) scan chain lengths are relatively equal. See “[Balancing Scan Chains Between Blocks](#)” on page 164.
- **EDT Logic Names Must Be Unique** — When multiple EDT blocks are integrated into a top-level netlist, all of the EDT logic file names and internal module/instance names must be unique. See “[Creation of EDT Logic Files](#)” on page 96.
- **Each EDT Block Must Have a Discrete Set of Scan Chains** — Scan chains cannot be shared between blocks.
- **Uncompressed Scan Chains Must be Connected to Top-Level Pins** — Uncompressed scan chains are scan chains not driven by or observed through the EDT logic. Uncompressed scan chains are supported if the inputs and outputs are connected directly to top-level pins. Uncompressed scan chains can also share top-level pins. See “[Inclusion of Uncompressed Scan Chains](#)” on page 54.
- **Only Certain Control Pins can be Shared with Functional Pins** — These pins can be shared within the same EDT block. See “[Functional/EDT Pin Sharing](#)” on page 85.
- **Control Signals can be Shared by EDT Blocks** — Control signals such as `edt_update`, `edt_clock`, `edt_reset`, `scan_enable` and `test_en` may be shared between EDT blocks; for example, the `edt_update` signals from different blocks could be connected to the same top-level pin.
- **Scan Channels Must Have Dedicated Top-Level pins** — Only input scan channels between identical EDT blocks can share top-level pins. See “[Sharing Input Scan Channels on Identical EDT Blocks](#)” on page 165.
- **Block-Level Signals Must be Connected in the Top-Level Netlist** — This includes connecting EDT logic signals to I/O pads and inserting any multiplexers needed for channel output signals shared with functional signals.
- **EDT Logic Must be Synthesized and Verified for Each Block** — See “[Synthesizing the EDT Logic](#)” on page 121 and “[Generating and Verifying Test Patterns](#)” on page 133.

Balancing Scan Chains Between Blocks	164
Sharing Input Scan Channels on Identical EDT Blocks	165

Channel Sharing for Non-Identical EDT Blocks	167
Overview of Channel Sharing Functionality	167
Compression Analysis	169
EDT IP Creation With Separate Control and Data Input Channels	170
Rules for Connecting Input Channels from Cores to Top	173
Channel Sharing Reporting.....	174
Limitations	174
Mixing Channel Sharing for Non-Identical EDT Blocks and Channel Broadcasting for Identical EDT Blocks.....	175
Generating Modular EDT Logic for a Fully Integrated Design	177
Estimating Test Coverage/Pattern Count for EDT Blocks	178
Legacy ATPG Flow	179

Balancing Scan Chains Between Blocks

Design blocks may contain a large amount of hardware with many internal blocks and many scan chains, so scan chain balance is very important for generating efficient test patterns. You should carefully plan the lengths of scan chains inside each design block so that all blocks have approximately the same scan chain length.

You should target the same compression for every block and apportion available tester channels according to the relative share of the overall design gate count contained in each block. Use the following two equations to calculate balanced scan chain lengths across multiple blocks:

$$\text{Scan Chain Length} \approx \frac{\# \text{ of Scan Cells in block}}{(\# \text{ of Channels for block}) \times (\text{Chain-to-channel ratio})}$$

$$\# \text{ of Channels for block} \approx \frac{\# \text{ of Scan Cells in block}}{\# \text{ of Scan Cells in chip}} \times \# \text{ of top-level Channels}$$

Tip

 Since different designers may perform scan insertion for different design blocks, it is important to work together to select a scan chain length target that works for all blocks.

Related Topics

[Variance in the Number of Scan Chains](#)

Sharing Input Scan Channels on Identical EDT Blocks

You can set up identical EDT blocks to share input scan channels and top-level pins when integrating modular design blocks into a top-level netlist.

When EDT blocks share input scan channels, test patterns are broadcast via shared top-level pins to all the identical EDT blocks simultaneously. This functionality reduces top-level pin requirements and increases the compression ratio for the input side of the EDT logic.

The Compression Analyzer in Tessent TestKompress fully supports channel broadcasting and can be used to assess the effectiveness of channel broadcasting in combination with other channel configurations. You run compression analysis with channel broadcasting at the top level of a design that has multiple identical EDT blocks, using the [analyze_compression](#) command.

The following switch has a special meaning for channel broadcasting:

- `-Broadcast_all_channels_to_identical_blocks [block1 block2 ...]` — defines the channel broadcasting group.

Where `[block1 block2 ...]` must be pre-existing identical EDT blocks.

Requirements

- EDT blocks must be identical as follows:
 - Number of input channels and output channels must match
 - Input, output, and compactor pipeline stages must match
 - Order of scan chains and the number of scan cells in each must match
 - Input channel/top-level pin inversions must match
- All corresponding input channels on identical EDT blocks must be shared in the corresponding order. For example the following channels can be shared:
 - input channel 1 of block1
 - input channel 1 of block2
 - input channel 1 of block3 and so on

Top-Level Dofile Modifications

You need to set up the input channel sharing when the block-level dofiles are integrated into a top-level dofile. Depending on the application, you can set up the input channel sharing in one of two ways:

- **Make top-level pins equivalent**

Use this method when a top-level pin exists for each input channel by defining the pins for the corresponding input channels on each block as equivalent. For example:

```
add_edt_blocks core1
set_edt_pins input 1 core1_edt_channels_in1
set_edt_pins input 2 core1_edt_channels_in2
add_edt_blocks core2
set_edt_pins input 1 core2_edt_channels_in1
set_edt_pins input 2 core2_edt_channels_in2
add_input_constraints -eq core1_edt_channels_in1 \
                      core2_edt_channels_in1
add_input_constraints -eq core1_edt_channels_in2 \
                      core2_edt_channels_in2
```

- **Physically share top-level pins**

Use this method when top-level pins need to be shared between input channels by explicitly specifying the top-level pins to be same. For example:

```
add_edt_blocks core1
set_edt_pins input 1 edt_channels_in1
set_edt_pins input 2 edt_channels_in2
add_edt_blocks core2
set_edt_pins input 1 edt_channels_in1
set_edt_pins input 2 edt_channels_in2
```

During DRC, the blocks that share input channels are reported. As long as the EDT blocks are identical and the channel sharing is set up properly, EDT DRCs should pass.

Use the [report_edt_configurations -All](#) command to display information on the EDT blocks set up to share input channels.

Channel Sharing for Non-Identical EDT Blocks

This section contains the following information:

Overview of Channel Sharing Functionality	167
Compression Analysis	169
EDT IP Creation With Separate Control and Data Input Channels	170
Rules for Connecting Input Channels from Cores to Top	173
Channel Sharing Reporting	174
Limitations	174

Overview of Channel Sharing Functionality

Identical EDT blocks have exactly the same scan chain and EDT structures. Therefore, you can generate scan patterns for one block and broadcast the pattern stimuli to the inputs of all identical blocks.

Tessent tools support pattern stimuli broadcast to identical blocks as described in “[Sharing Input Scan Channels on Identical EDT Blocks](#)” on page 165.

Non-identical EDT blocks cannot share all input channels. Tessent tools also provide support for using the same channel to drive multiple non-identical EDT blocks.

Channel sharing between non-identical EDT blocks enables you to improve data and time compression results for most designs that use a modular EDT approach. Specifically, the following scenarios can gain greater benefits from this feature:

- Designs with a limited number of top-level ports available for scan channel I/O
- Designs with a large pattern increase when comparing a single EDT block at the top level with multiple EDT blocks across the design
- Designs with a large number of EDT aborted faults due to high chain-to-channel ratios within individual EDT blocks

Support for channel sharing between non-identical EDT blocks does not have any impact on the output channels. The EDT hardware created for channel sharing uses existing functionality that uses dedicated (not shared) output channels.

Channel sharing between non-identical EDT blocks is supported by the compression analysis, and the standard EDT reporting capability.

You implement channel sharing across non-identical EDT blocks by separating the control and data input channels when creating the EDT IP. This allows the data channels to be shared across multiple non-identical blocks.

The default EDT hardware creates control data registers for Xpress compactor masking bits and low-power control bits (if they exist) in front of each EDT input channel. The diagram in [Figure 7-2](#) shows how test data (D) loaded into an EDT block is followed by control data for compactor masking (C) and low-power (LP) data.

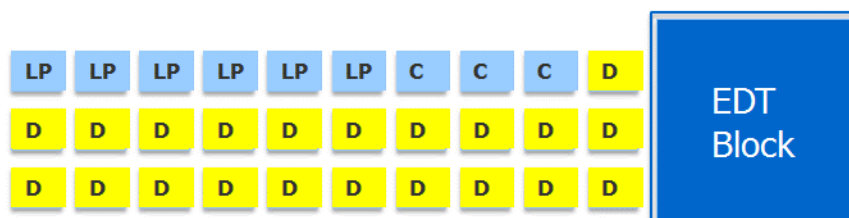
Note that the low-power registers always exist when you use the “set_edt_power_controller Shift Enabled | Disabled” command and switches. There is no low-power register created if you use the “set_edt_power_controller Shift None” command and switches.

Figure 7-2. Non-Separated Control Data Input Channels



You can create EDT hardware that separates control input channels from data input channels. The resulting hardware includes several input channels that only load scan test data into each EDT block, as illustrated in [Figure 7-3](#). By separating the control data that is specific to each block into dedicated input channels, the scan test data (D) input channels can be shared across multiple non-identical blocks. (With this option, an EDT block can no longer have only one input channel; it must have at least one control channel and one data channel.)

Figure 7-3. Separated Control Data Input Channels



Because the broadcast is only allowed to go to multiple non-control input channels, normally at least one dedicated control channel for each EDT block is still required, except for the special case in which an EDT block has a basic compactor but does not have a low-power controller.

In order to get the most benefit from input channel sharing, the number of input channels in each core should be maximized so that you share as many input channels as possible among multiple non-identical cores and take full advantage of all available top-level data input channels.

Channel sharing also results in a reduction in overall shift cycles. As shown in [Figure 7-3](#) on page 168, by moving the control data to a dedicated channel that is loaded with scan data, no extra shift cycles are added only for the purpose of masking or low-power control bits. This provides an additional increase in overall compression of test data and application time.

Also, as shown in [Figure 7-3](#) on page 168, the input control channels can also load scan test data (D) if the number of control bits (LPs and Cs) is smaller than the length of the longest scan chain. Similarly, if a design requires many control bits, the EDT block may require more than one control channel. The tool determines the appropriate number of control channels based on the number of masking and low-power control bits and the length of the longest scan chain.

Compression Analysis

The Compression Analyzer in Tessent TestKompress fully supports channel sharing and can be used to assess the effectiveness of channel sharing in combination with other channel configurations.

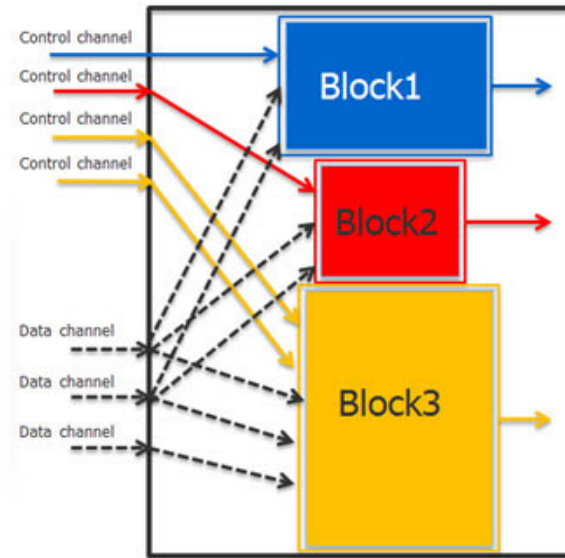
You run compression analysis with channel sharing at the top level of a design using the [analyze_compression](#) command. The following switches have special meaning for channel sharing:

- **-INPut_channels** — Defines the total number of control and data channels for each block.
- **-SHARE_data_channels[block1 block2 ...]** — Defines the channel sharing group.
- **-DATA_and_control_channels[int]** — Defines the total number of input channels, across all blocks, that can be shared among that group.

Optionally, you can allow this command to calculate the required number of input channels by using this command without specifying the total number of input channels.

For example, you can emulate the displayed configuration shown in [Figure 7-4](#) using the `analyze_compression` command with the `-input_channels` and the `-share_data_channels` `-data_and_control_channel` switches.

Figure 7-4. Channel Sharing Example



```
analyze_compression -edt_block \  
Block1 -input_channels 3 -output_channels 1 \  
-edt_block Block2 -input_channels 3 -output_channels 1 \  
-edt_block Block3 -input_channels 5 -output_channels 1 \  
-share_data_channels Block1 Block2 Block3 \  
-data_and_control_channels 7
```

All other `analyze_compression` command options are also supported, and you can use them to run various experiments.

EDT IP Creation With Separate Control and Data Input Channels

The only change you need to make to the EDT IP creation step is to separate the control and data input channels.

You can create the hardware that supports separate control and data input channels by using the “[set_edt_options -separate_control_data_channels ON](#)” command in setup mode as shown here:

```
SETUP> set_edt_options -separate_control_data_channels ON
```

By default, the `-separate_control_data_channels` is set to OFF. When enabled, the “`-separate_control_data_channels ON`” switch also modifies the generated EDT setup dofile to include information about the separate control and data channels.

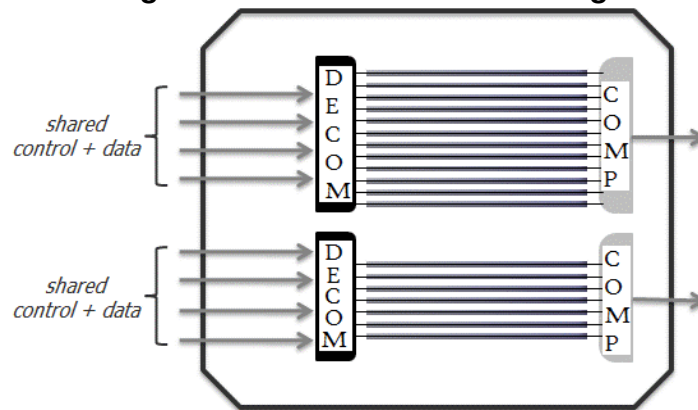
Typically, each EDT block needs to have at least one dedicated channel that cannot be shared, while all others can be shared. The dofiles generated in the EDT IP creation step will contain all of the information needed to fully describe the EDT hardware at each block. You do not need to make any changes to the pattern generation step.

For an EDT block with an Xpress compactor and/or low-power controller, you must have at least one dedicated control channel, and none of its control channels can be shared with other EDT blocks. The only exception to this requirement is an EDT block that has a basic compactor and does not have a low-power controller; in this case, the block is not required to have a control channel. The broadcast to shared channels is only allowed for data channels with no control bits.

Maximizing Block-Level Channels

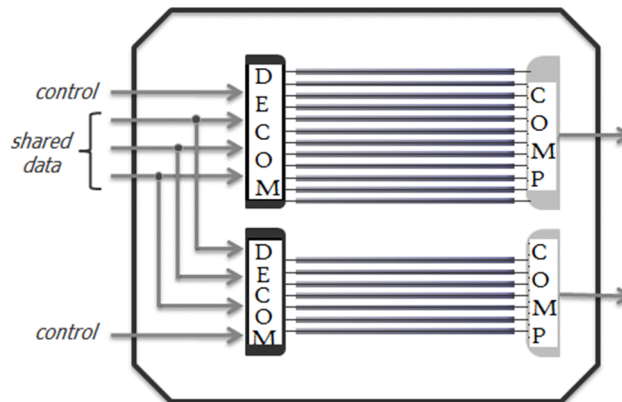
In order to maximize the benefits of channel sharing, you should maximize the number of input channels at the core level to take full advantage of all available top-level input channels. For example, in [Figure 7-5](#), the design has two EDT blocks, each block has four input channels with mixed control and data on each channel. Clearly, without channel sharing, the design requires eight input channels at the top-level.

Figure 7-5. Non-Channel Sharing



With channel sharing implemented, as shown in [Figure 7-6](#), the design requires only five input channels at the top-level, and each block still has four input channels (three data channels are shared by each core). This implementation mitigates the pin-limitation problem at the top-level, while maintaining the same bandwidth at the core-level.

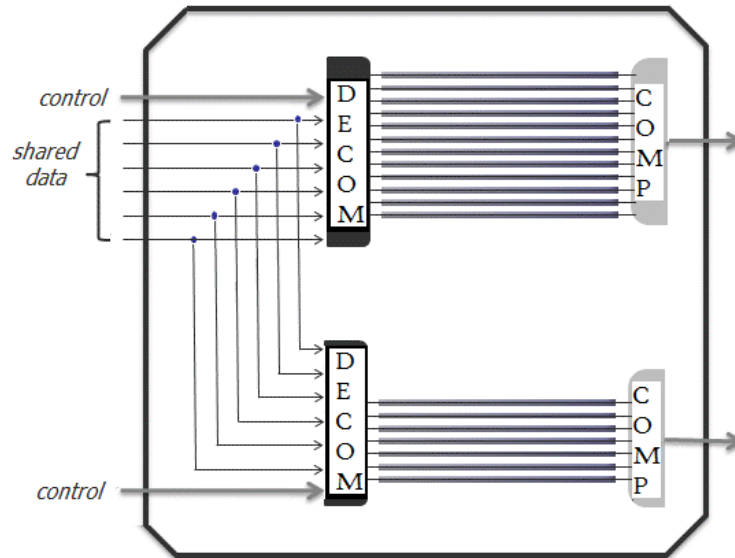
Figure 7-6. Channel Sharing Scenario 1



You can optimize input channel sharing by maintaining the same input pin count at the top-level, while increasing the number of input channels in each core to maximize the number of input channels shared among multiple non-identical cores.

In this channel sharing configuration, the design still has eight input channels at the top-level, as shown in [Figure 7-7](#), but each block now has seven input channels (six data channels are shared by each core); this can improve their bandwidth for each core, and thereby improve encoding efficiency and reduce the number of patterns.

Figure 7-7. Channel Sharing Scenario 2



Design Rule Checks for Channel Sharing

The K15 DRC verifies that all scan channels and control pins have the proper top-level pins. Each scan input channel requires a dedicated top-level pin, except for blocks (identical and non-identical) set up for input channel sharing. The K15 allows one top-level input port to broadcast to multiple EDT blocks.

For more information on the specific checks performed, see “[K15](#)” in the *Tessent Shell Reference Manual*.

Separate Control and Data Channels and Dual Compression Configuration

When you use the “[set_edt_options](#) -separate_control_data_channels on” command, the tool attempts to apply the setting to both high and low compression configurations.

If the high compression configuration does not have enough input channels to allow separate control and data channels, but the low compression configuration does, the tool will separate the

control and data channels for the low compression configuration, but not for the high compression configuration. In this case, the tool issues a warning. For example:

```
add_edt_block block1
set_edt_options -separate_control_data_channels on
add_edt_configurations high_comp
set_edt_options -channels 1
add_edt_configurations low_comp
set_edt_options -channels 5

// Warning: In configuration 'high_comp', the number of input channels (1)
//           is smaller than the required number of control and data
//           channels (1 control + 1 data).
//           The '-separate_control_data_channels' option has been
//           set to 'off' for this configuration.
//           Configuration 'low_comp' has separate control and data
//           input channels.
```

If both the high and low compression configurations lack sufficient input channels to allow for separated control and data channels, the tool will not separate the channels for either of the configurations. For example:

```
add_edt_block block1
set_edt_options -separate_control_data_channels on
add_edt_configurations high_comp
set_edt_options -channels 1
add_edt_configurations low_comp
set_edt_options -channels 2

// Warning: In configurations 'high_comp' and 'low_comp', of
//           EDT block 'block1' the numbers of input channels
//           (1 and 2 respectively) are smaller than the required
//           number of control and data channels (2 control + 1 data).
//           The '-separate_control_data_channels' option has been
//           set to 'off' for both configurations.
```

Rules for Connecting Input Channels from Cores to Top

For the non-core mapping for ATPG flow, during EDT IP creation, the control and data channels for each core are connected to the top-level ports with the command “set_edt_pins input_channel index [pin_name]”. For the shared data channels, you use the same port names.

In the core mapping for ATPG flow, you need to integrate the cores to the top level. In either case, data input channels should be connected based on the following rules:

- Data input channels should be shared so that top-level input channels are broadcast to EDT blocks.
- The same top-level input channel should not drive data into multiple channels on the same EDT block.

- Each top-level data input channel is NOT required to drive data into every EDT block. Different blocks may have a different number of data input channels.

Channel sharing has no impact on how output channels are connected.

Channel Sharing Reporting

The input channels to EDT decompressor can be divided into two categories: control channels that deliver control data, and data channels that deliver tests. (Note that the control channel may also be used to deliver a test if it is shorter than the data channels.) When broadcasting to non-identical blocks, the data channels can share the same inputs, but control channels cannot.

When channel sharing is used, it is desirable to report the channel sharing information. The tool provides the [report_edt_configurations](#) and the “[report_edt_pins](#) -Group_by_pin_name” commands to allow you to report channel sharing information. You can use these commands in the EDT IP Creation phase and also in the Pattern Generation phase when in either insertion or analysis mode.

Refer to the examples on the [report_edt_configurations](#) command reference page for an example of how channel sharing information is reported.

Limitations

The channel sharing functionality has the following limitations:

- Channel sharing is not allowed between EDT input channels and uncompressed chains inputs.
- Mapping compressed EDT patterns to bypass patterns is not allowed. That is, the `write_patterns -edt_bypass` and `-edt_single_bypass_chain` options are disabled for channel sharing.
- All core-level shared channels driven by the same top-level channel port must have the same number of external pipelining stages and the same input pin inversions.
- The non-overlapping clock setting should be the same for all blocks that are sharing input pins. That is, the “`set_edt_options -pulse_edt_before_shift_clocks`” option must be set the same for all blocks that are sharing input pins.
- When generating bypass uncompressed patterns, the generated patterns mimic Illinois scan patterns, since the existing hardware will be used for bypass patterns, which may cause coverage drop compared to the normal bypass patterns.

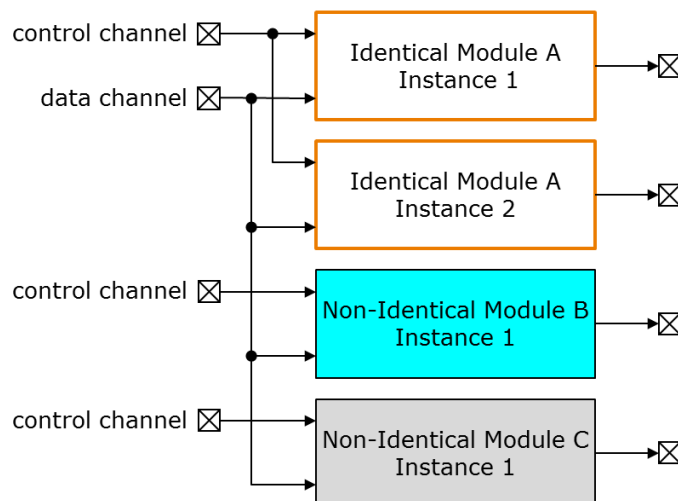
Mixing Channel Sharing for Non-Identical EDT Blocks and Channel Broadcasting for Identical EDT Blocks

You can mix channel sharing and channel broadcasting. The only sharing restriction is that each non-identical block must have its own dedicated control channel(s). The following sections present examples of different configurations of channel sharing and channel broadcasting between non-identical and identical EDT block.

Case 1: Identical Blocks Share Input Channels and Non-Identical Blocks Have Dedicated Control Channels

In the figure below, two identical blocks (instance 1 and instance 2 of Module A) share all input channels, and non-identical blocks (instance 1 of Module B and instance 1 of Module C) have their own dedicated control channels, but can share data channels with all other blocks. In this configuration, ATPG can run on the top-level of the design.

Figure 7-8. Case 1



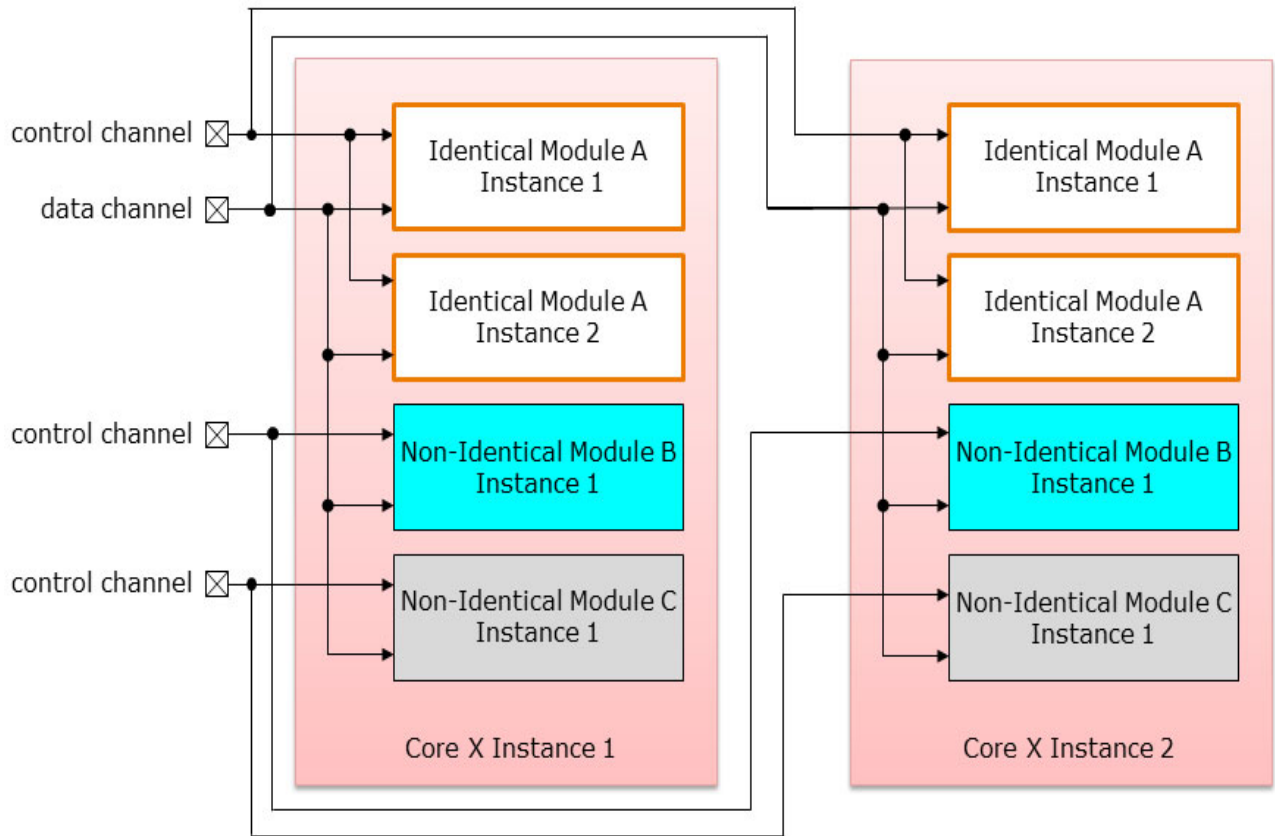
Case 2: Multiple Non-Identical and/or Identical EDT Blocks Inside One or More Identical Core Instances

It is common to have multiple non-identical and/or identical EDT blocks inside a core instance. You can use channel sharing and channel broadcasting inside this core instance to optimize access to the blocks in it. At the next level up, you may have multiple instances of those same cores and may want to broadcast channels to those identical core instances.

The following figure illustrates the general case of channel sharing across non-identical blocks and channel broadcasting between identical blocks in two identical instances of Core X. You may only have either channel sharing or channel broadcasting within a single core instance.

The tool supports both top-level ATPG and pattern retargeting in this case. However, if you are doing pattern retargeting, you must generate patterns for one core instance and broadcast those patterns to the multiple identical core instances.

Figure 7-9. Case 2

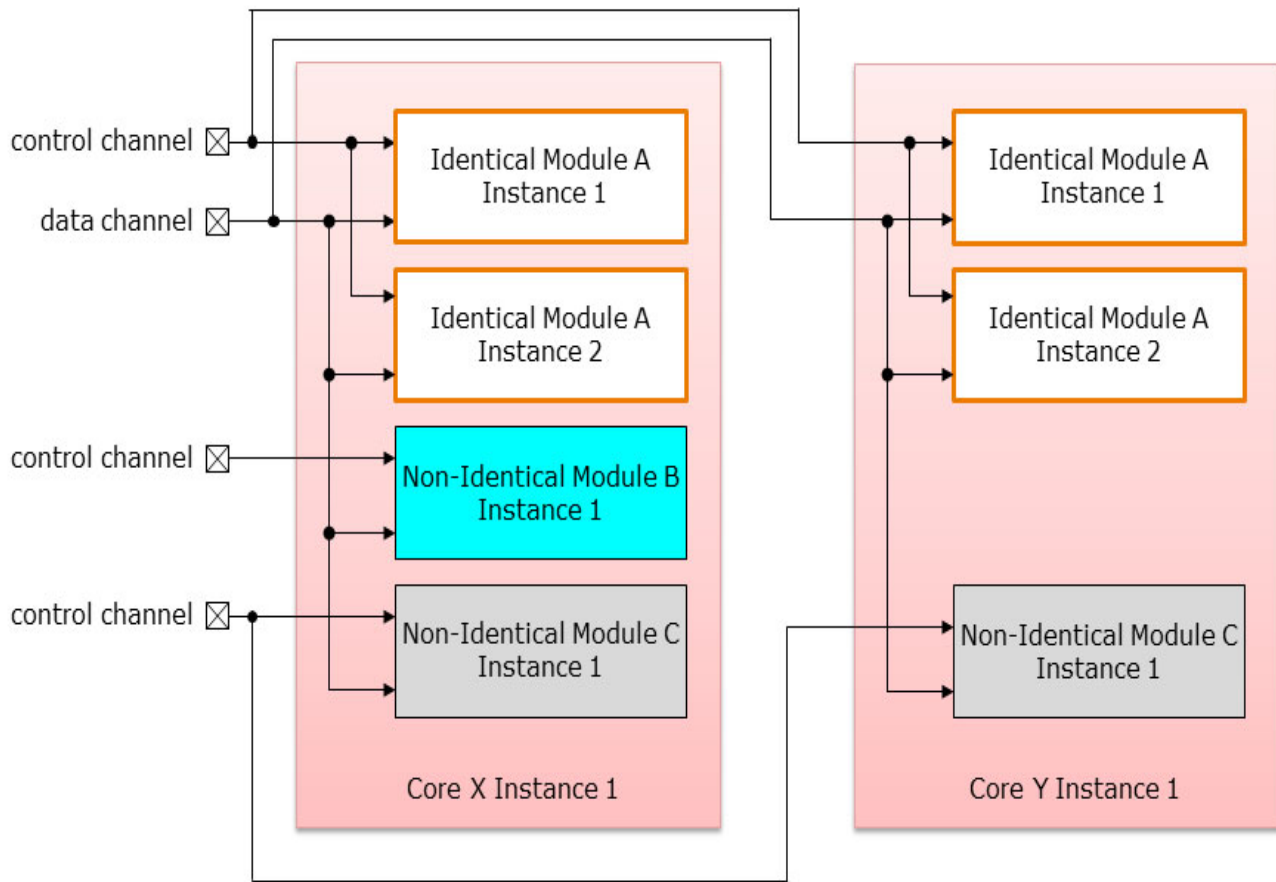


Case 3: Multiple Non-Identical and/or Identical EDT Blocks Inside Non-Identical Core Instances

The previous figure illustrates channel sharing across non-identical blocks and channel broadcasting between identical blocks in two *identical* instances of Core X.

Channel sharing and broadcasting are also allowed across non-identical cores as illustrated in the following figure. Core X and Core Y are non-identical cores because an instance of Module B is included in Core X but not included in Core Y. In this case, the tool still supports ATPG at the top-level. However, pattern retargeting is not allowed because it only supports channel broadcasting to identical core instances as enforced by the R3 DRC.

Figure 7-10. Case 3



Generating Modular EDT Logic for a Fully Integrated Design

Use this procedure to simultaneously generate modular EDT logic for all blocks within a fully integrated design. The resulting EDT logic can be set up as multiple instances within the design. If the integrated design shares top-level channels or requires any form of test scheduling, you must generate modular EDT logic one block at a time.

The files generated by this procedure support the same capabilities as the block by block modular flow.

Prerequisites

- The integrated design must be complete and fully functional.
- Each block must have dedicated input and output channels.

Procedure

1. Add each EDT block, one at a time, using the [add_edt_blocks](#) command.
2. Once an EDT block is added, set up the EDT logic for it with a [set_edt_options](#) command. The `set_edt_options` command only applies to the current EDT block. EDT control signals can be shared among blocks.
3. Once all the design blocks are added and set up, enter analysis mode. For more information, see the [set_system_mode](#) command.
4. Enter a [write_edt_files](#) command. A composite set of files is created including an RTL file, a synthesis script, a dofile/testproc file, and a bypass dofile/testproc file. All block-level EDT pins are automatically connected to the top level.
5. Use this composite set of files to synthesize EDT logic and generate test patterns.

Estimating Test Coverage/Pattern Count for EDT Blocks

After you create EDT logic for a block, you should use this procedure to get a more realistic coverage estimate before synthesis.

See “[Analyzing Compression](#)” on page 64.

Test coverage reported may be higher than when the EDT block is embedded in the design because the tool has direct access to the block-level inputs and outputs at this point.

Procedure

1. Constrain all functional inputs to X. For example:

```
add_input_constraints my_func_in -cx
```

Where the functional input *my_func_in* is constrained to X.

2. Mask all functional outputs. For example:

```
add_output_masks my_func_out1 my_func_out2
```

Where the two primary outputs *my_func_out1* and *my_func_out2* are masked.

Note

 Constraining inputs to X and masking the outputs produces very conservative estimates that negatively affect compression because all inputs become X sources when the CX constraints are added to the pins.

Note

 Because final test patterns are generated at the top-level of the design and are affected by all cores, the final test coverage and pattern count may vary.

Legacy ATPG Flow

This section describes the legacy functionality that enables you to integrate EDT blocks into the top level and generate top-level test patterns for them. This methodology requires that you manually generate chip-level test procedures.

You can use the Core Mapping for ATPG functionality that replaces this functionality, and automatically generates chip-level test procedures for you. For complete information, see “[Core Mapping for ATPG Process Overview](#)” in the *Tessent Scan and ATPG User’s Manual*.

Note

 If you are using the `set_edt_mapping` command in your dofiles, you should use this legacy functionality. The `set_edt_mapping` command and the EDT Mapping functionality have been superseded by the Core Mapping for ATPG functionality.

Generation of Top-Level Test Patterns

Generating test patterns for the top-level of a modular design is similar to creating test patterns in the standard flow except that you set one block up at a time.

Note

 To generate top-level patterns, you must have a top-level design netlist, dofile and test procedure file.

You use the following commands to generate test patterns for the top-level of a modular design:

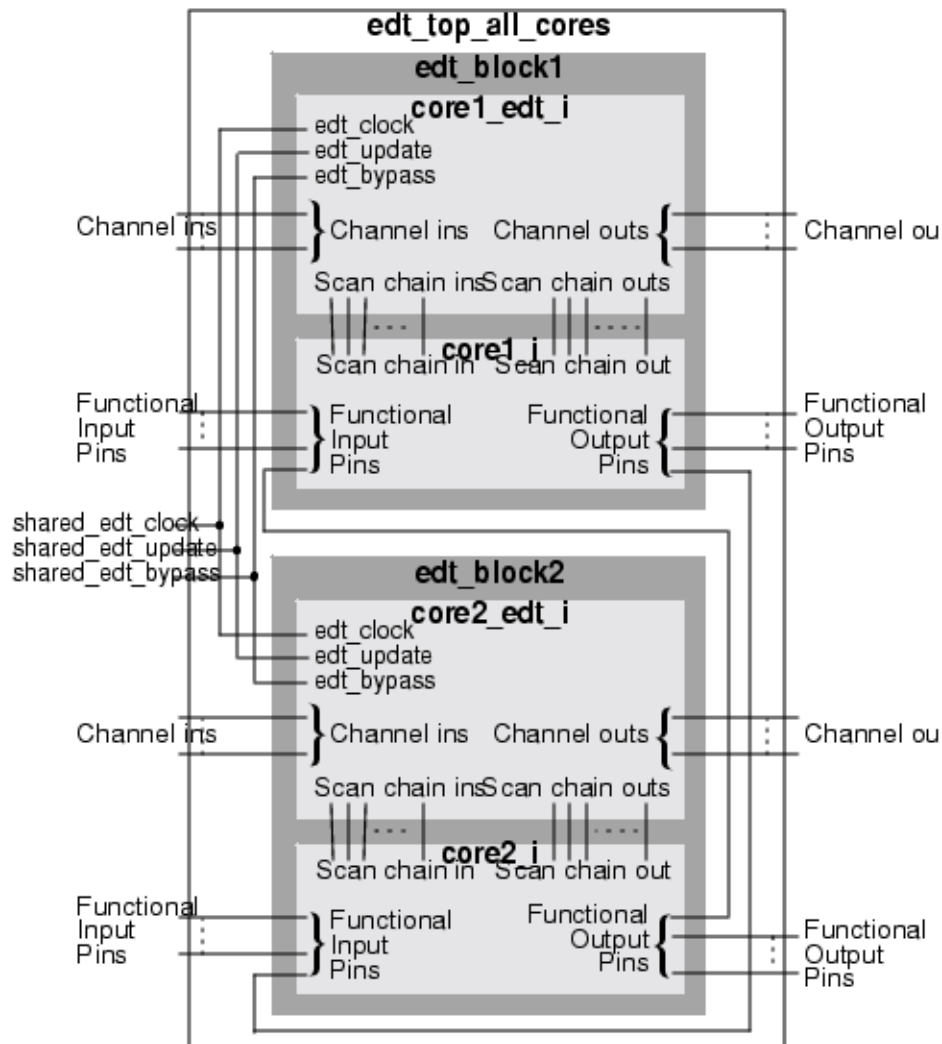
- [set_current_edt_block](#)— Applies EDT-specific commands and the `add_scan_chains` command to a particular EDT block. Restricting commands in this way enables you to re-specify the characteristics of an individual block without affecting other parts of the design.
- [report_edt_blocks](#)— Reports on EDT blocks currently defined in Tessent Shell memory.
- [delete_edt_blocks](#)— Deletes EDT blocks from Tessent Shell memory.

A few reporting commands also operate on the current EDT block by default, but provide an `-All_blocks` switch that enables you to report on the entire design. All other commands (`set_system_mode`, `create_patterns` and `report_statistics` for example) operate only on the entire design.

Example

This example demonstrates the commands used to integrate EDT blocks and generate test patterns. As shown in [Figure 7-11](#), EDT control signals are shared at the top level; each EDT block is created with the EDT logic and the scan-inserted core inside of a wrapper.

Figure 7-11. Netlist with Two Cores Sharing EDT Control Signals



1. Invoke Tessent Shell, set the context, and read in the design and library.
2. Perform necessary setup and then define scan chains, clocks and EDT logic for the first block. For example:

```
// Perform setup.
set_current_design edt_block1
...

// Define scan chains, clocks, and EDT hardware.
add_scan_groups grp1 group1.testproc
add_scan_chains chain1 grp1 edt_si1 edt_so1
add_scan_chains chain2 grp1 edt_si2 edt_so2
...
add_clocks clk1 0
set_edt_options -channels 6
set_system_mode analysis
```

3. Create EDT logic with unique module names based on the core module name for the first block. For example:

```
// Create EDT hardware with unique module names.  
write_edt_files created1 -replace
```

4. Delete the design using the `delete_design` command.
5. Return to setup mode using the `set_system_mode` command.
6. Read in the second block and repeat steps 2 and 3.
7. Using the DC script output during the EDT logic creation, synthesize the EDT logic for each block.
8. Verify that the EDT logic is instantiated properly by generating and simulating test patterns for each of the resultant gate-level netlists. This is done using the test bench created during test pattern generation and a timing-based simulator.
9. Verify that the block-level scan chains are balanced.
10. Create the top-level netlist, dofile, and test procedure files. The following example shows the top-level dofile. For more information, see [Generation of Top-Level Test Patterns](#).

Commands and options specific to *modular* compressed ATPG are shown in bold font.

```
// Define the top-level test procedure file to be used by all
// blocks.
add_scan_groups grp1 top_level.testproc

// Define top-level clocks and pin constraints here.
add_clocks...
add_read_controls...
add_write_controls...
add_input_constraints...
...

// Activate automatic mapping of commands from the block-level
// dofiles.
set_edt_mapping on

// Define the block tag (this is an arbitrary name) for an EDT block
// and automatically set it as the current EDT block.
add_edt_blocks cpu1

// Define the block by executing the commands in its block-level
// dofile.
dofile cpu1_edt.dofile

// Repeat the preceding procedure for another block.
add_edt_blocks cpu2
dofile cpu2_edt.dofile

// Once all EDT blocks are defined, create_patterns that use all the
// blocks simultaneously and generate patterns that target faults in
// the entire design.

// Flatten the design, run DRCs.
set_system_mode analysis

// Verify the EDT configuration.
report_edt_configurations -all_blocks

// Generate patterns.
create_patterns

// Create reports.
report_statistics
report_scan_volume

write_patterns...
exit
```

Modular Flow Command Reference

[Modular Compressed ATPG Command Summary](#) describes commands used for the modular design flow.

Table 7-2. Modular Compressed ATPG Command Summary

Command	Description
add_edt_blocks	Creates a name identifier for an EDT block instantiated in a netlist.

Table 7-2. Modular Compressed ATPG Command Summary (cont.)

Command	Description
delete_edt_blocks	Removes the specified EDT block(s) from the internal database.
report_edt_blocks	Displays current user-defined EDT block names.
report_edt_configurations	Displays the configuration of the EDT logic.
report_edt_instances	Displays the instance pathnames of the top-level EDT logic, decompressor, and compactor.
set_current_edt_block	Directs the tool to apply subsequent commands only to a particular EDT block, not globally.
set_edt_instances	Specifies the instance name or instance pathname of the design block that contains the EDT logic for DRC.
set_edt_mapping	Enables the automatic mapping necessary for block-level dofiles to be reused for top-level pattern creation.
write_design	If the design has been modified after executing the write_edt_files , you must update the netlist using this command.
write_edt_files	Writes all the EDT logic files required to implement the EDT technology in a design.

Chapter 8

Compressed ATPG Advanced Features

Additional advanced features are available for compressed ATPG.

Low-Power Test	187
Low-Power Shift	187
Setting Up Low-Power Test	192
Low Pin Count Test Controller	195
LPCT Controller Decision Tree	196
LPCT Limitations	208
LPCT Controller Types	210
LPCT Configuration Examples	222
Compression Bypass Logic	230
Structure of the Bypass Logic	230
Generating EDT Logic When Bypass Logic is Defined in the Netlist	231
Dual Bypass Configurations	233
Generation of Identical EDT and Bypass Test Patterns	234
Use of Bypass Patterns in Uncompressed ATPG	235
Creating Bypass Test Patterns in Uncompressed ATPG	238
Uncompressed ATPG (External Flow) and Boundary Scan	240
Boundary Scan Coexisting with EDT Logic	240
Drive Compressed ATPG with the TAP Controller	245
Use of Pipeline Stages in the Compactor	245
Use of Pipeline Stages Between Pads and Channel Inputs or Outputs	247
Channel Output Pipelining	247
Channel Input Pipelining	248
Clocks for Channel Input Pipeline Stages	249
Clocks for Channel Output Pipeline Stages	249
Input Channel Pipelines Must Hold Their Value During Capture	250
DRC for Channel Input Pipelining	251
DRC for Channel Output Pipelining	251
Input/Output Pipeline Examples	251
Change Edge Behavior in Bypass and EDT Modes	252
Understanding Lockup Cells	254
Lockup Cell Insertion	254
Lockup Cell Analysis For Bypass Lockup Cells Not Included as Part of the EDT Chains	256
Lockup Cell Analysis For Bypass Lockup Cells Included as Part of the EDT Chains ...	264

Lockups Between Channel Outputs and Output Pipeline Stages	272
Compression Performance Evaluation	274
Establishing a Point of Reference	275
Performance Measurement	276
Performance Improvement	277
Understanding Compactor Options.....	279
Understanding Scan Chain Masking in the Compactor	282
Fault Aliasing	285
About Reordering Patterns.....	287
Handling of Last Patterns.....	287
EDT Aborted Fault Analysis	288

Low-Power Test

Compressed ATPG with EDT can be configured to use low power during capture and/or shift cycles. When configured for low power, both EDT mode and bypass modes are affected.

A *low-power shift* application is based on the fact that test patterns typically contain only a small fraction of test-specific bits and the remaining scan cells or *don't care bits* are randomly filled with 0s and 1s; so, there are only a few scan chains with specified bits. In a low-power application, scan chains without any specified bits are filled with a constant value (0) to minimize needless switching as the test patterns are shifted through the core. For more information, see “[Low-Power Shift](#).”

A *low-power capture* application is based on the existing clock gaters in a design. In this case, clock gaters controlling untargeted portions of the design are turned off, while clock gaters controlling targeted portions are turned on. Power is controlled most effectively in designs that employ clock gaters, especially multiple levels of clock gaters (hierarchy), to control a majority of the state elements. Configuring low-power capture affects only the test patterns and is enabled with the [set_power_control](#) command during ATPG.

Note

 Low-power constraints are directly related to the number of test patterns generated in a low-power application. For example, using stricter low-power constraints results in more test patterns.

Low-Power Shift	187
Setting Up Low-Power Test	192

Low-Power Shift

Low-power shift is when you configure the low-power scheme to control the switching activity during “shift” to reduce power consumption. Setting up low-power shift includes two phases.

1. **Inserting power controller logic** — The power controller logic is configured/inserted during EDT logic creation based on the `-MIN_Switching_threshold_percentage value` specified with the [set_edt_power_controller](#) Shift command. This *value* must fall into one of the three threshold ranges described in “[Low-Power Shift and Switching Thresholds](#)” on page 188.

For example: To enforce a 20% switching threshold for shift, (assume a worse case switching activity of 50% for scan chains driven by the decompressor), the power controller is configured to drive up to 40% of the scan chains as shown here:

$$\begin{aligned} 20\% &= 50\% \text{ (max \% scan chains to switch of total scan chains)} \\ 20\% &= 50\% (40\%) \end{aligned}$$

The remaining scan chains (minimum of 60%) are loaded with a constant zero (0) value. So, in a case in which you have 300 scan chains, the maximum percentage of scan chains that will switch is 120, which is 40% of 300.

For more information, see “[Power Controller Logic](#)” on page 190 and “[Low-Power Shift and Switching Thresholds](#)” on page 188.

2. **Creating low-power test patterns** — When test patterns are generated, you must enable the power controller and specify the low-power switching threshold used during scan chain shifting with the [set_power_control](#) and [set_edt_power_controller](#) Shift commands. The specified switching threshold should not exceed the power controller hardware capabilities; out-of-range thresholds are supported but will generate a warning.

For example, if you configure the power controller hardware for a minimum switching threshold of 20%, you cannot set the test patterns to use a switching threshold of less than 12% or more than 24% as described in “[Low-Power Shift and Switching Thresholds](#)” on page 188.

In EDT bypass mode, the EDT logic and power controller are bypassed, and the low-power test patterns use a repeat-fill heuristic to load constant values into the *don't care bits* as they are shifted through the core. The repeat-fill heuristic minimizes needless transitions during bypass testing. This feature is only available in uncompressed ATPG or in the bypass mode of compressed ATPG.

Low-Power Shift and Switching Thresholds

The configuration/capability of the power controller hardware is determined by the -MIN_Switching_threshold_percentage value specified with the [set_edt_power_controller](#) command during EDT logic creation.

The *switching threshold percentage* is a percentage of the overall scan chain switching during shift. The *minimum switching threshold percentage* then represents the minimum switching threshold the power controller hardware can accommodate in a low-power application and determines the switching threshold percentage that can be used for test pattern generation.

You can use the following three threshold ranges to set up a low-power shift application. The bias value is set based on the threshold range you specify.

- < 12% (bias 2)
- >= 12% to < 25% (bias 1)
- >= 25% (bias 0)

Note



The term bias refers to “biased signal probability,” with a higher bias corresponding to an increase in the size of the power controller hardware.

If you specify a `-MIN_Switching_threshold_percentage` *value* that falls within one of these ranges, the tool generates a low-power controller that can generate shift patterns with low-power switching thresholds of the upper and lower bounds of the range. For example, if you specify a minimum threshold of 14, a low-power controller is generated that is capable of generating shift patterns with a low-power switching threshold of 12 to 24.

Both switching thresholds, the one for the power controller hardware and the one for low-power test patterns, must fall into the same switching threshold range. Low-power applications where power controller and test pattern thresholds fall in different ranges are not supported and may result in a higher test pattern count and decreased shift power control.

Note

 If reaching the specified threshold causes a drop in test coverage, the threshold is violated to maintain coverage. You can use the `set_power_control -rejection_threshold` switch to specify a hard limit on the switching activity and disregard the test coverage impact.

Pattern Generation and Switching Thresholds

During pattern generation, you use the `set_power_control` command to (1) enable the low-power logic and (2) set the low-power switching threshold to be used during scan chain shifting. The switching threshold you specify cannot exceed the power controller hardware capabilities.

The switching thresholds for both the power controller hardware and the low-power test patterns must fall into the same switching threshold range. A warning message is issued when a mismatch occurs between the software switching threshold and the power controller hardware threshold. The following example is an example warning message that reports a mismatch between the switching percentage threshold specified for shift and that specified for pattern generation:

```
// command: set_power_control shift on -switching_threshold_percentage 7
// Warning: Specified software switching threshold [7] is not consistent
// with the switching threshold used to generate the shift power control
// hardware [30] in block odd.
// The software and hardware thresholds should be in the same bias range
// (except for the full control case). The following are the valid bias
// ranges: [0-11], [12-24], [25-50].
```

Note

 Low-power applications where power controller and test pattern thresholds fall into different ranges are not supported and may result in a higher test pattern count and decreased shift power control.

Low-Power Shift and Test Patterns

An additional test pattern (`edt_setup`) is added before every test pattern set. This test pattern sets up the low-power mask registers before the load of the very first real test pattern. Similarly, the first real test pattern carries the low-power mask setup for the second pattern and so on. The unload values of the `edt_setup` pattern are not observed.

Power Controller Logic

The power controller loads constant values into the *don't care bits* within scan chains as the test patterns are uncompressed and shifted into the core.

The power controller must be enabled in both the EDT logic hardware and the test pattern generation software to use low-power ATPG. By default, the power controller is enabled. For more information, see [set_edt_power_controller](#). If you are not sure whether you need to use the low-power feature, you can insert a disabled controller and then, enable it if you need to lower power consumption. If you change the controller setting during pattern generation, remember to modify the generated test procedure file to force the `shift_const_en` signal to the appropriate value.

Note

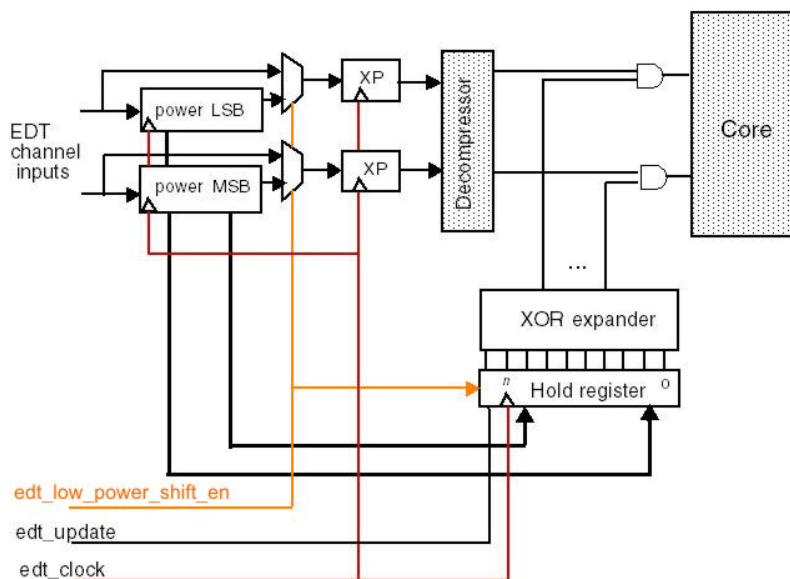


Low-power ATPG adds additional shift cycles to each test pattern, so the power controller should be disabled when it is not needed to prevent unnecessary cycles.

The `edt_low_power_shift_en` signal shown in [Figure 8-1](#) controls the low-power controller as follows:

- When `edt_low_power_shift_en` is asserted, the power controller is enabled and a control code is generated by pipeline stages at the channel inputs. The control code is loaded into a hold register and applied to the XOR expander to control whether the biasing AND gates are enabled. If the control code is 1, the AND gate is enabled and the decompressor drives the scan chain; if the control code is a 0, the AND gate is disabled and the 0 logic source drives the scan chain.
- When the `edt_low_power_shift_en` signal is forced off, the power controller is disabled, the input pipeline stages are bypassed, the hold register is filled with 1s, and the decompressor drives all the scan chains. For information on disabling the power controller, see [set_edt_power_controller](#).

For information on defining a signal for the power controller, see [set_edt_pins](#).

Figure 8-1. Low Power Controller Logic**Static Timing Analysis and Hold Violations From Low-Power Hold Registers**

Lockup cells are inserted on paths between the EDT decompressor and the scan cells to avoid clock skew issues. However, lockup cells are not required in the path between the low-power hold register and the first scan cell of each chain. This is because this path does not operate as a shift register due to the following:

- The low-power hold register only updates in the load_unload cycle, and the scan cells are not clocked in the load_unload cycle.
- The low-power hold register does not change during the shift cycle when the scan cells are clocked.

When you verify shift mode timing, both the `edt_clock` and the scan cell clocks are pulsed; this means that a static timing analysis (STA) tool will look for and report violations in the paths between low-power hold registers and the scan cells. You can prevent these violations from being reported by adding timing exceptions to your STA tool, directing it to ignore violations on these paths. An example of setting a timing exception is shown here:

```
set_multicycle_path -hold 1\
-from [get_cells edt_i/edt_contr_i/low_power_shift_contr_i/low_power_hold_reg_*_reg*]
```

Related Topics

[EDT Logic with Power Controller](#)

[Setting Up Low-Power Test](#)

Setting Up Low-Power Test

You can configure EDT logic with an enabled power controller, and programs the power controller for the desired level of shift control during test pattern generation. This also enables the low-power capture feature of the test patterns.

Prerequisites

- RTL or a gate-level netlist with scan chains inserted.
- DFT compression strategy for your design. A compression strategy helps define the most effective testing process for your design.

Procedure

1. Invoke Tessent Shell to perform EDT logic creation. For example:

```
<Tessent_Tree_Path>/bin/tessent -shell
```

Tessent Shell invokes in setup mode.

2. Set up for EDT logic creation. For example:

```
set_context dft -edt
read_verilog my_gate_scan.v
read_cell_library my_lib.atpg
set_current_design top
dofile edt_ip_creation.do
```

3. Define an *enabled* power controller with a minimum switching threshold. For example:

```
set_edt_power_controller shift enabled -min_switching_threshold_percentage 20
```

An enabled power controller with 20% minimum switching threshold is set up. If no minimum threshold is specified, 15% is used. For more information, see “[Low-Power Shift and Switching Thresholds](#)” on page 188.

You can use the [set_edt_pins](#) command to define a signal for the power controller.

Note

 You can configure the power controller as either enabled or disabled when generating the EDT controller. Whether it is enabled or disabled has an impact during pattern generation. If low power was enabled during IP generation and you want to generate patterns with low power disabled, you must issue the “set_edt_power_controller shift disabled” command in Setup mode. Similarly, if low power was disabled during IP generation and you want to generate low power patterns, you will need to enable it using the “set_edt_power_controller shift enabled” command in Setup mode. For complete information, see the [set_edt_power_controller](#) command.

4. Define the remaining EDT logic parameters. For more information, see “[Parameter Specification for the EDT Logic](#)” on page 72.

- Exit setup mode and run DRC. For example:

```
set_system_mode analysis
```

- Correct any DRC violations.

- Create the EDT logic. For example:

```
write_edt_files ../generated/low_power_enabled_edt -replace
```

- Exit Tessent Shell. For example:

```
exit
```

- Synthesize the EDT logic. For more information, see [“Synthesizing the EDT Logic”](#) on page 121.

- Invoke Tessent Shell in setup mode and then set context to perform test pattern generation.

```
set_context patterns -scan
```

- If you had generated the EDT controller with “`set_edt_power_controller shift disabled`,” you would need to enable the low power mode using the following command:

```
set_edt_power_controller shift enabled
```

- Program the power controller switching threshold. For example:

```
set_power_control shift on -switching_threshold_percentage 20 \  
-rejection_threshold_percentage 25
```

The switching during scan chain loading is minimized to 20% and any test patterns that exceed a 25% rejection threshold are discarded. For information on switching threshold constraints, see [“Low-Power Shift and Switching Thresholds”](#) on page 188.

By default, the switching threshold for ATPG is set to match the threshold used for the power controller hardware. For modular applications, the highest individual switching threshold is used.

- Report the power controller and switching threshold status. For example:

```
report_edt_configurations -all
```

```
// IP version: 4  
// External scan channels: 2  
// Compactor type: Xpress  
// Bypass logic: On  
// Lockup cells: On  
// Clocking: edge-sensitive  
// Low power shift controller:  
// Enabled and active  
// Min switching threshold:  
// 20%
```

Bold text indicates the output relevant to the power controller.

14. Turn on low-power capture. For example:

```
set_power_control capture on -switching_threshold_percentage 30 \  
-rejection_threshold_percentage 35
```

Switching during the capture cycle is minimized to 30% and any test patterns that exceed a 35% rejection threshold are discarded.

15. Exit setup mode and run DRC. For example:

```
set_system_mode analysis
```

16. Correct any DRC violations.

17. Create test patterns. For example:

```
create_patterns
```

Test patterns are generated and the test pattern statistics and power metrics display.

Note



If you had generated the IP logic with low power *disabled*, and now wanted to generate low power patterns, you would need to first enable low power using the “set_edt_power_controller shift enabled.”

18. Analyze reports, and adjust power and test pattern settings until power and test coverage goals are met. You can use the [report_power_metrics](#) command to report the capture and shift power usage associated with a specific instance or set of modules.
19. Save test patterns. For example:

```
write_patterns ../generated/patterns_edt_p.stil -stil -replace
```

Related Topics

[set_edt_power_controller \[Tessent Shell Reference Manual\]](#)

[set_power_control \[Tessent Shell Reference Manual\]](#)

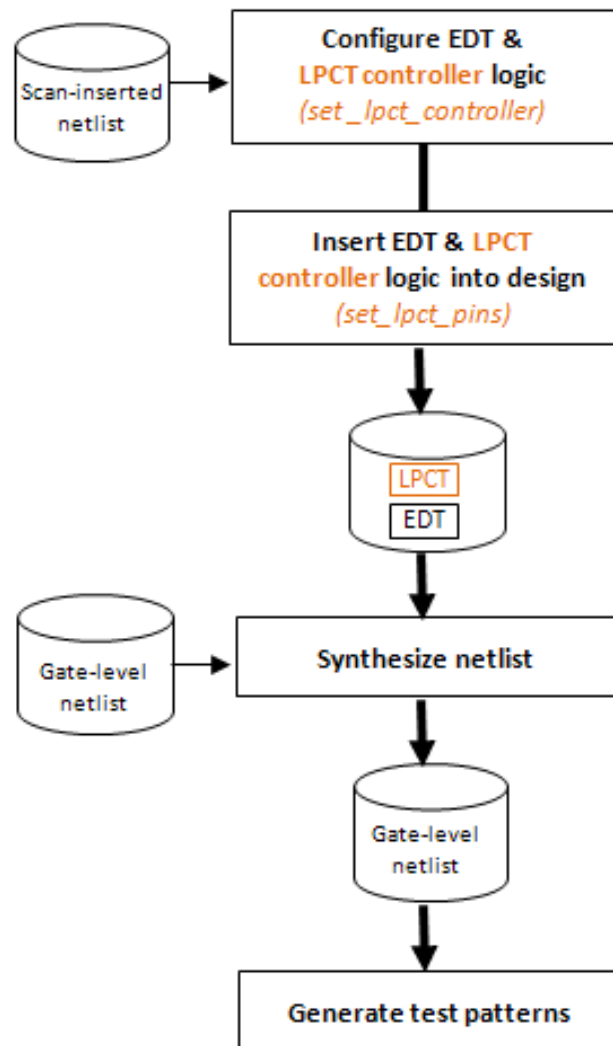
[Low-Power Test](#)

Low Pin Count Test Controller

The Low Pin Count Test (LPCT) controller minimizes the top-level pins required for the EDT application. The LPCT controller generates the control signals needed to operate the EDT logic and test the design core thereby making control signals from top-level pins unnecessary.

The LPCT controller is configured and embedded in the design along with the EDT logic according to the design process shown in [Figure 8-2](#).

Figure 8-2. LPCT Design Process

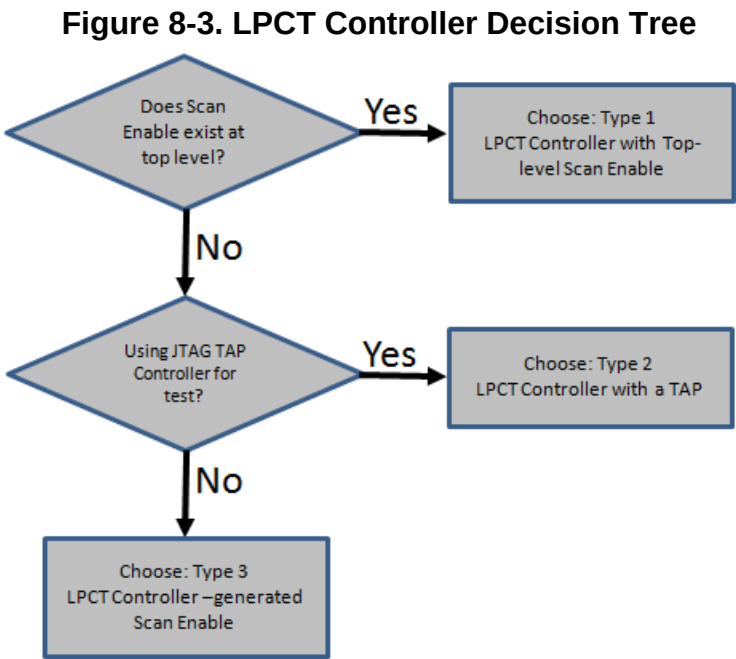


LPCT Controller Decision Tree	196
LPCT Limitations	208
LPCT Controller Types.....	210
LPCT Configuration Examples	222

LPCT Controller Decision Tree

You can implement an LPCT controller using one of three configuration types: Type 1, Type 2, or Type 3.

The three LPCT configuration types, Type 1, Type 2, and Type 3, are described in this section. Figure 8-3 illustrates the decision-making process for choosing which LPCT controller configuration fits your design constraints.



Note

Be aware that the Type 3 LPCT controller requires on-chip clock controller (OCC) logic in the design. Additionally, the OCC logic must be capable of turning off the clocks for capture which is necessary for ATPG to detect reset faults. OCC logic is supported for Type 1 and Type 2 LPCT controllers as well, but is not a requirement.

Test Mode Clock Multiplexer Requirement	197
Sharing of the LPCT Clock and a Top-Level Scan Clock	197
Shift Clock Control for LPCT Controllers	198
Type 1 LPCT Controller	202
Type 2 LPCT Controller	204
Type 3 LPCT Controller	206
Tessent OCC and LPCT Usage	208

Test Mode Clock Multiplexer Requirement

You need a multiplexer to choose between the clock controller output used during test and the original system functional clock source such as the output of a PLL. This is referred to as the *test mode clock multiplexer*.

Note



The test mode clock multiplexer is different than the *test clock multiplexer* which selects between the shift and capture clocks.

Internal capture clocks — If you are using an internal capture clock such as the output of a programmable clock controller with any of the LPCT controller types, the tool does not add this multiplexer because the tool only knows the test clock source (that is, clock controller output) and not the functional clock source (that is, PLL output). Therefore, you must add a test mode clock multiplexer for all internal capture clocks for all three types of LPCT controller and the tool assumes this multiplexer already exists in the design.

Shared LPCT and design clock — The test mode clock multiplexer is also needed when the LPCT clock is shared with the scan shift clock. In this case, in order to avoid breaking the functional clock path, you must provide the tool with the connection point to the test mode clock multiplexer. You do this using the “set_lpct_pins test_clock_connection” command.

Figure 8-4 shows an example of how the test mode clock multiplexer can be inserted and connected in the design prior to LPCT logic generation and insertion.

Sharing of the LPCT Clock and a Top-Level Scan Clock

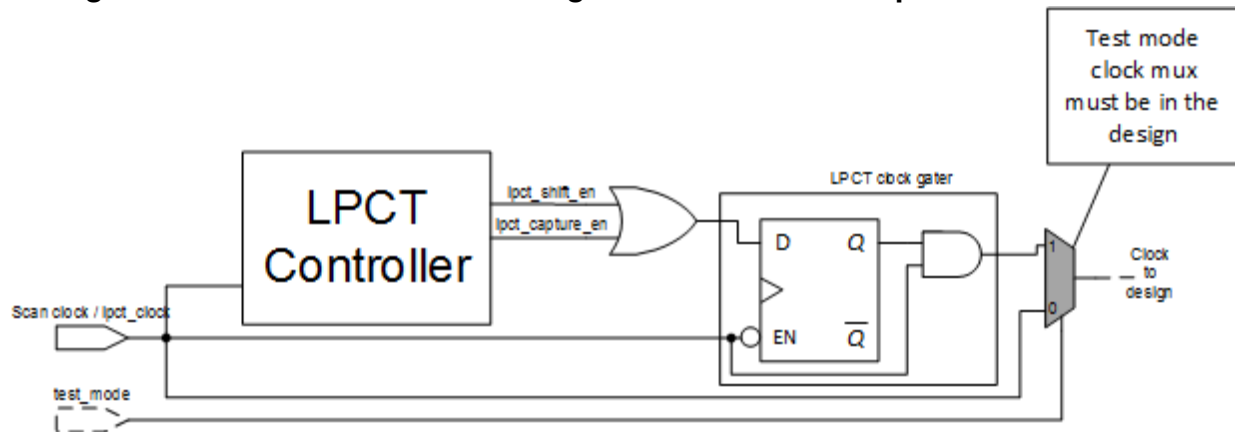
The Type 3 LPCT controller requires a dedicated LPCT clock that is different from other top-level scan clocks. The Type 1 and Type 2 LPCT controllers can use a top-level scan clock that is used for both shift and capture cycles as the LPCT clock.

When an internal scan clock is used, the tool inserts a clock mux to choose between the controller-generated shift clock during shift and the original internal clock during capture. For top-level clocks, the tool does not insert a mux because the clock can be controlled as needed during both shift and capture.

When a top-level scan clock is used as the LPCT clock and the -shift_control option is set to “clock”, the tool adds a clock gater for Type 1 and Type 2 controllers as shown in Figure 8-4. The clock gater is not added when it generates a shift enable signal. This clock gater is enabled for all shift and capture cycles, but disabled during the pre-shift and post-shift cycles.

The clock input of the clock gater is connected to the top-level clock, so ATPG has full control of the scan clock during capture. This allows ATPG to turn off the capture clock, for example when detecting asynchronous reset faults. Reusing a top-level scan clock as the LPCT clock is inferred when the defined LPCT clock is pulsed during shift in the incoming logic creation test procedure file.

Figure 8-4. Clock Gater for Sharing LPCT Clock with Top-Level Scan Clock



Shift Clock Control for LPCT Controllers

All LPCT controller types have the ability to generate and control the shift clocks.

You use the `set_lpct_controller -shift_control` option to specify how the LPCT controller generates the shift clock control signal. The three `-shift_control` options are

- **Enable** — The LPCT controller generates the `lpct_shift_en` enable signal to generate the shift clock. You can use this enable signal to create the shift clock for the design by gating it with a pulse-always clock. In this case, you must define the connections from the enable signal to the clock control logic. This is the default setting.
- **Clock** — The LPCT controller generates the shift clock. All necessary connections and gating are added so that shift clocks are controlled and driven from the LPCT controller. In the case of internal capture clocks, when the LPCT clock is shared with the scan clock and this option is used, the tool adds a clock gater in the clock path. In this case, the LPCT clock is used as both a shift clock and a capture clock.
- **None** — No signal is generated. You should use this option when shift clocks are available at the top level.

All LPCT controller types use the following signals:

- **lpct_clock_mux_select** — The select signal of a multiplexer that chooses between the shift clock and capture clock. This signal should be connected to the select signal of the existing multiplexer in the clock path.
- **lpct_shift_en** — An enable signal that can be used as a gating enable with a pulse-always clock to generate the shift clock.
- **lpct_capture_en** — An enable signal that indicates the tool is in capture mode. This signal can be used as a trigger to generate capture clock waveforms.

Use Cases for LPCT-Generated Clocks

You should specify the shift clock control option that is compatible with your design configuration. There are three `-shift_control` options for `set_lpct_controller` command, `enable`, `clock`, and `none`. There is a use case for each of three design configurations that describes the required criteria for a shift clock control option and illustrates the implementation of that option.

Use Case for “`set_lpct_controller -shift_control clock`”

If your design meets the following criteria:

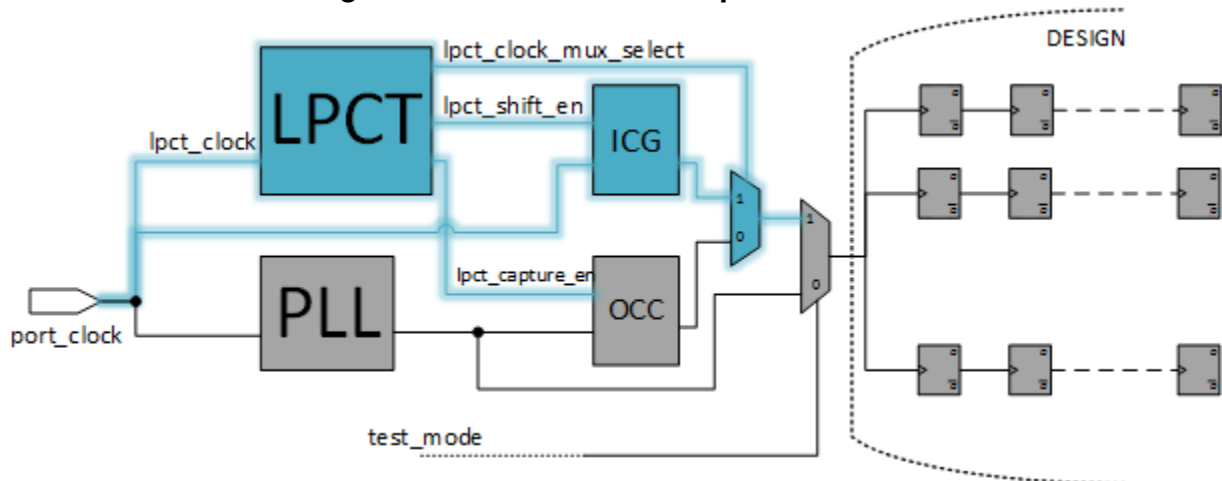
- Contains an OCC, and the test clock multiplexer is outside of the OCC.
- The shift clock is a top-level clock.
- An additional pin is not available for the `lpct_clock` signal.
- No shift clock gater exists in the design.

And, you want to eliminate the top-level shift clock pin and have the tool automatically insert the shift clock gater, you use the “`set_lpct_controller -shift_control clock`” option.

Additionally, if you have a test clock multiplexer already present in the design, you will need to use the “`set_lpct_pins -shift_clock`” command to connect the LPCT-generated shift clock to the input of the multiplexer. In this case, you specify the top-level shift_clock as “`lpct_clock`” which eliminates the need for a separate `lpct_clock` pin.

If you have internal clocks (that pulse during shift) in your design and the shift clock connection is not specified, the tool will insert a multiplexer to select between the defined internal clocks and the LPCT-generated shift clock as illustrated in [Figure 8-5](#).

Figure 8-5. -shift_control Option - clock



Note

-  Top-level capture clocks are not affected because they can be controlled with a test procedure during capture.
-

Use Case for “set_lpct_controller -shift_control enable”

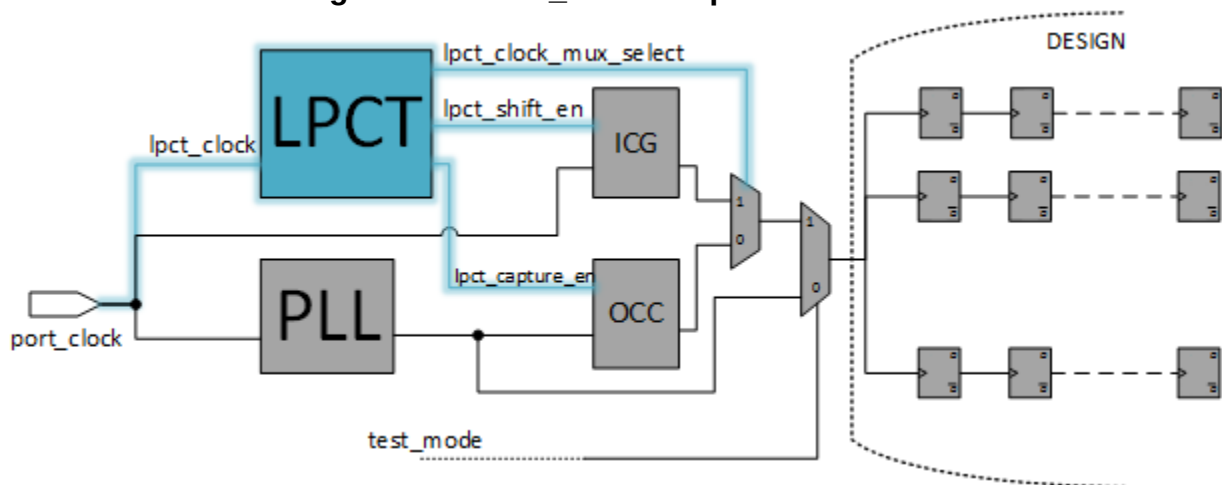
If your design meets the following configuration criteria:

- Contains an OCC, and the test clock multiplexer and clock gater are inside the OCC.
- The shift clock is a top-level clock.
- An additional pin is not available for the lpct_clock signal.
- A shift clock gater exists in the design.

And, you want to eliminate the top-level shift clock pin and have the tool connect a control signal to the existing clock gater, you use the “set_lpct_controller -shift_control **enable**” option.

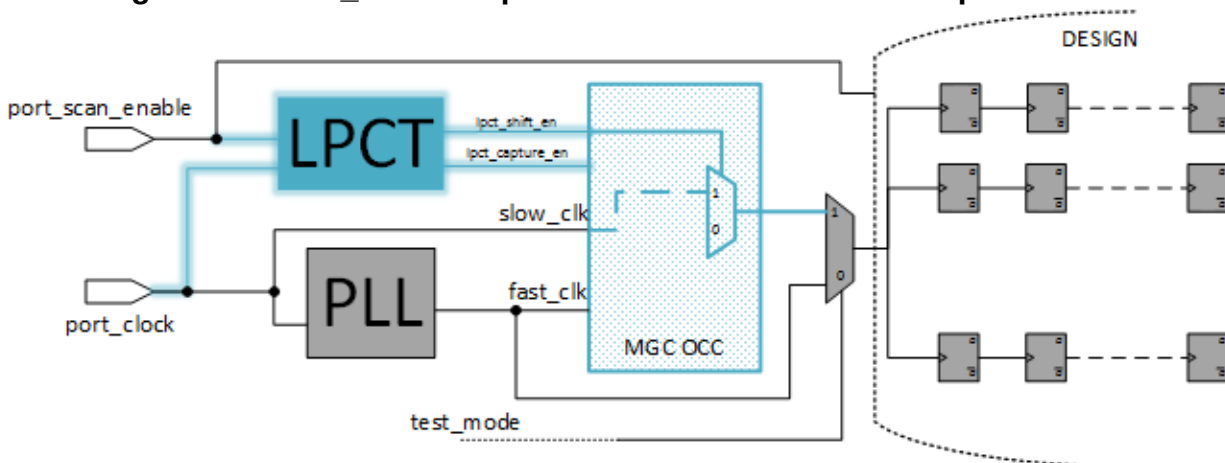
Additionally, you can use the lpct_clock_mux_select signal to drive the select input of the existing clock multiplexer as shown in [Figure 8-6](#). The design contains the clock gater (ICG) and the test clock multiplexer. In this case, the enable signal is generated from the LPCT controller. You must specify the connection point of the shift_enable signal.

Figure 8-6. -shift_control Option - enable



If you are using the Mentor Graphics OCC, you must specify the “-shift_control enable” option. You must also specify the connections of the LPCT-generated shift enable and LPCT-generated capture enable signals to the Mentor OCC as illustrated in [Figure 8-7](#).

Figure 8-7. shift_control Option -enable With Mentor Graphics OCC

**Note**

When the “-shift_control enable” option is specified, the tool does not add a clock multiplexer.

Use Case for “set_lpct_controller -shift_control none”

If your design meets the following configuration criteria:

Either:

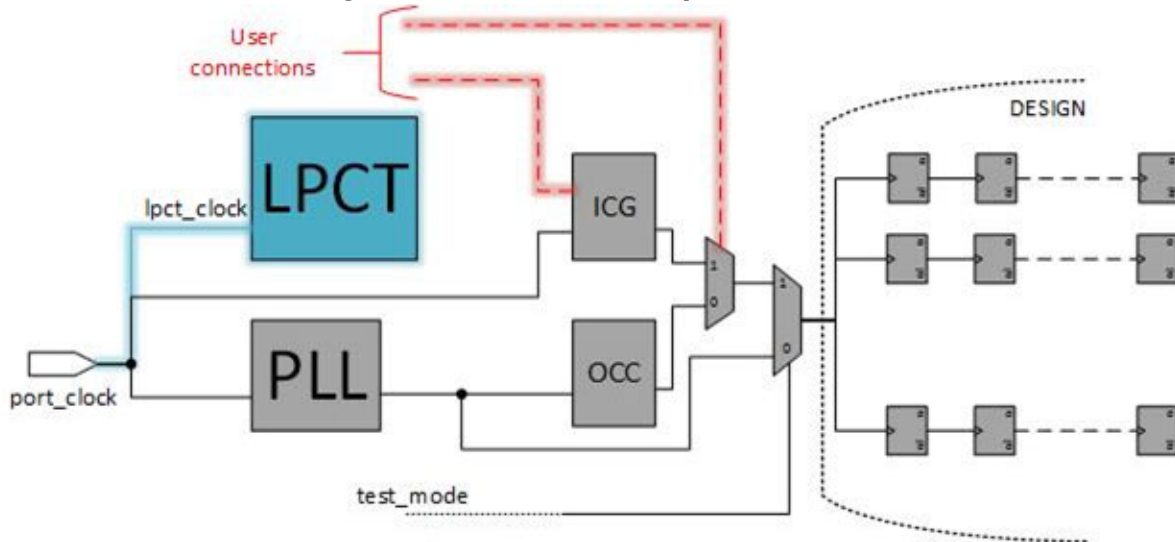
- The design is incomplete at the time of IP creation.
- The OCC and/or test clock multiplexers are not available at the time of LPCT generation.

Or:

- Clock structures are already in the design and/or all clocks are controllable from the top level.

You use the “set_lpct_controller -shift_control **none**” option to indicate that no clock generation is required from the tool. In this case, you are responsible for ensuring that the shift and capture clocks are correctly connected in the design prior to pattern generation as shown in [Figure 8-8](#).

Figure 8-8. Shift Clock Option - none



Type 1 LPCT Controller

If your design has a top-level scan enable pin, you can implement the Type 1 LPCT controller.

Configuration: Uses a top-level scan enable pin to generate the dynamic EDT signals `edt_update` and `edt_clock`.

Requirements: A top-level `scan_en` signal and a top-level `lpct_clock` signal.

LPCT Controller Configuration	Required Inputs	Generated Outputs
Type 1	<code>scan_en</code> <code>lpct_clock</code>	<code>edt_clock</code> <code>edt_update</code> <code>lpct_capture_en</code> <code>lpct_shift_clock</code> OR <code>lpct_shift_en</code> <code>lpct_clock_mux_select</code>

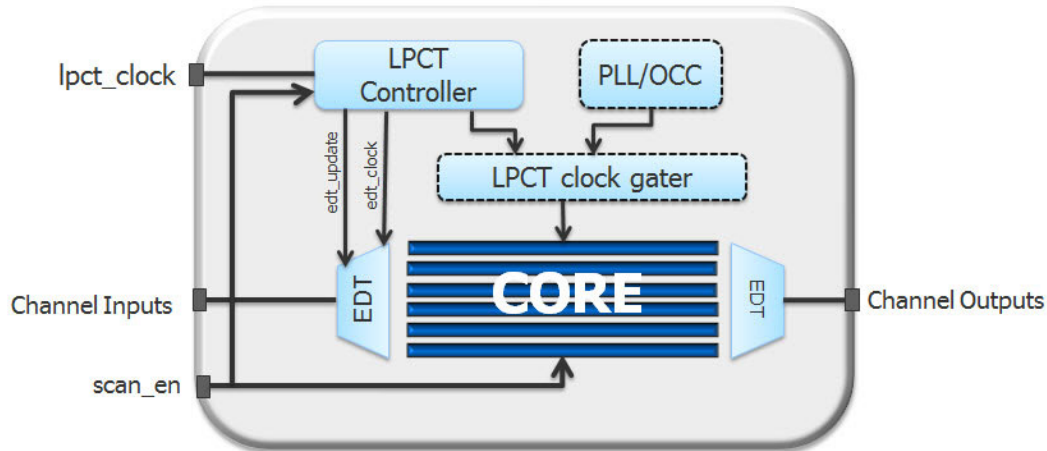
Description: If your design has a top-level scan enable pin, you can implement the Type 1 LPCT controller to generate the dynamic test signals `edt_clock`, `edt_update`, and shift clock from the `scan_en` and `lpct_clock` signals. All other static EDT-specific test signals (`edt_bypass`, `edt_low_power_shift_en`, and so on) are assumed to be available either from the top level or through user-provided test logic. You can choose to control them by some internal test data register. This LPCT controller does not generate any hardware to control any of these static signals.

Note

OCC logic is optional for the Type 1 LPCT controller.

Figure 8-9 shows the configuration of the Type 1 LPCT controller. For an in-depth description, see “Type 1 - LPCT Controller with Top-level Scan Enable”.

Figure 8-9. Type 1 LPCT Controller Configuration



Hardware area: The LPCT controller logic is approximately equal to 14 NAND gates and is independent of design size or test application.

Command: To generate a Type 1 controller use the following command:

```
set_lpct_controller on -generate_scan_enable off -tap_controller_interface off
```

Table 8-1 contains additional commands and switches that apply to the Type 1 controller.

Table 8-1. LPCT Controller Type 1 Commands and Switches

To generate a Type 1 LPCT Controller, use:	<code>set_lpct_controller</code>	<code>set_lpct_pins</code>	<code>set_lpct_condition_bits</code>
<code>set_lpct_controller</code> <code>-generate_scan_enable Off</code> <code>-tap_controller_interface Off</code>	<code>-shift_control</code>	clock input_scan_en (input) clock_mux_select (output) capture_en (output) shift_en (output) shift_clock (output) test_clock_connection (output)	None

Note

 When EDT channel outputs are shared with functional output pins, the tool adds an output channel sharing mux. The select signal of this mux is the scan enable signal specified using the “set_edt_pins scan_en” command. If you do not specify the scan enable signal for the Type-1 LPCT controller using the set_edt_pins command, the tool uses the specified LPCT input scan enable pin as the select signal for the mux.

Type 2 LPCT Controller

If your design uses an 1149.1 JTAG TAP controller at the top level to run compression, you can implement the Type 2 controller.

Configuration: Uses a TAP state machine to generate scan_enable and the dynamic EDT signals edt_update and edt_clock.

Requirement: 1149.1 JTAG TAP controller that is compliant with the IEEE 1687 standard:

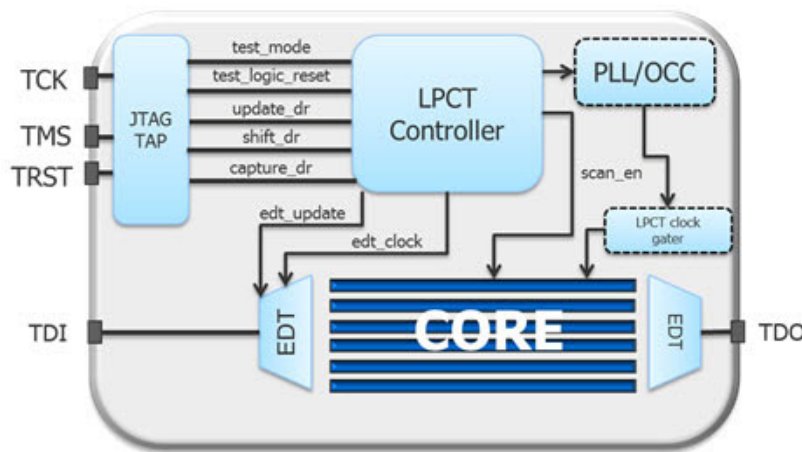
LPCT Controller Configuration	Required Inputs	Generated Outputs
Type 2	tck test_mode test_logic_reset update_dr shift_dr capture_dr	scan_en edt_clock edt_update lpct_capture_en lpct_shift_clock OR lpct_shift_en lpct_clock_mux_select

Note

 For the Type 2 LPCT controller, the top-level scan enable pin is removed and the internally-generated scan enable pin is used.

Description: If your design uses a 1149.1 JTAG TAP controller at the top level to run compression, you can implement the Type 2 controller to generate scan_en, edt_update and edt_clock on chip. All other static test signals can be controlled by the TAP controller. The LPCT controller only uses the shift_dr, capture_dr, update_dr, test_logic_reset and test_mode signals from the TAP controller. All other EDT-specific static signals (edt_bypass, edt_low_power_shift_en, and so on) are assumed to be available at the top level or are part of a user-defined data register in the JTAG TAP controller. This LPCT controller does not generate any hardware to control any of these static signals.

Figure 8-10 shows the configuration of the Type 2 LPCT controller. For an in-depth description, see “[Type 2 - LPCT Controller with a TAP](#)” on page 213.

Figure 8-10. Type 2 LPCT Controller Configuration

Hardware area: The LPCT controller logic is approximately equal to 20 NAND gates and is independent of design size or test application.

Command: To generate a Type 2 controller use the following command:

set_lpct_controller on -generate_scan_enable on -tap_controller_interface on

Table 8-2 contains additional commands and switches that apply to the Type 2 controller.

Table 8-2. LPCT Controller Type 2 Commands and Switches

To generate a Type 2 LPCT Controller, use:	set_lpct_controller	set_lpct_pins	set_lpct_condition_bits
set_lpct_controller -generate_scan_enable On -tap_controller_interface On	-shift_control	clock (input) test_mode (input) capture_dr (input) shift_dr (input) update_dr (input) tms (input) reset (input) clock_mux_select (output) capture_en (output) shift_en (output) shift_clock (output) output_scan_en (output) test_clock_connection (output)	None

Type 3 LPCT Controller

The Type 3 LPCT controller will internally generate the scan enable signal and all EDT-specific control signals.

Configuration: Scan enable signal and all other EDT-specific static and dynamic signals are generated by the LPCT controller.

Requirements: Generate all EDT-specific signals on chip including scan_en.

LPCT Controller Configuration	Required Inputs	Generated Outputs
Type 3	lpct_clock lpct_data_in (edt_channels_in1)	edt_update edt_clock scan_en edt_bypass edt_low_power_shift_en lpct_shift_clock OR lpct_shift_en lpct_capture_en edt_configuration

Note



For Type 3 controllers, the top-level scan enable pin is removed and the internally-generated scan enable pin is used.

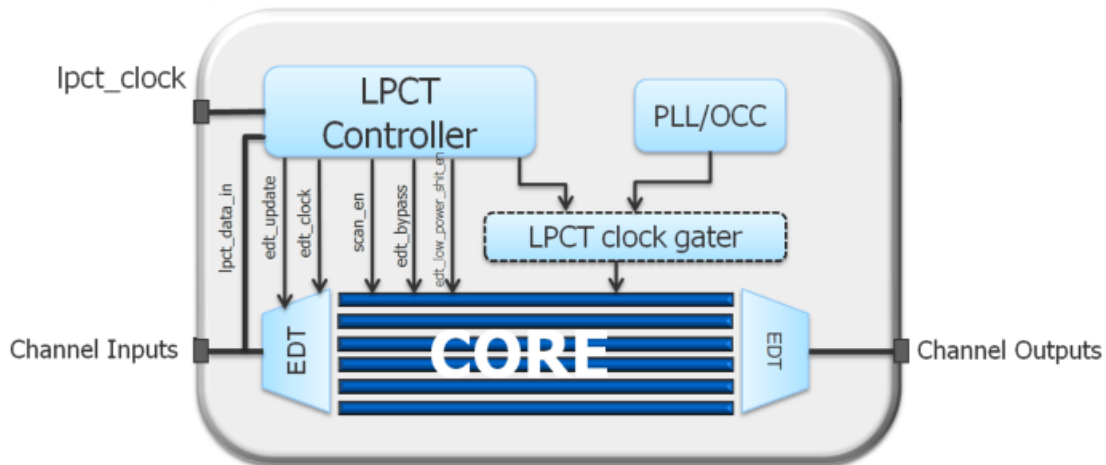
Note



OCC logic is required to detect reset faults for the design with a Type 3 LPCT controller.

Description: The LPCT controller will internally generate the scan enable signal and all EDT-specific control signals; this includes the dynamic signals edt_update, edt_clock, and scan_en and the static signals edt_bypass, edt_low_power_shift_en, and edt_configuration. If a design shift clock is not available at the top level, the LPCT controller can generate the shift clock from the LPCT clock.

Figure 8-11 shows the configuration of the Type 3 LPCT controller. For an in-depth description, see [“Type 3 - LPCT Controller-Generated Scan Enable”](#) on page 215.

Figure 8-11. Type 3 LPCT Controller Configuration

Hardware area: The LPCT controller logic is approximately equal to 1200 NAND gate equivalent and is independent of design size or test application.

Command:

set_lpct_controller on -generate_scan_enable on -tap_controller_interface off

Table 8-3 contains additional commands and switches that apply to the Type 3 controller.

Table 8-3. LPCT Controller Type 3 Commands and Switches

To generate a Type 3 LPCT Controller, use:	set_lpct_controller	set_lpct_pins	set_lpct_condition_bits
set_lpct_controller -generate_scan_enable On -tap_controller_interface Off	-max_shift_cycles -max_capture_cycles -max_scan_patterns -max_chain_patterns -test_mode_detect -shift_control -load_unload_cycles	clock (input) reset (input) data_in(input) test_mode (input) clock_mux_select (output) capture_en (output) shift_en (output) shift_clock (output) output_scan_en (output) reset_out (output) test_end (output)	-condition reset -condition scan_en

For an in-depth description of this configuration, see “[Type 3 - LPCT Controller-Generated Scan Enable](#)” on page 215.

Tessent OCC and LPCT Usage

Tessent On-Chip Clock Controllers (OCCs) support Type 1, Type 2, and Type 3 LPCT controllers.

Type 3 LPCT Controller

When using a Type 3 LPCT controller, there are two possible cases depending on when you added the Tessent OCC as follows:

- **OCC Core Instances Added During IP Creation** — In this case, the LPCT controller includes any TDR bits for the controller OCC static signals.
- **OCC Core Instances Not Added** — An example is the EDT skeleton flow. In this case, you must use the “-tessent_occ switch to the [set_lpct_controller](#) command to specify the OCC.

Refer to “[Tessent OCC Overview](#)” in the *Tessent Scan and ATPG Manual* for complete information.

LPCT Limitations

Be aware that the LPCT has certain limitations. There are design flow and hardware limitations, test pattern limitations (Type 3 LPCT), and a single shared LPCT controller for all EDT design blocks.

Design Flow/Hardware Limitations

The LPCT controller has the following design flow limitations:

- Compression logic inserted external to the design core within a top-level wrapper is not supported.
- LSSD architecture is not supported.
- The scan_en signal must be constrained to “0” during capture for the Type 1 LPCT controller.
- The number of pre-shift and post-shift cycles cannot be changed for any type controller during pattern generation. However the Type 3 controller allows changing of these cycles during IP creation.
- Pulsing edt_clock before shift is not supported because edt_clock and shift_clock are derived from the same clock source.
- When scan_en is available at the top level, the EDT static control signals such as *edt_bypass* and *edt_low_power_shift_en* are implemented as top-level pins. You are

responsible for connecting these pins to some internal test logic to avoid having them assigned as top-level pins.

Test Pattern Limitations When Using a Type 3 Controller

When using an LPCT controller-generated scan enable configuration, the controller has the following test pattern limitation:

- Multiple load type test patterns are not supported.

Single Shared LPCT Controller for All EDT Design Blocks

If you define all EDT blocks at the same time during IP creation (top-down), the tool correctly generates only one LPCT controller and drives all EDT blocks from this controller. In a design with multiple power domains, you should ensure that the LPCT controller is placed in an always-ON power domain. The EDT blocks can still be placed on the same power domain as the block level logic. You can use the [set_lpct_instances](#) command to control where the LPCT controller is instantiated.

If you use the integration flow (bottom-up), do not create LPCT logic along with block-level EDT logic. During the top-level integration, the tool can generate the LPCT controller while also making the connections for the block level EDT signals. In a design with multiple power domains, you should also ensure that the LPCT controller is placed in an always-ON power domain using the [set_lpct_instances](#) command.

LPCT Controller Types

This describes the three LPCT controller types, Type 1, Type 2, and Type 3, and their differences.

Note



All three LPCT controller types apply to scan and EDT control pins only and do not limit the number of EDT channels that can be used.

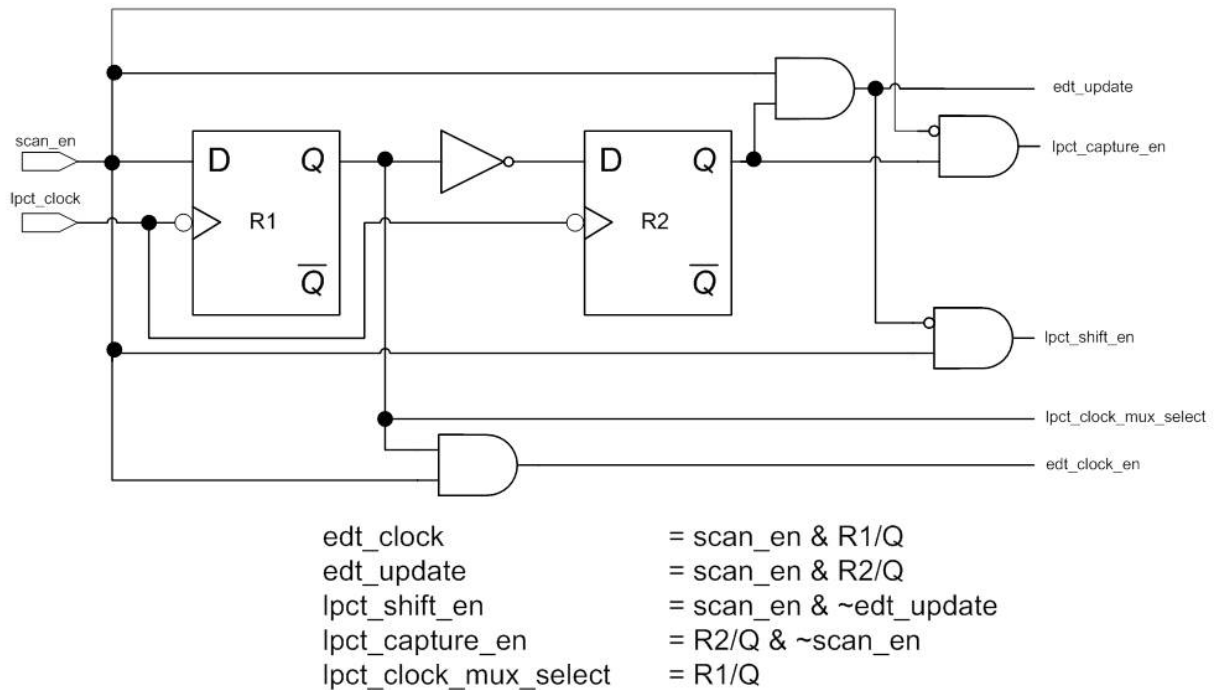
Type 1 - LPCT Controller with Top-level Scan Enable	210
Type 2 - LPCT Controller with a TAP	213
Type 3 - LPCT Controller-Generated Scan Enable	215

Type 1 - LPCT Controller with Top-level Scan Enable

When you implement an LPCT controller using a top-level scan_en pin, the LPCT controller generates the edt_update and edt_clock signals. However, it does not generate any of the EDT static control signals such as edt_bypass, edt_low_power_shift_en, and so on. To avoid having these signals assigned as top-level pins, you must connect them to some internal test logic.

When implementing the LPCT controller with a top-level scan enable signal (Type 1), the design must have a top-level clock. The clock can be a pulse-always reference clock or a tester-controllable top-level clock pin as shown in [Figure 8-12](#). If this clock is also a shift clock, and the shift control is set to “clock”, a clock gater is automatically inserted in this clock path to enable this clock to be used during shift and capture

[Figure 8-12](#) shows the controller logic. In this configuration, the scan_en signal is constrained to 0 in the capture cycle and set to 1 during the shift cycle, but is set to 0 during the post-shift cycles.

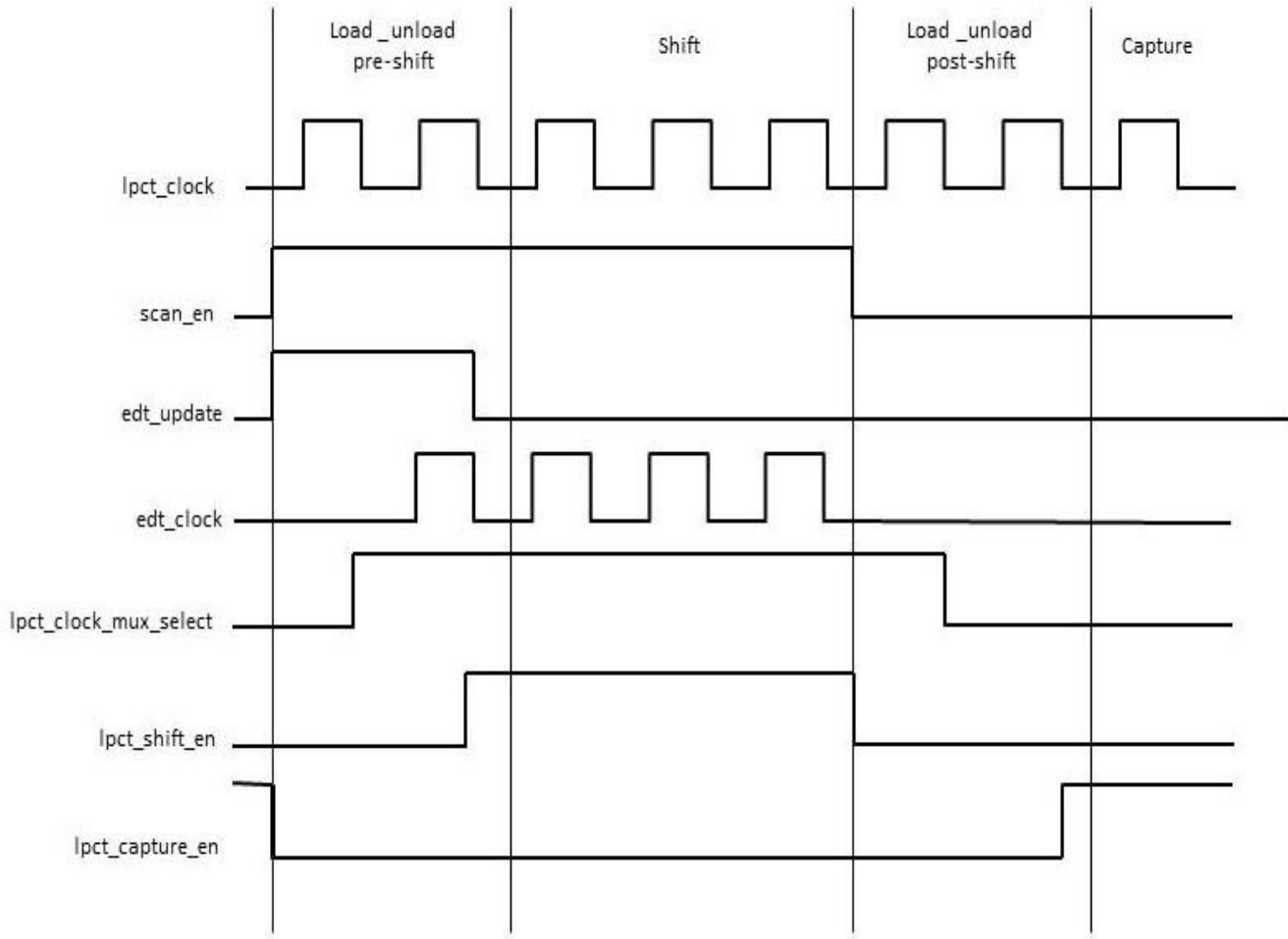
Figure 8-12. Type 1 LPCT Controller Operation

The Type 1 LPCT controller does not have a finite state machine or counters to track the test procedure states. The start of the load_unload procedure is inferred when the scan_en signal transitions from 0 to 1.

- For scan test patterns, the pin constraint on scan_en provides the initial 0 value for the transition.
- For chain test patterns, there are no capture cycles when using pulse-always clocks; the post-shift cycles in load_unload provide the initial 0 value for the transition.

Figure 8-13 shows the waveforms generated by Figure 8-12.

Figure 8-13. Signal Waveforms for Type 1 LPCT Controller



Two pre-shift and post-shift cycles are added to the load_unload procedures when using the Type 1 LPCT controller. These two cycles separate the transition between the shift clock, the capture clock, lpct_capture_en, and the lpct_clock_mux_select signals when transitioning from *capture to shift* and from *shift to capture*.

- **Pre-Shift Cycles** — At the beginning of the pre-shift cycles, the scan enable signal transitions from 0 (during capture) to 1 which allows the edt_update output from the LPCT controller to be asserted immediately. The edt_clock signal is generated one cycle later. However, the shift clock begins to pulse two cycles after the transition of the scan_en signal.
- **Post-Shift Cycles** — At the end of scan chain shifting, the scan_en signal is deasserted (transitions from 1 to 0). The signal scan_en is deasserted for both post-shift cycles and the clocks to the design are expected to be turned off as shown in the waveforms in [Figure 8-13](#). The lpct_clock_mux_select signal is deasserted at the negedge of the clock in the first post-shift cycle. The lpct_capture_en signal transitions one cycle later—on the negedge of the second post-shift cycle. The capture pulses can only be generated in

the cycle after the `lpct_capture_en` signal is asserted (after the 2nd post-shift cycle). During each of these cycles, only one of the signals, `lpct_clk_mux_select` and `lpct_capture_en`, transition at a time.

Type 2 - LPCT Controller with a TAP

When you implement an LPCT controller with a TAP, the LPCT controller generates the scan enable, `edt_update`, and `edt_clock` signals based on the output of the TAP controller. However, it does not generate any of the EDT static control signals such as `edt_bypass` and `edt_low_power_shift_en`. To avoid having these signals assigned as top-level pins, you must connect them to some internal test logic.

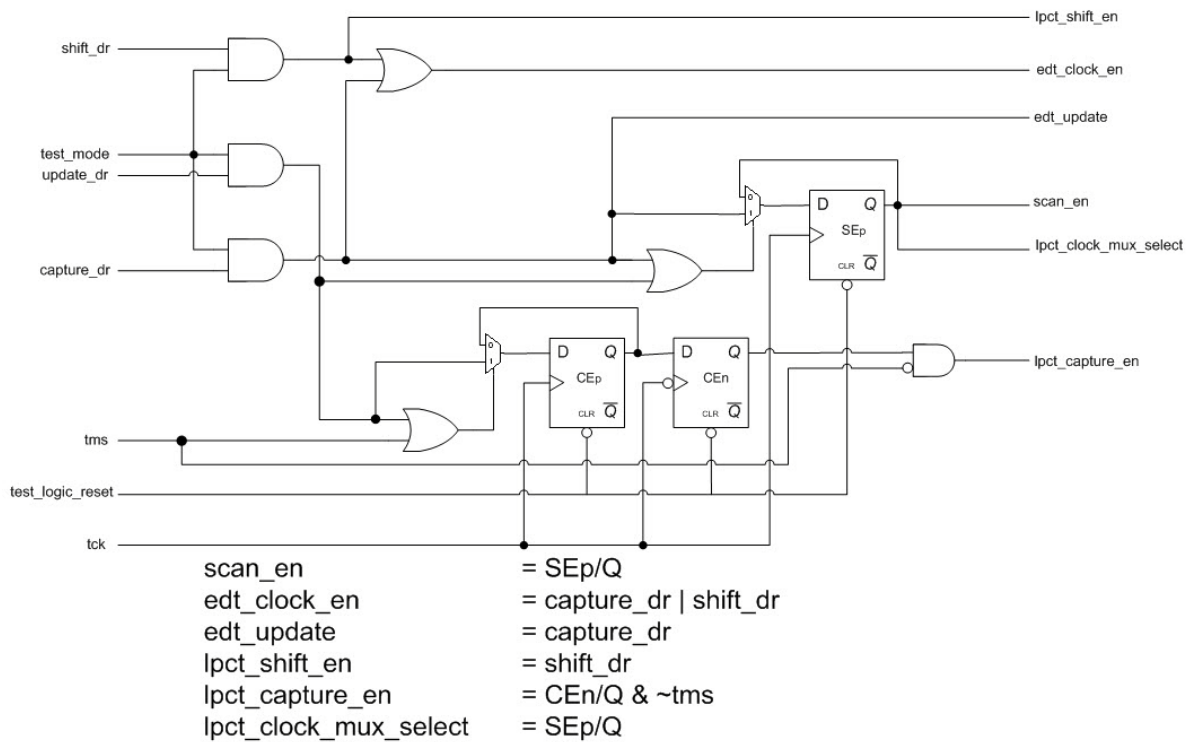
When implementing a Type 2 LPCT controller, the dynamic test control signals are generated based on the TAP controller state machine. In addition to the clock (`tck`) and test mode signal (`tms`), the enable signals corresponding to `capture_dr`, `test_logic_reset`, `shift_dr`, and `update_dr` are used as inputs to the controller. The `shift_dr` and `capture_dr` signals are assumed to change at the rising edge of the `tck` signal. These signals can be connected either to combinational tap output pins, or registered at the negedge of TCK in the TAP controller.

The `update_dr` signal is assumed to change at the falling edge of the `tck` signal. This signal can be connected either to a combinational tap output pin, or registered early at the prior posedge of TCK in the TAP controller.

These signal change edges are consistent with the IEEE 1149.1 standard.

Figure 8-14 shows the controller logic of the LPCT controller when using a TAP.

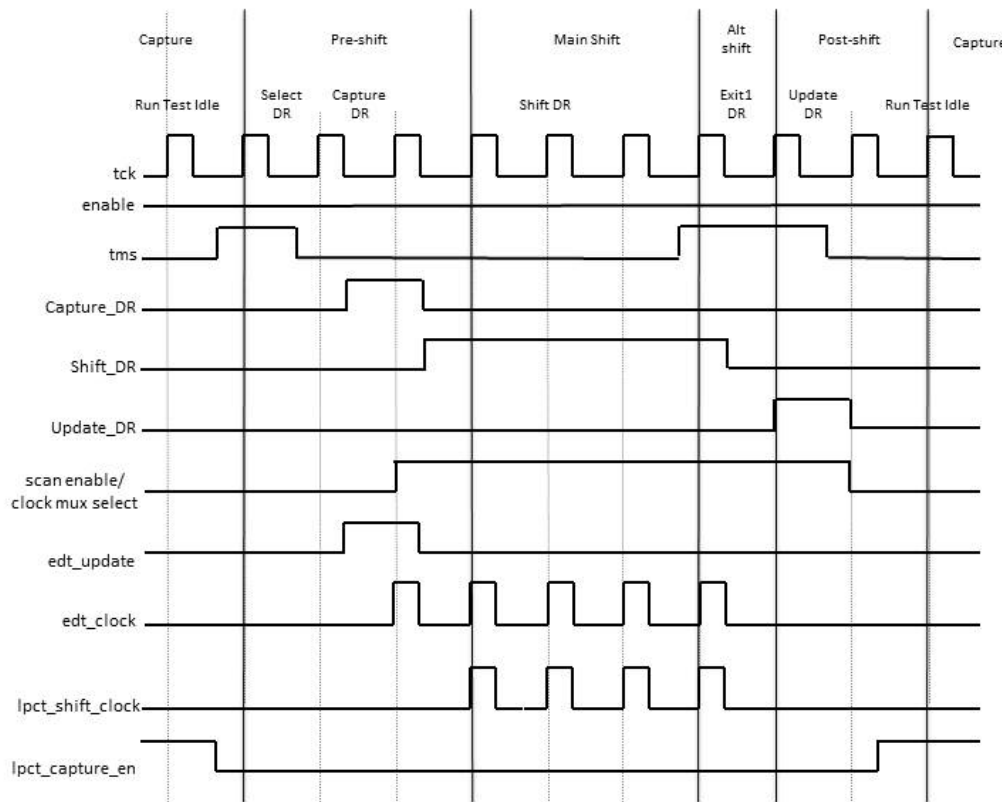
Figure 8-14. LPCT Controller with TAP



The TAP controller must provide an enable signal (`test_mode`) that signals the LPCT controller to enter test mode. This `test_mode` signal can be generated using a JTAG user-defined instruction.

The `test_mode` signal indicates whether the instruction corresponding to scan test is currently loaded in the instruction register. The signal `test_logic_reset` is used to asynchronously reset the flip-flops driving `scan_en` and `capture_en` because the design is required to go to a functional mode of operation immediately on reset of the TAP controller. The `scan_en` signal has a re-circulating mux to hold its ON value between `capture_dr` and `update_dr`; this allows the `scan_en` to not change unnecessarily when long shift sequences are broken using the `pause_dr` state.

Figure 8-15 illustrates the waveforms of the input signals from the TAP controller and the generated output signals. Capture is performed during the `run_test_idle` state; this allows for an arbitrary number of `tck` capture cycles by constraining `tms` to 0.

Figure 8-15. Signal Waveforms for TAP-based LPCT Controller

Type 3 - LPCT Controller-Generated Scan Enable

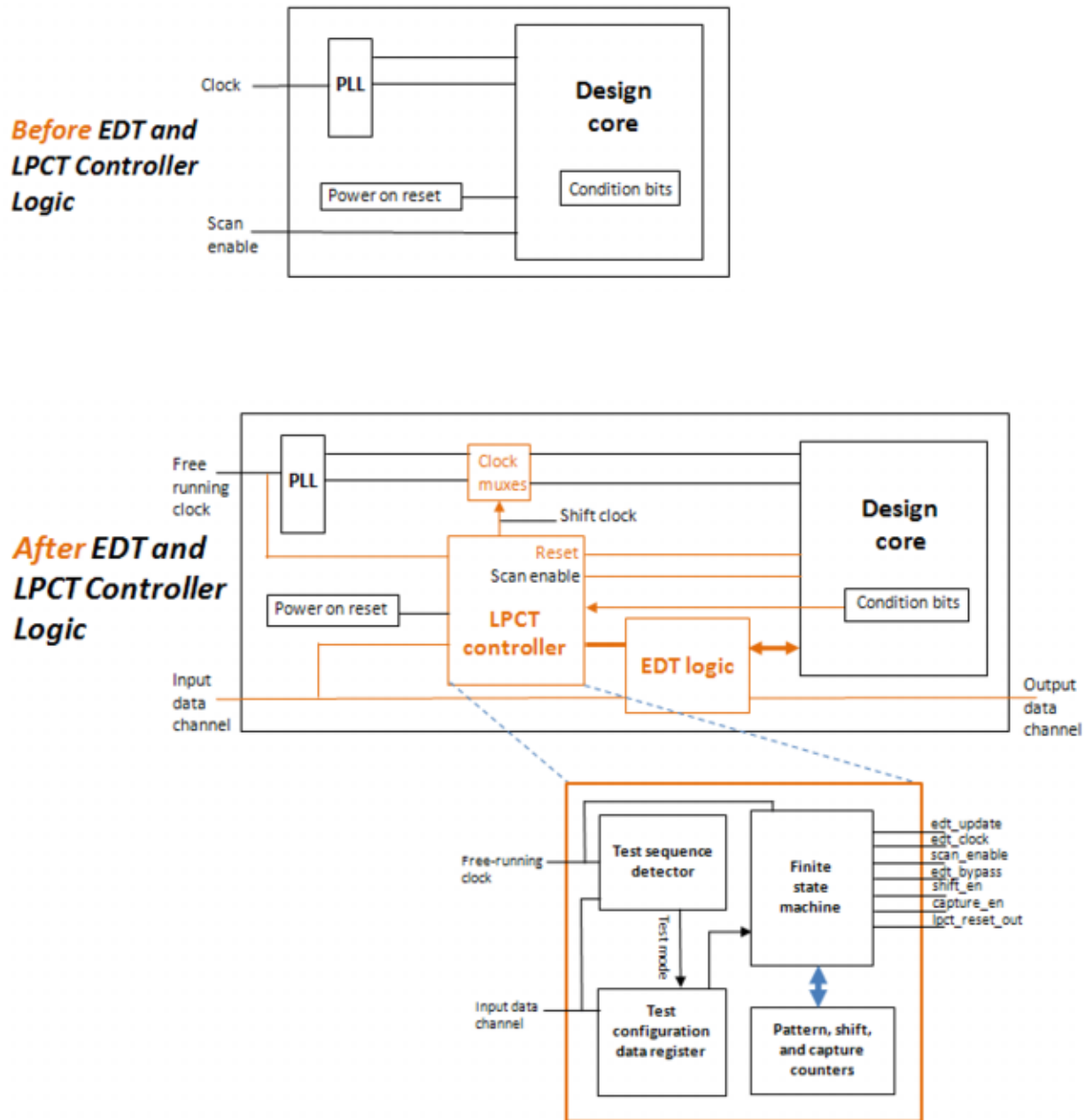
A Type 3 LPCT controller requires a minimum of three top-level pins including a pulse-always clock, an input data channel, and an output data channel.

As shown in [Figure 8-16](#), the pulse-always clock source can be either the reference clock from an on-chip PLL or the output of a PLL that is always running. The LPCT controller logic operates based on this clock; the EDT and capture clocks are derived from this clock.

Note

For Type 2 and Type 3 controllers, the top-level scan enable pin is removed and the internally-generated scan enable pin is used.

Figure 8-16. Before and After EDT and LPCT Controller Logic



In the Type 3 configuration, the LPCT controller contains the following components as shown in Figure 8-16:

- **Test Sequence Detector** — Detects a specified input sequence and produces a signal to enable test mode. This is optional depending on how you configure the test mode enable.
- **Test Configuration Data Register** — Contains information about the test pattern set such as the number of chain/scan tests, shift/capture cycles, and the EDT logic mode of operation including low-power, bypass, and dual configurations. The size of the test

configuration register is approximately 50 bits and is directly related to the size of the shift, capture, and pattern counters. The test configuration data is read once during the test_setup procedure.

- **Finite State Machine** — Generates the scan control signals during test pattern application and controls pattern shift and capture counters.
- **Pattern, Shift, And Capture Counters** — Track test pattern data for the finite state machine.

Test Mode Enable

Depending on the application, a signal from the LPCT controller to enable test mode must be configured using one of the following methods:

- **Test Mode Signal** — Test mode is enabled after the test mode signal is asserted for one cycle, and the test session end is determined by the test pattern counters. When this signal is a top-level pin, the correct test_setup procedure is automatically generated. When this signal is an internal pin, you must modify the test_setup procedure to ensure that the internal test mode signal is asserted as necessary and that the controller logic is reset before entering test mode.
- **Test Mode Sequence** — Test mode is enabled when a specific input sequence is detected within a specific number of cycles after the LPCT controller is reset. This is required when no top-level pin is used. You can specify the sequence/cycles with the [set_lpct_controller](#) command when setting up the LPCT controller. The generated test procedure file contains all the initialization cycles necessary to enter test mode when using sequence detection.

Note



Only one of these methods can be used to enable test mode for any single application.

Test Patterns and the Type 3 LPCT Controller

When generating test patterns for a Type 3 LPCT controller, you must take into account the following test pattern setups:

- **NCPs or Clock Control Definitions can be Used for Capture Cycles** — The LPCT controller hardware is configured for a fixed number of capture cycles as determined by the [set_lpct_controller](#) -max_capture_cycles command. Consequently, you must use NCPs to specify all possible clocking sequences and add additional cycles so all test patterns use the fixed number of capture cycles.
 - If NCPs are used, each one must have the same number of capture cycles.
 - If clock control definitions are used, the tool will automatically ensure that all patterns in the pattern set have the same number of capture cycles.

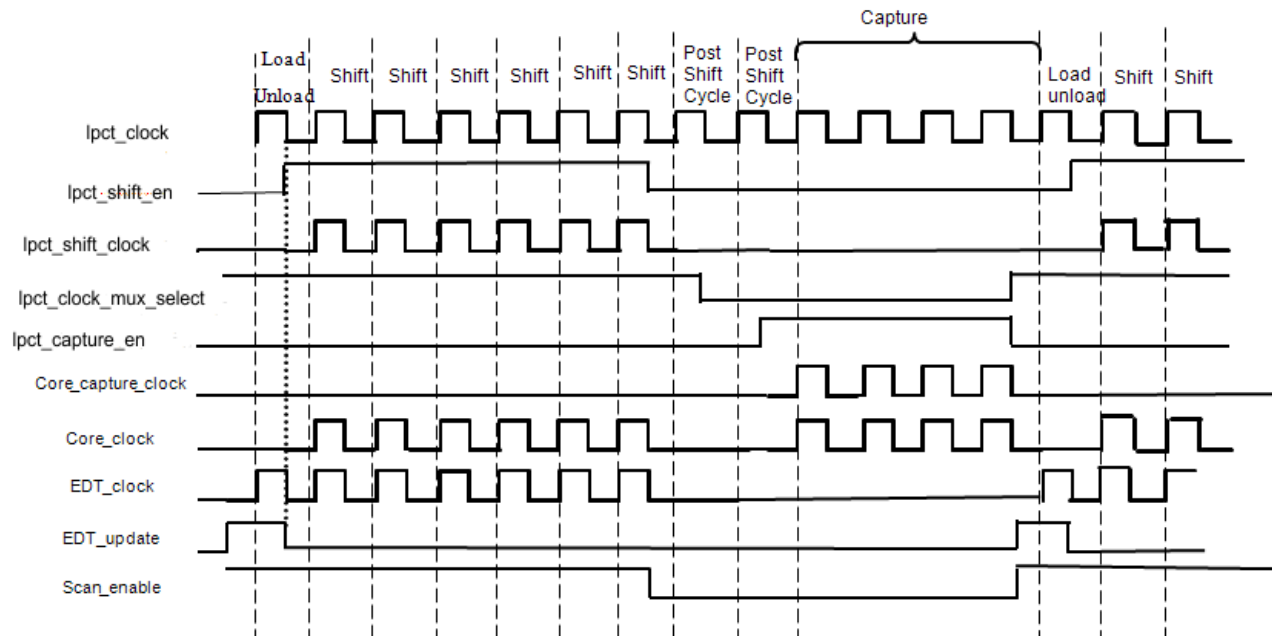
- The value of the capture cycle width portion of the test configuration data is automatically stored in the test patterns as part of the *test_setup* procedure.
- **Chain Test Patterns** — The LPCT controller includes separate counters for chain test and scan test patterns. The chain tests do not include a capture cycle, so the controller does not enter capture state for chain test patterns.
- **Iddq Test Patterns** — Iddq tests do not have a capture cycle, but there is a quiescent (dead) cycle between pattern loads. During this time, you must ensure that no functional/design clocks pulse. NCPs are not supported for iddq test patterns.
- **Parallel Test Patterns** — The LPCT controller includes internally-added primary input pins, so you must use the `-mode_internal` switch when saving parallel test patterns. For more information, see the [write_patterns](#) command.
- **WGL and Verilog Test Patterns** — If you are using the Type 3 LPCT controller, you cannot use the `write_patterns -mode LSI` switch when saving WGL or Verilog patterns. When you save WGL or Verilog patterns using the `-mode LSI` switch, the tool manipulates the patterns and adds extra cycles to ensure the same number of cycles in both pre- and post-shift. These modifications are inconsistent with the Type 3 LPCT hardware and cause the patterns to get out of sync with the LPCT Finite State machine. Because these same conditions can be enabled if you set the [ALL_FIXED_CYCLES](#) parameter to anything other than 0, you cannot save patterns with the `ALL_FIXED_CYCLES` parameter set to 1 or 2 when using the Type 3 LPCT controller.

[Figure 8-17](#) and show the waveforms for signals generated by the Type 3 LPCT controller configuration.

Note



The `edt_clock` signal is a gated version of the pulse-always clock that is always generated by the LPCT controller.

Figure 8-17. Scan Test Pattern Timing**Detecting Faults on Reset Lines**

When using the Type 3 LPCT controller, the functional reset for the design is also used to reset the LPCT controller. Therefore, the faults along the reset lines are not detected by ATPG because the reset is in the deasserted state during the entire ATPG session. You can use one of the following two methods to recover the coverage along the reset lines:

- **Assert Reset for the Entire Pattern Set** — With this method, the patterns to detect faults along the reset lines must be in a different pattern set with their own test_setup procedure. To use this method, make the following change in the dofile generated during the logic creation phase of the LPCT controller:

In the Pattern Generation dofile, specify pin constraints and register values as follows:

Change: `add_input_constraints reset_control -C0`

To: `add_input_constraints reset_control -C1`

Change: `add_register_value lpct_config_reset_control 0`

To: `add_register_value lpct_config_reset_control 1`

Note

When specifying an active low reset, using the `set_lpct_pins -reset -active low` command, you should reverse the pin constraint and register values. That is, you should flip the pin constraint from C1 to C0 and the register value from 1 to 0.

The `add_register_value` command holds the reset to the design in the asserted state while the reset to the controller is deasserted. Although this method requires two separate sets

of patterns each with their own test_setup procedure, no additional design requirements are needed to create these patterns.

- **Use LPCT Condition Bits** — With this method, you use a scan flop in the design as a control (condition) bit which allows ATPG to automatically justify the appropriate value to assert or deassert the reset signal to the design. With this method, only one pattern set is created for each fault model. To use this method, make the following changes:

In the IP Creation dofile, specify the condition scan cell as follows:

Add: `set_lpct_condition_bits -condition reset -from scancell_name`.

In the Pattern Generation dofile, specify pin constraints and register values as follows:

Change: `add_input_constraints reset_control -C0`

To: `add_input_constraints reset_control -C1`

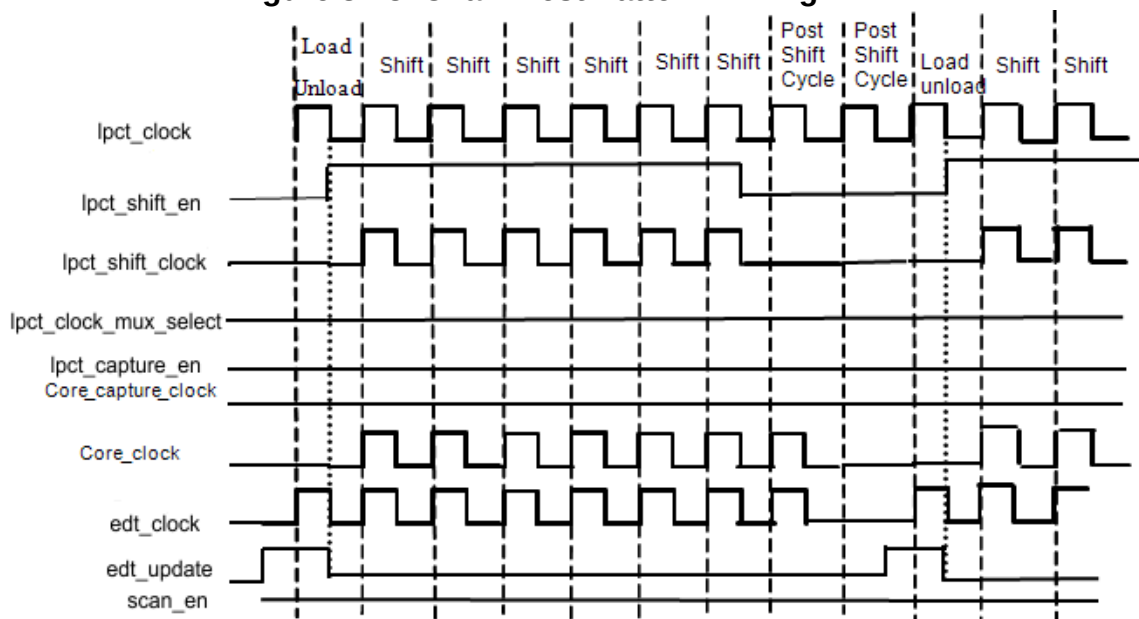
Change: `add_register_value lpct_config_reset_control 0`

To: `add_register_value lpct_config_reset_control 1`

Note

 When specifying an active low reset, using the `set_lpct_pins -reset -active low` command, you should reverse the pin constraint and register values. That is, you should flip the pin constraint from C1 to C0 and the register value from 1 to 0.

ATPG justifies the value in this scan cell in the last load cycle before capture to ensure the reset to the design is asserted or deasserted as needed. Using this method allows you to generate a single test pattern set to detect reset line faults as well as other design faults.

Figure 8-18. Chain Test Pattern Timing**Toggling Scan Enable**

When using the Type 3 LPCT controller, the scan enable signal generated by the LPCT controller (`lpct_scan_en`) is always driven to 0 during capture. For stuck-at patterns, if the scan enable signal to the design is required to toggle during capture, you must use dedicated scan cells (condition bits); these scan cells must be part of the scan chain. To toggle the scan enable signal, do the following:

In the IP Creation dofile, specify the condition scan cell:

Add: `set_lpct_condition_bits -condition scan_en -from scancell_name`.

In the Pattern Generation dofile, specify pin constraints and register values:

Change: `add_input_constraints scan_en_control -C0`

To: `add_input_constraints scan_en_control -C1`

Change: `add_register_value lpct_config_scan_en_control 0`

To: `add_register_value lpct_config_scan_en_control 1`

LPCT Configuration Examples

This section provides an example for creating each of the three LPCT configuration types and examples of the dofile and test procedure files generated for each configuration.

For more information on the differences between configuration types, see [LPCT Controller Types](#).

Type 1 Controller Generation Example	222
Type 2 Controller Generation Example	223
Type 3 Controller Example.....	225
Type 1 Controller LPCT Clock Example	227
Type 2 Controller Scan Shift Clock Example	228

Type 1 Controller Generation Example

This example generates a Type 1 LPCT controller.

Sample dofile:

```
// Group definition

add_scan_groups grp1 scan_setup.testproc

// Clock definitions

add_clocks 0 /occ/NX2 -pseudo_port_name NX2
add_clocks 0 /occ/NX1 -pseudo_port_name NX1

// Scan chain definitions

add_scan_chains chain1 grp1 scan_in1 scan_out1
add_scan_chains chain2 grp1 scan_in2 scan_out2
add_scan_chains chain3 grp1 scan_in3 scan_out3
add_scan_chains chain4 grp1 scan_in4 scan_out4
add_scan_chains chain5 grp1 scan_in5 scan_out5
add_scan_chains chain6 grp1 scan_in6 scan_out6
add_scan_chains chain7 grp1 scan_in7 scan_out7
add_scan_chains chain8 grp1 scan_in8 scan_out8
add_scan_chains chain9 grp1 scan_in9 scan_out9
add_scan_chains chain10 grp1 scan_in10 scan_out10
add_scan_chains chain11 grp1 scan_in11 scan_out11
add_scan_chains chain12 grp1 scan_in12 scan_out12
add_scan_chains chain13 grp1 scan_in13 scan_out13
add_scan_chains chain14 grp1 scan_in14 scan_out14
add_scan_chains chain15 grp1 scan_in15 scan_out15
add_scan_chains chain16 grp1 scan_in16 scan_out16

// EDT configuration
set_edt_options -channels 2 -location internal

// LPCT configuration
set_lpct_controller -generate_scan_enable off \
                    -tap_controller_interface off -shift_control clock

// LPCT Pin connections
set_lpct_pins clock refclk
set_lpct_pins input_scan_en scan_en

// Run DRC
set_system_mode analysis

// Insert EDT and LPCT controller logic in design
write_edt_files created -verilog -replace
// The command writes out the LPCT-specific TCD file
```

Type 2 Controller Generation Example

This example generates a Type 2 LPCT controller.

Sample dofile:

```
// Group definition

add_scan_groups grp1 scan_setup.testproc

// Clock definitions

add_clocks 0 /occ/NX2 -pseudo_port_name NX2
add_clocks 0 /occ/NX1 -pseudo_port_name NX1
add_clocks 0 tck

// Pin constraints

add_input_constraints trst -C1

// Scan chain definitions

add_scan_chains chain1 grp1 scan_in1 scan_out1
add_scan_chains chain2 grp1 scan_in2 scan_out2
add_scan_chains chain3 grp1 scan_in3 scan_out3
add_scan_chains chain4 grp1 scan_in4 scan_out4
add_scan_chains chain5 grp1 scan_in5 scan_out5
add_scan_chains chain6 grp1 scan_in6 scan_out6
add_scan_chains chain7 grp1 scan_in7 scan_out7
add_scan_chains chain8 grp1 scan_in8 scan_out8
add_scan_chains chain9 grp1 scan_in9 scan_out9
add_scan_chains chain10 grp1 scan_in10 scan_out10
add_scan_chains chain11 grp1 scan_in11 scan_out11
add_scan_chains chain12 grp1 scan_in12 scan_out12
add_scan_chains chain13 grp1 scan_in13 scan_out13
add_scan_chains chain14 grp1 scan_in14 scan_out14
add_scan_chains chain15 grp1 scan_in15 scan_out15
add_scan_chains chain16 grp1 scan_in16 scan_out16

// EDT configuration

set_edt_options -channels 1
set_edt_options -location internal

set_edt_pins input_channel 1 tdi m8051_i/edt_channels_in1
set_edt_pins output_channel 1 tdo tap_i/tap_edt_channel_reg_in

// LPCT configuration

set_lpct_controller -generate_scan_enable on \
                   -tap_controller_interface on -shift_control clock

// LPCT Pin connections to LPCT controller pins

set_lpct_pins clock tck pad_instance_1_i/po_pad_tck
set_lpct_pins reset - tap_i/U2/Z
set_lpct_pins capture_dr - tap_i/tap_ctrl_i/capturedr
set_lpct_pins shift_dr - tap_i/tap_ctrl_i/shiftdr
set_lpct_pins update_dr - tap_i/tap_ctrl_i/updatedr
set_lpct_pins test_mode - tap_i/instruction_decoder_i/edt_scan_inst
set_lpct_pins tms tms pad_instance_1_i/po_pad_tms
set_lpct_pins output_scan_en scan_en

// Run DRC
```



```
set_system_mode analysis  
  
// Insert EDT and LPCT controller logic in design  
  
write_edt_files created -replace
```

Type 3 Controller Example

This example generates a Type 3 LPCT controller and displays the associated pattern generation dofile and test procedure file generated by the tool.

Sample dofile:

```
set_context dft -edt
read_cell_library ../library/adk.tcelllib

# Read Verilog design (synthesized and scan inserted)
read_verilog ../des_chip_scan.v

# Set design top module
set_current_design des_chip
read_core_descriptions des_chip_rtl.tcd
add_core_instances -module des_chip_rtl_tessent_occ -parameter_values \
    {fast_cap_mode off}

# add scan chain definitions
dofile des_chip_scan.dofile

# Set up EDT options
set_edt_options on -location internal -bypass on
set_edt_options -input_channels 1 -output_channels 1

# Set up the LPCT controller (type 3)
set_lpct_controller on -generate_scan_enable on -tap_controller_interface off \
    -shift_control enable
set_lpct_controller on -test_mode sequence 110010010010001110 200 \
    -max_shift 5000 -max_capture 4

# Connections to/from LPCT controller.
# Primary input to attach as the continuous clock to the LPCT controller.
set_lpct_pins clock clk_st

# Scan_enable pin for design is needed as an input to Type 3 LPCT controller
set_lpct_pins output_scan_enable tmp_scan_en

# Connect lpct_capture_en output to the OCC.
set_lpct_pins capture_enable [get_pins capture_en -of_instance *_tessent_occ*inst]

# Connect lpct_shift_en output to the OCC.
set_lpct_pins shift_enable [get_pins occ_scan_en -of_instance *_tessent_occ*inst]

# Check design rules
set_system_mode analysis
report_edt_configuration -all
report_lpct_configuration
report_lpct_pins
report_edt_pins -all

# Generate and insert EDT and LPCT logic
write_edt_files des_chip -replace
exit -f
```

Sample pattern generation command sequence:

```
set_context patterns -scan

# Read scan inserted design
read_verilog des_chip_edt_top_gate.v

# read cell library
read_cell_library ../library/adk.tcelllib

# set current design
set_current_design des_chip

## NOTE: Assuming that the scan chains and the clock definitions are inaccurate
set_procedure_retargeting_options -scan on -ijtag off

# Read the TCD files for each of the instruments.
# In this case, there are two instruments - EDT and OCC
read_core_description des_chip_rtl_tessent_occ.tcd
read_core_description des_chip_des_chip_edt.tcd
read_core_description des_chip_des_chip_lpct.tcd
set_occ_module [get_modules {.*_tessent_occ(_[:digit:]]*)} -regexp]
set_lpct_module [get_modules {.*_lpct(_[:digit:]]*)} -regexp]
set_edt_module [get_modules {.*_edt(_[:digit:]]*)} -regexp]

# Associating the TCDs with each of the instances that it describes
# This automates the control of the pins and constraints required during test_setup for ATPG
# add_core_instance -module $edt_top_module
add_core_instance -module $occ_module -parameter_value {fast_cap_mode on}
add_core_instance -module $lpct_module
add_core_instance -module $edt_module
set_procfile_name ./top_level.testproc
dofile ./top_level_des_chip.dofile
set_external_capture_options -fixed 7

# Report the cores that are described
report_core_descriptions

# Check DRC rules
set_system_mode analysis

# Report DRC rules
report_drc_rules

# Add all faults
add_faults -all

# Create patterns
create_patterns

# Write patterns
write_patterns ./generated/pat.v -verilog -serial -param paramfile -replace
exit -f
```

Type 1 Controller LPCT Clock Example

This example generates a simple Type 1 controller that specifies a top-level scan clock as the LPCT clock.

```
set_context dft -edt
add_clock 0 clk //Note - there is a single clock in the design
add_scan_chains ...
set_lpct_controller on -shift_control clock
set_lpct_pin clock clk //LPCT clock is shared with scan clock
set_lpct_pin input_scan_enable scan_en
set_lpct_pin test_clock_connection test_mode_mux/B
set_system_mode analysis
report_lpct_pins
write_edt_files created -replace
```

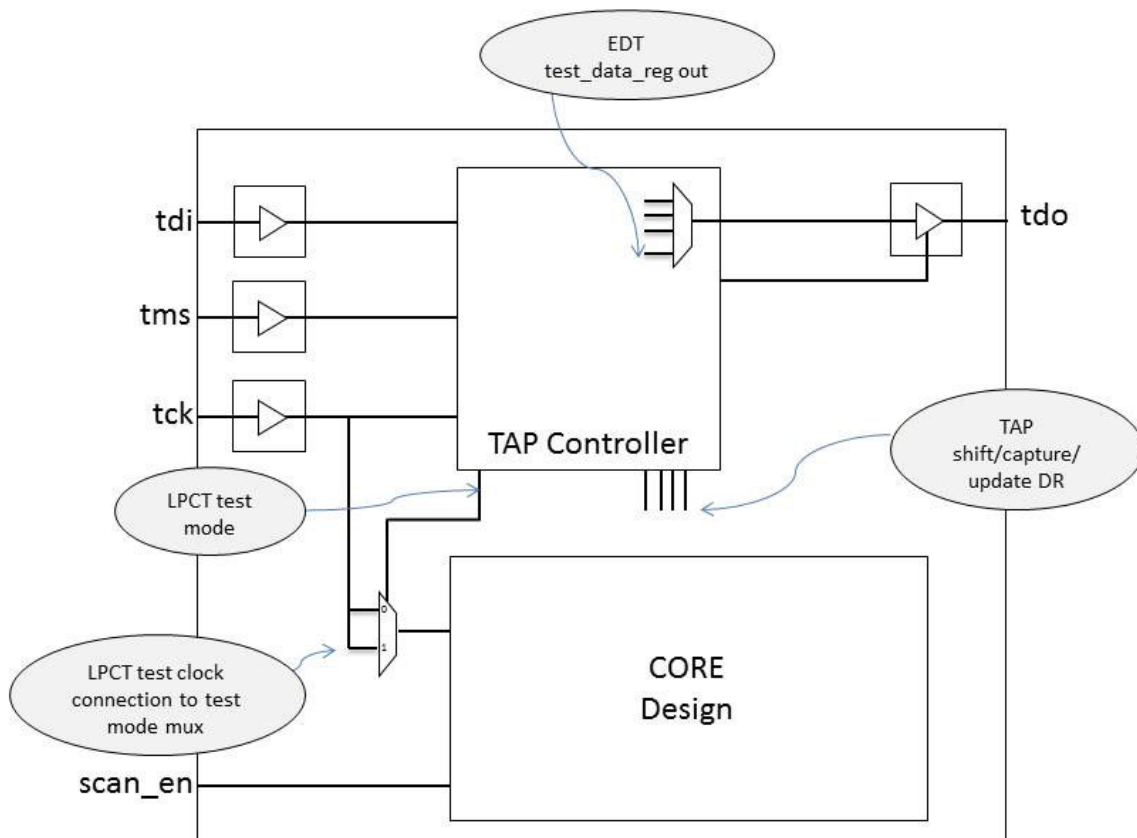
Type 2 Controller Scan Shift Clock Example

This example generates a Type 2 LPCT controller that uses tck as a scan shift clock. The test mode multiplexer, which chooses between the LPCT-generated scan clock and the functional clock, already exists in the design. The test_clock_connection pin on the mux is specified with the test_clock_connection pin type.

This example is illustrated in [Figure 8-19](#).

```
set_context dft -edt
add_clock 0 tck
add_scan_chains ...
set_lpct_controller -tap_controller_interface on
set_lpct_controller -shift_control clock
set_lpct_pins output_scan_enable scan_en
set_lpct_pins tck tck pad_tck/Z //LPCT clock is tck
set_lpct_pins tms tms pad_tms/Z
set_lpct_pins reset - tap_i/tlr
set_lpct_pins capture_dr - tap_i/capturedr
set_lpct_pins shift_dr - tap_i/shiftdr
set_lpct_pins update_dr - tap_i/updatedr
set_lpct_pins test_mode - tap_i/edt_scan_inst
set_lpct_pins test_clock_connection test_mode_mux/B
set_edt_options -channel 1
set_edt_pins input 1 tdi pad_tdi/Z
set_edt_pins output 1 tdo tap_i/tap_edt_channel_reg_in
set_system_mode analysis
report_lpct_pins
report_lpct_configuration
write_edt_files created -replace
```

Figure 8-19. Type 2 LPCT Design Example



Compression Bypass Logic

By default, bypass circuitry is included in the EDT logic. The bypass circuitry allows you to bypass the EDT logic and access uncompressed scan chains in the design core.

Bypassing the EDT logic enables you to apply uncompressed test patterns to the design to

- Debug compressed test patterns.
- Apply additional custom uncompressed scan chains.
- Apply test patterns from other ATPG tools.

Bypass logic can also be inserted in the core netlist at scan insertion time. This allows you to place the multiplexers and lockup cells required to operate the bypass mode inside the core netlist instead of the EDT logic. This option allows more effective design routing. For more information, see [“Insertion of Bypass Chains in the Netlist”](#) on page 54.

You can also set up two bypass scan chain configurations. In addition to the default configuration, you can create a second bypass configuration that concatenates all scan chains together into one bypass chain for use when hardware test channels are limited. For more information, see [“Dual Bypass Configurations”](#) on page 233.

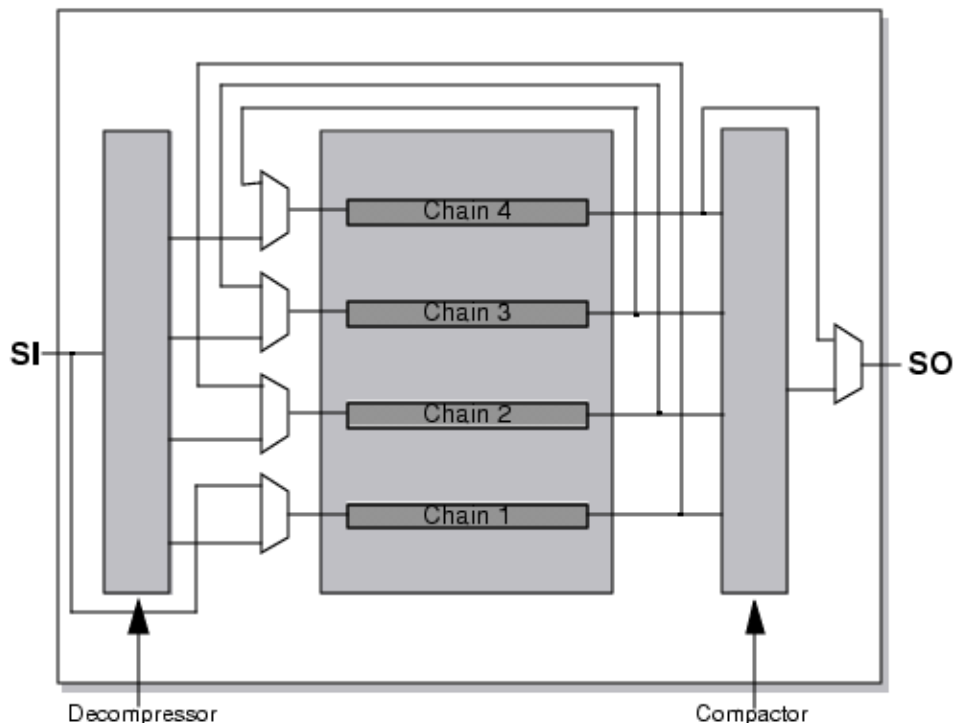
Structure of the Bypass Logic.....	230
Generating EDT Logic When Bypass Logic is Defined in the Netlist.....	231
Dual Bypass Configurations	233
Generation of Identical EDT and Bypass Test Patterns	234
Use of Bypass Patterns in Uncompressed ATPG	235
Creating Bypass Test Patterns in Uncompressed ATPG.....	238

Structure of the Bypass Logic

Because the number of core scan chains is relatively large, they are reconfigured into fewer, longer scan chains for bypass mode. For example, in a design with 100 core scan chains and four external channels, every 25 scan chains are concatenated to form one bypass chain. This bypass chain is then connected between the input and output pins of a given channel.

[Figure 8-20](#) illustrates conceptually how the bypass mode is implemented.

Figure 8-20. Bypass Mode Circuitry



Notice that the bypass logic is implemented with multiplexers. The tool includes the multiplexers and any lockup cells needed to concatenate scan chains in the EDT logic.

Note

When lockup cells are inserted as part of the bypass logic, the EDT logic requires a system clock. If the same bypass logic is placed in the netlist, the EDT logic does not require a system clock.

You can also set up the EDT clock to pulse before the scan chain shift clocks to avoid using a system clock. For more information, see the `-pulse_edt_before_shift_clocks` switch of the [set_edt_options](#) command.

The bypass circuitry is run from bypass mode in Tessent FastScan.

Generating EDT Logic When Bypass Logic is Defined in the Netlist

EDT technology supports netlists that contain two sets of pre-defined scan chains. Predefining two sets of scan chains allows you to insert both the bypass chains and the core chains into the core design with a scan-insertion tool.

Note

 Design blocks that contain bypass chains in the EDT logic and design blocks that contain bypass chains in the core can coexist in a design.

Restrictions and Limitations

- Bypass patterns cannot be created from compressed test patterns. You must generate bypass patterns from Tessent Shell. See “[Creating Bypass Test Patterns in Uncompressed ATPG](#)” on page 238.

Prerequisites

- Both bypass and core scan chains must be inserted in the design netlist. For more information, see “[Insertion of Bypass Chains in the Netlist](#)” on page 54.

Procedure

1. Invoke Tessent Shell. For example:

```
<Tessent_Tree_Path>/bin/tessent -shell
```

2. Load the design and library and set the context for EDT logic generation.

```
set_context dft -edt read_verilog my_gate_scan.v read_cell_library \
my_lib.aptg set_current_design top
```

3. Set up parameters for the EDT logic generation.

For more information, see “[Preparation for EDT Logic Creation](#)” on page 68.

4. Enable the tool to use existing bypass chains. For example:

```
set_edt_options -bypass_logic use_existing_bypass_chains
```

For more information, see the [set_edt_options](#) command.

5. Specify the number of bypass chains. For example:

```
set_edt_options -bypass_chain 2
```

For more information, see the [set_bypass_chains](#) command.

6. Specify the input and output pins for the bypass chains. For example:

```
set_bypass_chains 2 -pins scan_in2 scan_out2
```

For more information, see the [set_bypass_chains](#) command.

7. Generate the EDT logic. For more information, see “[Creation of EDT Logic Files](#)” on page 96.

Related Topics

[Synthesizing the EDT Logic](#)

[Creating Bypass Test Patterns in Uncompressed ATPG](#)

Dual Bypass Configurations

You can use the `set_edt_options -single_bypass_chain` command to output EDT logic with two bypass configurations.

The two bypass configurations are

- **Default Scan Chain Configuration** — All scan chains are evenly distributed and concatenated into scan chains equal to the number of input/output channels in the EDT logic. This configuration can also be specified with the “[set_edt_options -bypass_chains](#)” command and switch. For more information, see “[Compression Bypass Logic](#)” on page 230.
- **Single Bypass Scan Chain Configuration** — All scan chains are concatenated together to form one scan chain for bypass mode. A single bypass chain configuration can be used in test environments with hardware limitations.

When dual configurations are specified, an additional primary input `edt_single_bypass_chain` pin is created to enable and disable the single chain configuration. For more information, see “[Single Chain Bypass Logic](#)” on page 329.

An additional dofile `<design>_single_bypass_chain.dofile` is also produced to define the single top-level scan chain and force the `edt_single_bypass_chain` pin to 1.

Additional lockup cells are inserted as needed. For more information, see “[Lockups in the Bypass Circuitry](#)” on page 259.

By default only test patterns for the default configuration are saved. To save the test patterns for the single chain bypass configuration, you must use the `write_patterns edt_single_bypass_chain` command.

Note

 Single bypass chain configuration is associated with one compression block. To use this feature in a block-level architecture, you must manually integrate all single bypass chains together at the top-level.

The single bypass configuration is not included in reported test pattern statistics and scan chains. Only information about the default bypass configuration is reported.

Related Topics

[Structure of the Bypass Logic](#)

[Lockups in the Bypass Circuitry](#)

[Use of Bypass Patterns in Uncompressed ATPG](#)

[Structure of the Bypass Chains](#)

[EDT Logic Description](#)

Generation of Identical EDT and Bypass Test Patterns

The EDT technology supports the creation of uncompressed versions of each EDT pattern. The availability of uncompressed EDT patterns enables you to use uncompressed ATPG in bypass mode to directly load the scan cells with the same values that compressed ATPG loads. For debugging simulation mismatches in the core logic, it is sometimes helpful if you can apply the exact same patterns with uncompressed ATPG in bypass mode that you applied with compressed ATPG.

Note

 You can only convert EDT test patterns to uncompressed test patterns for bypass mode if the bypass scan chains are created with compressed ATPG. Otherwise, you must use uncompressed ATPG to generate bypass test patterns. See “[Creating Bypass Test Patterns in Uncompressed ATPG](#)” on page 238.

After you generate EDT patterns in the Pattern Generation phase, you can direct the tool to translate the EDT patterns into bypass mode uncompressed ATPG patterns and write the translated patterns to a file. The file format is the same as the regular uncompressed ATPG binary file format. You accomplish the translation and create the binary file by issuing the `write_patterns` command with the `-EDT_Bypass` and `-Binary` switches. For example:

```
write_patterns my_bypass_patterns.bin -binary -edt_bypass
```

You can then read the binary file into uncompressed ATPG, re-simulate the patterns in the analysis system mode to verify that the expected values computed in compressed ATPG are still valid in bypass mode, and save the patterns in any of the tool’s supported formats; WGL or Verilog for example. An example of this tool flow is provided in “[Use of Bypass Patterns in Uncompressed ATPG](#)” on page 235.

There are several reasons you cannot use EDT technology alone to create the EDT bypass patterns:

- The bypass operation requires a different set of test procedures. These are only loaded when running uncompressed ATPG and are unknown to EDT in the Pattern Generation phase.

If the bypass test procedures produce different tied cell values than the EDT test procedures, simulation mismatches can result if the EDT patterns are simply reformatted for bypass mode. An example of this would be if a boundary scan TAP controller were used to drive the EDT bypass signal. With the two sets of test procedures, the register driving the signal is forced to different values. As a result, the expected values computed for EDT would be incorrect for bypass mode.

- EDT would not have run any DRCs to ensure that the scan chains can be traced in bypass mode.

- You may need to verify that captured values do not change in bypass mode.

When it translates EDT patterns into bypass patterns, EDT changes the captured values on some scan cells to Xs to emulate effects of EDT compaction and scan chain masking. For example, if two scan cells are XOR'd together in the compactor and one of them had captured an X, the tool sets the captured value of the other to X so no fault can be detected on those cells, incorrectly credited, then lost during compaction.

Similarly, if a scan chain is masked for a given pattern, the tool sets captured values on all scan cells in that chain to X. When translating the EDT patterns, the tool preserves those Xs so the two pattern sets are identical. While this can lower the “observability” possible with the bypass patterns, it emulates EDT test conditions. For more information on how EDT uses masking, refer to “[Understanding Scan Chain Masking in the Compactor](#)” on page 282.

Chain Test Pattern Handling for Bypass Operation

The EDT technology saves only the translated EDT scan patterns in the binary file. The enhanced *chain + EDT logic* test patterns are not saved. The purpose of the enhanced test patterns is to verify the operation of the EDT logic as well as the scan chains. Because no shifting occurs through the EDT logic when it is bypassed, regular chain test patterns are sufficient to verify the scan chains work in bypass configuration; the regular chain test patterns are appended to the compressed test pattern set when you write out the bypass patterns.

Note



Because the EDT pattern set contains the enhanced test patterns and the bypass pattern set does not, the number of patterns in the EDT and bypass pattern sets are different.

You can use the bypass test patterns with uncompressed ATPG to debug problems in the core design and scan chains but not in the EDT logic. If the enhanced tests fail in compressed ATPG and the bypass chain test passes in uncompressed ATPG, the problem is probably in the EDT logic or the interface between the EDT logic and the scan chains.

Use of Bypass Patterns in Uncompressed ATPG

After you save the bypass patterns, invoke Tessent Shell, read the design, and use the dofile and test procedure file generated when the EDT logic is created. You then read into Tessent Shell the binary pattern file you previously saved from compressed ATPG. You should re-simulate the patterns in the analysis system mode to verify that the expected values computed with compressed ATPG are still valid in bypass mode. Then save the patterns in any of the tool's supported formats, WGL or Verilog for example.

Bypass Pattern Flow Example

The following example demonstrates how to use bypass patterns in ATPG mode.

Note

 The following steps assume that, as part of a normal flow, you already have run Tessent Shell to create the EDT logic, followed by Design Compiler to synthesize it. You must complete both steps in order to run Tessent Shell with uncompressed ATPG in bypass mode. The bypass dofile and the bypass test procedure file generated by compressed ATPG are required by uncompressed ATPG in order to correctly apply a bypass pattern set.

In the compressed ATPG Pattern Generation phase, issue a
“write_patterns -binary -edt_bypass” command to write bypass patterns. For example:

```
write_patterns my_bypass_patterns.bin -binary -edt_bypass
```

Notice that the -Binary and the -Edt_bypass switches are both required in order to write bypass patterns.

Tessent Shell Setup in Uncompressed ATPG

Invoke Tessent Shell in setup mode and invoke the bypass dofile generated by compressed ATPG. Place the design in the same state in uncompressed ATPG that you used in compressed ATPG, then run DRC.

Note

 Placing the design in the same state in uncompressed ATPG as in compression ATPG ensures the expected test values in the bypass patterns remain valid when the design is configured for bypass operation.

The following example uses the bypass dofile, created_bypass.dofile, described in “[Creation of EDT Logic Files](#)” on page 96:

```
dofile created_bypass.dofile  
set_system_mode analysis
```

Verify that no DRC violations occurred.

Processing of the Bypass Patterns

To simulate the bypass patterns and verify the expected values, you can enter commands similar to the following:

```
read_patterns my_bypass_patterns.bin  
report_failures -pdet
```

Note

 The expected values in the binary pattern file mirror those with which compressed ATPG observes EDT patterns. Therefore, if compressed ATPG cannot observe a scan cell (for example, due to scan chain masking or compaction with a scan cell capturing an X), the expected value of the cell is set to X even if it can be observed by uncompressed ATPG in bypass mode.

Saving of the Patterns with Compressed ATPG Observability

To save the patterns in another format using the expected values in the binary pattern file, issue the `write_patterns` command with the `-External` switch. For example, to save ASCII patterns:

```
read_patterns my_bypass_patterns.bin  
write_patterns my_bypass_patterns.ascii -external
```

Saving of the Patterns with Uncompressed ATPG Observability

Alternatively, you can save expected values based on what is observable by uncompressed ATPG when the design is in bypass operation. Some scan cells which had X expected values in compressed ATPG, due to scan chain masking or compaction with an X in another scan cell, may be observed by uncompressed ATPG. To write `write_patterns` where the expected values reflect uncompressed ATPG observability, first simulate the patterns as follows:

```
set_system_mode analysis  
read_patterns my_bypass_patterns.bin  
simulate_patterns -store_patterns all
```

Note

 The preceding command sequence will cause the Xs that emulate the effect of compaction in EDT to disappear from the expected values. The resultant bypass patterns will no longer be equivalent to the EDT patterns; only the stimuli will be identical in the two pattern sets. For a given EDT pattern, therefore, the corresponding bypass pattern will no longer provide test conditions identical to what the EDT pattern provided in compressed ATPG.

Using the `-Store_patterns` switch in analysis system mode when specifying the external file as the pattern source causes uncompressed ATPG to place the simulated patterns in the tool's internal pattern set. The simulated patterns include the load values read from the external pattern source and the expected values based on simulation.

Note

 If you fault simulate the patterns loaded into uncompressed ATPG, the test coverage reported may be slightly higher than it actually is in compressed ATPG. This is because uncompressed ATPG recomputes the expected values during fault simulation rather than using the values in the external pattern file. The recomputed values do not reflect the effect of the compactors and scan chain masking that are unique to EDT. Therefore, there likely will be fewer Xs in the recomputed values, resulting in the higher coverage number.

When you subsequently save these patterns, take care not to use the `-External` switch with the `write_patterns` command. The `-External` switch saves the current external pattern set rather than the internal pattern set containing the simulated expected values. The following example saves the simulated expected values in the internal pattern set to the file, `my_bypass_patterns.ascii`:

```
write_patterns my_bypass_patterns.ascii
```

Creating Bypass Test Patterns in Uncompressed ATPG

Generate test patterns for the bypass chains located in your netlist or in the EDT logic.

Prerequisites

- If a signal other than the `edt_bypass` signal is used for the mux select that enables the bypass chains, the test procedure file for the bypass chains must be modified to allow bypass chains to be traced.
- EDT logic must be created and synthesized into your netlist and test procedure files generated by compressed ATPG are available.

Procedure

1. Invoke Tessent Shell. The setup prompt displays.

```
<Tessent_Tree_Path>/bin/tessent -shell -logfile \  
../transcripts/edt_pattern_gen.log -replace
```

2. Set the context, read in the design library, read and set current design.

```
SETUP> set_context patterns -scan  
SETUP> read_verilog created_edt_top_gate.v  
SETUP> read_cell_library atpg.lib  
SETUP> set_current_design top
```

3. If a Tessent Shell Database (TSDB) is not available, read the TCD file for EDT IP using the `read_core_description` command. For example:

```
SETUP> read_core_description created_cpu_edt.tcd
```

If a TSDB is available, the tool will automatically read the TCD for the EDT logic.

4. Define the parameter values to automatically configure the EDT logic using the `add_core_instances` command. In this case, you want to create bypass patterns, so use the “`edt_bypass on`” switch. For example

```
SETUP> add_core_instances -core cpu_edt \  
-parameter_values edt_bypass on
```

5. Add top-level clocks driving the scan changes using the `add_clocks` command.

```
add_clocks 0 clk1 clk2 clk3 clk4 ramclk
```

6. Provide the top-level test procedure file using the `set_procfile_name` command.

```
SETUP> set_procfile_name created_cpu_edt.testproc
```

7. Change to analysis system mode, which runs DRC.

```
SETUP> set_system_mode analysis
```

8. Check for and debug any DRC violations.

9. Create uncompressed ATPG patterns as you would for a design without EDT. For example:

```
add_faults /my_core  
create_patterns  
report_scan_volume
```

Be sure to add faults only on the core of the design (assumed to be “/my_core” in this example) and disregard the EDT logic.

The [report_scan_volume](#) command provides information for analyzing pattern data and achieved compression.

Uncompressed ATPG patterns that utilize the bypass circuitry are generated.

10. Save the results to the TSDB.

```
ANALYSIS> write_tsdb_data -replace
```

This command writes the results of the TSDB, including the flat model, the TCD, the PatDB pattern file, and the fault list. This information is useful for pattern retargeting and diagnosis.

11. Save the patterns in parallel and serial Verilog format.

```
ANALYSIS> write_patterns ../generated/patterns_edt_p.v -verilog -replace -parallel
```

```
ANALYSIS> set_pattern_filtering -sample 2
```

```
ANALYSIS> write_patterns ../generated/patterns_edt_s.v -verilog -replace -serial
```

12. Save the patterns in tester format. For example WGL.

```
ANALYSIS> write_patterns ../generated/test_patterns.wgl -wgl -replace
```

Related Topics

[Use of Bypass Patterns in Uncompressed ATPG](#)

[Test Pattern Generation](#)

[Preparation for Test Pattern Generation](#)

[Simulation of the Generated Test Patterns](#)

Uncompressed ATPG (External Flow) and Boundary Scan

When using boundary scan with the external compressed pattern flow, you can use any tool to insert boundary scan. There are two approaches that are typically used to configure the TAP controller when you insert boundary scan.

- Drive the minimal amount of the EDT control circuitry with the TAP controller, so the boundary scan simply coexists with EDT.
- Drive the EDT logic clock, update, and bypass signals with the TAP controller.

Note

 As mentioned previously, boundary scan cells must not be present in your design before you add the EDT logic. This requirement also applies to I/O pads. You must enable compressed ATPG to create the EDT logic as a wrapper around your core design.

For more information on the overall external compressed pattern flow, see “[Compressed Pattern External Flow](#)” on page 45.

Boundary Scan Coexisting with EDT Logic	240
Drive Compressed ATPG with the TAP Controller.....	245

Boundary Scan Coexisting with EDT Logic

This section describes how EDT logic can coexist with boundary scan and provides a flow reference for this methodology. This approach enables the EDT logic to be controlled by primary input pins and not by the boundary scan circuitry. In test mode, the boundary scan circuitry just needs to be reset. Also, all PIs and POs are directly accessible.

1. [Preparation for Synthesis of Boundary Scan and EDT Logic](#)
2. [Modification of the Dofile and Procedure File for Boundary Scan](#)

Preparation for Synthesis of Boundary Scan and EDT Logic

Prior to synthesizing the EDT logic and boundary scan circuitry, you should ensure any scripts used for synthesis include the boundary scan circuitry. For example, the Design Compiler synthesis script that compressed ATPG generates needs the following modifications (shown in bold font) to ensure the boundary scan circuitry is synthesized along with the EDT logic:

Note

 The modifications are to the example script shown in “[Design Compiler Synthesis Script External Flow](#)” on page 103.

```

/*****
**  Synopsys Design Compiler synthesis script for created_edt_bs_top.v
**
*****/

/* Read input design files */
read -f verilog created_core_blackbox.v
read -f verilog created_edt.v
read -f verilog created_edt_top.v
read -f verilog edt_top_bscan.v
    /*ADDED*/

current_design edt_top_bscan          /*MODIFIED*/

/* Check design for inconsistencies */
check_design

/* Timing specification */
create_clock -period 10 -waveform {0,5} edt_clock
create_clock -period 10 -waveform
{0,5} tck /*ADDED*/

/* Avoid clock buffering during synthesis. However, remember */
/* to perform clock tree synthesis later for edt_clock */
set_clock_transition 0.0 edt_clock
set_dont_touch_network edt_clock
set_clock_transition 0.0 tck
    /*ADDED*/
set_dont_touch_network tck
    /*ADDED*/

/* Avoid assign statements in the synthesized netlist.
set_fix_multiple_port_nets -feedthroughs -outputs -buffer_constants

/* Compile design */
uniquify
set_dont_touch cpu
compile -map_effort medium

/* Report design results for EDT logic */
report_area > created_dc_script_report.out
report_constraint -all_violators -verbose >>
    created_dc_script_report.out
report_timing -path full -delay max >> created_dc_script_report.out
report_reference >> created_dc_script_report.out

/* Remove top-level module */
remove_design cpu

/* Read in the original core netlist */
read -f verilog gate_scan.v
current_design edt_top_bscan
    /*MODIFIED*/
link

/* Write output netlist using a
new file name*/
write -f verilog -hierarchy -o created_edt_bs_top_gate.v /*MODIFIED*/

```

After you have made any required modifications to the synthesis script to support boundary scan, you are ready to synthesize the design—see “[The EDT Logic Synthesis Script](#)” on page 121.

Modification of the Dofile and Procedure File for Boundary Scan

To correctly operate boundary scan circuitry, you need to edit the dofile and test procedure file created by compressed ATPG.

Note



The information in this section applies only when the design includes boundary scan.

Typical changes include:

- The internal scan chains are one level deeper in the hierarchy because of the additional level added by the boundary scan wrapper. This needs to be taken into consideration for the [add_scan_chains](#) command.
- The boundary scan circuitry needs to be initialized. This typically requires you to revise both the dofile and test procedure file.
- You may need to make additional changes if you drive compressed ATPG signals with the TAP controller.

In the simplest configuration, the EDT logic is controlled by primary input pins, not by the boundary scan circuitry. In test mode, the boundary scan circuitry just needs to be reset.

Following is the same dofile shown in “[EDT IP Generation Dofiles](#)” on page 370, except now it includes the changes (shown in bold font) necessary to support boundary scan when configured simply to coexist with EDT logic. The boundary scan circuitry is assumed to include a TRST asynchronous reset for the TAP controller.

```
add_scan_groups grp1 modified_edt.testproc

add_scan_chains -internal chain1 grp1 /core_i/cpu_i/edt_si1
/core_i/cpu_i/edt_so1
add_scan_chains -internal chain2 grp1 /core_i/cpu_i/edt_si2
/core_i/cpu_i/edt_so2
add_scan_chains -internal chain3 grp1 /core_i/cpu_i/edt_si3
/core_i/cpu_i/edt_so3
add_scan_chains -internal chain4 grp1 /core_i/cpu_i/edt_si4
/core_i/cpu_i/edt_so4
add_scan_chains -internal chain5 grp1 /core_i/cpu_i/edt_si5
/core_i/cpu_i/edt_so5
add_scan_chains -internal chain6 grp1 /core_i/cpu_i/edt_si6
/core_i/cpu_i/edt_so6
add_scan_chains -internal chain7 grp1 /core_i/cpu_i/edt_si7
/core_i/cpu_i/edt_so7
add_scan_chains -internal chain8 grp1 /core_i/cpu_i/edt_si8
/core_i/cpu_i/edt_so8

add_clocks 0 clk
add_clocks 0 edt_clock

add_input_constraints tms -C1

add_write_controls 0 ramclk

add_read_controls 0 ramclk

add_input_constraints edt_clock -C0

set_edt_options -channels 1 -ip_version 1
```

The test procedure file, `created_edt.testproc`, shown in “[EDT IP Generation Dofiles](#)” on page 370, must also be changed to accommodate boundary scan circuitry that you configure to simply coexist with EDT logic. Here is that file again, but with example changes for boundary scan added (in bold font). This modified file was saved with the new name `modified_edt.testproc`, the name referenced in the fifth line of the preceding dofile.

```
set time scale 1.000000 ns ;
set strobe_window time 100 ;

timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 100 ;
    pulse clk 200 100;
    pulse edt_clock 200 100;
    pulse ramclk 200 100;
    period 400 ;
end;

procedure capture =
    timeplate gen_tp1 ;
    cycle =
        force_pi ;
        measure_po ;
        pulse_capture_clock ;
    end;
end;

procedure shift =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force_sci ;
        force edt_update 0 ;
        measure_sco ;
        pulse clk ;
        pulse edt_clock ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force clk 0 ;
        force edt_bypass 0 ;
        force edt_clock 0 ;
        force edt_update 1 ;
        force ramclk 0 ;
        force scan_en 1 ;
        pulse edt_clock ;
    end ;
    apply shift 26;
end;

procedure test_setup =
    timeplate gen_tp1 ;
    cycle =
        force edt_clock 0 ;
        ...
        force tms 1;
        force tck 0;
        force trst 0;
    end;
    cycle =
```

```
        force trst 1;  
    end;  
end;
```

Drive Compressed ATPG with the TAP Controller

You can drive one or more compressed ATPG signals from the TAP controller; however, there are a few more requirements and restrictions than in the simplest case where the boundary scan just coexists with EDT logic.

Some of these requirements and restrictions apply when you set up the boundary scan circuitry, others when you generate patterns:

- If you want to completely drive the EDT logic from the TAP controller, you first should decide on an instruction to drive the EDT channels.
- To ensure the TAP controller stays in the proper state for shift as well as capture during EDT pattern generation, you should specify TCK as the capture clock. This requires a “set_capture_clock TCK -atpg” command in the EDT dofile that causes the capture clock TCK to be pulsed only once during the capture cycle.
- Also, the TAP controller must step through the Exit1-DR, Update-DR, and Select-DR-Scan states to go from the Shift-DR state to the Capture-DR state. This requires three intervening TCK pulses between the pulse corresponding to the last shift and the capture. These three pulses need to be suppressed for the clock supplied to the core.
- The EDT update signal is usually asserted during the first cycle of the load/unload procedure, so as not to restrict clocking in the capture window. Typically, the EDT clock must be in its off state in the capture window. Because there is already a restriction in the capture window due to the “set_capture_clock TCK -atpg” command, you can supply the EDT clock from the same waveform as the core clock without adding any more constraints. To update the EDT logic, the EDT update signal must now be asserted in the capture window. You can use the Capture-DR signal from the TAP controller to drive the EDT update signal.
- You should also modify any synthesis scripts to include the boundary scan circuitry. For an example of a Design Compiler script with the necessary changes, see “[Preparation for Synthesis of Boundary Scan and EDT Logic](#)” on page 240.

Use of Pipeline Stages in the Compactor

Pipeline stages can sometimes improve the overall rate of data transfer through the logic in the compactor by increasing the scan shift frequencies. Pipeline stages are flip-flops that hold intermediate values output by a logic level so that values entering that logic level can be updated earlier in a clock cycle. Because the EDT logic is relatively shallow, most designs do need compactor pipeline stages to attain the desired shift frequency. The limiting factors on shift frequencies are usually the performance of the scan chains and power considerations.

You can enable the addition of pipeline stages in the compactor with the [set_edt_options](#) -pipeline_logic_levels_in_compactor command when creating the EDT logic. Pipeline stages added to the compactor use the EDT clock and lockup cells as described in “[Lockups Between Scan Chain Outputs and Compactor](#)” on page 258.

Note



The -pipeline_logic_levels_in_compactor switch specifies the maximum number of combinational logic levels (XOR gates) between compactor pipeline stages, not the number of pipeline stages. The number of logic levels between any two pipeline stages controls the propagation delay between pipeline stages.

Use of Pipeline Stages Between Pads and Channel Inputs or Outputs

When the signal propagation delay between a pad and the corresponding channel input or output is excessive, you may want to add pipeline stages. Use the guidelines provided in this section to add pipeline stages between a top-level channel input pin/pad and the corresponding decompressor input, or between a compactor output and the corresponding channel output pin/pad. The number of pipeline stages on each input/output channel can vary.

Typically, pipeline stages are inserted throughout the design during top-level design integration. Pipeline stages are generally not placed within the EDT logic.

Note

 You must use the `set_edt_pins -pipeline_stages` command during test pattern generation to enable channel pipeline stages. You must also modify the associated test procedure file as described in this section.

Lockups Between Channel Outputs and Output Pipeline Stages

Channel Output Pipelining	247
Channel Input Pipelining	248
Clocks for Channel Input Pipeline Stages	249
Clocks for Channel Output Pipeline Stages	249
Input Channel Pipelines Must Hold Their Value During Capture	250
DRC for Channel Input Pipelining	251
DRC for Channel Output Pipelining	251
Input/Output Pipeline Examples	251

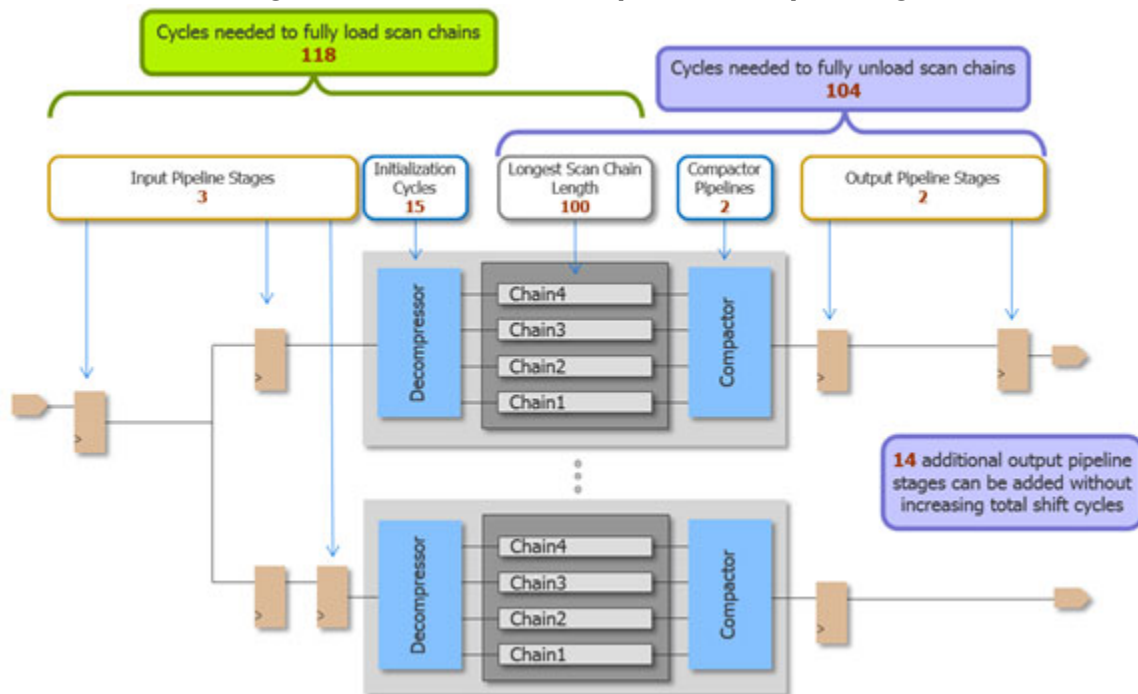
Channel Output Pipelining

To support channel output pipelines, the tool ensures there are enough shift cycles per pattern to flush out the pipeline and observe all scan chains.

The number of cycles needed to fully load the scan chains determines the limit of how many output pipeline stages can be added without increasing the number of shift cycles.

Figure 8-21 illustrates the factors that determine the number of pipeline stages that can be added without any shift penalty; the number of stages depends on the number of decompressor initialization cycles and the number of input pipeline stages.

Figure 8-21. Channel Outputs and Pipelining



In the figure, 118 cycles are required to fully load the scan chains because the data has to shift from the primary channel input (on the left) through the last scan cell in the longest scan chain. Only 104 cycles are needed to fully unload the scan chains from the first scan cell in the chains to the primary channel output. Since 118 cycles are needed to fully load and unload the scan chains, up to 14 additional output pipeline stages can be added without any shift penalty.

Channel Input Pipelining

While the contents of the channel output pipeline stages at the beginning of shifting each pattern are irrelevant since they will be flushed out, the contents of the channel input pipeline stages do matter because they will go to the decompressor when shifting begins (just after the decompressor is initialized in the load_unload procedure).

The tool adds an additional test pattern before every test pattern set. This test pattern initializes the channel input pipelining stages before the load of the very first real test pattern.

The number of additional shift cycles is typically incremented by the number of channel input pipeline stages. If the number of additional shift cycles is four without input pipelining, and the channel input with the most pipeline stages has two stages, the number of additional shift cycles in each test pattern is incremented to six.

If you have a choice between using either input or output pipeline stages, you should choose output stages for the following reasons:

- The number of shift cycles for the same number of pipeline stages is higher when the pipeline stages are on the input side.
- You must ensure that input channel pipelines hold their value during the capture cycle. For information on how to do this, see [“Input Channel Pipelines Must Hold Their Value During Capture”](#) on page 250.

Clocks for Channel Input Pipeline Stages

If you use channel input pipelining, you must ensure there is no clock skew between the channel input pipeline and the decompressor. If you use channel output pipelining, you must ensure there is no clock skew between the compactor (if you also use compactor pipelining) and the channel output pipeline, or between the scan chain outputs (if no compactor pipelining is used) and the channel output pipeline.

On the input side, the pipeline stages are connected to the decompressor, which is clocked by the leading edge of the EDT clock. If the channel input pipeline is not clocked by the EDT clock, a lockup cell must be inserted between the pipeline and the decompressor.

Note

 EDT patterns saved for application through bypass mode ([write_patterns](#) -edt_bypass) may not work correctly if the first cell of a chain, driven by channel input pipeline stages in bypass mode, captures on the trailing edge of the clock. This is because that first cell of the chain, which is normally a master, becomes a copy of the last input pipeline stage in bypass mode. To resolve this, you must add a lockup cell that is clocked on the trailing edge of a shift clock at the end of the pipeline stages for a particular channel input. This ensures that the first cell in the scan chain remains a master.

Clocks for Channel Output Pipeline Stages

On the output side, the last state element driving the channel output is either a compactor pipeline stage clocked by the EDT clock or the last elements of the scan chains when the compactor has no pipelining. In addition to ensuring no clock skew between the chains/ compactor and the pipeline stages, you must ensure that the first pipeline stages capture on the leading edge (LE) when no compactor pipelining is used. This is because if the last scan cell in a chain captures on the LE and the path from the last scan cell to the channel pipeline is combinational, and the channel pipeline stage captures on the trailing edge (TE), the pipeline stage is essentially a copy during shift and the last scan cell no longer gets observed.

To ensure there is no clock skew between the pipeline stages and the compactor outputs, you can use the `set_edt_pins -change_edge_on_compactor_output` command to specify whether compactor output data changes on the LE or TE of the EDT clock. For example, specify the

compactor output changes at the trailing edge of the clock before feeding LE pipeline stages. Depending on your application, compressed ATPG automatically inserts lockup cells and output channel pipeline stages as needed. For more information, see [set_edt_pins](#) in the *Tessent Shell Reference Manual*.

If you use pipeline stages clocked with the rising edge of the `edt_clock`, the tool inserts lockup cells in the IP Creation phase to balance clock skew on the output side pipeline registers. For more information, see “[Lockups in the Bypass Circuitry](#)” on page 259.

Note

If the clock used for the pipeline stages is not a shift clock, it must be pulsed in the shift procedure.

Input Channel Pipelines Must Hold Their Value During Capture

The tool adds an additional test pattern before every test pattern set to initialize channel input pipelining stages before the load of the first test pattern. You must ensure that the values that get shifted into the input pipeline stages at the *end* of shift (for every pattern) are not changed during capture.

As mentioned earlier in “[Channel Input Pipelining](#)” on page 248, following the initialization pattern, the tool ensures that every generated pattern has sufficient trailing zeros (ones for channels with pad inversion) to set the pipeline stages to zeros/ones after every pattern is shifted in.

You can ensure this in one of the following ways:

- Constrain the clock used for the pipeline stages off.
- Constrain the channel input pin to 0 (or 1 in case of channel inversion).

Note

During scan pattern retargeting or when EDT Mapping or EDT Finder is enabled (EDT Finder is enabled by default), TestKompress automatically adds proper constraints to input channels if pipelines are detected and their clocks are not constrained off during capture. For more information on EDT mapping and EDT Finder, see [set_edt_mapping](#) and [set_edt_finder](#) in the *Tessent Shell Reference Manual*. For more information on scan pattern retargeting, see “[Scan Pattern Retargeting](#)” in the *Tessent Scan and ATPG User’s Manual*.

Because the EDT clock is already constrained during the capture cycle, and drives the decompressor (no clock skew), using the EDT clock to control the input pipeline stages is recommended.

Note

 If the pipeline stages use the EDT clock, the channel pins must be forced to zero (or one if there is channel inversion) in load_unload as well, since the EDT clock is pulsed there as well (to reset the decompressor and update the mask logic). TestKompress will automatically add the needed force statements in the load_unload procedure if they are not already added by the user.

DRC for Channel Input Pipelining

The K19 and K22 design rules detect errors in initializing the channel input pipeline stages. If the pipeline is not correctly initialized for the first pattern, K19 reports mismatches on the EDT block channel inputs - assuming the hierarchy is not dissolved and the EDT logic is identified. If the EDT logic channel inputs cannot be located, for example because the design hierarchy was dissolved, K19 reports that Xs are shifted out of the decompressor. On the EDT logic channel inputs, the simulated values would mismatch within the first values shifted out, while the rest of the bits subsequently applied would match.

If the pipeline is correctly initialized for the first pattern and K19 passes, but the pipeline contents change (during capture or the following load_unload prior to shift) such that it no longer contains zeros, K22 fails. K19 and K22 detect these cases if input channel pipelining is defined and issue warnings about the possible problems related to channel pipelining.

DRC for Channel Output Pipelining

The K20 rule check considers channel output pipelining, in addition to any compactor pipelining that may exist. K20 reports any discrepancy between the number of identified and specified pipeline stages between the scan chains and pins (including compactor and channel output pipelines).

If the first stage of the channel output pipeline is TE instead of LE, this will result in one less cycle of delay than expected, which will also trigger a K20 violation. If the first stage is TE, and you specify one less pipeline stage, those two errors may mask each other and no violation may be reported. However, this may result in mismatches during serial pattern simulation.

Input/Output Pipeline Examples

These pipeline examples demonstrate an input with two pipeline stages and an output with one pipeline stage and the modified load_unload procedure with the user-supplied events to support pipelining.

The following command defines two pipeline stages for input channel 1:

```
set_edt_pins input_channel 1 -pipeline_stages 2
```

This example sets the EDT context to core1 (EDT context is specific to modular compressed ATPG and is explained in “[Modular Compressed ATPG](#)” on page 159), and then specifies that all output channels of the core1 block have one pipeline stage:

```
set_current_edt_block core1  
set_edt_pins output_channel -pipeline_stages 1
```

Following is the modified load_unload procedure for a design with two channels having input pipelining; edt_channel1 has inversion and edt_channel2 does not. The input pipeline stages are clocked by the EDT clock, edt_clock. The user-added events that support pipelining are shown in bold and comments are shown in italics.

```
procedure load_unload =  
  scan_group grp1 ;  
  timeplate gen_tpl ;  
  cycle =  
    // To ensure the values shifted into the input pipeline stages at  
    // the end of shift are not changed during capture, you must force  
    // channel pins with pipelines to zero (or one if there is channel  
    // inversion) since edt_clock is pulsed in load_unload and is  
    // also used for the pipeline stages.  
    force edt_channel1 1 ;  
    force edt_channel2 0 ;  
    force system_clk 0 ;  
    force edt_bypass 0 ;  
    force edt_clock 0 ;  
    force edt_update 1 ;  
    force ramclk 0 ;  
    force scan_en 1 ;  
    pulse edt_clock ;  
  end;  
  apply shift 21 ;  
end;
```

Change Edge Behavior in Bypass and EDT Modes

The output side compaction logic combines the scan outputs of multiple internal scan chains into an EDT channel output. In the general case, the last scan cell of scan chains may be clocked by different clocks and edges. Tessent TestKompress can add logic to ensure a uniform change edge at the compactor output. By default, the tool uses the trailing edge (TE) as the change edge for both bypass mode (multi- and single-mode bypass chains) and EDT mode (compactor output).

Note



The default TE change edge is optimal for channel pipelining. If you choose to change the default to the leading edge or any edge, ensure that no channel pipeline stages will be added later in the flow as this could cause timing issues.

In bypass mode, the tool adds a retiming cell at the end of every bypass chain as needed to ensure the same edge as EDT mode. Similarly, the tool also ensures that the default capture edge (for the first cell) for every bypass chain is changed to LE. That is, the tool adds an LE retiming flop at the beginning of every bypass chain, as needed. The added bypass chain mode input capture and output change edge cell will be clocked by the same clock driving the first or last scan cell, respectively. This clock waveform may not be aligned with the EDT clock waveform.

In EDT mode, the default TE compactor change edge is not suitable for the following situations:

- The channel output pipeline register is TE and clocked by system clock. In this case, there may be clock skew issues between the compactor change edge register clocked by `edt_clock` and the channel pipeline register clocked by system clock.

To avoid this case, change the channel output pipeline register clock to LE.

- When the design has a JTAG controller and TDO output is used as a channel output. In this case, there may be clock skew issues between compactor change edge register clocked by `edt_clock` and TDO change TE register clocked by `tck`.

To avoid this case, specify change edge leading for this channel output. Assuming the first channel output is TDO, you can specify this with the following commands:

```
set_edt_pins output_channel -change_edge_at_compactor_output leading \  
for {set channel 2} {$channel <= $n_channels} {incr channel} \  
{ set_edt_pins output_channel $channel \  
-change_edge_at_compactor_output trailing}
```

Understanding Lockup Cells

The tool analyzes the timing relationships of the clocks that control the sequential elements between the scan chains and the EDT logic and inserts edge-triggered flip-flops (lockup cells) when necessary to synchronize the clocks and ensure data integrity.

You can use the [report_edt_lockup_cells](#) command to display a detailed report of the lockup cells the tool has inserted.

Lockup Cell Insertion 254

**Lockup Cell Analysis For Bypass Lockup Cells Not Included as Part of the EDT Chains
256**

Lockup Cell Analysis For Bypass Lockup Cells Included as Part of the EDT Chains . 264

Lockups Between Channel Outputs and Output Pipeline Stages 272

Lockup Cell Insertion

The tool analyzes the relationship between the clock that controls each sequential element sourcing data (source clock) and the clock that controls the sequential element receiving the data (destination clock).

The tool inserts a lockup cell when the source and destination clocks overlap as follows:

- Both clocks have identical waveform timing within a tester cycle; clocks are *on* at the same time and their edges are aligned.
- The active edge of the destination clock occurs later in the cycle than the active edge of the source clock.

When clocks are non-overlapping, data is protected by the timing sequence and no lockup cells are inserted.

Note



Partially overlapping clocks are not supported.

You can set up the EDT logic clock and scan chain shift clocks to be non-overlapping by pulsing the EDT clock before the shift clock of each scan chain. When the EDT logic is set up in this manner, there is no need for lockup cells between the EDT logic and scan chains. However, a lockup cell driven by the EDT clock is still inserted between all bypass scan chains. For more information, see “[Pulse EDT Clock Before Scan Shift Clocks](#)” on page 81.

If your design contains a mix of overlapping and non-overlapping clocking, or the shift clocks are pulsed before the EDT logic clock, you must let the tool analyze the design and insert lockup cells (default behavior), as described in “[Lockup Cell Analysis For Bypass Lockup Cells](#)”

[Not Included as Part of the EDT Chains](#)” on page 256 and [“Lockup Cell Analysis For Bypass Lockup Cells Included as Part of the EDT Chains](#)” on page 264.

Lockup Cell Analysis For Bypass Lockup Cells Not Included as Part of the EDT Chains

Lockup cell analysis is performed for bypass lockup cells that are not included as part of the EDT chains. This happens for lockups between decompressor and scan chain inputs, lockups between scan chain outputs and the compactor, and lockups in the bypass circuitry.

Lockups Between Decompressor and Scan Chain Inputs	256
Lockups Between Scan Chain Outputs and Compactor	258
Lockups in the Bypass Circuitry	259

Lockups Between Decompressor and Scan Chain Inputs

The decompressor is located between the scan channel input pins and the scan chain inputs. It contains sequential circuitry clocked by the EDT clock. As the off state of the EDT clock (at the EDT logic module port) is always 0, leading edge triggered (LE) flip-flops are used in this sequential circuitry. Scan chain clocking does not utilize the EDT clock. Therefore, there is a possibility of clock skew between the decompressor and the scan chain inputs.

For each scan chain, the tool analyzes the clock timing of the last sequential element in the decompressor stage (source) and the first active sequential element in the scan chain (destination).

Note

The first sequential element in the scan chain could be an existing lockup cell (a transparent latch for example) and may not be part of the first scan cell in the chain.

The tool analyzes the need for lockup cells on the basis of the waveform edge timing (change edge and capture edge, respectively) of the source and destination clocks. The change edge is typically the first time at which the data on the source scan cell's output may update. The capture edge is the capturing transition at which data is latched on the destination scan cell's output. The tool inserts lockup cells between the decompressor and scan chains based on the following rules:

- A lockup cell is inserted when a source cell's change edge coincides with the destination cell's capture edge.
- A lockup cell is inserted when the change edge of the source cell precedes the capture edge of the destination cell.

In addition, the tool attempts to place lockup cells in a way that introduces no additional delay between the decompressor and the scan chains and tries to minimize the number of lockup cells at the input side of the scan chains. The lockup cells are driven by the EDT clock to reduce routing of the system clocks from the core to the EDT logic.

[Table 8-4](#) summarizes the relationships and the lockup cells the tool inserts on the basis of the preceding rules, assuming there is no pre-existing lockup cell (transparent latch) between the decompressor and the first scan cell in each chain.

Table 8-4. Lockup Cells Between Decompressor and Scan Chain Inputs

Clock Waveforms	Source Clock	Dest. Clock	Source¹ Change Edge	Dest.^{1, 2} Capture Edge	# Lockups Inserted	Lockup³ Edge(s)
Overlapping	EDT clock	Scan clock	LE	LE	1	TE
	EDT clock	Scan clock	LE	TE	2	TE, LE
	EDT clock	Scan clock	LE	active high (TE)	2	TE, LE
	EDT clock	Scan clock	LE	active low (LE)	1	TE
Non-Overlapping ⁴	EDT clock	Scan clock	LE	LE	2	TE, LE
	EDT clock	Scan clock	LE	TE	2	TE, LE
	EDT clock	Scan clock	LE	active high (TE)	2	TE, LE
	EDT clock	Scan clock	LE	active low (LE)	2	TE, LE

1. LE = Leading edge, TE = Trailing edge.

2. Active high/low = Active clock level when destination is a latch. Active high means the latch is active when the primary input (PI) clock is on. Active low means the latch is active when the PI clock is off. (LE) or (TE) indicates the clock edge corresponding to the latch's capture edge.

3. Lockup cells are driven by the EDT clock.

4. These are cases for which the tool determines that the source edge precedes the destination edge. (Lockups are unnecessary if the destination edge precedes the source edge).

To minimize the number of lockup cells added, the tool always adds a trailing edge triggered (TE) lockup cell at the output of the Linear Feedback Shift Machine (LFSM) in the decompressor. The tool adds a second LE lockup cell at the input of the scan chain only when necessary, as shown in [Table 8-4](#).

Note

 If there is a pre-existing transparent latch between the decompressor and the first scan cell, a single lockup cell (LE) is added between the decompressor and the latch. This ensures the correct value is captured into the first scan cell from the decompressor.

Lockups Between Scan Chain Outputs and Compactor

When compactor pipeline stages are inserted, lockup cells are inserted as needed in front of the first pipeline stage. Pipeline stages are LE flip-flops clocked by the EDT clock, similar to the sequential elements in the decompressor.

The clock timing between the last active sequential element in the scan chain (source) and the first sequential element (first pipeline stage) that it feeds in the compactor (destination) is analyzed. Similar to the input side of the scan chains, the tool analyzes the need for lockup cells on the basis of the waveform edge timings (change edge and capture edge, respectively, of the source and destination clocks). The change edge is typically the first time at which the data on the source scan cell's output may update. The capture edge is the capturing transition at which data is latched on the destination scan cell's output.

Lockup cells driven by the EDT clock are added according to the following rules:

- A lockup cell is inserted when a source cell's change edge coincides with the destination cell's capture edge.
- A lockup cell is inserted when the change edge of the source cell precedes the capture edge of the destination cell.

In addition, the tool attempts to place lockup cells in a way that introduces no additional delay between the scan chains and the compactor pipeline stages. It also tries to minimize the number of lockup cells at the output side of the scan chains. The lockup cells are driven by the EDT clock so as to reduce routing of the system clocks from the core to the EDT logic.

Table 8-5 shows how the tool inserts lockup cells in the compactor.

Table 8-5. Lockup Cells Between Scan Chain Outputs and Compactor

Clock Waveforms	Source Clock	Dest. Clock	Source ^{1,2} Change Edge	Dest. ¹ Capture Edge	# Lockups Inserted	Lockup ³ Edge(s)
Overlapping	Scan clock	EDT clock	LE	LE	1	TE
	Scan clock	EDT clock	TE	LE	none	-
	Scan clock	EDT clock	active high (LE)	LE	1	TE
	Scan clock	EDT clock	active low (TE)	LE	none	-

Table 8-5. Lockup Cells Between Scan Chain Outputs and Compactor (cont.)

Clock Waveforms	Source Clock	Dest. Clock	Source^{1,2} Change Edge	Dest.¹ Capture Edge	# Lockups Inserted	Lockup³ Edge(s)
Non-Overlapping ⁴	Scan clock	EDT clock	LE	LE	1	TE
	Scan clock	EDT clock	TE	LE	1	TE
	Scan clock	EDT clock	active high (LE)	LE	1	TE
	Scan clock	EDT clock	active low (TE)	LE	1	TE

1. LE = Leading edge, TE = Trailing edge.

2. Active high/low = Active clock level when source is a latch. Active high means the latch is active when the primary input (PI) clock is on. Active low means the latch is active when the PI clock is off. (LE) or (TE) indicates the clock edge corresponding to the latch's change edge.

3. Lockup cells are driven by the EDT clock.

4. These are cases for which the tool determines that the source edge precedes the destination edge. (Lockups are unnecessary if the destination edge precedes the source edge).

Lockups in the Bypass Circuitry

The number and location of lockup cells the tool inserts in the bypass logic depend on the active edges (change edge and capture edge, respectively) of the source and destination clocks. The change edge is typically the first time at which the data on the source scan cell's output may update. The capture edge is the capturing transition at which data is latched on the destination scan cell's output.

The number and location of lockup cells also depend on whether the first and last active sequential elements in the scan chain are clocked by the same clock. The first and last active sequential elements in a scan chain could be existing lockup cells and may not be part of a scan cell. The tool inserts the lockup cells between source and destination scan cells according to the following rules:

- A lockup cell is inserted when a source cell's change edge coincides with the destination cell's capture edge and the cells are clocked by different clocks.
- A lockup cell is inserted when the change edge of the source cell precedes the capture edge of the destination cell.
- If multiple lockup cells are inserted, the tool ensures that:
 - A master/copy scan cell combination is always driven by the same clock. This prevents the situation where captured data in the master cell is lost because a different clock drives the copy cell and is not pulsed in a particular test pattern.

- The earliest data capture edge of the last lockup cell is not before the latest time when the destination cell can capture new data. This makes the first scan cell of every chain a master and prevents D2 DRC violations.
- If the earliest time when data is available at the output of the source is before the earliest data capture edge of the first lockup, the first lockup cell is driven with the same clock that drives the source.
- If a lockup cell already exists at the end of a scan chain, the tool learns its behavior and treats it as the source cell.

Table 8-6 summarizes how the tool inserts lockup cells in the bypass circuitry.

Table 8-6. Bypass Lockup Cells

Clock Waveforms	Source¹ Clock	Dest.¹ Clock	Source^{2,3} Change Edge	Dest.^{2,3} Capture Edge	# Lockups Inserted	Lockup Edge(s)
Overlapping	clk1	clk1	LE	LE	none	-
	clk1	clk1	LE	TE	1	TE clk1
	clk1	clk1	TE	TE	none	-
	clk1	clk1	TE	LE	none	-
Overlapping	clk1	clk2	LE	LE	1	TE clk1
	clk1	clk2	LE	TE	2	LE clk1, TE clk2
	clk1	clk2	TE	TE	2	LE clk1, TE clk2
	clk1	clk2	TE	LE	none	-
Non-Overlapping ⁴	clk1	clk2	LE	LE	2	LE clk1, TE clk2
	clk1	clk2	LE	TE	2	LE clk1, TE clk2
	clk1	clk2	TE	TE	2	LE clk1, TE clk2
	clk1	clk2	TE	LE	2	LE clk1, TE clk2
Overlapping	clk1	clk1	active high (LE)	active high (TE)	1	TE clk1
	clk1	clk1	active high (LE)	active low (LE)	1	TE clk1
	clk1	clk1	active low (TE)	active low (LE)	none	-
	clk1	clk1	active low (TE)	active high (TE)	none	-

Table 8-6. Bypass Lockup Cells (cont.)

Clock Waveforms	Source¹ Clock	Dest.¹ Clock	Source^{2, 3} Change Edge	Dest.^{2, 3} Capture Edge	# Lockups Inserted	Lockup Edge(s)
Overlapping	clk1	clk2	active high (LE)	active high (TE)	2	LE clk1, TE clk2
	clk1	clk2	active high (LE)	active low (LE)	1	TE clk1
	clk1	clk2	active low (TE)	active low (LE)	none	-
	clk1	clk2	active low (TE)	active high (TE)	2	LE clk1, TE clk2
Non-Overlapping ⁴	clk1	clk2	active high (LE)	active high (TE)	2	LE clk1, TE clk2
	clk1	clk2	active high (LE)	active low (LE)	2	LE clk1, TE clk2
	clk1	clk2	active low (TE)	active low (LE)	2	LE clk1, TE clk2
	clk1	clk2	active low (TE)	active high (TE)	2	LE clk1, TE clk2
Overlapping	clk1	clk1	LE	active high (TE)	1	TE clk1
	clk1	clk1	LE	active low (LE)	none	-
	clk1	clk1	active high (LE)	LE	none	-
	clk1	clk1	active low (TE)	LE	none	-

Table 8-6. Bypass Lockup Cells (cont.)

Clock Waveforms	Source ¹ Clock	Dest. ¹ Clock	Source ^{2,3} Change Edge	Dest. ^{2,3} Capture Edge	# Lockups Inserted	Lockup Edge(s)
Overlapping	clk1	clk2	LE	active high (TE)	2	LE clk1, TE clk2
	clk1	clk2	LE	active low (LE)	1	TE clk1
	clk1	clk2	active high (LE)	LE	1	TE clk1
	clk1	clk2	active low (TE)	LE	none	-
Non-Overlapping ⁴	clk1	clk2	LE	active high (TE)	2	LE clk1, TE clk2
	clk1	clk2	LE	active low (LE)	2	LE clk1, TE clk2
	clk1	clk2	active high (LE)	LE	2	LE clk1, TE clk2
	clk1	clk2	active low (TE)	LE	2	LE clk1, TE clk2
Overlapping	clk1	clk1	TE	active high (TE)	none	-
	clk1	clk1	TE	active low (LE)	none	-
	clk1	clk1	active high (LE)	TE	1	TE clk1
	clk1	clk1	active low (TE)	TE	none	-

Table 8-6. Bypass Lockup Cells (cont.)

Clock Waveforms	Source¹ Clock	Dest.¹ Clock	Source^{2,3} Change Edge	Dest.^{2,3} Capture Edge	# Lockups Inserted	Lockup Edge(s)
Overlapping	clk1	clk2	TE	active high (TE)	2	LE clk1, TE clk2
	clk1	clk2	TE	active low (LE)	2	LE clk1, TE clk2
	clk1	clk2	active high (LE)	TE	2	LE clk1, TE clk2
	clk1	clk2	active low (TE)	TE	2	LE clk1, TE clk2
Non-Overlapping ⁴	clk1	clk2	TE	active high (TE)	2	LE clk1, TE clk2
	clk1	clk2	TE	active low (LE)	2	LE clk1, TE clk2
	clk1	clk2	active high (LE)	TE	2	LE clk1, TE clk2
	clk1	clk2	active low (TE)	TE	2	LE clk1, TE clk2

1. clk1 & clk2 are the functional (scan) clocks.

2. LE = Leading edge, TE = Trailing edge.

3. Active high/low = Active clock level when source or destination is a latch. Active high means the latch is active when the primary input (PI) clock is on. Active low means the latch is active when the PI clock is off. (LE) or (TE) indicates the clock edge corresponding to the latch's change/capture edge.

4. These are cases for which the tool determines the source edge precedes the destination edge. (Lockups are unnecessary if the destination edge precedes the source edge).

Lockup Cell Analysis For Bypass Lockup Cells Included as Part of the EDT Chains

The tool adds lockup cells at the scan chain boundary to eliminate bypass-only lockup cells.

Note

See “[Differences Based on Inclusion/Exclusion of Bypass Lockup Cells in EDT Chains](#)” on page 266 for a thorough explanation of the differences that result when bypass lockup cells are included in the EDT chain as opposed to when they are not.

The tool analyzes the clocking of first and last active scan elements and adds lockup cells at scan chain inputs and outputs as required. These cells are added to ensure each scan chain starts with a LE register and ends with a TE register. These lockup cells are included as part of both EDT and EDT-bypass scan chains. They avoid clock skew problems between the decompressor and scan chains, scan chains and compactor, as well as when concatenating EDT scan chains into bypass chains. They also provide the ability to map EDT mode patterns into bypass mode.

As an exception, when all of the first and last scan elements are driven by the LE of the same clock and the compactor has no sequential registers, scan chain output lockup cells are added only for the last internal chain grouped into bypass chains.

EDT Lockup and Scan Chain Boundary Lockup Cells	264
Differences Based on Inclusion/Exclusion of Bypass Lockup Cells in EDT Chains ...	266
Lockup Cell Functionality Limitations	269
Comparison of Bypass Lockup Cell Insertion Results.....	270

EDT Lockup and Scan Chain Boundary Lockup Cells

When lockup cells at chain boundaries are inserted, the tool combines the analysis of decompressor and compactor lockup cells along with the scan chain input/output bypass lockup cells.

[Table 8-7](#) summarizes how the tool adds lockup cells for different clocking configurations.

Table 8-7. EDT Lockup and Scan Chain Boundary Lockup Cells

Clock (source → destination) (last cell → first cell)	EDT Decompressor Lockup Cells ¹	Scan Chain Input Lockup	Scan Chain Output Lockup	Compactor Lockup Cell ¹
Same source and destination clocks				
LE clk → LE clk	TE edt_clock	-	TE clk	-
LE clk → LE clk ²	TE edt_clock	-	-	-
LE clk → TE clk	TE edt_clock	LE clk	TE clk	-

Table 8-7. EDT Lockup and Scan Chain Boundary Lockup Cells (cont.)

Clock (source → destination) (last cell → first cell)	EDT Decompressor Lockup Cells¹	Scan Chain Input Lockup	Scan Chain Output Lockup	Compactor Lockup Cell¹
TE clk → LE clk	TE edt_clock	-	-	-
TE clk → TE clk	TE edt_clock	LE clk	-	-
Overlapping clocks, clkS and clkD				
LE clkS → LE clkD	TE edt_clock	-	TE clkS	-
LE clkS → TE clkD	TE edt_clock	LE clkD	TE clkS	-
TE clkS → LE clkD	TE edt_clock	-	-	-
TE clkS → TE clkD	TE edt_clock	LE clkD	-	-
Non-overlapping clocks, clkS overlaps with edt_clock, clkD later than edt_clock & clkS				
LE clkS → LE clkD	TE edt_clock	LE clkS	TE clkS	-
LE clkS → TE clkD	TE edt_clock	LE clkS	TE clkS	-
TE clkS → LE clkD	TE edt_clock	LE clkS	-	-
TE clkS → TE clkD	TE edt_clock	LE clkS	-	-
Non-overlapping clocks, clkS and clkD (either same or different clocks) later than edt_clock				
LE clkS → LE clkD	TE, LE edt_clock	-	TE clkS	-
LE clkS → TE clkD	TE, LE edt_clock	LE clkD	TE clkS	-
TE clkS → LE clkD	TE, LE edt_clock	-	-	-
TE clkS → TE clkD	TE, LE edt_clock	LE clkD	-	-
Overlapping clocks, same or different				
active high clkS (LE) → active high clkD (TE)	TE edt_clock	LE clkD	TE clkS	-
active high clkS (LE) → active low clkD (LE)	-	-	TE clkS	-
active low clkS (TE) → active high clkD (TE)	TE edt_clock	LE clkD	-	-
active low clkS (TE) → active low clkD (LE)	TE edt_clock	-	-	-
LE clkS → active high clkD (TE)	TE edt_clock	LE clkD	TE clkS	-

Table 8-7. EDT Lockup and Scan Chain Boundary Lockup Cells (cont.)

Clock (source → destination) (last cell → first cell)	EDT Decompressor Lockup Cells¹	Scan Chain Input Lockup	Scan Chain Output Lockup	Compactor Lockup Cell¹
LE clkS → active low clkD (LE)	TE edt_clock	-	TE clkS	-
TE clkS → active high clkD (TE)	TE edt_clock	LE clkD	-	-
TE clkS → active low clkD (LE)	TE edt_clock	-	-	-
active high clkS (LE) → LE clkD	TE edt_clock	-	TE clkS	-
active high clkS (LE) → TE clkD	TE edt_clock	LE clkD	TE clkS	-
active low clkS (TE) → LE clkD	TE edt_clock	-	-	-
active low clkS (TE) → TE clkD	TE edt_clock	LE clkD	-	-

1. Decompressor and compactor lockup cells are not included as part of the EDT scan chains.

2. Special case where all scan cells are clocked by a single LE clock. This optimization is applicable only when lockup cells are not required in the compactor, except for making compactor change edge as trailing. Any of compactor pipelining, TK/LBIST (due to MISR lockup), dual configuration may necessitate compactor lockup.

Differences Based on Inclusion/Exclusion of Bypass Lockup Cells in EDT Chains

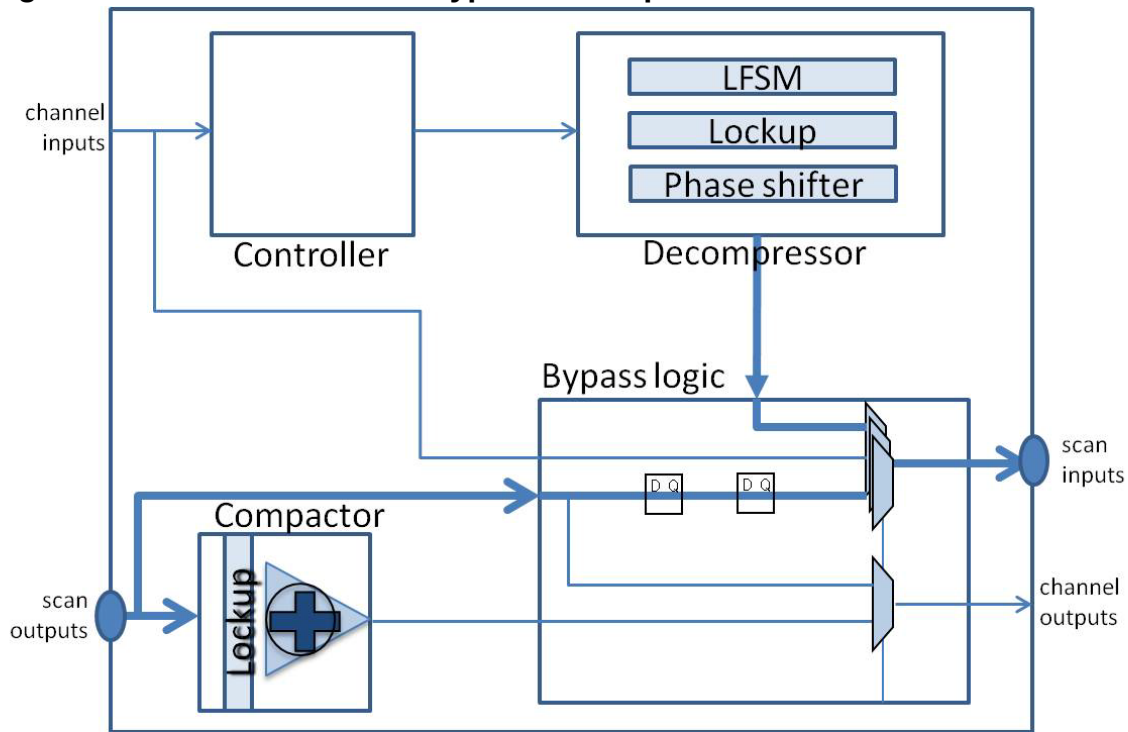
The behavior of the tool in adding decompressor/compressor lockup cells and scan chain lockup cells is based on factors such as whether lockup cells are included as part of the EDT scan chains during pattern generation, the edges of the scan cells in the chains, and whether single bypass chain functionality is specified.

Complete information on how the tool adds lockup cells when they are not included in EDT chains is presented in “[Lockup Cell Analysis For Bypass Lockup Cells Not Included as Part of the EDT Chains](#)” on page 256.

Internal Scan Chain Definition

[Figure 8-22](#) illustrates the internal scan chain definition anchor points (scan inputs and scan outputs) during pattern generation when bypass lockup cells are not included as part of the EDT scan chains.

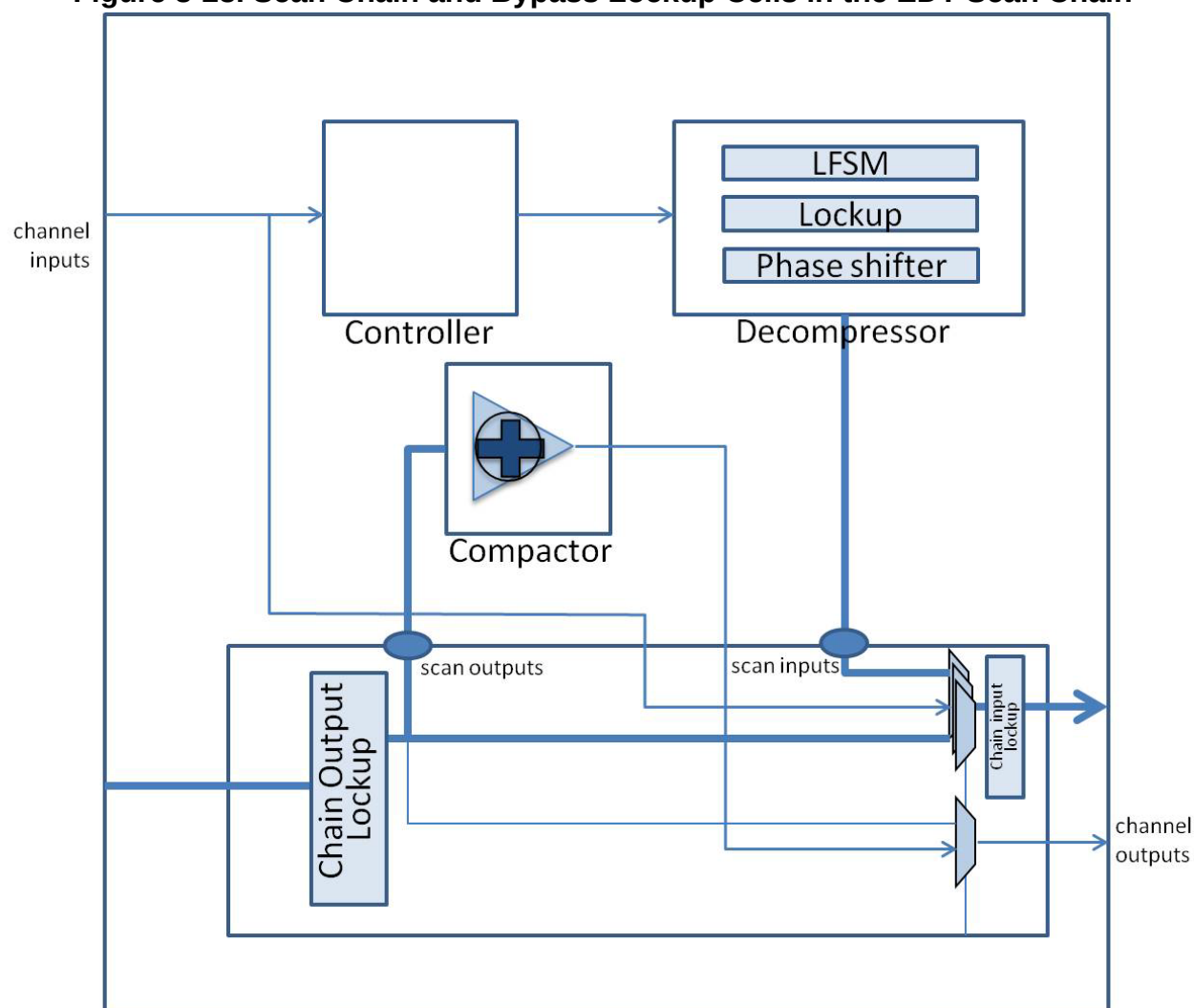
Figure 8-22. Scan Chain and Bypass Lockup Cells Not in the EDT Scan Chain



You can insert bypass lockup cells such that they are included as part of the EDT scan chains. This allows the tool to see the actual bypass lockup cells and account for them correctly.

Figure 8-23 illustrates the internal scan chain definition anchor points (scan inputs and scan outputs) when bypass lockup cells are included as part of the EDT scan chains.

Figure 8-23. Scan Chain and Bypass Lockup Cells in the EDT Scan Chain



Note

The lockup cells inside bypass logic are now included as part of the EDT scan chains as well. The first level lockup cells for the decompressor are still excluded from the scan chain definition as is the case when bypass lockup cells are not included as part of the EDT scan chains.

Insertion Algorithm When Bypass Lockup Cells are Included at the Boundary of the EDT Chains

As shown in [Figure 8-23](#), when bypass lockup cells are included in the EDT scan chain, TestKompress does the following:

- If the last scan cell is a LE scan cell, the tool adds a TE lockup cell clocked by the last scan cell clock to the scan chain output.

- If the first scan cell is a TE scan cell, the tool adds a LE lockup cell to the scan chain input.
- If all of the scan chains are clocked by the LE of the same clock, the tool makes an exception and adds lockup cells only for the last internal scan chain of each bypass chain. This facilitates the concatenation of the bypass chains of an EDT block at a higher level.
- The LE lockup cell at the scan chain input is clocked by the source scan clock if it has an early waveform compared with the destination scan clock; otherwise, the lockup cell is clocked by the destination scan clock.
- The second decompressor lockup cell is not required when the destination scan cell is a TE with the same waveform as the EDT clock. When the first scan cell has a late clock, the second decompressor lockup cell is included only if the lockup cell at the last scan chain output is pulsed with a late clock.
- The new lockup cell can influence EDT and compactor lockup cells because these new lockup cells are visible in the EDT path and are cumulative with dedicated EDT-only lockup cells in the decompressor and compactor.
- Compactor lockup cell analysis includes the source lockup cell at the scan chain output. In particular, if a TE source lockup cell is needed for a bypass lockup cell, it will also be used for a compactor lockup cell.

Single Bypass Chain

When using this functionality, bypass lockup cells are also added to the input of the first and output of the last internal chains grouped into a bypass chain. This enables the regular bypass chains to be easily concatenated to form the single bypass chain for the entire EDT block.

The lockup cells for bypass mode concatenation also allow concatenating the single bypass chain of all EDT blocks declared in the tool during IP creation. TestKompress does not actually concatenate the single bypass chains of the EDT blocks; rather TestKompress facilitates the process for some other tool to make such a concatenation.

You can concatenate the single bypass chains of all the EDT blocks in a design to construct a system-wide single bypass chain, even across blocks not declared in IP creation. In such cases, if the source clock from the preceding EDT block is pulsed earlier than the destination clock from the succeeding EDT block in the system-wide single chain concatenation order, these scan cells will become a master-copy pair. This should be properly accounted for when translating EDT mode patterns into the single system-wide bypass chain patterns.

Lockup Cell Functionality Limitations

The lockup cell functionality cannot be used with certain features.

The following features are not compatible with lockup cells:

- **Generating a blackbox for the EDT logic using “set_edt_options -blackbox on”** — When including bypass lockup cells in EDT scan chains, the scan chains are defined on the EDT decompressor and compactor instance pins in the EDT logic. The instance pins are not available in a blackbox description of the EDT module.
- **Pulsing EDT clock before shift clock** — Since the bypass lockup cells are clocked by edt_clock in this case, including them as part of the scan chains will result in D1 violations on all the lockup cells at scan chain inputs.

Comparison of Bypass Lockup Cell Insertion Results

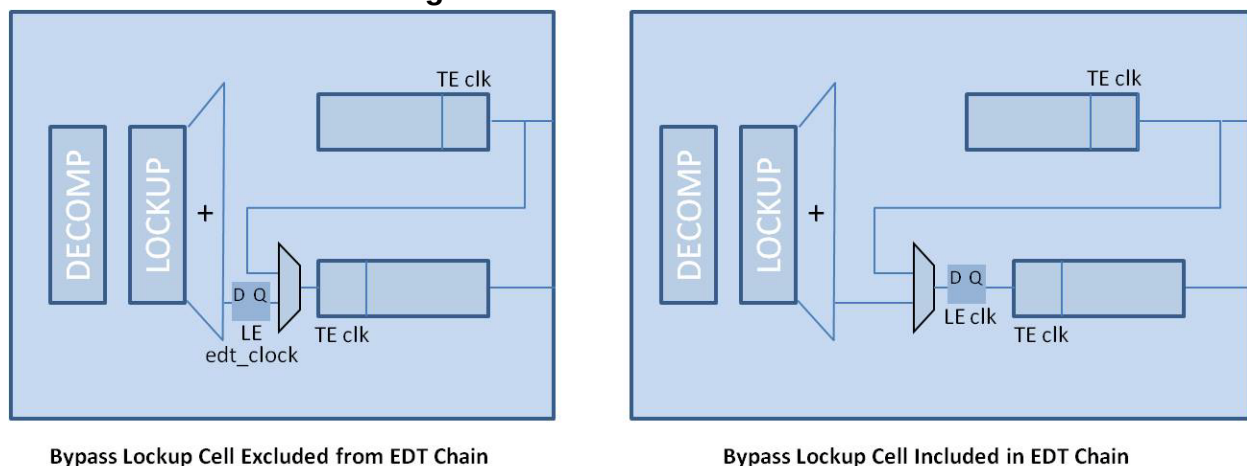
This section compares the circuitry created by bypass lockup cell insertion depending upon whether the bypass lockup cell is included or excluded from the EDT chain.

Case 1: No Bypass Lockup Cell

Figure 8-24 illustrates the circuitry when the bypass lockup cell insertion algorithm does not insert any lockup cells: on the left is the circuitry when bypass lockup cells are excluded from EDT chains, and on the right is the circuitry when they are included in EDT chains.

When both the last and first scan cells are TE and clocked by the same clock, a LE lockup cell is added to the destination scan chain input. In this case, the second decompressor lockup cell in the EDT decompressor is not added. This is shown in Figure 8-24. This is an example that demonstrates the case when the bypass lockup cell affects the EDT decompressor lockup cell.

Figure 8-24. TE CLK to TE CLK

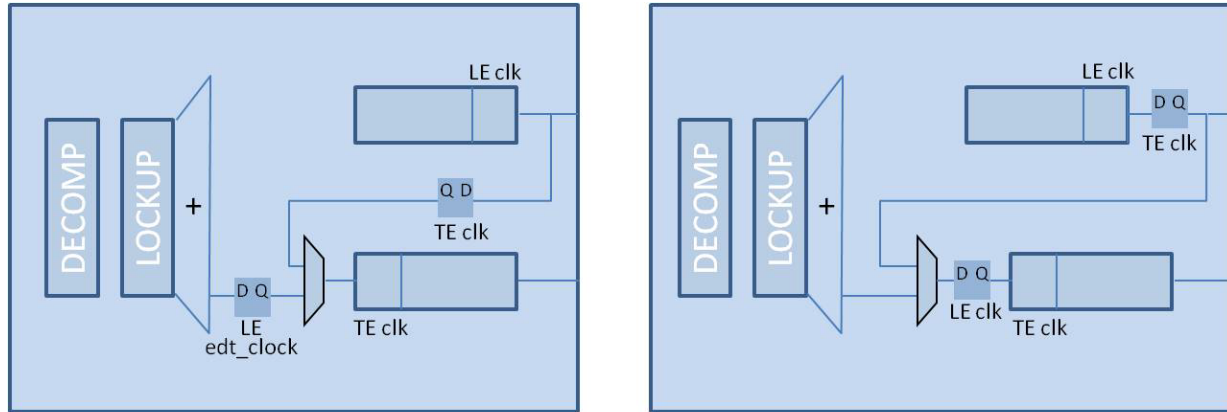


Case 2: One Bypass Lockup Cell

When bypass lockup cells are excluded from the EDT chain, the tool inserts one bypass lockup cell in the following cases:

- If LE clk to TE clk, the tool inserts a TE clk lockup as illustrated, on the left, in [Figure 8-25](#). Note, the absence of the second decompressor lockup cell, on the right, in [Figure 8-25](#).

Figure 8-25. LE Clk to TE Clk

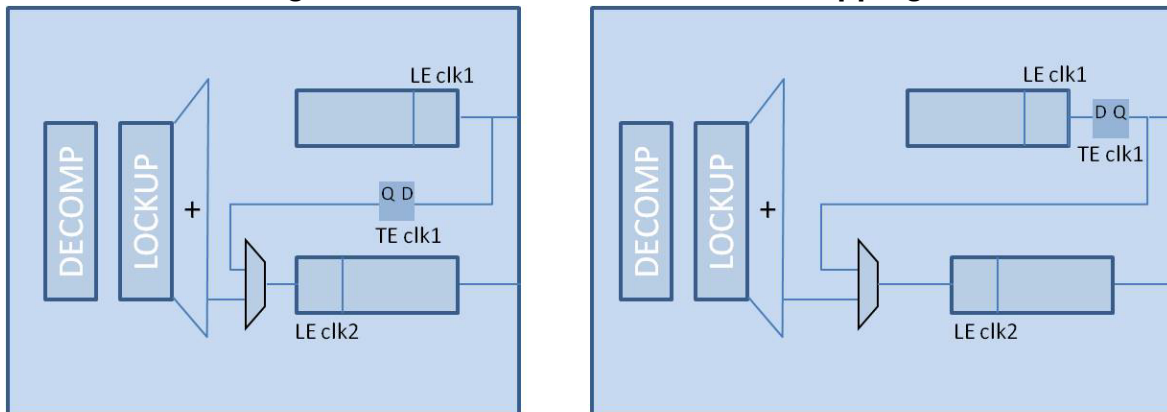


Bypass Lockup Cell Excluded from EDT Chain

Bypass Lockup Cell Included in EDT Chain

- If LE clk1 to LE clk2, the tool inserts a TE clk1 lockup as illustrated, on the left, in [Figure 8-26](#).

Figure 8-26. LE Clk1 to LE Clk2 Overlapping



Bypass Lockup Cell Excluded from EDT Chain

Bypass Lockup Cell Included in EDT Chain

Case 3: Two Bypass Lockup Cells

Figure 8-27 illustrates the case where the tool infers two lockup cells in both cases, but the clock edges of the lockup cells are different. This case applies when both clkS and clkD are overlapping with the EDT clock, and when clkS overlaps with the EDT clock but clkD has a late waveform.

Figure 8-27. LE ClkS to TE ClkD

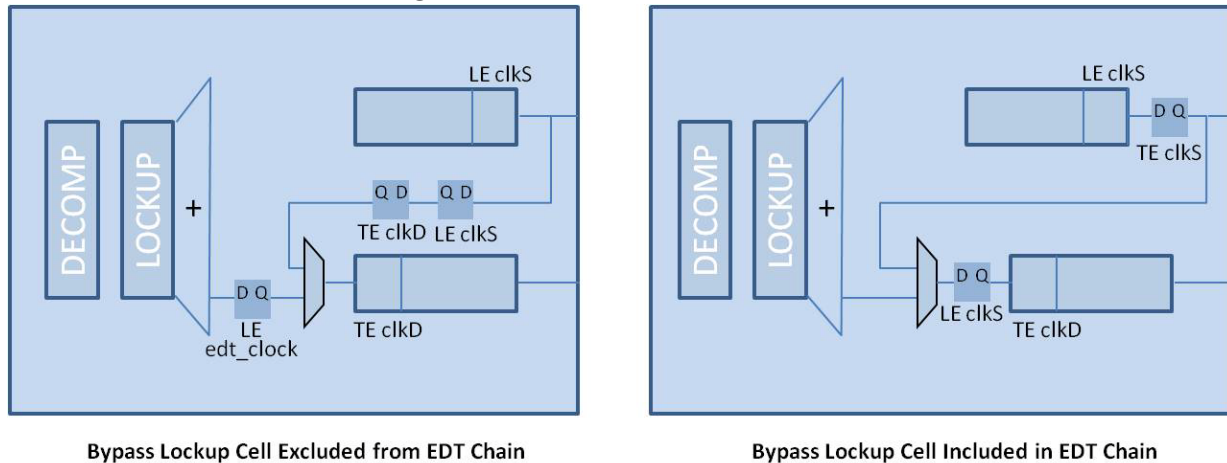
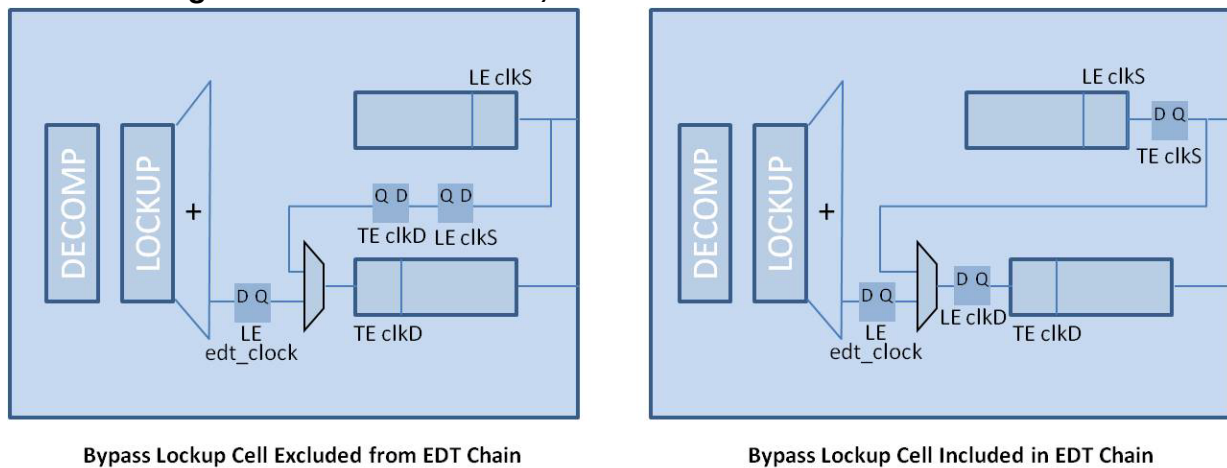


Figure 8-28 illustrates the case when the destination cell is TE, but the same situation applies when the destination cell is LE.

Figure 8-28. ClkS to ClkD, Both Clocks Later Than EDT Clock



Lockups Between Channel Outputs and Output Pipeline Stages

During the top-level design integration process, clocking requirements may require you to insert lockup cells between the EDT logic and pad terminals. If the clocking of the last scan cells compacted into an output channel and the clocking of the output pipeline stage (outside the EDT logic) overlap, you must add a lockup cell (outside the EDT logic). Tessent Shell in the EDT IP Creation phase does not insert these lockup cells.

However, if internal compactor pipelining is enabled in the EDT logic, and the output pipeline stages are active on the leading edge (LE) of the EDT clock, no lockup cells are necessary because the internal compactor pipeline stages also use the leading edge (LE) of the EDT clock.

If the output pipeline stages use a different edge or clock, the existing lockup cells may be insufficient, and you must specify the change edge for the compactor outputs or insert lockup cells manually. When you specify the change edge for the compactor outputs, the tool inserts pipeline stages and lockup cells as needed to ensure the compactor outputs change as specified.

You use the [set_edt_pins](#) command with the -CHange_edge_at_compactor_output option to specify the change edge for the compactor outputs. Depending on the change edge specified for the compactor outputs, the tool inserts lockup cells between the compactor output and output channels as described in [Table 8-8](#).

Table 8-8. Lockup Insertion Between Channel Outputs and Output Pipeline

-CHange_edge_at_compactor_ Output	Compactor Pipeline Stages	Lockup Between Scan Chain and Compactor	Last Scan Cell	Lockup Inserted Between Compactor Output and Output Channels
LEading_edge_of_edt_clock	LE ¹	NA ²	NA	none ³
	none	LE	NA	none
	none	TE	NA	LE
	none	none	LE	none
	none	none	TE ⁴	LE
TRailing_edge_of_edt_clock	LE	NA	NA	TE
	none	LE	NA	TE
	none	TE	NA	none
	none	none	LE	TE
	none	none	TE	none

1. *LE* indicates the leading edge of the clock pulse.

2. *NA* indicates the state of that column has no effect on the resulting action described in the right-most column (Lockup inserted between compactor output and output channels).

3. *None* indicates the object does not exist or is not inserted.

4. *TE* indicates the trailing edge of the clock pulse.

Related Topics

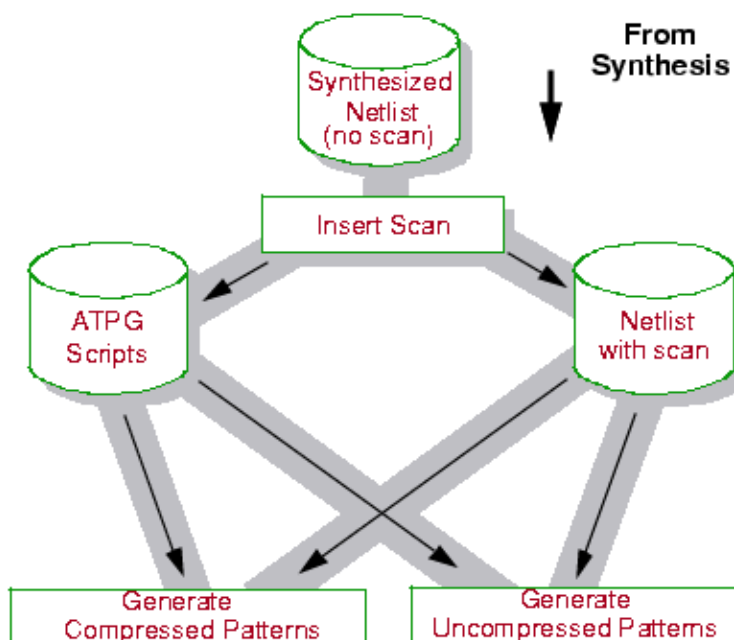
[Use of Pipeline Stages Between Pads and Channel Inputs or Outputs](#)

Compression Performance Evaluation

Focus on the parts of the compressed ATPG flow that are necessary to perform experiments on compression rates and performance so you can make informed choices about how to fine-tune performance.

Figure 8-29 illustrates the typical evaluation flow.

Figure 8-29. Evaluation Flow



The complete Tessent TestKompress flow is described in “[Top-Down Design Flows](#)” on page 42.

In an experimentation flow, where your intention is to verify how well EDT works in a design, you generate compressed patterns and use these patterns to verify coverage and pattern count, but not to perform final testing. Consequently, you do not need to write out the hardware description files. The first thing you should do, though, to make the data you obtain from running compressed ATPG meaningful, is establish a point of reference using uncompressed ATPG.

Establishing a Point of Reference.....	275
Performance Measurement.....	276
Performance Improvement.....	277

Establishing a Point of Reference

The following steps describe the flow for establishing a point of reference for evaluating the performance of the compression. A design configured with eight scan chains is assumed.

To illustrate how you establish a point of reference using uncompressed ATPG, assume as a starting point that you have both a non-scan netlist and a netlist with eight scan chains. You would calculate the test data volume for measuring compression performance in the following way:

$$\begin{aligned}\text{Test Data Volume} &= (\text{\#scan loads}) \times (\text{volume per scan load}) \\ &= (\text{\#scan loads}) \times (\text{\#shifts per patterns}) \times (\text{\#scan channels})\end{aligned}$$

Note



#patterns may provide a reasonable approximation for #scan loads, but be aware that some patterns require multiple scan loads.

For a regular scan-based design without EDT, the volume per scan load will remain fairly constant for any number of scan chains because the number of shifts decreases when the number of chains increases. Therefore, it does not matter which scan chain configuration you use when you establish the reference point.

Procedure

1. Invoke Tessent Shell on your design.

```
<Tessent_Tree_Path>/bin/tessent -shell
```

2. Set the context, read in the netlist with eight scan chains and a library, and set the current design.

```
set_context patterns -scan  
read_verilog mydesign_scan_8.v  
read_cell_library my_lib.atpg  
set_current_design top
```

3. Execute the dofile that performs basic setup.

```
dofile atpg_8.dofile
```

4. Run DRC and verify that no DRC violations occur.

```
set_system_mode analysis
```

5. Generate patterns. Assuming the design does not have RAMs, you can just generate basic patterns. To speed up the process, use fault sampling. It is important to use the same fault sample size in both the uncompressed and compressed runs.

```
add_faults /cpu_i
set_fault_sampling 10
create_patterns
report_statistics
report_scan_volume
```

6. Note the test coverage and the total data volume as reported by the [report_scan_volume](#) command.

Performance Measurement

In the compressed and uncompressed runs, you should examine some statistics to assist with your evaluation.

Specifically, the numbers you will want to examine are

- Test coverage ([report_statistics](#))
- CPU time ([report_statistics](#))
- Scan data volume ([report_scan_volume](#))
- Observable X sources ([E5](#)) violations which can explain lower compression performance.
- Runs to compare results with and without fault sampling.

Performance Improvement

There are some analyses you can do if the measured performance is not as expected.

Table 8-9 lists some suggested analyses.

Table 8-9. Summary of Performance Issues

Unsatisfactory Result	Suggested Analysis
Compression	- Many observable X sources. Examine E5 violations. - Too short scan chain vs. # of additional shift cycles.¹ Verify the # of additional shift cycles, and scan chain length using the <code>report_edt_configurations</code> command.
Run time	- Untestable/hard to compress patterns. If they cause a high runtime for uncompressed ATPG, they will also cause a high runtime for compressed ATPG. - If compressed ATPG has a much larger runtime than uncompressed ATPG, examine X sources, E5 violations.
Coverage	- Shared scan chain I/Os. Scan pins are masked by default. These pins should be dedicated. - Too aggressive compression (chain-to-channel ratio too high), leading to incompressible patterns. Use the <code>report_aborted_faults</code> command to debug. Look for EDT aborted faults.

1. Additional shift cycles refers to the sum of the initialization cycles, masking bits (when using Xpress), and low-power bits (when using a low-power decompressor).

Variance in the Number of Scan Chains	277
Variance in the Number of Scan Channels	278
Determining the Limits of Compression	279
Speed up the Process	279

Variance in the Number of Scan Chains

The effective compression depends primarily on the ratio between the number of internal scan chains and the number of external scan channels. In most cases, it is sufficient to just do an approximate configuration. For example, if the number of scan channels is eight and you need 4X compression, you can configure the design with 38 chains. This will typically result in 3.5X to 4.5X compression.

In certain cases, such a rough estimate is not enough. Usually, the number of scan channels is fixed because it depends on characteristics of the tester. Therefore, to experiment with different compression outcomes, different versions of the netlist (each with a different number of scan chains) are necessary.

Related Topics

[Balancing Scan Chains Between Blocks](#)

[Variance in the Number of Scan Channels](#)

Variance in the Number of Scan Channels

Another alternative to varying the number of scan chains in order to evaluate compression performance, is to use a design with a relatively high number of scan chains and experiment with different numbers of channels. You can do these experiments, varying the chain-to-channel ratio. Then, when you find the optimum ratio, reconfigure the scan chains to match the number of scan channels you want. You can achieve similar test data volume reduction for a 100:10 configuration as for a 50:5 configuration.

For example, assume you have a design with 350,000 gates and 27,000 scan cells. If a certain tester requires the chip to have 16 scan channels, and your compression goal is to have no less than 4X compression, you might proceed as follows:

1. Determine the approximate number of scan chains you need. This example assumes a reasonable estimate is 60 scan chains.
2. Use Tessent Scan to configure the design with many more scan chains than you estimated, say, 100 scan chains.
3. Run the tool for 30, 26, 22, and 18 scan channels. Notice that these numbers are all between 1-2X the 16 channels you need.

Note

 Use the same commands with compressed ATPG that you used with uncompressed ATPG when you established a point of reference, with one exception: with compressed ATPG, you must use the [set_edt_options](#) command to reconfigure the number of scan channels.

Suppose the results show that you achieve 4X compression of the test data volume using 22 scan channels. This is a chain-to-channel ratio of 100:22 or 4.55. For the final design, where you want to have 16 scan channels, you would expect approximately a 4X reduction with $16 \times 4.55 = 73$ scan chains.

Related Topics

[Variance in the Number of Scan Chains](#)

Determining the Limits of Compression

You will find that the maximum amount of compression you can attain is limited by the ratio of scan chains to channels. If the number of scan channels is fixed, the number of scan chains in your design becomes the limiting factor.

For example, if your design has eight scan chains, the most compression you can achieve under optimum conditions will be less than 8X compression. To exceed this maximum, you would need to reconfigure the design with a higher number of scan chains.

Related Topics

[Scan Chain Insertion](#)

[Compression Analysis](#)

Speed up the Process

If you need to perform multiple iterations, either by changing the number of scan chains or the number of scan channels, you can speed up the process by using fault sampling. When you use fault sampling, first perform uncompressed ATPG with fault sampling. Then, use the same fault sample when generating compressed patterns.

Note

 You should always use the entire fault list when you do the final test pattern generation. Use fault sampling only in preliminary runs to obtain an estimate of test coverage with a relatively short test runtime. Be aware that sampling has the potential to produce a skewed result and is a means of estimation only.

Related Topics

[Analyzing Compression](#)

Understanding Compactor Options

There are two compactors available in compressed ATPG, Xpress and basic.

- Xpress

The Xpress compactor is the second generation compactor generated by default. The Xpress compactor optimizes compression for all designs but is especially effective for designs that generate X values. The Xpress compactor observes all chains with known values and masks out scan chains that contain X values. This X handling results in fewer test patterns being required for designs that generate X values.

Depending on the application, the EDT logic generated with the Xpress compactor requires additional clocking cycles. The additional clocking cycles are determined by

the ratio of scan chains to output channels and are relatively few when compared with the total shift cycles.

- Basic

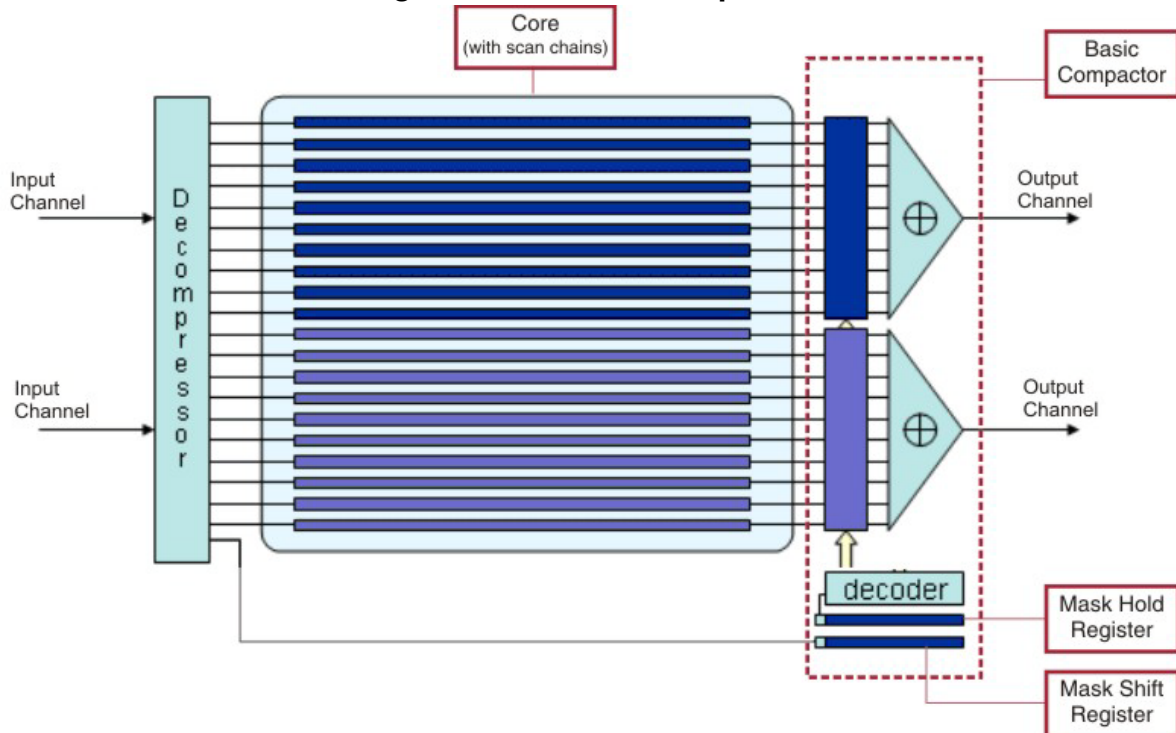
The basic compactor is the first generation compactor enabled with the `-compactor_type basic` switch with the `set_edt_options` command.

The basic compactor should be used for designs that do not generate many unknown (X) values. Due to scan cell masking, the basic compactor is significantly less effective on designs that generate unknown (X) values in scan cells when a test pattern is applied.

The EDT logic generated when the basic compactor is used may be up to 30% smaller than EDT logic generated when the Xpress compactor is used. However, when X values are present, more test patterns may be required.

Basic Compactor Architecture

Figure 8-30. Basic Compactor



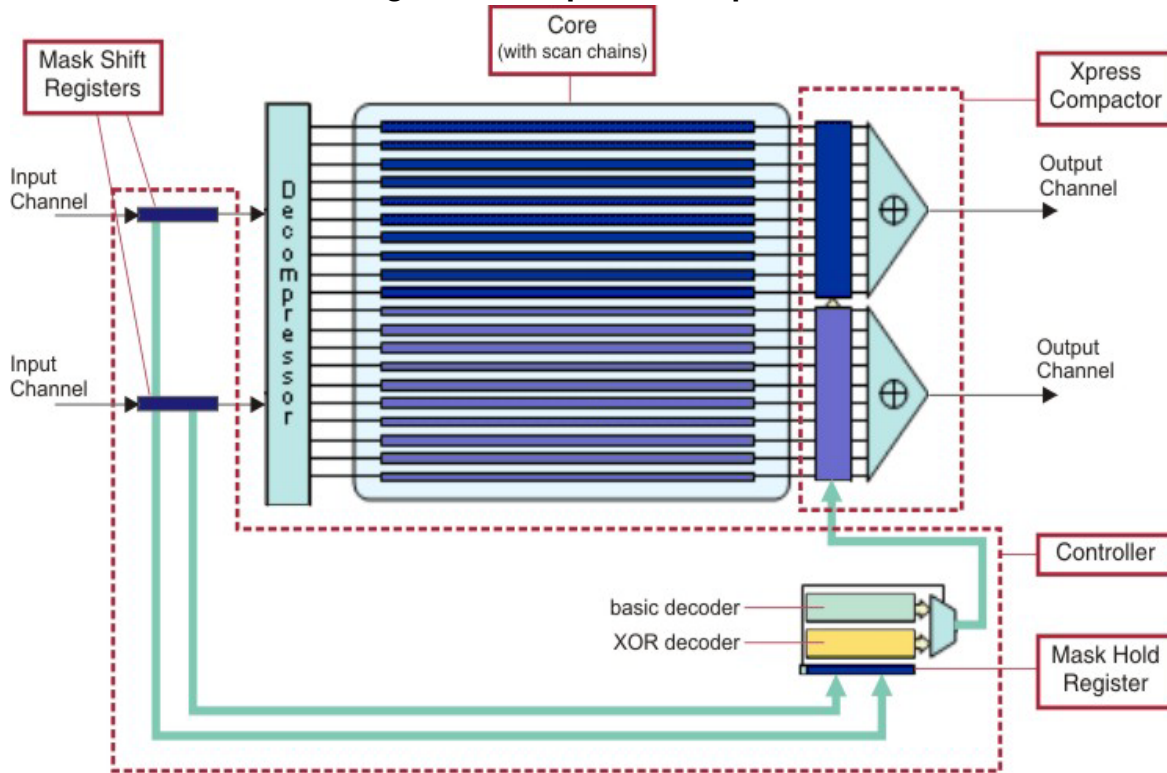
A mask code (prepend with a decoder mode bit) is generated with each test pattern to determine which scan chains are masked or observed. The basic compactor determines which chains to observe or mask using the mask code as follows:

1. The decompressor loads the mask code into the mask shift register.
2. The mask code is parallel-loaded into the mask hold register, where the decoder mode bit determines the observe mode: either one scan chain or all scan chains.

3. The mask code in the mask hold register is decoded and each bit drives one input of a masking AND gate in the compactor. Depending on the observe mode, the output of these AND gates is either enabled or disabled.

Xpress Compactor Architecture

Figure 8-31. Xpress Compactor



A mask code (prepending with a decoder mode bit) is generated with each test pattern to determine which scan chains are masked or observed. The Xpress compactor determines which chains to observe or mask using the mask code as follows:

1. Each test pattern is loaded into the decompressor through a mask shift register on the input channel.
2. The mask code is appended to each test pattern and remains in the mask shift register once the test pattern is completely loaded into the decompressor.
3. The mask code is then parallel-loaded into the mask hold register, where the decoder mode bit determines whether the basic decoder or the XOR decoder is used on the mask code.
 - o The basic decoder selects only one scan chain per compactor. The basic decoder is selected when there is a very high rate of X values during scan testing or during chain test to allow failing chains to be fully observed and easy to diagnose.

- The XOR decoder masks or observes multiple scan chains per compactor, depending on the mask code. For example, if the mask code is all 1s, then all the scan chains are observed.
4. The decoder output is shifted through a multiplexer, and each bit drives one input on the masking AND gates in the compactor to either disable or enable the output, depending on the decoder mode and bit value.

Understanding Scan Chain Masking in the Compactor

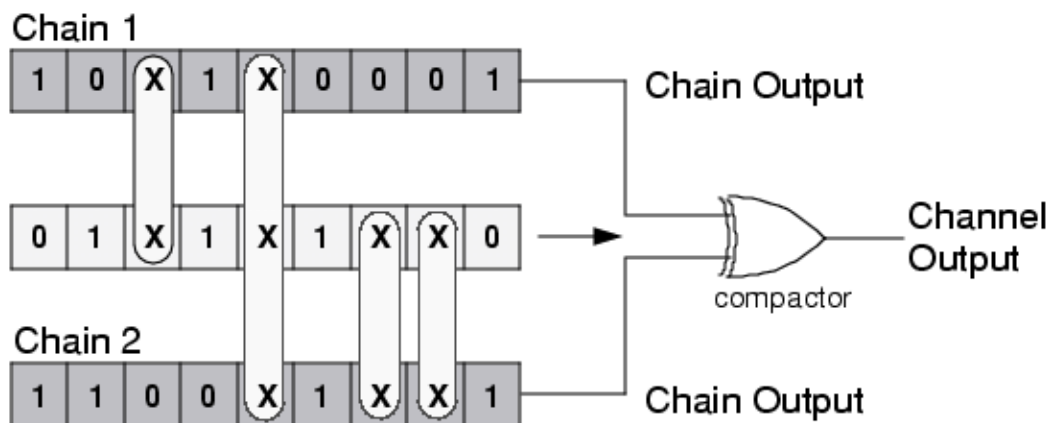
It is important to use scan chain masking in the compactor in order to ensure accurate scan chain observations.

Why Masking is Needed

To facilitate compression, the tool inserts a compactor between the scan chain outputs and the scan channel outputs. In this circuitry, one or more stages of XOR gates compact the response from several chains into each channel output. Scan chains compacted into the same scan channel are said to be in the same compactor group.

One common problem with different compactor strategies is handling of Xs (unknown values). Scan cells can capture X values from unmodeled blocks, memories, non-scan cells, and so forth. Assume two scan chains are compacted into one channel. An X captured in Chain 1 will then block the corresponding cell in Chain 2. If this X occurs in Chain 1 for all patterns, the value in the corresponding cell in Chain 2 will never be measured. This is illustrated in [Figure 8-32](#), where the row in the middle shows the values measured on the channel output.

Figure 8-32. X-Blocking in the Compactor



The tool records an X in the pattern file in every position made unmeasurable as a result of the actual occurrence of an X in the corresponding cell of a different scan chain in the same compactor group. This is referred to as X blocking. The capture data for Chain 1 and Chain 2

that you would see in the ASCII pattern file for this example would look similar to [Figure 8-33](#). The Xs substituted by the tool for actual values, unmeasurable because of the compactor, are shown in red.

Figure 8-33. X Substitution for Unmeasurable Values

Chain 1								
1	0	X	1	X	0	X	X	1

Chain 2								
1	1	X	0	X	1	X	X	1

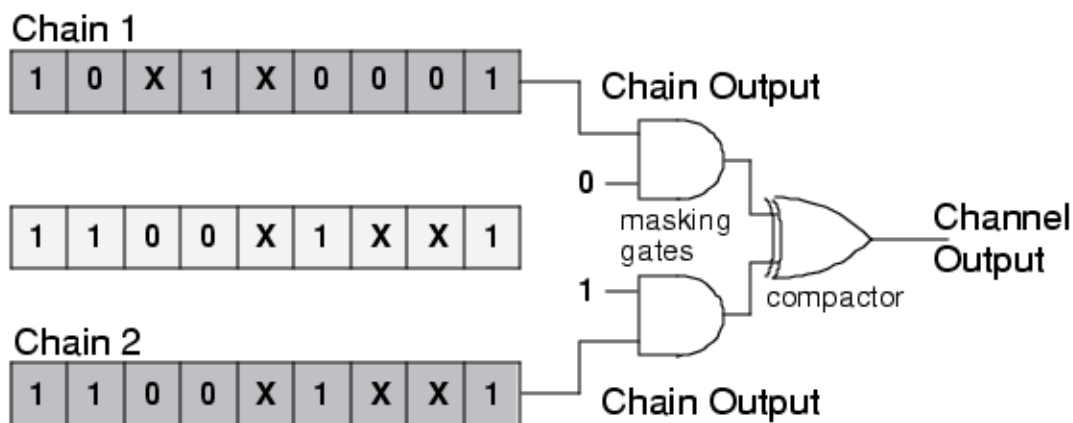
Resolving X Blocking with Scan Chain Masking

The solution to this problem is a mechanism utilized in the EDT logic called “scan chain masking.” This mechanism allows selection of individual scan chains on a per-pattern basis. Two types of scan chain masking are used: 1-hot masking and flexible masking.

- With 1-hot masking, only one chain is observed via each scan channel's compaction network. All the other chains in that compactor are masked so they produce a constant 0 to the input of the compactor. This allows observation of fault effects for the observed chains even if there are Xs in the observation cycles for the other chains. 1-hot masking patterns are only generated for a few ATPG cycles at points when the non-masking and flexible masking algorithms fail to detect any significant number of faults.
- Flexible masking patterns allow multiple chains to be observed via each scan channel's compaction network. Flexible masking is not fully non-masking; with fully non-masking patterns, none of the chains are masked so Xs in some cycles of some chains can block the observation of the fault effects in some other chain. The Xpress compactor observes all chains with known values and masks out those scan chains that contain X values so they do not block observation of other chains. With Xpress flexible masking, only a subset of the chains is masked to maximize the fault detection profile while reducing the impact on pattern count. When a fault effect cannot be observed at the channel output under any of the flexible masking configurations, the tool uses 1-hot masking to guarantee the detection of such faults.

[Figure 8-34](#) shows how scan chain masking would work to resolve X blocking for the case in “[Why Masking is Needed](#)” on page 282. For one pattern, only the values of Chain 2 are measured on the scan channel output. This way, the Xs in Chain 1 will not block values in Chain 2. Similar patterns would then also be produced where Chain 2 is disabled while the values of Chain 1 are observed on the scan channel output.

Figure 8-34. Example of Scan Chain Masking



When using scan chain masking, the tool records the actual measured value for each cell in the unmasked, selected scan chain in a compactor group. The tool masks the rest of the scan chains in the group, which means the tool changes the values to all Xs. With masking, the capture data for Chain 1 and Chain 2 that you would see in the ASCII pattern file would look similar to [Figure 8-35](#), assuming Chain 2 is to be observed and Chain 1 is masked. The values the tool changed to X for the masked chain are shown in red.

Figure 8-35. Handling of Scan Chain Masking



Following is part of the transcript from a pattern generation run for a simple design where masked patterns were used to improve test coverage. The design has three scan chains, each containing three scan cells. One of the scan chain pins is shared with a functional pin, contrary to recommended practice, in order to illustrate the negative impact such sharing has on test coverage.

```
// -----
// Simulation performed for #gates = 134 #faults = 68
// system mode = analysis pattern source = internal patterns
// -----
// #patterns test #faults #faults #eff. #test
// simulated cvrg in list detected patterns patterns
// deterministic ATPG invoked with abort limit = 30
// ---
// 32 82.51% 16 47 6 6
// ---
// Warning: Unsuccessful test for 10 faults.
// deterministic ATPG invoked with abort limit = 30
// EDT with scan masking.
// ---
// 96 91.26% 0 16 6 12
// ---
```

The transcript shows six non-masked and six masked patterns were required to detect all faults. Here's an excerpt from the ASCII pattern file for the run showing the last unmasked pattern and the first masked pattern:

```
pattern = 5;
apply "edt_grp1_load" 0 =
    chain "edt_channel1" = "000110000000";
end;
force "PI" "100XXX0" 1;
measure "PO" "1XXX" 2;
pulse "/CLOCK" 3;
apply "grp1_unload" 4 =
    chain "chain1" = "1X1";
    chain "chain2" = "1X1";
    chain "chain3" = "0X1";
end;

pattern = 6;
apply "edt_grp1_load" 0 =
    chain "edt_channel1" = "110000000000";
end;
force "PI" "110XXX0" 1;
measure "PO" "0XXX" 2;
pulse "/CLOCK" 3;
apply "grp1_unload" 4 =
    chain "chain1" = "XXX";
    chain "chain2" = "111";
    chain "chain3" = "XXX";
end;
```

The capture data for Pattern 6, the first masked pattern, shows that this pattern masks chain1 and chain3 and observes only chain2.

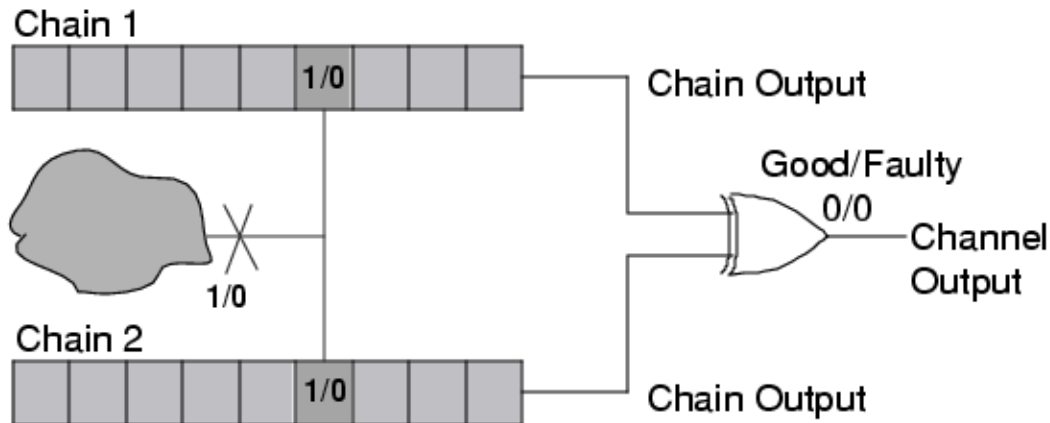
Fault Aliasing

Another potential issue with the compactor used in the EDT logic is called fault aliasing. Assume one fault is observed by two scan cells, and that these scan cells are located in two scan

chains that are compacted to the same scan channel. Further, assume that these cells are in the same locations (columns) in the two chains and neither chain is masked.

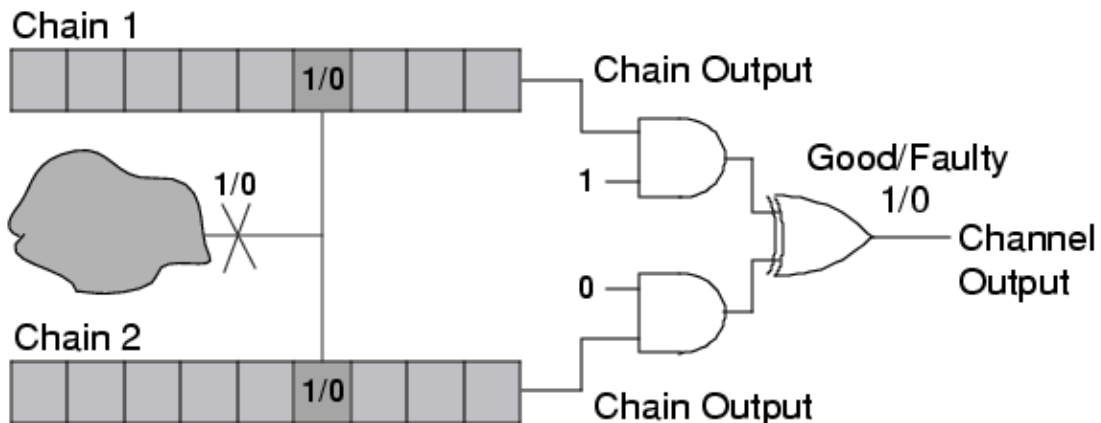
Figure 8-36 illustrates this case. Assume that the good value for a certain pattern is a 1 in the two scan cells. This corresponds to a 0 measured on the scan channel output, due to the XOR in the compactor. If a fault occurs on this site, 0s are measured in the scan cells, which also result in a 0 on the scan channel output. For this unique scenario, it is not possible to see the difference between a good and a faulty circuit.

Figure 8-36. Example of Fault Aliasing



The solution to this problem is to utilize scan chain masking. The tool does this automatically. In compressed ATPG, a fault that is aliased will not be marked detected for the unmasked pattern (Figure 8-36). Instead, the tool uses a masked pattern as shown in Figure 8-37. This mechanism guarantees that all potentially aliased faults are securely detected. Cases in which a fault is always aliased and requires a masking pattern to detect it are rare.

Figure 8-37. Using Masked Patterns to Detect Aliased Faults



About Reordering Patterns

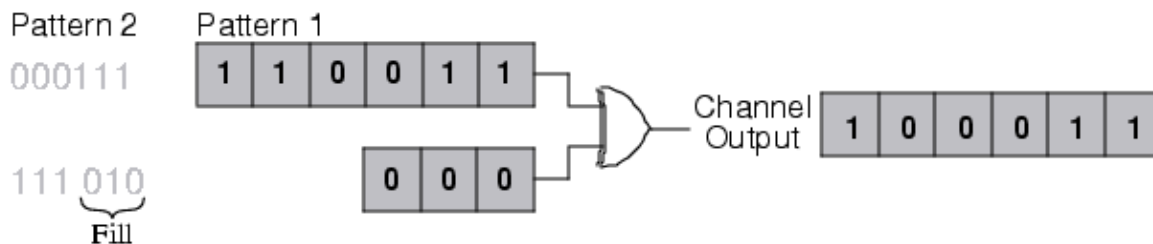
In Tessent Shell, you can reorder patterns using static compaction.

You reorder patterns with the [compress_patterns](#) command, and pattern optimization with the [order_patterns](#) command. You can also use split pattern sets by, for example, reading a binary or ASCII pattern file back into the tool, and then saving a portion of it using the -Begin and -End options to the [write_patterns](#) command.

The tool does not support reordering of serial EDT patterns by a third-party tool, after the compressed patterns are saved.

This has to do with what happens in the compactor when two scan chains have different lengths. Suppose two scan chains are compacted into one channel, as illustrated in [Figure 8-38](#). Chain 1 is six cells long and Chain 2 is three cells long. The captured values of the last three bits of Chain 1 are going to be XOR'd with the first three values of the next pattern being loaded into Chain 2. For regular ATPG, this problem does not occur because the expected values on Chain 2, after you shift three positions, are all Xs. So you never observe the values being loaded as part of the next pattern. But, if that is done with EDT, the last three positions of Chain 1 are XOR'd with X and faults observed on these last cells are lost. Because the padding data for the shorter scan chains is derived from the scan-in data of the next pattern, avoid reordering serial patterns to ensure valid computed scan-out data.

Figure 8-38. Handling Scan Chains of Different Length



Handling of Last Patterns

In order to completely shift out the values contained in the final capture cycle, the tool shifts in the last pattern one additional time so that the output matches the calculated value.

When the design contains chains of different lengths, the tool pads the shorter chains using the values generated by the decompressor during next pattern load. The calculated expected values on the last pattern unload are based on loading the last pattern one more time. Changing the last pattern load will result in simulation mismatches.

Note



If the last pattern load is modified, mismatches will occur.

EDT Aborted Fault Analysis

When creating compressed patterns, some faults may not be detected and become classified as EDT Aborted (EAB). While a fault may become EAB due to insufficient encoding capacity, linear dependency or other reason, it is important to understand what design characteristics may be contributing to coverage loss.

Analysis Overview

A fault is classified as EDT Aborted (EAB) when the test cube (EAB test cube) for the fault is not compressible. Normally, the test coverage loss caused by EAB faults is small but in some special situations there may be many EAB faults, causing notable test coverage loss. Using EAB analysis, you can identify the common scan cell(s) that are involved in many EAB test cubes.

Tessent TestKompress records a number of EAB test cubes during each [create_patterns](#) session for analysis after the session. By default, the tool analysis a minimum of 1000 EAB test cubes. Using the [set_edt_abort_analysis_options](#) command, you can change the default. After issuing the [create_patterns](#) command, you can report the analysis results of the stored EAB test cubes using the [report_edt_abort_analysis](#) command. Using this command, you can customize the report output so that it contains the most specified shift positions, the most specified scan cells, and details to inspect the EAB test cubes.

Note



For some designs, the number of EAB faults analyzed by the tool may be higher than the number of EAB faults in the output of the [report_statistics](#) command. This can happen when a fault is considered EAB during pattern generation (when this analysis is performed) but it is later detected during simulation of a subsequent pattern.

A design may contain multiple EDT blocks and test cube encoding failure may occur in more than one EDT block. The analysis, however, is only performed for the first failing EDT block. For example, if an EAB test cube has specified bits in 3 EDT blocks and the tool found that the bits in the first EDT block cannot successfully be encoded, the thereafter analysis for this EAB test cube will only consider the bits specified in this block. Bits from other blocks are ignored for the rest of the analysis.

Results Analysis

When performing EAB fault analysis, focus your efforts on data that can most efficiently help identify problems and that can be addressed in the design, EDT configuration, or tool setup. As such, it is important to understand the following scenarios:

- If an EAB test cube specifies many more bits in an EDT block than the total number of variables the tool can supply for that block in one pattern, it is expected that the test cube will not be compressible. Given that the number of specified bits are simply too large, it may not be worthwhile to perform analysis for this type of test cubes. The implemented

command allows the user to specify the analysis range for the collected EAB test cubes so that such test cubes can be skipped.

- Note that a variable is one bit shifted from the tester into the EDT decompressor, and ultimately used to provide the decompressed data shifted into scan chains. If the number of bits specified by ATPG exceeds about 90% of the variables shifted into the decompressor, there is a high probability that the test will not be compressible.
- An EAB test cube may specify a large number of bits in the failing block but not all bits are relevant in terms of compressibility. It is possible that only a few bits in this block cannot be compressed when specified by themselves. These bits are the real problem sources that should be analyzed and understood. The focus of the new tool feature is to provide detailed report statistics for these bits. Let's call these responsible bits of an EAB test cube, the smallest EAB test cube.
- When reporting a bit, the tool also reports the property of the bit, if such information is available. For example, if the bit has a cell constraint, is a clock control condition bit, or if the bit is part of the condition bits of an NCP. This information can help locate the problem directly.

Chapter 9

Integrating Compression at the RTL Stage

You can create EDT logic during the RTL design phase, rather than waiting for the complete synthesized gate-level design netlist. Creating the EDT logic early allows you to consider the EDT logic earlier in the floor-planning, placement, and routing phases.

IP Generation and Insertion Using EDT Specification	292
Basic Flow	292
Pipeline Stage Insertion	293
Bused EDT Channel Input and Output Connections	294
Lockup Cells on the Input Side of the EDT Controller	295
Lockup Cells on the Output Side of the EDT Controller	295
Lockup Cells Clock Connections	296
EDT Specification Wrapper Creation	296
Validating the EDT Specification and Creating the EDT IP	298
Legacy Skeleton RTL Flow.....	301
Skeleton Flow Overview	301
Skeleton Design Input and Interface Files	304
Creation of the EDT Logic for a Skeleton Design	309
Integration of the EDT Logic into the Design	310
Skeleton Flow Example	312

IP Generation and Insertion Using EDT Specification

The primary way of creating and, optionally, inserting EDT logic during the RTL stage is using the configuration-based specification EDT wrapper contained in the DftSpecification wrapper.

Basic Flow	292
Pipeline Stage Insertion	293
Bused EDT Channel Input and Output Connections	294
Lockup Cells on the Input Side of the EDT Controller	295
Lockup Cells on the Output Side of the EDT Controller	295
Lockup Cells Clock Connections	296
EDT Specification Wrapper Creation	296
Validating the EDT Specification and Creating the EDT IP	298

Basic Flow

The EDT specification flow is used to create and optionally insert the EDT IP into your RTL. In general, this flow consists of creating the EDT specification, validating the specification, and generating the EDT IP.

At its most basic level, an EDT specification is an ASCII file that describes your EDT IP using configuration data syntax to encode the specification. You input this EDT specification into Tessent Shell, which creates the EDT IP and, if desired, inserts the IP into your RTL.

Flow Limitations

With the EDT specification RTL flow, you cannot specify the first and last scan cell of each chain. EDT IP will be created with the assumption that all scan chains will be inserted with a leading-edge cell for the first scan cell and a trailing-edge cell for the last scan cell. This is a current limitation with this flow.

Requirements

To create EDT logic during the RTL stage, you must know the following parameters for your design:

- Number of external scan channels.
- Number of internal scan chains.
- Longest scan chain length range. This is an estimate of the minimum number of scan cells and maximum number of scan cells the tool can expect in the longest scan chain.

You should also have knowledge about the design interface if you are creating/inserting the EDT logic external to the design core.

Flow Overview

The EDT specification flow with Tessent Shell consists of the following steps:

1. Create the EDT specification wrapper that describes the EDT IP you require using the configuration-based specification—see “[EDT Specification Wrapper Creation](#)” on page 296.
2. Validate and process the EDT specification. During this step, you create in IJTAG network to create connection points for the EDT IP and generate the EDT IP. You can also inserted the created EDT IP into your RTL during this step—see “[Validating the EDT Specification and Creating the EDT IP](#)” on page 298.

Pipeline Stage Insertion

When using configuration-based specification EDT, the pipeline stage insertion order depends on the order of the PipelineStage configuration data wrappers.

The tool inserts the pipeline stages in the design from left-to-right starting from the first specified [PipelineStage](#) wrapper. For a single EDT input or output channel, you specify the first pipeline stage using the wrapper placed above all other PipelineStage wrappers within the scope of its parent wrapper. The same rule applies to the last pipeline stage, which is specified by the wrapper placed below all other PipelineStage wrappers of the same configuration-based specification data scope.

For EDT *input* channels the pipeline stage inserted closest to the EDT decompressor is the last specified pipeline stage as in the following example configuration-based specification wrapper:

```
EdtChannelsIn(1) {
  port_pin_name: top_channel_in1 ;
  PipelineStage {
    leaf_instance_name: pipe1 ;
  }
  PipelineStage {
    leaf_instance_name: pipe2 ;
  }
  PipelineStage {
    leaf_instance_name: pipe3 ;
  }
}
```

In the above example, the insertion process creates three pipeline stages on the first EDT input channel with “pipe3” placed closest to the EDT decompressor.

For EDT *output* channels, the tool inserts the first specified pipeline stage closest to the EDT compactor as in the following example configuration-based specification wrapper:

```
EdtChannelsOut(1) {  
  port_pin_name: top_channel_out1 ;  
  PipelineStage {  
    leaf_instance_name: pipe1 ;  
  }  
  PipelineStage {  
    leaf_instance_name: pipe2 ;  
  }  
  PipelineStage {  
    leaf_instance_name: pipe3 ;  
  }  
}
```

In the above example, the insertion process creates three pipeline stages on the first EDT output channel with “pipe1” placed closest to the EDT compactor.

BusEDT Channel Input and Output Connections

Using the configuration-based specification EDT flow, you can specify busEDT channel input and output connections in a single wrapper.

The [EdtChannelsIn](#) and [EdtChannelsOut](#) accept multiple EDT channel index values using the (*id*) component of the wrapper. When specified, the tool connects them to a bus or a list of ports or pins by using a single [EdtChannelsIn](#) or [EdtChannelsOut](#) configuration-based specification data wrapper.

For example:

```
EdtChannelsIn(4:1) {  
  port_pin_name : top_bus[3:0] ;  
}
```

The tool connects the first four EDT input channels to the *top_bus[3:0]* port object. The first EDT channel connects to *top_bus[0]* and the fourth channel connects to *top_bus[3]*.

When specifying multiple index values using either [EdtChannelsIn\(id\)](#) or [EdtChannelsOut\(id\)](#), the order of the EDT IP connections is determined by the order of the channel index range. The tool performs the insertion “left to left” and “right to right”; given this, you should check if the order of the channel index range is the same as the order of the bus or port list. If either the channel index range or the connection bus range has a different order the created connections, then the tool “swaps” the connections, that is the highest EDT IP channel is connected to the LSB of the connection object.

The [process_dft_specification](#) command for EDT creates bus ports for EDT channel connections on the block level as specified in the configuration data. If the bus ports exist

already, they are used for EDT channel connections. If they do not exist, they will be created on the block level by process_dft_specification.

Limitation

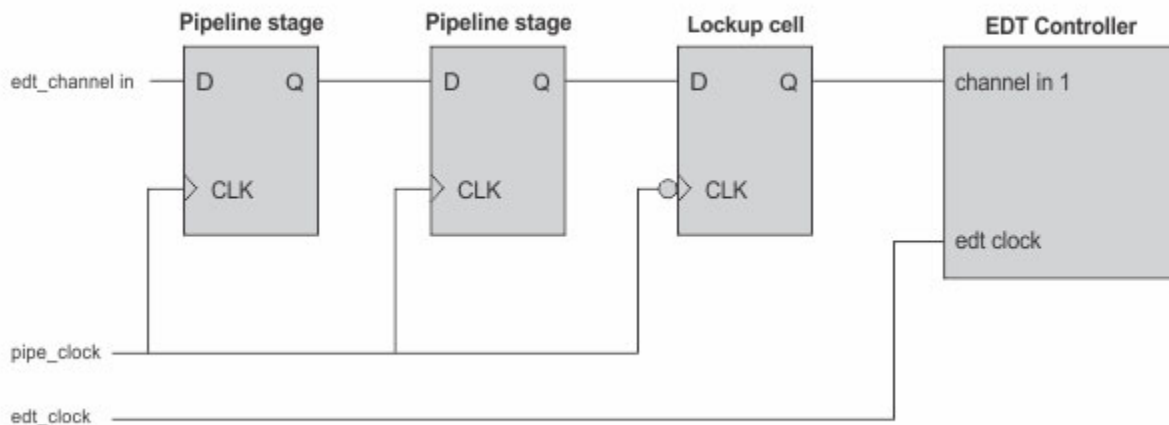
When you specify at least one input connection, you must specify all values and ranges.

Lockup Cells on the Input Side of the EDT Controller

If pipeline stages are inserted on the input side then a lockup cell will be added between the last input pipeline stage (counting from the top level EDT input channel port) and the EDT logic only if the clock driving the last pipeline stage is different than the clock driving the EDT logic.

Figure 9-1 provides an example of a lockup cell on the input side:

Figure 9-1. Lockup Cell EDT Controller Input Side

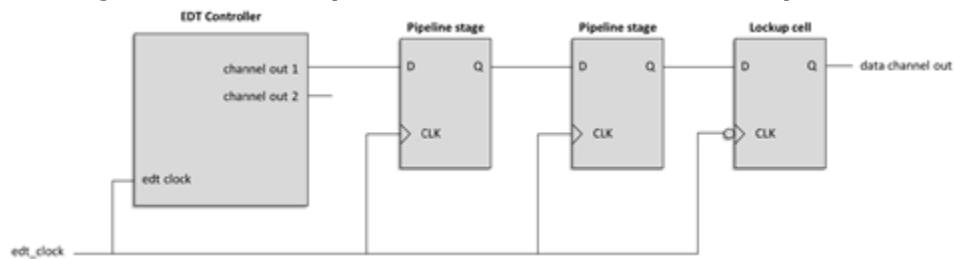


Lockup Cells on the Output Side of the EDT Controller

If pipeline stages are inserted on the output side then a lockup cell is always inserted after the last output pipeline stage (counting from the EDT output channel pin on the EDT Controller).

Figure 9-2 provides an example of a lockup cell on the output side.

Figure 9-2. Lockup Cells on EDT Controller Output Side



Lockup Cells Clock Connections

Every pipeline stage can be clocked by a different source; however, when the tool automatically inserts lockup cells, these cells use the clocks that drive the pipelines.

The tool connects the clock pin of every inserted lockup cell to the clock that drives the last pipeline stage of EDT input or output channel as follows:

- On the input side, if the tool inserts a lockup cell, then the cell is driven by the clock source of the pipeline stage *closest* to the EDT IP.
- On the output side, if the tool inserts a lockup cell, then the cell is driven by the clock of the pipeline stage *farthest* from the EDT IP.

EDT Specification Wrapper Creation

The first step to integrating EDT IP into your RTL is creating the EDT specification wrapper.

The [EDT wrapper](#) and [DftSpecification wrapper](#) sections in the *Tessent Shell Reference Manual* provide the complete syntax and options you use when creating your EDT specification wrapper.

Once you have nested the EDT wrapper within the DftSpecification wrapper, then you use Tessent Shell to process and validate your wrapper and create the EDT IP. Tessent Shell writes the EDT IP output to the [Tessent Shell Data Base \(TSDB\)](#), a structured directory containing subdirectories and files.

EDT Specification Example

You manually create the EDT wrapper using the syntax described in “EDT” in the *Tessent Shell Reference Manual*. The following example shows a basic EDT specification wrapper nested in a *DftSpecification* wrapper you can use as a guide:

```

DftSpecification(CoreA,rtl) +{
  EDT +{
    Controller(1) +{
      scan_chain_count : 16;
      input_channel_count : 2;
      output_channel_count : 2;
      longest_chain_range : 100, 110;
      separate_control_data_channels : on;
      leaf_instance_name : edt_inst_new;
      BypassChains {
        present : on;
        bypass_chain_count : 2;
        single_bypass_chain : on;
      }
      Compactor {
        type : xpress;
        pipeline_logic_levels_in_compactor : 5;
      }
      Clocking {
        type : edge;
        lockup_cells : on;
        retime_chain_boundaries : off;
        reset_signal : off;
      }
      HighCompressionConfiguration {
        present : on;
        input_channel_count : 1;
        output_channel_count : 1;
      }
      ShiftPowerOptions {
        present : on ;
        full_control : on ;
        min_switching_threshold_percentage : 25 ;
      }
      Connections +{
        edt_clock : clock1 ;
        EdtChannelsIn(2) {
          port_pin_name : user_control1_chin2;
        }
        EdtChannelsOut(1) {
          port_pin_name : user_control1_chout1;
        }
      }
    }
  }
}

```

In the *DftSpecification* wrapper, you can only define the nested EDT specification once. Within each Controller sub-wrapper, you specify using required configuration parameters, for example basic scan chain and EDT channel information, for each EDT IP Controller you define.

Validating the EDT Specification and Creating the EDT IP

Use Tessent Shell to validate the DftSpecification containing the EDT specification and create the EDT IP. The tool writes the EDT IP to TSDB. The first step of the EDT IP RTL flow is inserting the IJTAG network. The IJTAG Network Insertion functionality enables you to connect existing instruments and insert SIBs, TDRs, and ScanMuxes to create your own IJTAG network. You can optionally insert the EDT IP into the RTL.

Prerequisites

- EDT Specification. See “[EDT Specification Wrapper Creation](#)” on page 296.
- A list of IDs that specify the Scan Resource Instrument (SIB) that will be used to create the Sib(id) wrappers in the IJTAG network. Refer to [create_dft_specification](#) in the *Tessent Shell Reference Manual* for more information and “[IJTAG Network Insertion](#)” in the *Tessent IJTAG User’s Manual*.
- Scan cell estimates. Refer to “[Requirements](#)” on page 292.

Procedure

1. From a shell, invoke Tessent Shell.

```
% tessent -shell
```

2. Set the context to “dft-rtl”.

```
SETUP> set_context dft-rtl
```

3. Read in the RTL.

```
SETUP> read_verilog CoreA.v
```

4. Set the current design and the design level.

```
SETUP> set_current_design rtl_design
```

```
SETUP> set_design_level physical_block
```

5. Set the TSDB directory location if other than the present working directory.

```
SETUP> set_tsdb_output_dir tsdbA
```

6. Add static DFT control signals. For example:

```
SETUP> add_dft_signals ltest_en int_ltest_en ext_ltest_en int_mode ext_mode
```

The [add_dft_signals](#) command specifies DFT signals used to control aspects of DFT logic.

7. Add dynamic DFT control signals. For example:

```
SETUP> add_dft_signals scan_en -source_node scan_en
```

```
SETUP> add_dft_signals {test_clock edt_update} -source_nodes test_clock \  
edt_update
```

8. If you are using fast capture, then define functional clocks. For example:

```
SETUP> foreach clk [dict keys $FUNC_CLOCKS] {
    add_clocks 0 $clk -period [dict get $FUNC_CLOCKS $clk]
}
```

9. Change from setup to analysis mode.

```
SETUP> check_design_rules
```

In RTL context, [check_design_rules](#) synthesizes the design in parts before creating the flat model. The DRC rules specific to the current context are then executed. If no DRC with a severity of error fails, the tool enters analysis mode.

10. Create the IJTAG network, specifying the Sib IDs. For example:

```
ANALYSIS> create_dft_specification -sri_sib_list "edt"
```

Specifying the -sri_sib_list switch and string pair with create_dft_specification creates connection points for the inserted EDT to connect to.

11. Obtain scan cell estimate to figure out number of EDT chains. For example:

```
set scannable_flop_count \
[ sizeof_collection [get_gate_pins -filter \
{primitive_name==DFF && pin_index==0 && !is_non_scannable}] ]

set scannable_flop_count \
[expr {$scannable_flop_count + 4*[llength [dict keys $FUNC_CLOCKS]]}]

puts "Estimated scan cell count: $scannable_flop_count"

Estimated scan cell count: 7766

set chain_count \
[expr {int(ceil($scannable_flop_count / $SCAN_CHAIN_LENGTH) \
* (1 + $CHAIN_COUNT_MARGIN))}]

(code) puts "Chain count: $chain_count"

Chain count: 41
```

12. Add EDT and the EDT Controller to the configuration tree created in the previous step.

```
ANALYSIS > add_config_element EDT -in_wrapper dftspec
ANALYSIS > add_config_element Controller(1) -in_wrapper dftspec
```

13. Connect to the EDT SIB. For example, the following Tcl proc:

```
set_config_value ijtag_host_interface "Sib(edt)" -in_wrapper $edt_cont
set_config_value scan_chain_count -in $edt_cont $chain_count
set_config_value input_channel_count -in $edt_cont $::EDT_IN_CHANNELS
set_config_value output_channel_count -in $edt_cont $::EDT_OUT_CHANNELS
set_config_value longest_chain_range -in $edt_cont $::LONGEST_CHAIN_RANGE
```

14. Validate and create the EDT IP based on the EDT specification.

```
ANALYSIS> process_dft_specification -no_insertion
```

The tool processes the DftSpecification and writes the EDT IP to the [TSDB directory](#).

If required, you can also insert the EDT IP into your RTL by omitting the `-no_insertion` switch to the `process_dft_specification` command.

15. Extract the ICL network:

ANALYSIS > extract_icl

Related Topics

[add_config_element \[Tessent Shell Reference Manual\]](#)
[add_dft_signals \[Tessent Shell Reference Manual\]](#)
[check_design_rules \[Tessent Shell Reference Manual\]](#)
[create_dft_specification \[Tessent Shell Reference Manual\]](#)
[extract_icl \[Tessent Shell Reference Manual\]](#)
[process_dft_specification \[Tessent Shell Reference Manual\]](#)
[read_verilog \[Tessent Shell Reference Manual\]](#)
[set_config_value \[Tessent Shell Reference Manual\]](#)
[set_context \[Tessent Shell Reference Manual\]](#)
[set_current_design \[Tessent Shell Reference Manual\]](#)
[set_design_level \[Tessent Shell Reference Manual\]](#)
[set_tsdb_output_directory \[Tessent Shell Reference Manual\]](#)

Legacy Skeleton RTL Flow

The legacy skeleton RTL flow uses the `create_skeleton_design` utility to create a skeleton design.

Note

 While you can still use the skeleton flow, Mentor Graphics recommends moving to the “[IP Generation and Insertion Using EDT Specification](#)” on page 292 to integrate EDT logic at the RTL stage.

Skeleton Flow Overview	301
Skeleton Design Input and Interface Files.....	304
Skeleton Design Input File	305
Skeleton Design Interface File	308
Creation of the EDT Logic for a Skeleton Design.....	309
Longest Scan Chain Range Estimate	309
Integration of the EDT Logic into the Design.....	310
Skeleton Flow Example.....	312
Input File.....	313

Skeleton Flow Overview

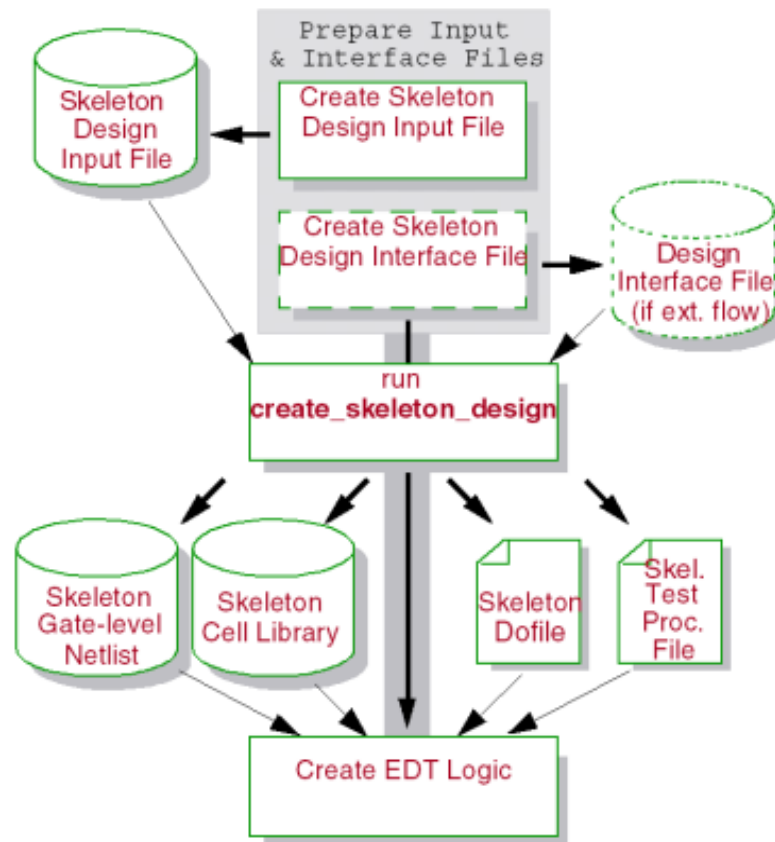
The `create_skeleton_design` utility is used to create a skeleton design.

[Figure 9-3](#) shows the IP Creation RTL stage flow. The utility, `create_skeleton_design` is used to create a skeleton design. This utility writes out a gate-level *skeleton* Verilog design and several related files required to create EDT logic.

To use the `create_skeleton_design` utility, you must create a Skeleton Design Input File. The Skeleton Design Input File contains the requisite number of scan chains with the first and last cell of each of these chains driven by the appropriate clocks. For more information, see “[Skeleton Design Input File](#)” on page 305.

If you are creating/inserting the EDT logic external to the design core, you must also create a Skeleton Design Interface File. For more information, see “[Skeleton Design Interface File](#)” on page 308.

Figure 9-3. EDT IP Creation RTL Stage Flow



Use the following steps to create EDT logic for an RTL design:

1. Create a Skeleton Design Input File. For more information, see “[Skeleton Design Input File](#)” on page 305.
2. If you are inserting the EDT logic external to the core design ([Compressed Pattern External Flow](#)), create a Design Interface File to provide the interface description of the core design in Verilog format. For more information, see “[Skeleton Design Interface File](#)” on page 308.
3. Run the [create_skeleton_design](#) utility. For example:

- o Internal Flow:

```
create_skeleton_design -o output_file_prefix \
-i skeleton_design_input_file
```

- o External Flow:

```
create_skeleton_design -o output_file_prefix \
-i skeleton_design_input_file -design_interface \
file_name
```

The utility writes out the following four files:

`<output_file_prefix>.v` — Skeleton design netlist

`<output_file_prefix>.dofile` — Dofile

`<output_file_prefix>.testproc` — Test procedure file

`<output_file_prefix>.atpglib` — Tessent cell library

For a complete example showing [create_skeleton_design](#) input files and the resultant output files, see the “[Skeleton Flow Example](#)” on page 312.

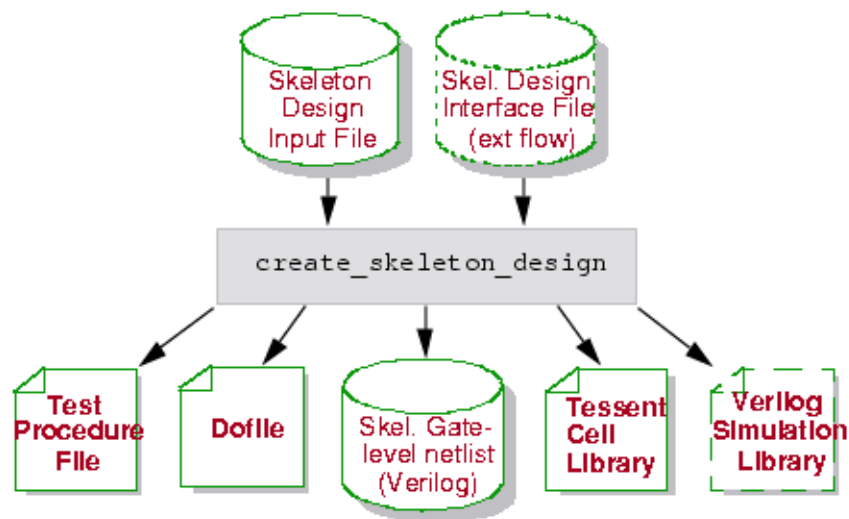
4. Invoke Tessent Shell, set the correct context, and read the skeleton design netlist and the Tessent cell library.
5. Provide compression setup commands.
 - Run the dofile and test procedure file to set up the scan chains for the EDT logic.
 - Issue the [set_edt_options](#) command to specify the number of scan channels. You should use the `-Longest_chain_range` switch with this command to specify an estimated length range (*min_number_cells* and *max_number_cells*) for the longest scan chain in the design. For additional information, refer to “[Longest Scan Chain Range Estimate](#)” on page 309.
6. Provide EDT DRC, configuration, and logic creation commands.
 - Use the [set_system_mode](#) analysis command to flatten the design and run DRCs.
 - Issue other configuration commands as needed.
 - Write out the RTL description of the EDT logic with the [write_edt_files](#) command.

Skeleton Design Input and Interface Files

This section describes the inputs and outputs for the `create_skeleton_design` utility.

These inputs and outputs are illustrated in [Figure 9-4](#). The [Skeleton Design Input File](#) is always required. The [Skeleton Design Interface File](#) is needed only if you will be creating the EDT logic external to the core design (see “[Compressed Pattern External Flow](#)” on page 45). You must create both files using the format and syntax described in the following subsections.

Figure 9-4. Create_skeleton_design Inputs and Outputs



Skeleton Design Input File	305
Input File Format	305
Input File Example	307
Skeleton Design Interface File	308

Skeleton Design Input File

Create the skeleton design input file using the rules described in the next section.

Input File Format	305
Input File Example	307

Input File Format

This section describes the format of the input file for `create_skeleton_design`.

Example [Skeleton Design Input File Format](#) shows the format of the skeleton design input file. Required keywords are highlighted in bold. This file contains distinct sections that are described after the example. shows a small working example.

Figure 9-5. Skeleton Design Input File Format

```
// Description of scan pins and LSSD system clock with design interface
// (required)
scan_chain_input <prefix> <bused|indexed> [<starting_index_if_indexed>]
scan_chain_output <prefix> <bused|indexed> [<starting_index_if_indexed>]
lssd_system_clock <clock_name> // Any system clock for LSSD designs
scan_enable <scan_enable_name> // Any scan_enable pin name

// Clock definitions (required)
begin_clocks // Keyword to begin clock definitions
    <clock_name> <off_state> // Clock name and off state
    <clock_name> <off_state> // Clock name and off state
end_clocks // Keyword to end clock definitions

// Scan chain specification (required)
begin_chains // Keyword to begin chain
definitions
// first_chain_number and last_chain_number specify range of chains
// MUXD chain
<first_chain_number> <last_chain_number> <chain_length> \
    <TE|LE> <first_cell_clock> <TE|LE> <last_cell_clock>

//LSSD chain
<first_chain_number> <last_chain_number> <chain_length> \
    LA <first_cell_master_clock> <first_cell_slave_clock> \
    LA <last_cell_master_clock> <last_cell_slave_clock>
end_chains // Keyword to end chain definitions
```

Scan Pins and LSSD System Clock Specification Section

Note

 This section is required when you use the `-Design_interface` switch with [create_skeleton_design](#) to enable the tool to create a correct instantiation of the core in the top-level EDT wrapper (“[Compressed Pattern External Flow](#)” on page 45). If the scan pins specified in this section are not present in the design interface, the utility automatically adds them to the skeleton design. You can omit this section if you are not using the `-Design_interface` switch.

In this section, specify the scan chain pin name prefix and the type, bused or indexed, using the keywords, “`scan_chain_input`” and “`scan_chain_output`”. The bused option will result in scan chain pins being declared as vectors, i.e., `<prefix>[Max-1:0]`. The indexed option will result in scan chain pins being declared as scalars, numbered consecutively beginning with the specified starting index, and named in “`<prefix><index>`” format.

If you intend to share channel outputs, you can specify the name of a scan enable pin using the “`scan_enable`” keyword. If you do not specify a scan enable pin, the tool will automatically add a default pin named “`scan_en`” to the output skeleton design.

If the design contains LSSD scan cells, you can optionally use the `lssd_system_clock` keyword to specify the name of any one LSSD system clock. If you do not specify a name, the tool will use the default name, “`lssd_system_clock`”.

Clock Definition Section

In this section, specify clock names and their corresponding off states. The utility uses these off states to create a correct skeleton dofile and skeleton test procedure file. (See the [add_clocks](#) command for additional details about the meaning of clock off states.)

Scan Chain Specification Section

The scan chain specification section is the key section. Here, you specify the number of scan chains, length of the chains, and clocking of the first and last scan cell.

Note

 If the EDT logic clock is pulsed before the scan chain shift clock, you do not need to account for the clocking of the first and last cell in each scan chain as this information will not be evaluated. For more information, see “[Pulse EDT Clock Before Scan Shift Clocks](#)” on page 81.

To simplify and shorten this section, you can list, on one line, a range of chains that have the same specifications. Each line should contain the chain number of the first chain in the range, the chain number of the last chain in the range, length of the chains, and the edge and clock information of the first and last scan cell. For IP creation with the skeleton flow, the length of the scan chains can be any value not less than 2, but typically 2 suffices for the purpose of

creating appropriate EDT logic. In the created skeleton design, all chains in this range will be the same length and contain a first and last scan cell with the same clocking.

The edge specification must be one of the following:

- LE for a scan cell whose output changes on the leading edge of the specified clock
- TE for a scan cell whose output changes on the trailing edge of the specified clock
- LA for an LSSD scan cell

When you specify the clock edge of the last scan cell, it is critical to include the lockup cell timing as well. For example, if a leading edge (LE) triggered scan memory element is followed by a lockup cell, the edge specification of the scan *cell* must be TE (not LE) since the cell contains a scan memory element followed by a lockup cell and the scan cell output changes on the trailing edge (TE) of the clock. Specifying incorrect edges will result in the tool inserting improper lockup cells and you may need to regenerate the EDT logic later.

Note



When the scan chain specification indicates the first and last scan cell have master/slave or master/copy clocking (for example, an LE first scan cell and a TE last scan cell), the [create_skeleton_design](#) utility will increase that chain's length by one cell in the skeleton netlist it writes out. This is done to satisfy a requirement of lockup cell analysis and will not alter the EDT logic; the length of the scan chains seen by the tool after it reads in the skeleton netlist will be as specified in the skeleton design input file.

Comment Lines

You can place comments in the file by beginning them with a double slash (/). Everything after a double slash on a line is treated as a comment and ignored.

Input File Example

The following example utilizes bused scan chain input and output pins. It also defines two clocks, clk1 and clk2, with off-states 0 and 1, respectively.

A total of eight scan chains are specified. Chains 1 through 4 are of length 2, with the first cell being LE clk1 triggered and the last cell being TE clk1 triggered. Chains 5 and 6 are of length 3, with the first cell being LE clk2 triggered and the last cell being TE clk2 triggered. Chains 7 and 8 are also of length 3, with the first and last cells being of LSSD type, clocked by master and slave clocks, mclk and sclk, respectively.

Figure 9-6. Skeleton Design Input File Example

```
// Double slashes (//) mean everything following on the line is a comment.
//
// edt_si[7:0] and edt_so[7:0] pins are created for scan chains.
scan_chain_input edt_si bused
scan_chain_output edt_so bused
begin_clocks
    clk1 0
    clk2 1
    mclk 0
    sclk 0
end_clocks
begin_chains
// chains 1 to 4 have the following characteristics (Mux scan)
    1 4 2 LE clk1 TE clk1
// chains 5 and 6 have the following characteristics (Mux scan)
    5 6 3 LE clk2 TE clk2
// chains 7 and 8 have the following characteristics (LSSD)
    7 8 3 LA mclk sclk LA mclk sclk
end_chains
```

Skeleton Design Interface File

You should create a skeleton design interface file if you are creating EDT logic that is inserted external to the design core. It should contain only the interface description of the core design in Verilog format; that is, only the module port list and declarations of these ports as input, output, or inout.

For an example of this file, see “[Interface File](#)” on page 313.

Tip

i The interface file ensures the files written out by the **create_skeleton_design** utility contains the information the tool needs to write out valid core blackbox (*_core_blackbox.v) and top-level wrapper (*_edt_top.v) files.

Creation of the EDT Logic for a Skeleton Design

After invoking Tessent Shell and reading the skeleton design, you must set up the following parameters with the `set_edt_options` command:

- Number of external scan channels
- Estimate of the longest scan chain length (optional). This value allows flexibility when configuring scan chains. For more information, see “[Longest Scan Chain Range Estimate](#)” on page 309.

For example:

```
set_edt_options -channels 2
set_edt_options -longest_chain_range 75 125
```

For more information on setting up and creating the EDT logic, see “[Creation of EDT Logic Files](#)” on page 96.

Longest Scan Chain Range Estimate 309

Longest Scan Chain Range Estimate

The longest scan chain range estimate defines a range for the length of the longest scan chain in the design. The EDT logic is then configured to allow the longest scan chain in the design to fall within this range without requiring the EDT logic to be regenerated.

This builds in flexibility in cases, such as the RTL flow, where the scan chains may change after the EDT logic is created as follows:

- **min_number_cells** — Specifies the lower bound of the longest scan chain range. You should avoid specifying an artificially low value for the `set_edt_options “min_number_cells”` command option. Specifying an artificially low value results in the creation of an EDT logic configuration that can result in incompressible patterns.

Note that the `set_edt_options “longest_chain_range”` switch defines a range for the length of the longest scan chain in your design — this does *not* mean the range of lengths of all the scan chains in your design. Setting the `min_number_cells` option based on these considerations enables the tool to configure the EDT logic to assure robust pattern compression.

For more information on compactors, see “[Understanding Compactor Options](#)” on page 279.

- **max_number_cells** — Specifies the higher bound of the longest scan chain range and is used to configure the phase shifter in the decompressor. The phase shifter is configured to separate the bit streams provided to the scan chains by at least as many cycles as specified by the `max_number_cells` value. This reduces linear dependencies among the bit streams supplied to the internal scan chains.

The flexibility of this restriction is determined by the linear dependencies present in a design and the number of scan cells specified for the longest scan chain. Some designs tolerate up to a 25% increase in scan chain length before the EDT logic is affected.

Integration of the EDT Logic into the Design

After you create the EDT logic, integrating it into the design is a manual process.

- For EDT logic created external to the design core (“[Compressed Pattern External Flow](#)” on page 45):

If you provided the `create_skeleton_design` utility with the recommended interface file when it generated the skeleton design, you can continue with the compressed pattern external flow (optionally insert I/O pads and boundary scan, then synthesize the I/O pads, boundary scan, and EDT logic).

If you did not use an interface file, you will need to manually provide the interface and all related interconnects needed for the functional design before synthesizing the EDT logic.

- For EDT logic created within the design core (“[Compressed Pattern Internal Flow](#)” on page 48):

Integrating the EDT logic into the design is a manual process you perform using your own tools and infrastructure to stitch together different blocks of the design to create a top level design.

Note

 The Design Compiler synthesis script that the tool writes out does not contain information for connecting the EDT logic to design I/O pads, as the tool did not have access to the complete netlist when it created the EDT logic.

Knowing When to Regenerate the EDT Logic

By the time the gate-level netlist is available, there may be changes to the design that affect the EDT logic as described in the following list. When one of these changes occurs in the design, the safest approach is to always regenerate the EDT logic and compare the new RTL with the previous RTL to determine if the EDT logic is changed.

- **Number of Channels or Chains has Changed** — In this case, the EDT logic must be regenerated.
- **Clocking of a First or Last Scan Cell has Changed** — Whether the EDT logic actually needs to be regenerated depends on whether the clock edge that triggers the first or last scan cell has changed and whether lockup cells are inserted for bypass mode scan chains. You should regenerate the EDT logic any time the clocking of the first or last scan cell changes. Note, this scan chain clocking information is not relevant (not a cause

for regenerating EDT logic) if you set up the EDT clock to pulse before the scan chain shift clocks. For more information, see “[Pulse EDT Clock Before Scan Shift Clocks](#)” on page 81.

- **Length of the Longest Scan Chain is less than the min_number_cells Specified with the set_edt_options -Longest_chain_range Switch** — If the EDT logic uses the Xpress compactor (default), this value does not affect the architecture and the EDT logic does not need to be regenerated.

However, if the EDT logic uses the Basic compactor, this parameter is used to configure the length of the mask register in the compactor. In this case, you should regenerate the EDT logic. For more information, see “[Longest Scan Chain Range Estimate](#)” on page 309”.

- **Length of the Longest Scan Chain is Greater than the max_number_cells Specified with the set_edt_options -Longest_chain_range Switch** — Whether the EDT logic actually changes or not depends on whether the phase shifter in the decompressor needs to be redesigned or not. The flexibility of this restriction is determined by the linear dependencies present in a design and the number of scan cells specified for the longest scan chain. Some designs tolerate up to a 25% increase in scan chain length before the EDT logic is affected. For more information, see “[Longest Scan Chain Range Estimate](#)” on page 309”.

Skeleton Flow Example

This section shows example skeleton design input and interface files and the output files the **create_skeleton_design** utility generated from them.

Input File **313**

Input File

The following example skeleton design input file, *my_skel_des.in*, utilizes indexed scan chain input and output pins. The file defines two clocks, NX1 and NX2, with off-states 0, and specifies a total of 16 scan chains, most of which are 31 scan cells long. Notice the clocking of the first and last scan cell in each chain is specified, but no other scan cell definition is required. This is because the utility has built-in ATPG models of simple mux-DFF and LSSD scan cells that are sufficient for it to write out a skeleton design (and for the tool to use later to create the EDT logic).

Note



If you will be creating the EDT logic within the core design (“[Compressed Pattern Internal Flow](#)” on page 48), this file is the only input the utility needs.

```
scan_chain_input scan_in indexed 1
scan_chain_output scan_out indexed 1

begin_clocks
  NX1 0
  NX2 0
end_clocks

begin_chains
  1 1 31 TE NX1 TE NX1
  2 2 30 TE NX1 TE NX1
  3 3 30 TE NX1 TE NX1
  4 4 31 TE NX1 TE NX1
  5 5 31 TE NX1 TE NX1
  6 6 32 LE NX2 LE NX2
  7 7 31 LE NX2 LE NX2
  8 8 31 LE NX2 LE NX2
  9 9 31 LE NX2 LE NX2
  10 10 31 LE NX2 LE NX2
  11 11 31 LE NX2 LE NX2
  12 12 31 LE NX2 LE NX2
  13 13 31 LE NX2 LE NX2
  14 14 31 LE NX2 LE NX2
  15 15 31 LE NX2 LE NX2
  16 16 31 LE NX2 LE NX2
end_chains
```

Interface File	313
Outputs	314

Interface File

The following shows an example interface file *nemo6_blackbox.v* for the design described in the preceding input file.

Use of an interface file is recommended if you intend to create the EDT logic as a wrapper external to the core design (“[Compressed Pattern External Flow](#)” on page 45).

```

module nemo6 ( NMOE , NMWE , DLM , ALE , NPSEN , NALEN , NFWE , NFOE ,
              NSFRWE , NSFROE , IDLE , XOFF , OA , OB , OC , OD , AE ,
              BE , CE , DE , FA , FO , M , NX1 , NX2 , RST , NEA ,
              NESFR , ALEI , PSEI , AI , BI , CI , DI , FI , MD ,
              scan_in1 , scan_out1 , scan_in2 , scan_out2 , scan_in3 ,
              scan_out3 , scan_in4 , scan_out4 , scan_in5 , scan_out5 ,
              scan_in6 , scan_out6 , scan_in7 , scan_out7 , scan_in8 ,
              scan_out8 , scan_in9 , scan_out9 , scan_in10 , scan_out10 ,
              scan_in11 , scan_out11 , scan_in12 , scan_out12 ,
              scan_in13 , scan_out13 , scan_in14 , scan_out14 ,
              scan_in15 , scan_out15 , scan_in16 , scan_out16 , scan_en);
input  NX1 , NX2 , RST , NEA , NESFR , ALEI , PSEI , scan_in1 , scan_in2 ,
       scan_in3 , scan_in4 , scan_in5 , scan_in6 , scan_in7 , scan_in8 ,
       scan_in9 , scan_in10 , scan_in11 , scan_in12 , scan_in13 ,
       scan_in14 , scan_in15 , scan_in16 , scan_en ;
input  [7:0] AI ;
input  [7:0] BI ;
input  [7:0] CI ;
input  [7:0] DI ;
input  [7:0] FI ;
input  [7:0] MD ;
output NMOE , NMWE , DLM , ALE , NPSEN , NALEN , NFWE , NFOE , NSFRWE ,
       NSFROE , IDLE , XOFF , scan_out1 , scan_out2 , scan_out3 ,
       scan_out4 , scan_out5 , scan_out6 , scan_out7 , scan_out8 ,
       scan_out9 , scan_out10 , scan_out11 , scan_out12 , scan_out13 ,
       scan_out14 , scan_out15 , scan_out16 ;
output [7:0] OA ;
output [7:0] OB ;
output [7:0] OC ;
output [7:0] OD ;
output [7:0] AE ;
output [7:0] BE ;
output [7:0] CE ;
output [7:0] DE ;
output [7:0] FA ;
output [7:0] FO ;
output [15:0] M ;
endmodule

```

Outputs

This section shows examples of the four ASCII files written out by the **create_skeleton_design** utility when run on the preceding input and interface files using the following shell command:

```

create_skeleton_design -o bb1 -design_interface nemo6_blackbox.v \
                      -i my_skel_des.in

```

The utility wrote out the following files:

bb1.v bb1.dofile bb1.testproc bb1.atpglib

Skeleton Design

Following is the gate-level skeleton netlist that resulted from the example input and interface files of the preceding section. For brevity, lines are not shown when content is readily apparent

from the structure of the netlist. Parts attributable to the interface file are highlighted in bold; the utility would not have included them if there had not been an interface file.

Note



The utility obtains the module name from the interface file, if available. If you do not use an interface file, the utility names the module “skeleton_design_top”.

```
module nemo6 (NMOE, NMWE, DLM, ALE, NPSEN, NALEN, NFWE,
NFOE, NSFRWE,
NSFROE, IDLE, XOFF, OA, OB, OC,
OD, AE, BE, CE, DE, FA, FO, M, NX1,
NX2,
RST, NEA, NESFR, ALEI, PSEI, AI,
BI, CI, DI, FI, MD, scan_in1,
scan_in2, ..., scan_in16, scan_out1, scan_out2, ..., scan_out16, scan_en);

output NMOE;
output NMWE;
output DLM;
output ALE;
output NPSEN;
output NALEN;
output NFWE;
output NFOE;
output NSFRWE;
output NSFROE;
output IDLE;
output XOFF;
output [7:0] OA;
output [7:0] OB;
output [7:0] OC;
output [7:0] OD;
output [7:0] AE;
output [7:0] BE;
output [7:0] CE;
output [7:0] DE;
output [7:0] FA;
output [7:0] FO;
output [15:0] M;
input NX1;
input NX2;
input RST;
input NEA;
input NESFR;
input ALEI;
input PSEI;
input [7:0] AI;
input [7:0] BI;
input [7:0] CI;
input [7:0] DI;
input [7:0] FI;
input [7:0] MD;

input scan_in1;
input scan_in2;
...
input scan_in16;
output scan_out1;
output scan_out2;
...
output scan_out16;
input scan_en;

wire NX1_inv;
```

```
    wire chain1_cell1_out;
    wire chain1_cell2_out;
    ...
    wire chain1_cell31_out;
    wire chain2_cell11_out;
    wire chain2_cell12_out;
    ...
    wire chain2_cell30_out;
    .
    .
    .
    wire chain16_cell11_out;
    wire chain16_cell12_out;
    ...
    wire chain16_cell31_out;

    inv01 NX1_inv_inst ( .Y(NX1_inv), .A(NX1));

    muxd_cell chain1_cell0 ( .Q(scan_out1), .SI(chain1_cell1_out),
        .D(1'b0), .CLK(NX1_inv), .SE(scan_en) );
    muxd_cell chain1_cell1 ( .Q(chain1_cell1_out), .SI(chain1_cell2_out),
        .D(1'b0), .CLK(NX1_inv), .SE(scan_en) );
    ...
    muxd_cell chain1_cell30 ( .Q(chain1_cell30_out), .SI(scan_in1),
        .D(1'b0), .CLK(NX1_inv), .SE(scan_en) );

    muxd_cell chain2_cell0 ( .Q(scan_out2), .SI(chain2_cell1_out),
        .D(1'b0), .CLK(NX1_inv), .SE(scan_en) );
    muxd_cell chain2_cell1 ( .Q(chain2_cell1_out), .SI(chain2_cell2_out),
        .D(1'b0), .CLK(NX1_inv), .SE(scan_en) );
    ...
    muxd_cell chain2_cell29 ( .Q(chain2_cell29_out), .SI(scan_in2),
        .D(1'b0), .CLK(NX1_inv), .SE(scan_en) );
    .
    .
    .

    muxd_cell chain16_cell0 ( .Q(scan_out16), .SI(chain16_cell1_out),
        .D(1'b0), .CLK(NX2), .SE(scan_en) );
    muxd_cell chain16_cell1 ( .Q(chain16_cell1_out),
        .SI(chain16_cell2_out), .D(1'b0), .CLK(NX2), .SE(scan_en) );
    ...
    muxd_cell chain16_cell30 ( .Q(chain16_cell30_out), .SI(scan_in16),
        .D(1'b0), .CLK(NX2), .SE(scan_en) );

endmodule
```

Skeleton Design Dofile

The generated dofile includes most setup commands required to create the EDT logic. Following is the example dofile `bb1.dofile` the utility wrote out based on the previously described inputs:

```
add_scan_groups grp1 bb1.testproc
add_scan_chains chain1 grp1 scan_in1 scan_out1
add_scan_chains chain2 grp1 scan_in2 scan_out2
add_scan_chains chain3 grp1 scan_in3 scan_out3
add_scan_chains chain4 grp1 scan_in4 scan_out4
add_scan_chains chain5 grp1 scan_in5 scan_out5
add_scan_chains chain6 grp1 scan_in6 scan_out6
add_scan_chains chain7 grp1 scan_in7 scan_out7
add_scan_chains chain8 grp1 scan_in8 scan_out8
add_scan_chains chain9 grp1 scan_in9 scan_out9
add_scan_chains chain10 grp1 scan_in10 scan_out10
add_scan_chains chain11 grp1 scan_in11 scan_out11
add_scan_chains chain12 grp1 scan_in12 scan_out12
add_scan_chains chain13 grp1 scan_in13 scan_out13
add_scan_chains chain14 grp1 scan_in14 scan_out14
add_scan_chains chain15 grp1 scan_in15 scan_out15
add_scan_chains chain16 grp1 scan_in16 scan_out16
add_clocks 0 NX1
add_clocks 0 NX2
```

Skeleton Design Test Procedure File

The utility also writes out a test procedure file that has the test procedure steps needed to create EDT logic. Following is the example test procedure file *bb1.testproc* the utility wrote out using the previously described inputs:

```
set time scale 1.000000 ns ;
timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 10 ;
    pulse NX1 40 10;
    pulse NX2 40 10;
    period 100 ;
end;

procedure shift =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force_sci ;
        measure_sco ;
        pulse NX1 ;
        pulse NX2 ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force NX1 0 ;
        force NX2 0 ;
        force scan_en 1 ;
    end ;
    apply shift 2 ;
end ;
```

Skeleton Design Tessent Cell Library

The Tessent cell library written out by the utility contains the models used to create the skeleton design. You must use this library when you perform EDT IP Creation on the skeleton design in Tessent Shell.

```
model inv01(A, Y) (  
    input (A) ()  
    output(Y) (primitive = _inv(A, Y); )  
)  
  
// muxd_scan_cell is the same as sff in adk library.  
model muxd_scan_cell (D, SI, SE, CLK, Q, QB) (  
    scan_definition (  
        type = mux_scan;  
        data_in = D;  
        scan_in = SI;  
        scan_enable = SE;  
        scan_out = Q, QB;  
    )  
    input (D, SI, SE, CLK) ()  
    intern(_D) (primitive = _mux (D, SI, SE, _D);)  
    output(Q, QB) (primitive = _dff(, , CLK, _D, Q, QB); )  
)  
  
model lssd_scan_cell (D, SYS_CLK, SI, MCLK, SCLK, Q, QB) (  
    scan_definition (  
        type = lssd;  
        data_in = D;  
        scan_in = SI;  
        scan_master_clock = MCLK;  
        scan_slave_clock = SCLK;  
        scan_out = Q;  
    )  
    input (D, SYS_CLK, SI, MCLK, SCLK) ()  
    intern(MOUT) (primitive = _dlat master ( , , SYS_CLK, D, MCLK, SI,  
MOUT, );)  
    output(Q, QB) (primitive = _dlat slave ( , , SCLK, MOUT, Q, QB );)  
)
```

Note



You can get the utility to write out a Verilog simulation library that matches the Tessent cell library by including the optional -Simulation_library switch in the shell command.

There are several ways to get help when setting up and using Tessent software tools. Depending on your need, help is available from documentation, online command help, and Mentor Graphics Support.

The Tessent Documentation System	321
Mentor Support Services.....	322

The Tessent Documentation System

At the center of the documentation system is the InfoHub that supports both PDF and HTML content. From the InfoHub, you can access all locally installed product documentation, system administration documentation, videos, and tutorials. For users who want to use PDF, you have a PDF bookcase file that provides access to all the installed PDF files.

For information on defining default HTML browsers, setting up browser options, and setting the default PDF viewer, refer to the “[Documentation Options](#)” in the *Mentor Documentation System* manual.

You can access the documentation in the following ways:

- **Shell Command** — On Linux platforms, enter **mgcdocs** at the shell prompt or invoke a Tessent tool with the **-manual** invocation switch.
- **File System** — Access the Tessent InfoHub or PDF bookcase directly from your file system, without invoking a Tessent tool. For example:

HTML:

```
firefox $MGC_DFT/docs/infohubs/index.html
```

PDF

```
acroread $MGC_DFT/docs/pdffdocs/_bk_tessent.pdf
```

- **Application Online Help** — You can get contextual online help within most Tessent tools by using the “**help -manual**” tool command. For example:

> help dofile -manual

This command opens the appropriate reference manual at the “**dofile**” command description.

Mentor Support Services

Mentor provides a range of industry-leading support services that keep design teams productive and up-to-date with Mentor products.

A Mentor support contract includes the following:

- **Software Updates** — Get the latest releases and product enhancements to keep your environment current.
- **Mentor Graphics Support Center** — Access our online knowledge base, personalized to your Mentor products.
- **Support Forums** — Learn, share, and connect with other Mentor users.
- **Technical Support** — Collaborate with Mentor support engineers to solve complex design challenges.
- **Regular Communications** — Receive the latest knowledge base articles and announcements for your Mentor products.
- **Mentor Ideas** — Share ideas and vote for your favorites to shape future products.

More information is available here:

<https://support.mentor.com>

If your site is under a current support contract, but you do not have a Support Center login, register today:

<https://support.mentor.com/register>

Appendix B

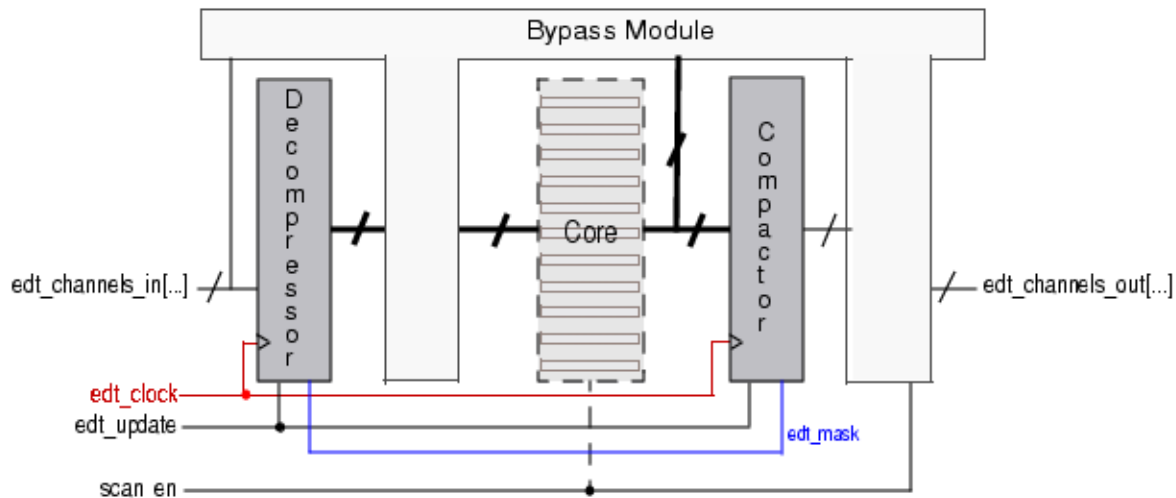
EDT Logic Specifications

This section contains illustrations of EDT logic specifications.

EDT Logic with Basic Compactor and Bypass Module	323
EDT Logic with Xpress Compactor and Bypass Module	324
Decompressor Module with Basic Compactor	325
Decompressor Module with Xpress Compactor	325
Input Bypass Logic	326
Compactor Module	327
Output Bypass Logic	328
Single Chain Bypass Logic	329
Basic Compactor Masking Logic	330
Xpress Compactor Controller Masking Logic	331
Dual Compression Configuration Input Logic	332
Dual Compression Configuration Output Logic.....	334
EDT Logic with Power Controller	334

EDT Logic with Basic Compactor and Bypass Module

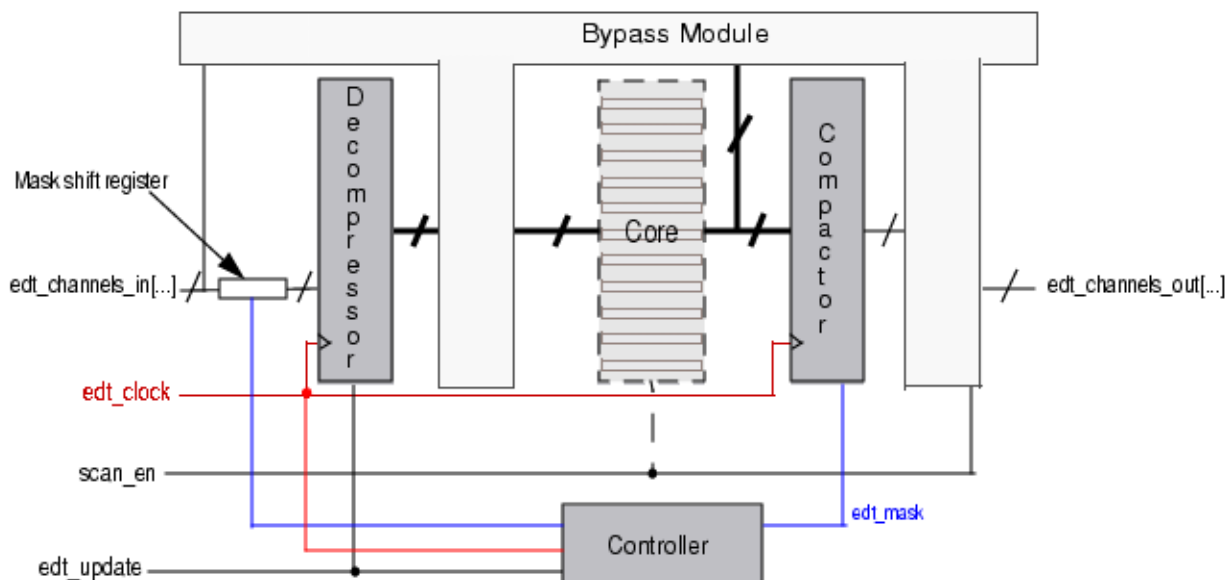
Illustration of the basic compactor and bypass module.



NOTE: Functional pins not shown

EDT Logic with Xpress Compactor and Bypass Module

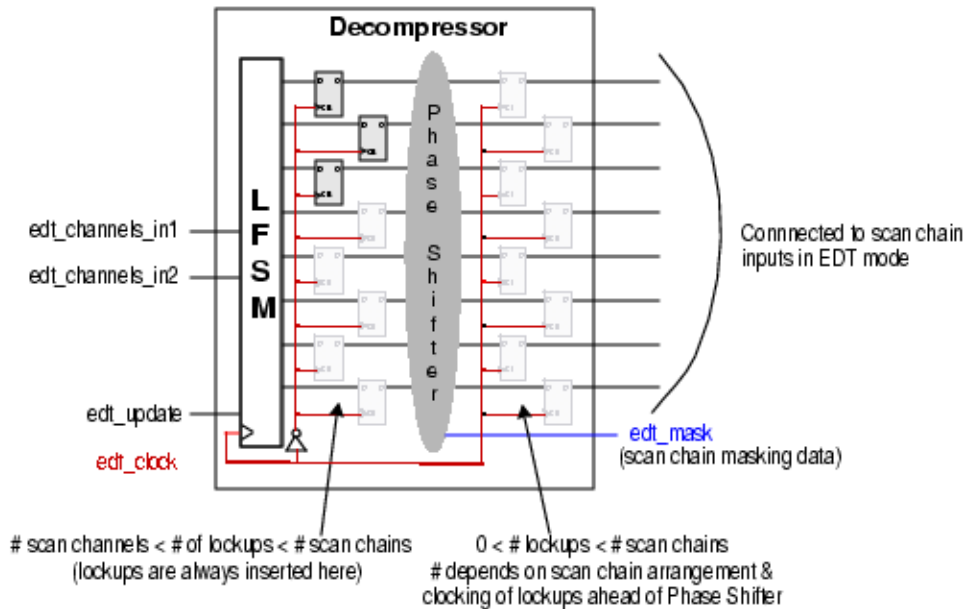
Illustration of the Xpress compactor and bypass module.



NOTE: Functional pins not shown

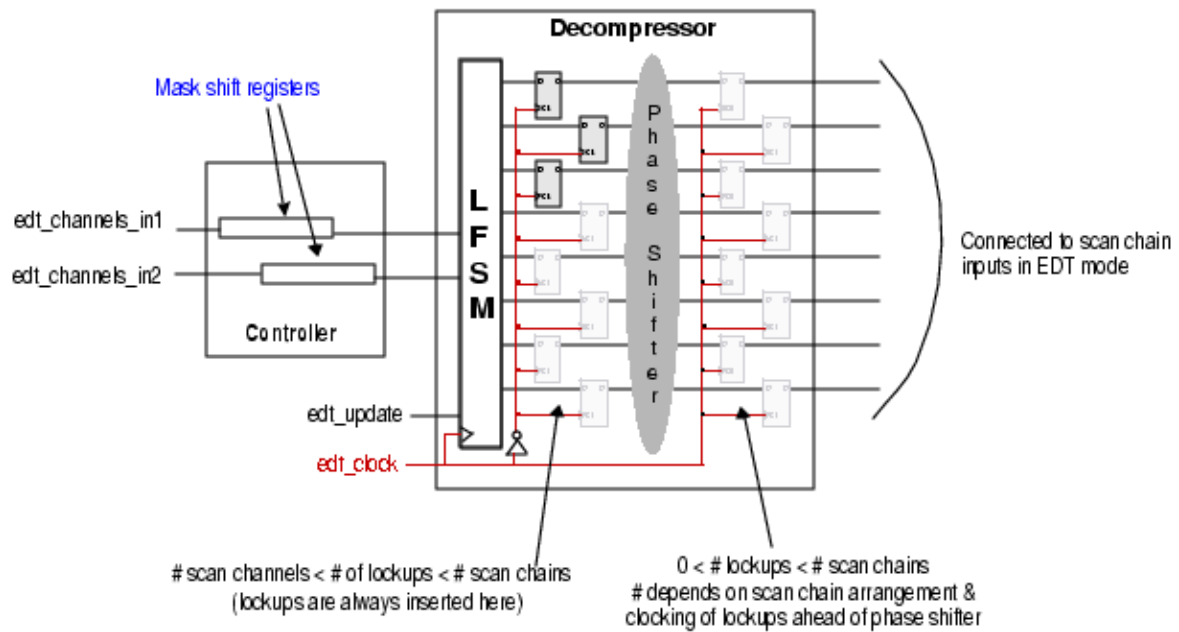
Decompressor Module with Basic Compactor

Illustration of the details for a decompressor used with a basic compactor, eight scan chains, and two scan channels.



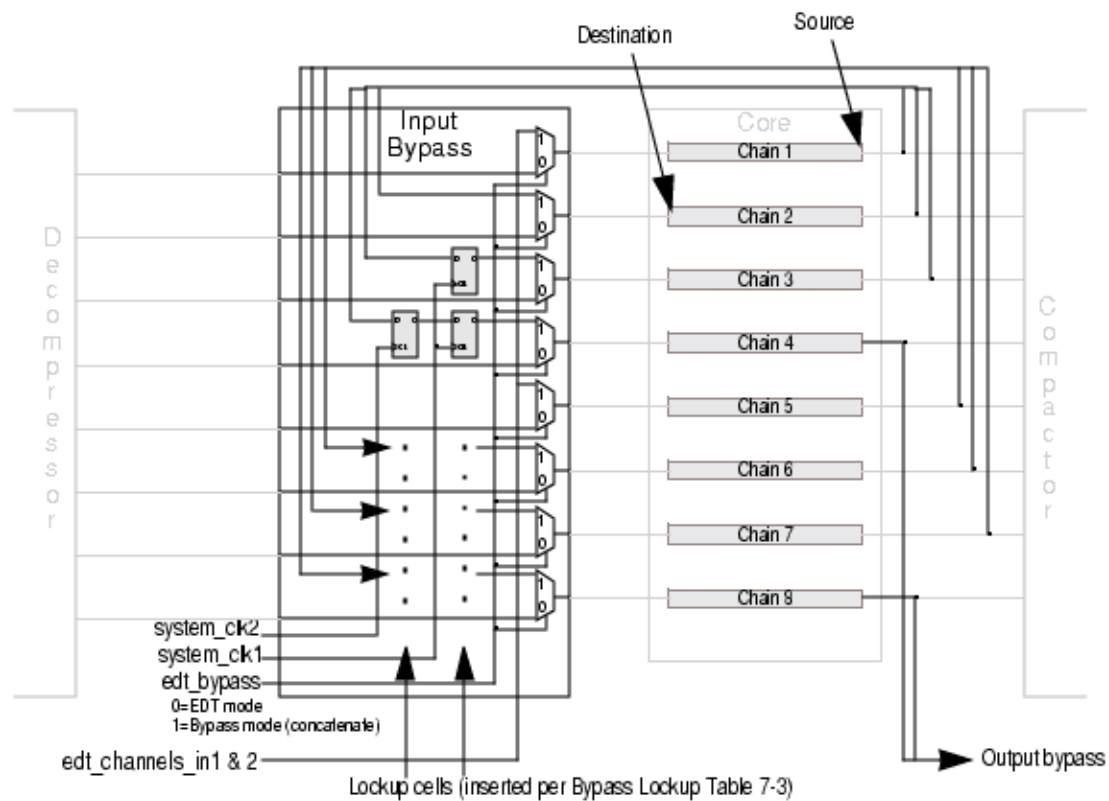
Decompressor Module with Xpress Compactor

Illustration of the details for a decompressor used with an Xpress compactor, eight scan chains, and two scan channels.



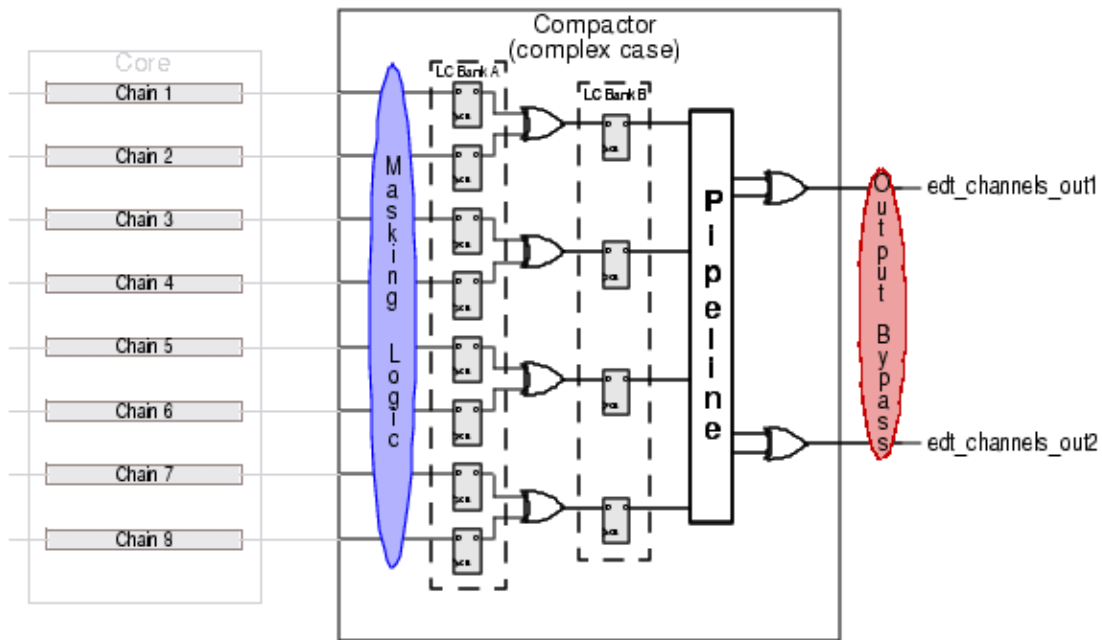
Input Bypass Logic

Illustration of input bypass logic.



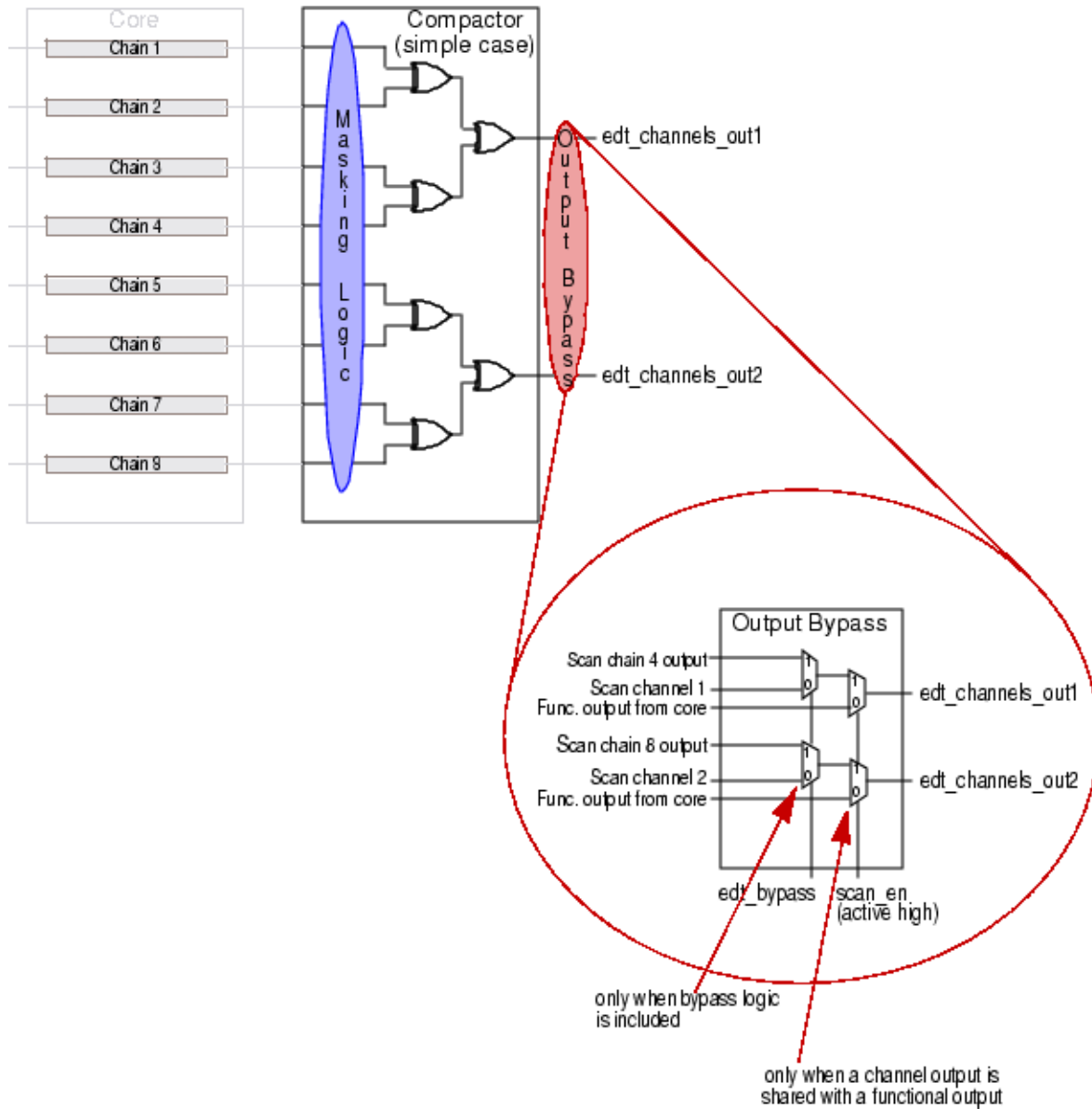
Compactor Module

Illustration of the compactor module.



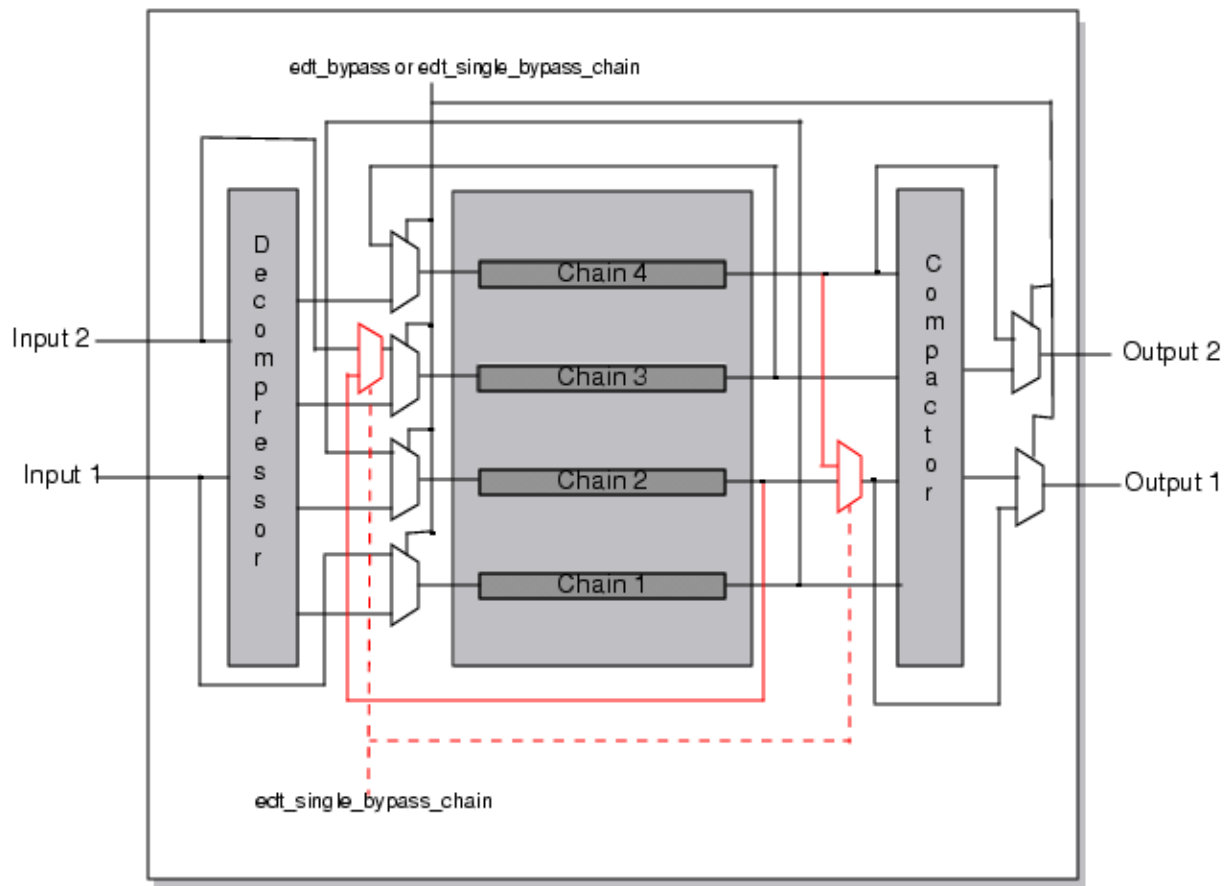
Output Bypass Logic

Illustration of output bypass logic.



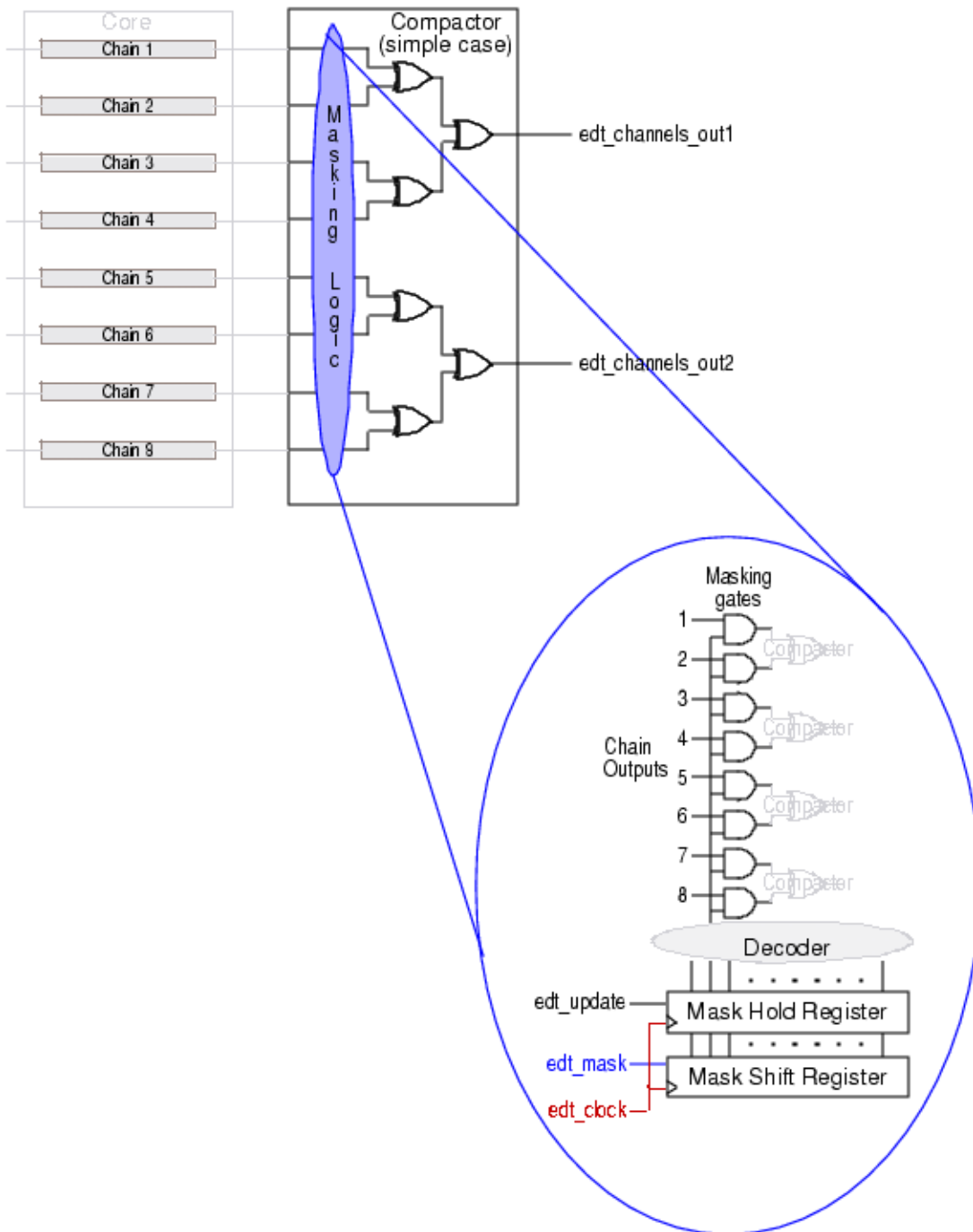
Single Chain Bypass Logic

Illustration of single chain bypass logic.



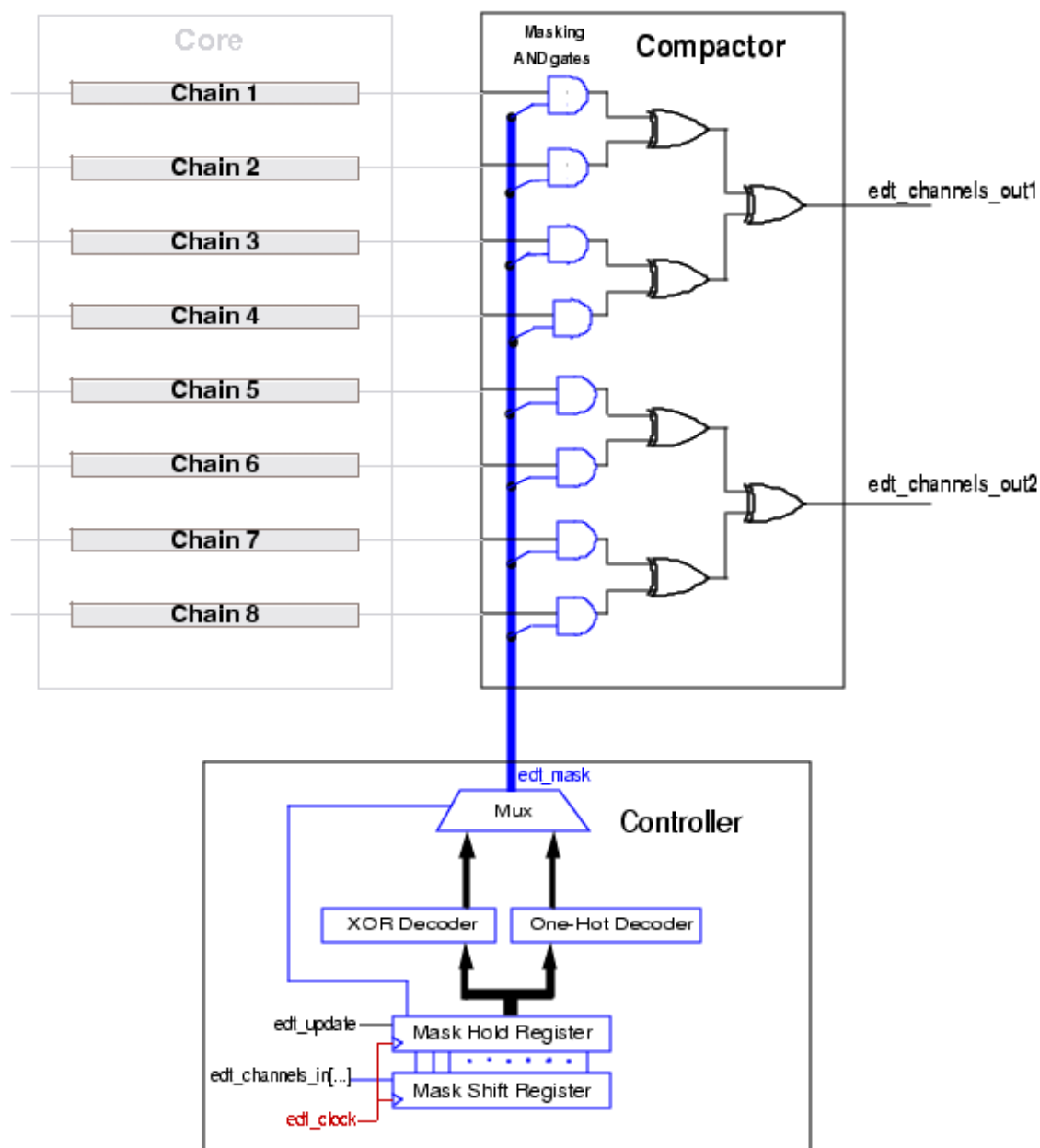
Basic Compactor Masking Logic

Illustration of basic compactor masking logic.



Xpress Compactor Controller Masking Logic

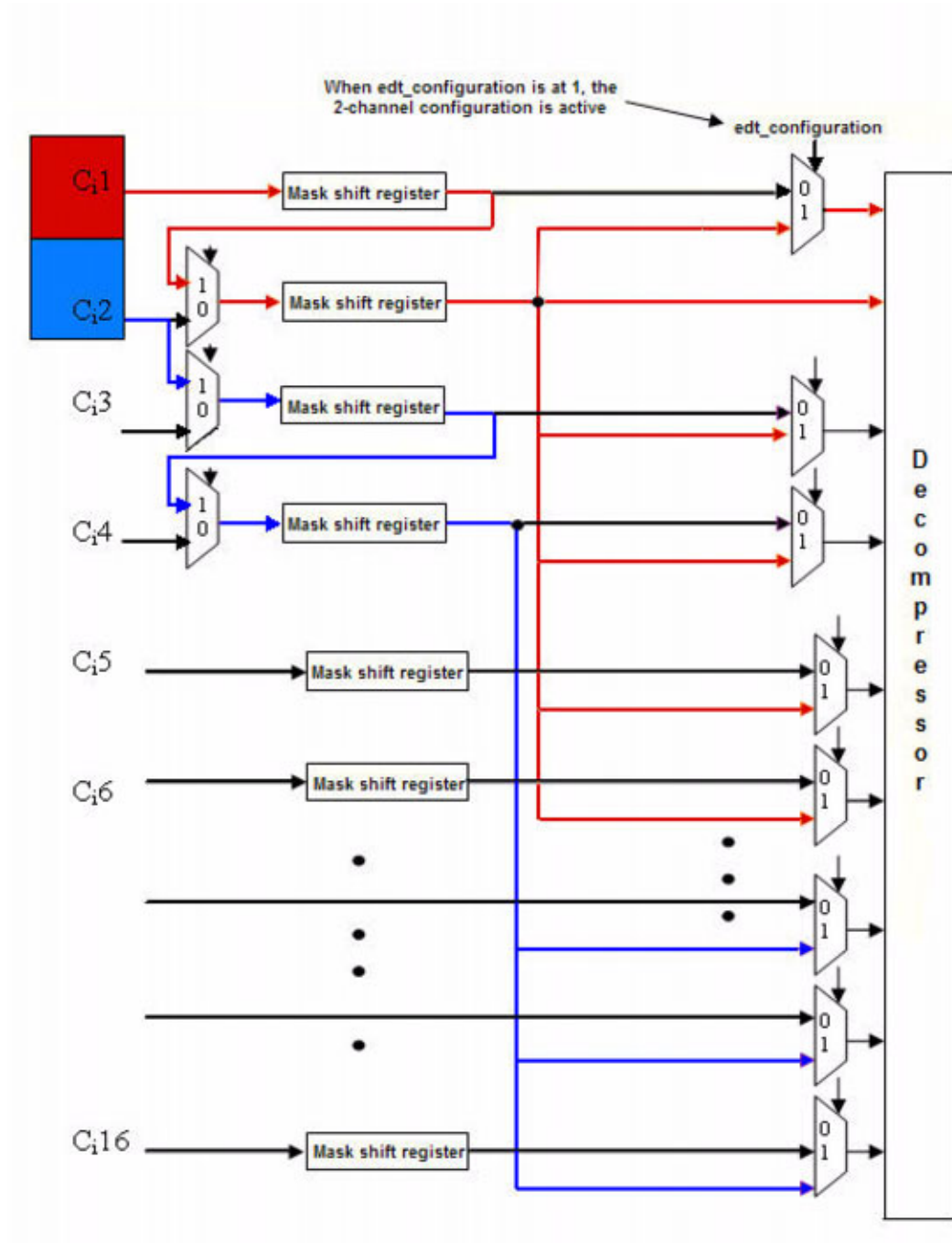
Illustration of Xpress compactor controller masking logic.



Dual Compression Configuration Input Logic

The illustration shows input logic details when both a 2-channel and a 16-channel compression configuration are defined. Note that the first 2 channels of the 16-channel configuration are always used for the 2-channel configuration.

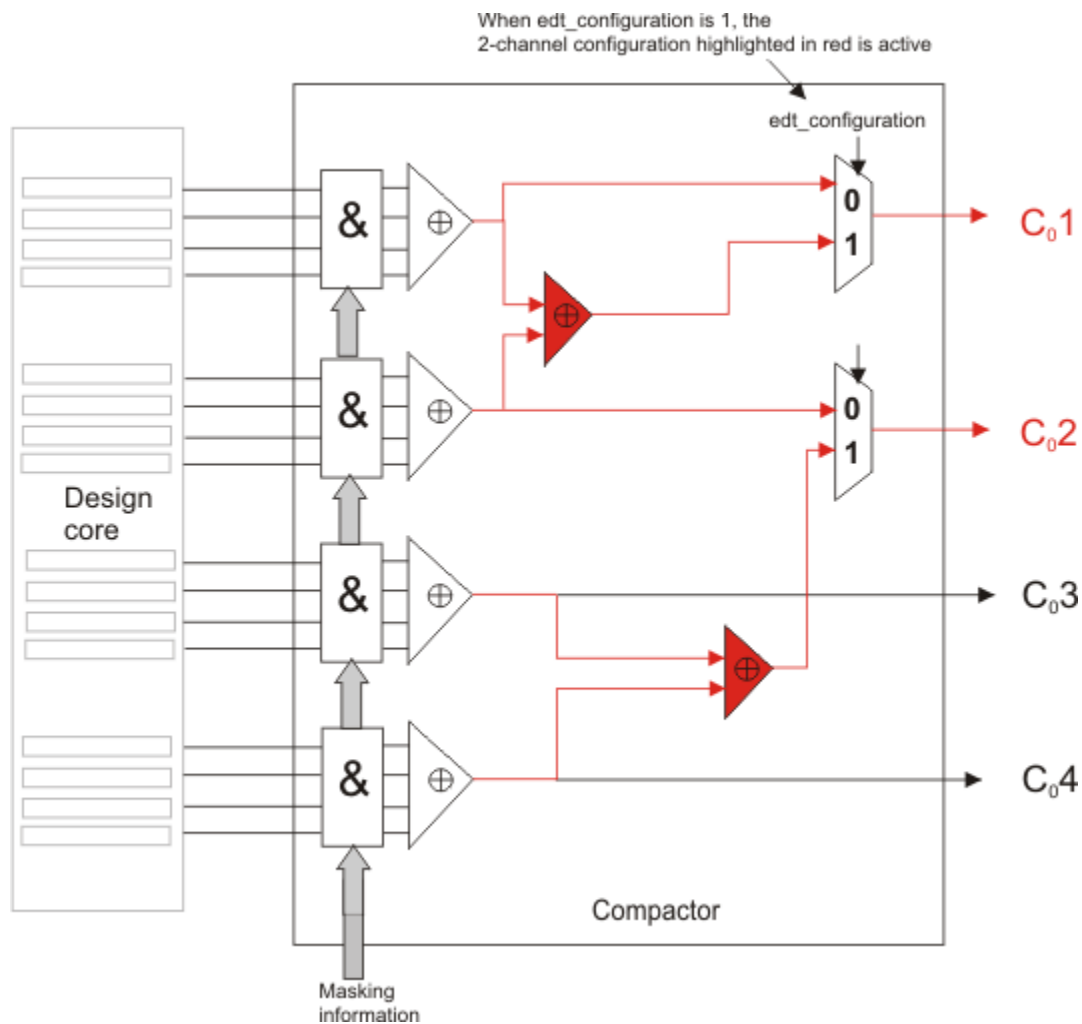
Red highlights the path for channel 1 when the 2-channel configuration is active. Blue highlights the path for channel 2 when the 2-channel input configuration is active.



Dual Compression Configuration Output Logic

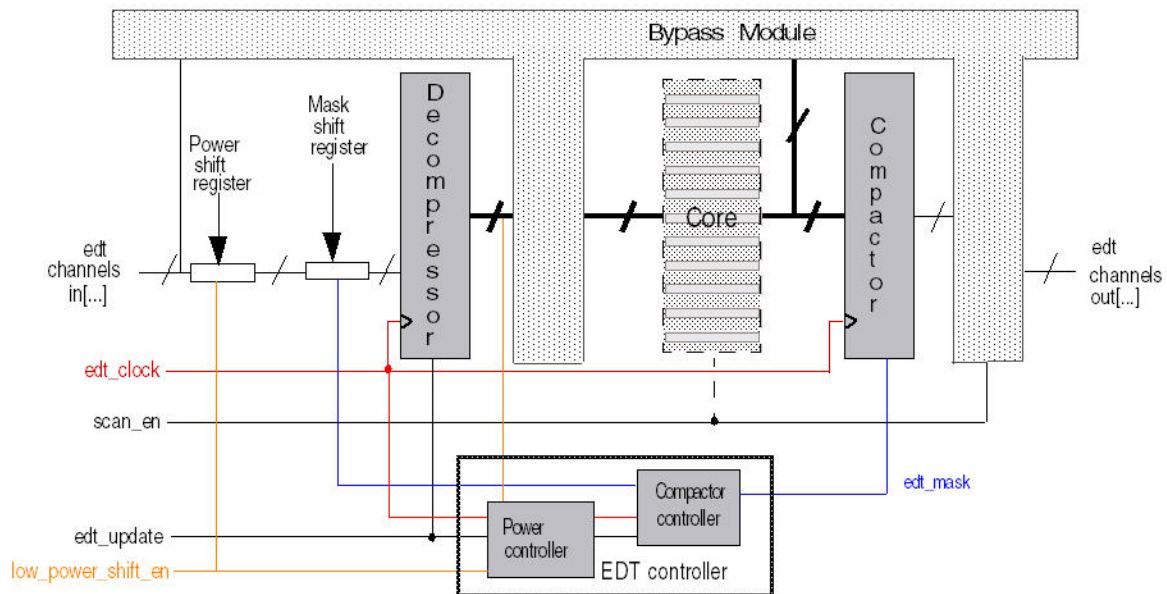
The illustration shows output logic details when both a 2-channel and a 4-channel compression configuration are defined.

Note that the first 2 channels of the 4-channel configuration are always used for the 2-channel configuration.



EDT Logic with Power Controller

Illustration of EDT logic with a power controller.



Appendix C

Troubleshooting

This appendix is divided into three parts.

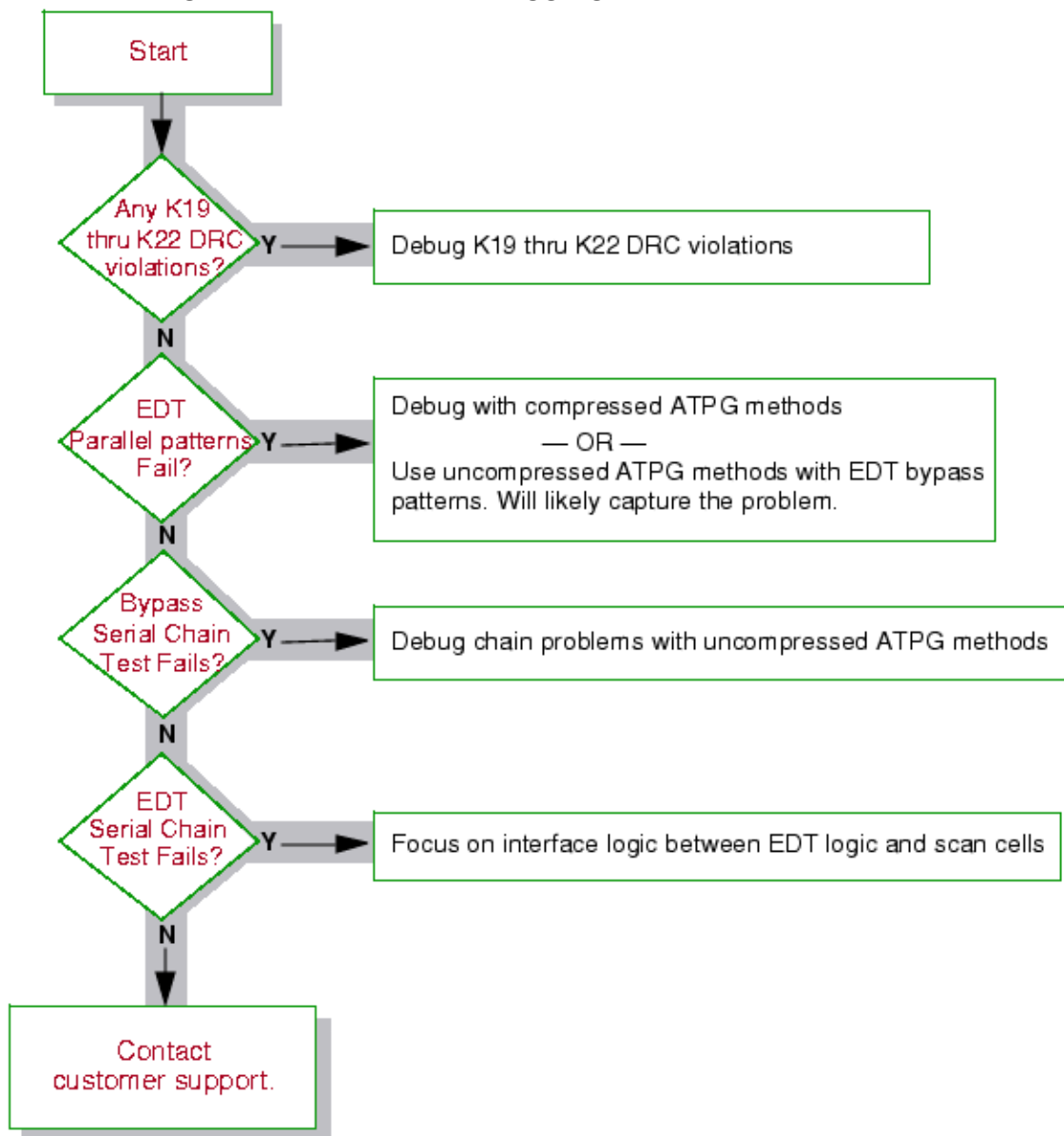
Debugging Simulation Mismatches	337
Resolving DRC Issues	339
K19 through K22 DRC Violations	339
Debugging Best Practices	341
Miscellaneous	364
Incorrect References in Synthesized Netlist	364
Limiting Observable Xs for a Compact Pattern Set	365
Applying Uncompressable Patterns Thru Bypass Mode	365
If Compression is Less Than Expected	366
If Test Coverage is Less Than Expected	366
If there are EDT aborted faults	367
Internal Scan Chain Pins Incorrectly Shared with Functional Pins	367
Masking Broken Scan Chains in the EDT Logic	367

Debugging Simulation Mismatches

This section provides a suggested flow for debugging simulation mismatches in a design that uses EDT.

You are assumed to be familiar with the information provided in “[Potential Causes of Simulation Mismatches](#)” of the Tessent Scan and ATPG User’s Manual, so that information is not repeated here. Your first step with EDT should be to determine if the source of the mismatch is the EDT logic or the core design. [Figure C-1](#) shows a suggested flow to help you begin this process.

Figure C-1. Flow for Debugging Simulation Mismatches



If the core design is the source of the mismatch, then you can use uncompressed ATPG troubleshooting methods to pinpoint the problem. This entails saving bypass patterns from compressed ATPG, which you then process and simulate in uncompressed ATPG with the design configured to operate in bypass mode. Alternatively, you can invoke Tessent Shell with the circuit (configured to run in bypass mode) and generate another set of uncompressed patterns. For more information, refer to "[Compression Bypass Logic](#)" on page 230.

Resolving DRC Issues

This section supplements the DRC information in the reference manual with some suggestions to help you reduce the occurrence of certain DRC violations.

Full descriptions of the EDT-specific “K” rules, K19 through K22 DRC Violations, are provided in “[Design Rule Checking](#)” in the *Tessent Shell Reference Manual*.

K19 through K22 DRC Violations	339
Debugging Best Practices	341

K19 through K22 DRC Violations

K19 through K22 are simulation-based DRCs. They verify the decompressor and compactor through zero-delay serial simulation and analyze mismatches to try to determine the source of each mismatch. As a troubleshooting aid, these DRCs transcript detailed messages listing the gates where the tool’s analysis determined each mismatch originated, and specific simulation results for these gates.

The tool can provide the most debugging information if you have preserved the EDT logic hierarchy, including pin pathnames and instance names, during synthesis. When this is not the case and either rule check fails, the tool transcripts a message that begins with the following reminder (K22 would be similar):

Warning: Rule K19 can provide the most debug information if the EDT logic hierarchy, including pin and instance names, is preserved during synthesis and can be found by Tessent TestKompress.

The message then lists specifics about instance(s) and/or pin pathname(s) the tool cannot resolve, so you can make adjustments in tool setups or your design if you choose. For example, if the message continues:

```
The following could not be resolved:
  EDT logic top instance "edt_i" not found.
  EDT decompressor instance "edt_decompressor_i" not found.
```

you can use the [set_edt_instances](#) command to provide the tool with the necessary information. Use the [report_edt_instances](#) command to double-check the information.

If the tool can find the EDT logic top, decompressor and compactor instances, but cannot find expected EDT pins on one or more of these instances, the specifics would tell you about the pins as in this example for an EDT design with two channels:

```
The following could not be resolved:
  EDT logic top instance "edt_i" exists, but could not find
    2-bit channel pin vector "edt_channels_in" on the instance.
  EDT decompressor instance "edt_decompressor_i" exists, but
    could not find 2-bit channel pin vector "edt_channels_in"
    on the instance.
```

When the tool is able to find the EDT logic top, decompressor and compactor instances, but cannot resolve a pin name within the EDT logic hierarchy, it is typically because the name was changed during synthesis of the EDT RTL. To help prevent interruptions of the pattern creation flow to fix a pin naming issue, you are urged to preserve during synthesis, the pin names the tool created in the EDT logic hierarchy. For additional information about the synthesis step, refer to [“The EDT Logic Synthesis Script”](#) on page 121.

Debugging Best Practices

For most common K19 and K22 debug tasks, you can report gate simulation values with the `set_gate_report Drc_pattern` command.

Typical debug tasks include checking for correct values on:

- EDT control signals (`edt_clock`, `edt_update`, `edt_bypass`, `edt_reset`)
- Sensitized paths from:
 - Input channel pins to the decompressor and from the decompressor to the scan chains during shift. (K19)
 - Scan chains to the compactor and from the compactor to the output channel pins during shift. (K22)

When you use the `Drc_pattern` option the gate simulation data for different procedures in the test procedure file display. For more information on the use of `Drc_pattern` reporting, refer to “[State Stability Issues](#)” in the *Tessent Scan and ATPG User’s Manual*.

In rare cases, you may need to see the distinct simulation values applied in every shift cycle. For these special cases, you can force the tool to simulate every event specified in the test procedure file by issuing the `set_gate_report` command with the K19 or K22 argument while in setup mode.

Tip

 Use `set_gate_report` with the K19 or K22 argument only when necessary. Because the tool has to log simulation data for all simulated setup and shift cycles, `set_gate_report K19/K22` reporting can slow EDT DRC run time and increase memory usage compared to `set_gate_report Drc_pattern` reporting.

The following two subsections provide detailed discussion of the K19 and K22 DRCs, with debugging examples utilizing the `Drc_pattern`, K19 and K22 options to the `set_gate_report` command.

Understanding K19 Rule Violations	342
Incorrect Control Signals	345
Inverted Signals	348
Incorrect EDT Channel Signal Order	349
Incorrect Scan Chain Order	350
X Generated by EDT Decompressor	352
Using <code>set_gate_report K19</code>	353
Understanding K22 Rule Violations	354
Inverted Signals	357

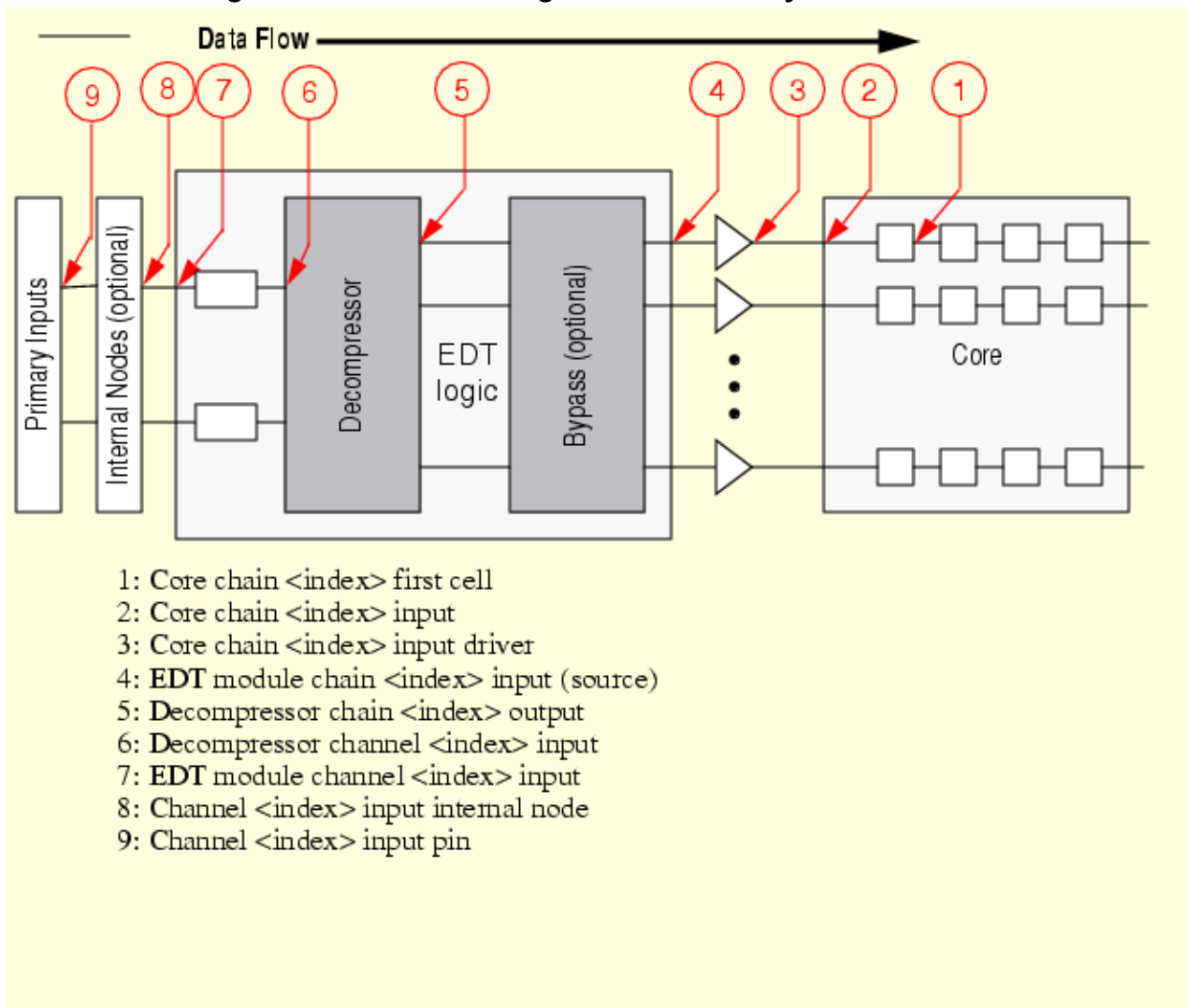
Incorrect Scan Chain Order	359
Masking Problems	361
Using set_gate_report K22	363

Understanding K19 Rule Violations

DRC K19 simulates the test_setup, load_unload, shift and capture procedures as defined in the test procedure file. By default, this simulation is performed with constrained pins initialized to their constrained values. To speed up simulation times, however, the rule simulates only a small number of shift cycles. If the first scan cell of each scan chain is loaded with the correct values, then the EDT decompressor works properly and this rule check passes.

If the first scan cell of any scan chain is loaded with incorrect data, the K19 rule check fails. The tool then automatically performs an initial diagnosis to determine where along the path from the channel inputs to the core chain inputs the problem originated. [Figure C-2](#) shows the data flow through the decompressor and where in this flow the K19 rule check validates the signals.

Figure C-2. Order of Diagnostic Checks by the K19 DRC



For example, if the K19 rule detected erroneous data at the output of the first scan cell (1) in scan chain 2, the rule would check whether data applied to the core chain input (2) is correct. If the data is correct at the core chain input, the tool would issue an error message similar to this:

```
Erroneous bit(s) detected at core chain 2 first cell
/cpu_i/option_reg_2/DFP1/ (7021).
Data at core chain 2 input /cpu_i/edt_si2 (43) is correct.
Expected: 0011101011101001X
Simulated:01100110001110101
```

The error message reports the value the tool expected at the output of the first cell in scan chain 2 for each shift cycle. For comparison, the tool also lists the values that occurred during the DRC's simulation of the circuitry. If the data is correct at the first scan cell (1) and at the core chain inputs (2), the rule next checks the data at the outputs of the core chain input drivers (3).

Note

 The term, “core chain input drivers” refers to any logic that drives the scan chain inputs. Usually, the core chain input drivers are part of the EDT logic. However, if a circuit designer inserts logic between the EDT logic and the core scan chain inputs, the drivers might be outside the EDT module.

The signals at (3) should always be the same as the signals at the core chain inputs (2). The tool checks that this is so, however, because the connection between these two points is emulated and not actually a physical connection.

Note

 Due to the tool’s emulation of the connection between points (2) and (3), you cannot obtain the gate names at these points by tracing between them with a “report_gates -backward” or “report_gates -forward” command. However, reporting a gate that has an emulated connection to another gate at this point will display the name and gate ID# of the other gate; you can then issue report_gates for the other gate and continue the trace from there.

If the data at the outputs of the core chain input drivers (3) is correct, the rule next checks the chain input data at the outputs of the EDT module (4). For each scan chain, if the data is correct at (4), but incorrect at the core chain input (2), the tool issues a message similar to the following:

```
Erroneous bit(s) detected at core chain 1 input /tiny_i/scan_in1 (11).
Data at EDT module chain 1 input (source) /edt_i/edt_bypass_logic_i/ix31/Y
(216) is correct.
Expected: 10011101011101001
Simulated:10110011000111010
```

In this message, “EDT module chain 1 input (source)” refers to the output of the EDT module that drives the “core chain 1 input.” The word “source” indicates this is the pattern source for chain 1. Also, notice the gate name “/edt_i/edt_bypass_logic_i/ix31/Y” for the EDT module chain 1 input. Because the tool simulates the flattened netlist and does not model the hierarchical module pins, the tool reports the gate driving the EDT module output.

Note

 The K19 and K22 rules always report gates driving EDT module inputs or outputs. Again, this is because in the flattened netlist there is no special gate that represents module pins.

The K19 rule verifies the data at the EDT module chain inputs (4) only if the EDT module hierarchy is preserved. If the netlist is flattened, or the EDT module name or pin names are changed during synthesis, the tool will no longer be able to identify the EDT module and its pins.

Tip

i Preserving the EDT module during synthesis allows for better diagnostic messages if the simulation-based DRCs (K19 and K22) fail during the Pattern Generation Phase.

The K19 rule continues comparing the simulated data to what is expected for all nine locations shown in [Figure C-2](#) until it finds a location where the simulated data matches the expected data. The tool then issues an error message that describes where the problem first occurred, and where the data was verified successfully.

This rule check not only reports erroneous data, but also reports unexpected X or Z values, as well as inverted signals. This information can be very useful when you are debugging the circuit.

Examples of some specific K19 problems, with suggestions for how to debug them, are detailed in the Related Topics table.

Related Topics

[Incorrect Control Signals](#)

[Incorrect Scan Chain Order](#)

[Inverted Signals](#)

[X Generated by EDT Decompressor](#)

[Incorrect EDT Channel Signal Order](#)

[Using set_gate_report K19](#)

Incorrect Control Signals

Fixing incorrect values on EDT control signals often resolves other K19 violations. Problems with control signals may be detected by other K rules, so it is a good practice to check for these in the transcript prior to the K19 failure(s) and fix them first. At minimum, the other K rule failures may provide clues that help you solve the K19 issues.

If K19 detects incorrect values on an EDT control signal, the tool will issue a message similar to this one for the EDT bypass signal (edt_bypass by default):

```
1 EDT module control signals failed. (K19-1)
Inverted data detected at EDT module bypass /edt_bypass (37).
Expected: 000000000000000000000000
Simulated: 111111111111111111111111
```

Because the edt_bypass signal is a primary input, and the message indicates it is at a constant incorrect value, it is reasonable to suspect that the load_unload or shift procedure in the test procedure file is applying an incorrect value to this pin. The edt_bypass signal should be 0

during load_unload and shift (see [Figure 6-1](#)), so you could use the following command sequence to check the pin's value after DRC.

1. set_gate_report drc_pattern load_unload
2. report_gates /edt_bypass
3. set_gate_report drc_pattern shift
4. report_gates /edt_bypass

The following transcript excerpt shows an example of the use of this command sequence, along with examples of procedures you would be examining for errors:

set_gate_report drc_pattern load_unload

report gate /edt_bypass

```
// /edt_bypass primary_input
//   edt_bypass 0 (0001) /cpu_bypass_logic_i/ix23/S0...

timeplate gen_tp1 =
  force_pi 0;
  measure_po 10;
  pulse tclk 20 10;
  pulse edt_clock 20 10;
  period 40;
end;

procedure load_unload =
  scan_group grp1;
  timeplate gen_tp1;
  cycle =
    force clear 0 ;
    force edt_update 1;
    force edt_clock 0;
    force edt_bypass 0;
    force scan_en 1;
    pulse tclk 0;
    pulse edt_clock;
  end;
apply shift 22;
end;
```

The values reported for the load_unload are okay, but in the first “apply shift” (shown in bold font), edt_bypass is 1 when it should be 0. This points to the shift procedure as the source of the problem.

You can use the following commands to confirm:

set_gate_report drc_pattern shift

report_gate /edt_bypass

```
// /edt_bypass primary_input
//   edt_bypass 0 (111) /cpu_bypass_logic_i/ix23/S0...

timeplate gen_tpl =
  force_pi 0;
  measure_po 10;
  pulse tclk 20 10;
  pulse edt_clock 20 10;
  period 40;
end;

procedure shift =
  scan_group grp1;
  timeplate gen_tpl;
  cycle =
    force_sci;
    force edt_bypass 1;
    force edt_update 0;
    measure_sco;
    pulse tclk;
    pulse edt_clock;
  end;
end;
```

The DRC simulation data for the shift procedure shows it is forcing the `edt_bypass` signal to the wrong value (1 instead of 0). The remedy is to change the force statement to “force `edt_bypass` 0”.

Following is another example of the tool’s K19 messaging—for an incorrect value on the EDT update signal (highlighted in bold).

```
EDT update pin "edt_update" is not reset before pulse of EDT clock pin
"edt_clock" in shift procedure. (K18-1)
1 error in test procedures. (K18)
...
1 EDT module control signals failed.
(K19-1)
Inverted data detected at EDT module
update /edt_update (36).
  Expected: 000000000000000000000000
  Simulated: 111111111111111111111111
4 of 4 EDT decompressor chain outputs (bus
/cpu_edt_i/cpu_edt_decompressor_i/edt_scan_in) failed. (K19-2)
Erroneous bit(s) detected at EDT decompressor chain 1 output
/cpu_edt_i/cpu_edt_decompressor_i/ix97/Y (282).
Data at EDT module channel inputs (signal /cpu_edt_i/edt_channels_in)
is correct.
  Expected: 110101101111010100001X
  Simulated: 000000000000000000000000
...
```

Notice that earlier in the transcript there is a K18 message that mentions the same control signal and describes an error in the shift procedure. A glance at [Figure 6-1](#) shows the EDT update

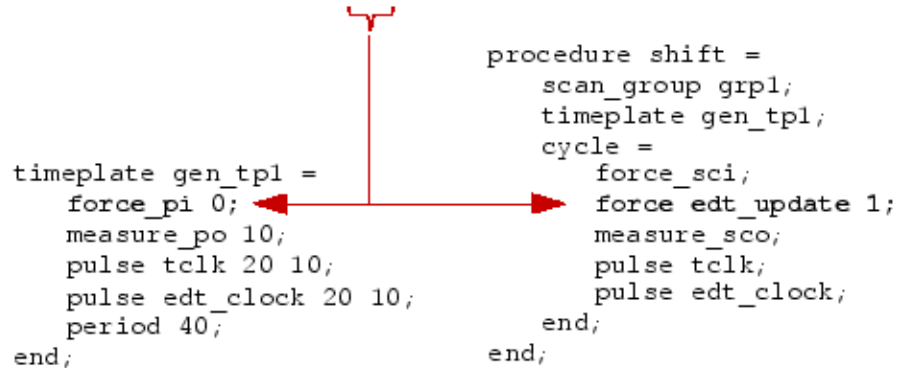
signal should be 1 during load_unload and 0 for shift. You could now check the value of this signal as follows (relevant procedure file excerpts are shown below the example commands):

set_gate_report drc_pattern shiftreport_gate /edt_update

```
// /edt_update primary_input
//   edt_update 0 (111) /cpu_bypass_logic_i/ix23/S0...

timeplate gen_tpl =
  force_pi 0;
  measure_po 10;
  pulse tclk 20 10;
  pulse edt_clock 20 10;
  period 40;
end;

procedure shift =
  scan_group grp1;
  timeplate gen_tpl;
  cycle =
    force_sci;
    force edt_update 1;
    measure_sco;
    pulse tclk;
    pulse edt_clock;
  end;
end;
```



The output of the gate report for the shift procedure shows the EDT update signal is 1 during shift. The reason is an incorrect force statement in the shift procedure, shown in the procedure excerpt below the example. Changing “force edt_update 1;” to “force edt_update 0;” in the shift procedure would resolve these K18 and K19 violations.

Inverted Signals

You can use inverting input pads to drive the EDT decompressor.

However, you must specify the inversion using the [set_edt_pins](#) command. (This actually is true of any source of inversion added on the input side of the decompressor.) Without this information, the decompressor will generate incorrect data and the K19 rule check will transcript a message similar to this:

```
1 of 1 EDT module channel inputs (signal /cpu_edt_i/edt_channels_in)
  failed. (K19-1)
Inverted data detected at EDT module channel 1 input /U$1/Y (237) .
Data at channel 1 input pin /edt_channels_in1 (38) is correct.
  Expected:  1000001011011000010000
  Simulated: 0111110100100111101111
```

The occurrence message lists the name and ID of the gate where the inversion was detected (point 6 in [Figure C-2](#)). It also lists the upstream gate where the data was correct (point 8 in [Figure C-2](#)). To debug, trace back from point 6 looking for the source of the inversion. For example:

report_gates /U\$1/Y

```
// /U$1  inv02
//      A      I /edt_channels_in1
//      Y      O /cpu_edt_i/cpu_edt_decompressor_i/ix199/A1
//              /cpu_edt_i/cpu_edt_decompressor_i/ix191/A1
//              /cpu_edt_i/cpu_edt_decompressor_i/ix183/A1
```

b

```
// /edt_channels_in1 primary_input
//      edt_channels_in1  O  /U$1/Y
```

The trace shows there are no gates between the primary input where the data is correct and the gate (an inverter) where the inversion was detected, so the latter is the source of this K19 violation. You can use the *-Inv* switch with the [set_edt_pins](#) command to solve the problem.

report_edt_pins

```
//
// Pin description      Pin name      Inversion
// -----
// Clock                edt_clock      -
// Update               edt_update     -
// Scan channel 1 input  edt_channels_in1  -
// "      "      " output  edt_channels_out1  -
//
```

set_edt_pins input_channel 1 -inv
report edt pins

```
//
// Pin description      Pin name      Inversion
// -----
// Clock                edt_clock      -
// Update               edt_update     -
// Scan channel 1 input  edt_channels_in1  inv
// "      "      " output  edt_channels_out1  -
//
```

Incorrect EDT Channel Signal Order

If you manually connect the EDT module to the core scan chains, it is easy to connect signals in the wrong order. If the K19 rule check detects incorrectly ordered signals at any point, it issues

messages similar to the following; notice the statement that signals appear to be connected in the wrong order:

```
2 of 2 EDT module channel inputs (bus /edt_i/edt_channels_in) failed.
(K19-1)
Erroneous bit(s) detected at EDT module channel 1 input
/edt_channels_in2 (9).
Data at channel 1 input pin /edt_channels_in1 (8) is correct.
Expected: 0100000000
Simulated: 0000000000
Erroneous bit(s) detected at EDT module channel 2 input
/edt_channels_in1 (8).
Data at channel 2 input pin /edt_channels_in2 (9) is correct.
Expected: 0000000000
Simulated: 0100000000
2 signals appear to be connected in the wrong
order at EDT module
channel inputs (bus /edt_i/edt_channels_in). (K19-2)
Data at EDT module channel 2 input /edt_channels_in1 (8) match those
expected at EDT module channel 1 input /edt_channels_in2 (9).
Data at EDT module channel 1 input /edt_channels_in2 (9) match those
expected at EDT module channel 2 input /edt_channels_in1 (8).
```

DRC reports this as two K19 occurrences, but the same signals are mentioned in both occurrence messages. Notice also that the Expected and Simulated values are the same, but reversed for each signal, a corroborating clue. The fix is to reconnect the signals in the correct order in the netlist.

Incorrect Scan Chain Order

The tool enables you to add and delete scan chain definitions with the commands `add_scan_chains` and `delete_scan_chains`. If you use these commands, it is mandatory that you keep the scan chains in exactly the same order in which they are connected to the EDT module.

For example, the input of the scan chain added first must be connected to the least significant bit of the EDT module chain input port (point 4 in [Figure C-2](#)). Deleting a scan chain with the `delete_scan_chains` command and then adding it back again with `add_scan_chains` will change the defined order of the scan chains, resulting in K19 violations. If scan chains are not added in the right order, the K19 rule check will issue a message similar to the following:

```
2 signals appear to be connected in the wrong order at core chain
inputs. Check if scan chains were added in the wrong order. (K19-2)
Data at core chain 6 input /cpu_i/edt_si6 (39)
match those expected at core chain 5 input /cpu_i/edt_si5 (40).
Data at core chain 5 input /cpu_i/edt_si5 (40)
match those expected at core chain 6 input /cpu_i/edt_si6 (39).
```

To check if scan chains were added in the wrong order, issue the `report_scan_chains` command and compare the displayed order with the order in the dofile the tool wrote out when the EDT logic was created. For example:

```
report_scan_chains
```

```
chain = chain1    group = grp1
    input = /cpu_i/scan_in1  output = /cpu_i/scan_out1  length = unknown
chain = chain2    group = grp1
    input = /cpu_i/scan_in2  output = /cpu_i/scan_out2  length = unknown
...
chain = chain6    group = grp1
    input = /cpu_i/scan_in6  output = /cpu_i/scan_out6  length = unknown
chain = chain5    group = grp1
    input = /cpu_i/scan_in5  output = /cpu_i/scan_out5  length = unknown
```

shows chains 5 and 6 reversed from the order in this excerpt of the original tool-generated dofile:

```
//
// Define the instance names of the decompressor, compactor, and the
// container module which instantiates the decompressor and compactor.
// Locating those instances in the design allows DRC to provide more debug
// information in the event of a violation.
// If multiple instances exist with the same name, substitute the instance
// name of the container module with the instance's hierarchical path
// name.

set_edt_instances -edt_logic_top test_design_edt_i
set_edt_instances -decompressor test_design_edt_decompressor_i
set_edt_instances -compactor test_design_edt_compactor_i

add_scan_groups grp1 testproc
add_scan_chains -internal chain1 grp1 /cpu_i/scan_in1 /cpu_i/scan_out1
add_scan_chains -internal chain2 grp1 /cpu_i/scan_in2 /cpu_i/scan_out2
...
add_scan_chains -internal chain5 grp1 /cpu_i/scan_in5 /cpu_i/scan_out5
add_scan_chains -internal chain6 grp1 /cpu_i/scan_in6 /cpu_i/scan_out6
```

The easiest way to solve this problem is either to delete all scan chains and add them in the right order:

```
delete_scan_chains -all
add_scan_chains -internal chain1 grp1 /cpu_i/scan_in1/cpu_i/scan_out1
add_scan_chains -internal chain2 grp1 /cpu_i/scan_in2 /cpu_i/scan_out2
...
add_scan_chains -internal chain5 grp1 /cpu_i/scan_in5 /cpu_i/scan_out5
add_scan_chains -internal chain6 grp1 /cpu_i/scan_in6 /cpu_i/scan_out6
```

or exit the tool, correct the order of add_scan_chains commands in the dofile, and start the tool with the corrected dofile.

X Generated by EDT Decompressor

Xs should never be applied to the scan chain inputs. If this occurs, the K19 rule check issues a message similar to this:

```
X detected at EDT module chain 1 input (source)
  /edt_i/edt_bypass_logic_i/U86/Z (3303) .
Data at EDT decompressor chain 1 output
  /edt_i/edt_decompressor_i/U83/Z (2727) is correct.
Expected: 101000100000000010001
Simulated: X0X000X000000000X000X
```

Provided the EDT module hierarchy is preserved, the message describes the origin of the X signals. The preceding message, for example, indicates the EDT bypass logic generates X signals, while the EDT decompressor works properly.

To debug these problems, check the following:

- Are the core chain inputs correctly connected to the EDT module chain input port? Floating core chain inputs could lead to an X.
- Are the channel inputs correctly connected to the EDT module channel input ports? Floating EDT module channel inputs could lead to an X.
- Are the EDT control signals (edt_clock, edt_update and edt_bypass by default) correctly connected to the EDT module? If the EDT decompressor is not reset properly, X signals might be generated.
- Is the EDT update signal (edt_update by default) asserted in the load_unload procedure so that the decompressor is reset? If the decompressor is not reset properly, X signals might be generated.
- Is the EDT bypass signal (edt_bypass by default) forced to 0 in the shift procedure? If the edt_bypass signal is not 0, X signals from un-initialized scan chains might be switched to the inputs of the core chains.
- If the EDT control signals are generated on chip (by means of a TAP controller, for example), are they forced to their proper values so the decompressor is reset in the load_unload procedure?

You can report the K19 simulation results for gates of interest by issuing “set_gate_report k19” in setup system mode, then using “report_gates” on the gates after the K19 rule check fails. You can also use an HDL simulator like ModelSim. In order to do that, ignore failing K19 DRCs by issuing a “set_drc_handling k19 ignore” command. Next, generate three random patterns in analysis system mode and save the patterns as serial Verilog patterns. Then simulate the circuit with an HDL simulator and analyze the signals of interest.

Using set_gate_report K19

If you issue a `set_gate_report` command with the K19 argument prior to DRC, you can use `report_gates` to view the simulated values for the entire sequence of events in the test procedure file for any K19-simulated gate. The K19 argument also has several options that enable you to limit the content of the displayed data.

Tip

 Use `set_gate_report` with the K19 argument only when necessary. Because the tool has to log simulation data for all simulated setup and shift cycles, `set_gate_report K19` reporting can slow EDT DRC run time and increase memory usage compared to `set_gate_report Drc_pattern` reporting.

The following shows how you might report on the simulated values for the “core chain 2 first cell” mentioned in the first error message example of this section (see “[Understanding K19 Rule Violations](#)” on page 342):

set_gate_report k19

```
// Warning: Data will be accessible after running DRC.
```

set_system_mode analysis

```
...
Erroneous bits detected at core chain 2 first cell
/cpu_i/option_reg_2/DFF1/ (7021).
Data at core chain 2 input /cpu_i/edt_si2 (43) is correct.
Expected: 0011101011101001X
Simulated:01100110001110101
...
```

report_gates 7021

```
// /cpu_i/option_reg_2/DFF1 (7021) DFF
// "S"   I   50-
// "R"   I   46-
// CLK   I   1-/clk
// "D0"  I   1774-
// "OUT" O   52- 53-
//
// Proc: ts ld_u sh 1 sh 2 sh 3 sh 4 sh 5 sh 6... cap
// -----
// Time:  i   234   123   123   123   123   123   123... o o
//        n0 0000 0000 0000 0000 0000 0000 0000... fXf
// -----
// Sim:  XX XXXX XX00 0001 0011 0010 1110 1111... XXX
// Emu:  -- ---- ---0 ---0 ---1 ---1 ---1 ---0... ---
// Mism:                *          *          *          *
// Monitor: core chain 1 first cell.
//
// Inputs:
// S      00 0000 0000 0000 0000 0000 0000 0000 0000... 0X0
// R      00 0000 0000 0000 0000 0000 0000 0000 0000... 0X0
// CLK    X0 0000 0010 0010 0010 0010 0010 0010 0010... 0X0
// DO     XX XXXX XX00 0001 0011 0010 1110 1111... XXX
```

You can see from this report the effect each event in each shift cycle had on the gate's value during simulation. The time numbers (read vertically) indicate the relative time events occurred within each cycle, as determined from the procedure file. If the gate is used by DRC as a reference point in its automated analysis of K19 mismatches, the report lists the value the tool expected at the end of each cycle and whether it matched the simulated value. The last line reminds you the gate is a monitor gate (a reference point in its automated analysis) and tells you its location in the data path. These monitor points correspond to the eight points illustrated in [Figure C-2](#).

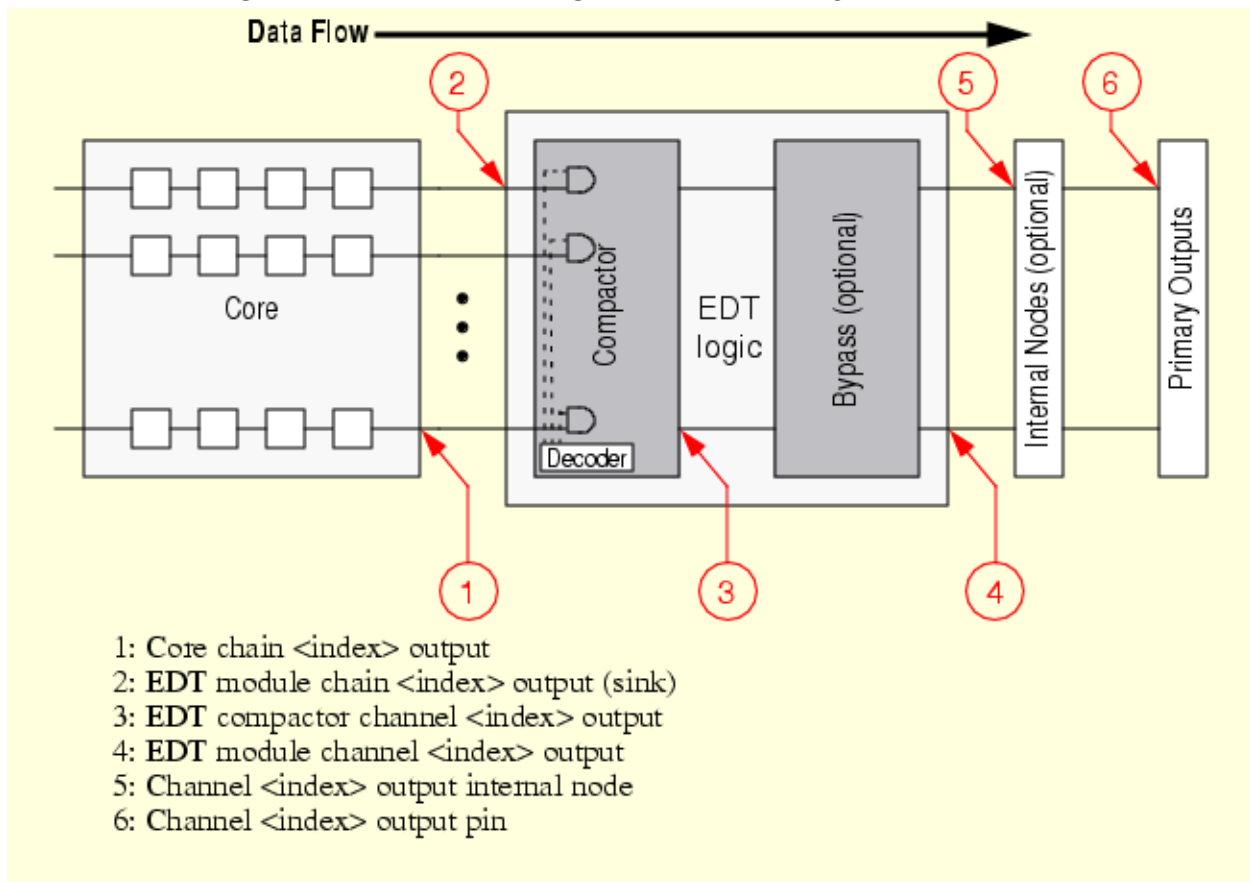
Understanding K22 Rule Violations

Like DRC K19, the K22 rule check simulates the test_setup, load_unload and shift procedures, as defined in the test procedure file. But the K22 rule check performs more simulations than K19; one simulation in non-masking mode and a number of simulations in masking mode. If the correct values are shifted out of the channel outputs in both modes, then the EDT compactor works properly and this rule check passes.

If erroneous data is observed at any channel output, either in non-masking or masking mode, the K22 rule check fails. The tool then automatically performs an initial diagnosis to determine where along the path from the core scan chains to the channel outputs the problem originated.

[Figure C-3](#) shows the data flow through the compactor and where in this flow the K22 rule check validates the signals.

Figure C-3. Order of Diagnostic Checks by the K22 DRC



For example, if the K22 rule detected erroneous data at the channel outputs (6), the tool would begin a search for the origin of the problem. First, it checks if the core chain outputs (1) have the correct values. If the data at (1) is correct, the tool next checks the data at the inputs of the EDT

module (2). If the simulated data does not match the expected data here, the tool stops the diagnosis and issues a message similar to the following:

```
Error:Non-masking mode: 1 of 8 EDT module chain outputs (sink)
(bus /edt_i/edt_scan_out) failed. (K22-1)
Erroneous bit(s) detected at EDT module chain 3 output (sink)
/cpu_i/stack2_reg_8/Q (1516).
Data at core chain 3 output /cpu_i/edt_so3 (7233) is correct.
Check if core chain 3 output is properly connected to EDT module
chain 3 output (sink).
Expected: 1111010011011001000000000001000100000
Simulated: 110100100101101000101011010111001111

Error:Masking mode (mask 3): 1 of 8 EDT module chain outputs (sink)
(bus /edt_i/edt_scan_out) failed. (K22-2)
Erroneous bit(s) detected at EDT module chain 3 output (sink)
/cpu_i/stack2_reg_8/Q (1516).
Data at core chain 3 output /cpu_i/edt_so3 (7233) is correct.
Check if core chain 3 output is properly connected to EDT module
chain 3 output (sink).
Expected: 11000100101100000000000000000000110001
Simulated: 000110001110100000000001111001101100
```

In this message, “EDT module chain 3 output (sink)” refers to the input of the EDT module that is driven by the “core chain 3 output.” The word “sink” indicates this is the sink for the responses captured in chain 3. Also, notice the gate name “/cpu_i/stack2_reg_8/Q” for the EDT module chain 3 output. Because the tool simulates the flattened netlist and does not model hierarchical module pins, the tool reports the gate driving the EDT module’s input.

Note



The K19 and K22 rules always report `_gates` driving EDT module inputs or outputs. This is because in the flattened netlist there is no special gate that represents module pins.

The message has two parts; the first part reporting problems in non-masking mode, the second reporting problems in masking mode. The preceding example tells you the masking mode fails when the mask is set to 3; that is, when the third core chain is selected for observation.

Note



In masking mode, only one core chain per compactor group is observed at the channel output for the group. In non-masking mode, the output from all core chains in a compactor group are compacted and observed at the channel output for the group.

Given the error message, it is easy to debug the problem. Check the connection between the core chain output (1 in [Figure C-3](#) on page 355) and the EDT module, making sure any logic in between is controlled correctly. Usually, there is no logic between the core chain outputs and the EDT module.

The K22 rule verifies data at the EDT module chain outputs (2) only if the EDT module hierarchy is preserved. If the netlist is flattened or the EDT module's name or pin names are changed during synthesis, the tool will no longer be able to identify the EDT module and its pins.

Note



Preserving the EDT module during synthesis allows for better diagnostic messages if the simulation-based DRCs (K19 and K22) fail during the Pattern Generation Phase.

If the data at the EDT module chain outputs (2) is correct, the K22 rule continues comparing the simulated data to the expected data for the EDT compactor outputs (3), the EDT module channel outputs(4), and so on until the tool identifies the source of the problem. This approach is analogous to that used for the K19 rule checks described in “[Understanding K19 Rule Violations](#)” on page 342.

For guidance on methods of debugging incorrect or inverted signals, X signals, and signals or scan chains in the wrong order, the discussion of these topics in “[Understanding K19 Rule Violations](#)” on page 342 is good background information for K22 rule violations.

Inverted Signals

You can use inverting pads on EDT channel outputs.

However, you must specify the inversion using the `set_edt_pins` command. (This actually is true of any source of inversion added on the output side of the compactor.) Without this information,

the compactor will generate incorrect data and the K22 rule check will transcript a message similar to this (for a design with one scan channel and four core scan chains):

```
Non-masking mode: 1 of 1 channel output pins failed. (K22-1)
Inverted data detected at channel 1 output pin /edt_channels_out1 (564).
Data at EDT module channel 1 output /cpu_edt_i/edt_bypass_logic_i/ix23/Y
(458) is correct.
Expected:  X000001101110000100111
Simulated: X111110010001111011000
```

```
Masking mode (mask 1): 1 of 1 channel output pins failed. (K22-2)
Inverted data detected at channel 1 output pin /edt_channels_out1 (564).
Data at EDT module channel 1 output /cpu_edt_i/edt_bypass_logic_i/ix23/Y
(458) is correct.
Expected:  X111101001010010011001
Simulated: X000010110101101100110
```

```
Masking mode (mask 2): 1 of 1 channel output pins failed. (K22-3)
Inverted data detected at channel 1 output pin /edt_channels_out1 (564).
Data at EDT module channel 1 output /cpu_edt_i/edt_bypass_logic_i/ix23/Y
(458) is correct.
Expected:  X111111110000000010010
Simulated: X000000001111111101101
```

```
Masking mode (mask 3): 1 of 1 channel output pins failed. (K22-4)
Inverted data detected at channel 1 output pin /edt_channels_out1 (564).
Data at EDT module channel 1 output /cpu_edt_i/edt_bypass_logic_i/ix23/Y
(458) is correct.
Expected:  X010001010000110011101
Simulated: X101110101111001100010
```

```
Masking mode (mask 4): 1 of 1 channel output pins failed. (K22-5)
Inverted data detected at channel 1 output pin /edt_channels_out1 (564).
Data at EDT module channel 1 output /cpu_edt_i/edt_bypass_logic_i/ix23/Y
(458) is correct.
Expected:  X110101011110011101110
Simulated: X001010100001100010001
```

Notice the separate occurrence messages are identifying the same problem.

The occurrence messages list the name and ID of the gate where the inversion was detected (point 6 in [Figure C-3](#)). It also lists the upstream gate where the data was correct (point 4 in [Figure C-3](#)). To debug, simply trace back from point 6 looking for the source of the inversion. For example:

```
report_gates /edt_channels_out1

// /edt_channels_out1 primary_output
//      edt_channels_out1  I  /ix77/Y

b

// /ix77  inv02
//      A      I  /cpu_edt_i/edt_bypass_logic_i/ix23/Y
//      Y      O  /edt_channels_out1
```

The trace shows there are no gates between the primary output where the inversion was detected and the gate (an inverter) where the data is correct, so the latter is the source of this K22 violation. You can use the -Inv switch with the [set_edt_pins](#) command to solve the problem.

report_edt_pins

```
//
// Pin description      Pin name      Inversion
// -----
// Clock                edt_clock      -
// Update               edt_update     -
// Scan channel 1 input  edt_channels_in1  -
// " " " output         edt_channels_out1 -
//
```

set_edt_pins output_channel 1 -inv report edt pins

```
//
// Pin description      Pin name      Inversion
// -----
// Clock                edt_clock      -
// Update               edt_update     -
// Scan channel 1 input  edt_channels_in1  -
// " " " output         edt_channels_out1  inv
//
```

Incorrect Scan Chain Order

You can add and delete scan chain definitions with the commands `add_scan_chains` and `delete_scan_chains`. If you use these commands, it is mandatory that you keep the scan chains in exactly the same order in which they are connected to the EDT module.

For example, the output of the scan chain added first must be connected to the least significant bit of the EDT module chain output port (point 2 in [Figure C-3](#)). Deleting a scan chain with the `delete_scan_chains` command, then adding it again with `add_scan_chains`, will change the defined order of the scan chains, resulting in K22 violations. If scan chains are not added in the right order, the K22 rule check will issue a message similar to the following:

```
4 signals appear to be connected in the wrong order at EDT module chain
outputs (sink) (bus/cpu_edt_i/edt_so). (K22-8)
Data at EDT module chain 2 output (sink) /cpu_i/dataai/uul/Y (254)
  match those expected at EDT module chain 1 output (sink)
  /cpu_i/dataao/uul/Y (256).
Data at EDT module chain 3 output (sink) /cpu_i/dataai/uul/Y (253)
  match those expected at EDT module chain 2 output (sink)
  /cpu_i/dataai/uul/Y (254).
Data at EDT module chain 4 output (sink) /cpu_i/addr_0/uul/Y (245)
  match those expected at EDT module chain 3 output (sink)
  /cpu_i/dataai/uul/Y (253).
Data at EDT module chain 1 output (sink) /cpu_i/dataao/uul/Y (256)
  match those expected at EDT module chain 4 output (sink)
  /cpu_i/addr_0/uul/Y (245).
```

To check if scan chains were added in the wrong order, issue the [report_scan_chains](#) command and compare the displayed order with the order in the dofile the tool wrote out when the EDT logic was created. For example:

report_scan_chains

```
chain = chain2 group = grp1
    input = /cpu_i/scan_in2 output = /cpu_i/scan_out2 length = unknown
chain = chain3 group = grp1
    input = /cpu_i/scan_in3 output = /cpu_i/scan_out3 length = unknown
chain = chain4 group = grp1
    input = /cpu_i/scan_in4 output = /cpu_i/scan_out4 length = unknown
chain = chain1 group = grp1
    input = /cpu_i/scan_in1 output = /cpu_i/scan_out1 length = unknown
```

shows chain1 added last instead of first, chain2 added first instead of second, and so on; not the order in this excerpt of the original tool-generated dofile:

```
//
// Define the instance names of the decompressor, compactor, and the
// container module which instantiates the decompressor and compactor.
// Locating those instances in the design allows DRC to provide more debug
// information in the event of a violation.
// If multiple instances exist with the same name, substitute the instance
// name of the container module with the instance's hierarchical path
// name.

set_edt_instances -edt_logic_top test_design_edt_i
set_edt_instances -decompressor test_design_edt_decompressor_i
set_edt_instances -compactor test_design_edt_compactor_i

add_scan_groups grp1 testproc
add_scan_chains -internal chain1 grp1 /cpu_i/scan_in1 /cpu_i/scan_out1
add_scan_chains -internal chain2 grp1 /cpu_i/scan_in2 /cpu_i/scan_out2
add_scan_chains -internal chain3 grp1 /cpu_i/scan_in3 /cpu_i/scan_out3
add_scan_chains -internal chain4 grp1 /cpu_i/scan_in4 /cpu_i/scan_out4
...
```

The easiest way to solve this problem is either to delete all scan chains and add them in the right order:

```
delete_scan_chains -all
add_scan_chains -internal chain1 grp1 /cpu_i/scan_in1 /cpu_i/scan_out1
add_scan_chains -internal chain2 grp1 /cpu_i/scan_in2 /cpu_i/scan_out2
add_scan_chains -internal chain3 grp1 /cpu_i/scan_in3 /cpu_i/scan_out3
add_scan_chains -internal chain4 grp1 /cpu_i/scan_in4 /cpu_i/scan_out4
```

or exit the tool, correct the order of add_scan_chains commands in the dofile and start the tool with the corrected dofile.

Note

 When the tool is set up to treat K19 violations as errors, the invocation default, incorrect scan chain order will be detected by the K19 rule check, since the tool performs K19 checks before K22—see “[Incorrect Scan Chain Order](#)” on page 350 in the K19 section for example tool messages. In this case, the tool will stop before issuing any K22 messages related to the incorrect order.

If the issue was actually one of incorrect signal order only at the outputs of the internal scan chains and the inputs were in the correct order, you would get K22 messages similar to the preceding and no K19 messages about scan chains being “added in the wrong order.”

Masking Problems

Most masking problems are caused by disturbances in the operation of the mask hold and shift registers.

One such problem results in the following message for the decoded masking signals:

```
Non-masking mode: 4 of 4 EDT decoded masking signals failed. (K22-1)
Constant X detected at EDT decoded masking signal 1
/cpu_edt_i/cpu_edt_compactor_i/decoder1/ix63/Y (343) .
Expected: 111111111111111111111111
Simulated: XXXXXXXXXXXXXXXXXXXXXXXX
```

You can usually find the source of masking problems by analyzing the mask hold and shift registers. In this example, you could begin by tracing back to find the source of the Xs:

set_gate_level primitive
set_gate_report drc_pattern state_stability
report_gates /cpu_edt_i/cpu_edt_compactor_i/decoder1/ix63/Y

```
// /cpu_edt_i/cpu_edt_compactor_i/decoder1/ix63 (343) NAND
//      (ts) ( ld) (shift) (cap) (stbl)
//  "I0"  I  ( X) (XXX) (XXX~X) (XXX) (   X) 294-
//  B0    I  ( X) (XXX) (XXX~X) (XXX) (   X) 291- ../decoder1/ix107/Y
//  Y     O  ( X) (XXX) (XXX~X) (XXX) (   X) 419- ../ix41/A1
```

b

```
// /cpu_edt_i/cpu_edt_compactor_i/decoder1/ix63 (294) OR
//      (ts) ( ld) (shift) (cap) (stbl)
//  A0    I  ( X) (XXX) (XXX~X) (XXX) (   X) 208- ../reg_masks_hold_reg_0_/Q
//  A1    I  ( X) (XXX) (XXX~X) (XXX) (   X) 214- ../reg_masks_hold_reg_1_/Q
//  "OUT" O  ( X) (XXX) (XXX~X) (XXX) (   X) 343-
```

b

```
// /cpu_edt_i/cpu_edt_compactor_i/reg_masks_hold_reg_0_ (208) BUF
//      (ts) (ld) (shift) (cap) (stbl)
//  "IO"  I  ( X) (XXX) (XXX~X) (XXX) (  X) 538-
//  Q      O  ( X) (XXX) (XXX~X) (XXX) (  X) 235- ../ix102/A0
//                                           292- ../decoder1/ix57/A0
//                                           293- ../decoder1/ix113/A
//                                           346- ../decoder1/ix61/A0
//                                           294- ../decoder1/ix63/A0
```

b

```
// /cpu_edt_i/cpu_edt_compactor_i/reg_masks_hold_reg_0_ (538) DFF
//      (ts) (ld) (shift) (cap) (stbl)
//  "S"   I  ( 0) (000) (000~0) (000) (  0) 48-
//  "R"   I  ( 0) (000) (000~0) (000) (  0) 150-
//  CLK   I  ( 0) (000) (000~0) (000) (  0) 47-
//  D     I  ( X) (XXX) (XXX~X) (XXX) (  X) 235- ../ix102/Y
//  "OUT" O  ( X) (XXX) (XXX~X) (XXX) (  X) 208- 209-
```

The trace shows the clock for the mask hold register is inactive. Trace back on the clock to find out why:

report_gates 47

```
// /cpu_edt_i (47) TIE0
//      (ts) (ld) (shift) (cap) (stbl)
//  "OUT" O  ( 0) (000) (000~0) (000) (  0) 541-../reg_masks_hold_reg_1_/CLK
//                                           540-../reg_masks_shift_reg_1_/CLK
//                                           539-../reg_masks_shift_reg_0_/CLK
//                                           538-../reg_masks_hold_reg_0_/CLK
//                                           537 ../reg_masks_shift_reg_2_/CLK
//                                           536-../reg_masks_hold_reg_2_/CLK
```

The information for the clock source shows it is tied. As the EDT clock should be connected to the hold register, you could next report on the EDT clock primary input at the compactor and check for a connection to the hold register:

report_gates /cpu_edt_i/cpu_edt_compactor_i/edt_clock. . .

Based on the preceding traces, you would expect to find that the EDT clock was *not* connected to the hold register. Because an inactive clock signal to the mask hold register would cause masking to fail, check the transcript for corroborating messages that indicate multiple similar

masking failures. These DRC messages, which preceded the K22 message in this example, provide such a clue:

```
Pipeline identification for channel output pins failed. (K20-1)
Non-masking mode: Failed to identify pipeline stage(s) at channel 1 output
pin /edt_channels_out1 (563).
Masking mode (mask 1, chain1): Failed to identify pipeline stage(s) at
channel 1 output pin /edt_channels_out1 (563).
Masking mode (mask 2, chain2): Failed to identify pipeline stage(s) at
channel 1 output pin /edt_channels_out1 (563).
Masking mode (mask 3, chain3): Failed to identify pipeline stage(s) at
channel 1 output pin /edt_channels_out1 (563).
Masking mode (mask 4, chain4): Failed to identify pipeline stage(s) at
channel 1 output pin /edt_channels_out1 (563).

Error during identification of pipeline stages. (K20)
Rule K21 (lockup cells) not performed for the compactor side since
pipeline identification failed.
```

Notice the same failure was reported in masking mode for all scan chains. To fix this particular problem, you would need to connect the EDT clock to the mask hold register in the netlist.

Using set_gate_report K22

The set_gate_report command has a K22 argument.

This K22 argument is similar to the K19 argument described in “[Using set_gate_report K19](#)” on page 353. If you issue the command prior to DRC, you can use “report_gates” to view the simulated values for the entire sequence of events in the test procedure file for any K22 simulated gate. Like the K19 argument, the K22 argument also has several options that enable you to limit the content of the displayed data.

Tip

 Use set_gate_report with the K22 argument only when necessary. Because the tool has to log simulation data for all simulated setup and shift cycles, “set_gate_report k22” reporting can slow EDT DRC run time and increase memory usage compared to “set_gate_report drc_pattern” reporting.

Miscellaneous

This section contains the following troubleshooting procedures:

Incorrect References in Synthesized Netlist	364
Limiting Observable Xs for a Compact Pattern Set.....	365
Applying Uncompressable Patterns Thru Bypass Mode	365
If Compression is Less Than Expected	366
If Test Coverage is Less Than Expected	366
If there are EDT aborted faults	367
Internal Scan Chain Pins Incorrectly Shared with Functional Pins.....	367
Masking Broken Scan Chains in the EDT Logic	367

Incorrect References in Synthesized Netlist

Use the information in this section to troubleshoot problems that cause Design Compiler to insert ****TSGEN**** references in a synthesized netlist.

Run Design Compiler to synthesize the netlist and verify that no errors occurred and check that tri-state buffers were correctly synthesized. For certain technologies, Design Compiler is unable to correctly synthesize tri-state buffers and inserts an incorrect reference to “****TSGEN****” instead. You can run the **grep** command to check for TSGEN:

```
grep TSGEN created_edt_bs_top_gate.v
```

If TSGEN is found, as shown in bold font in the following example Verilog code,

```
module tri_enable_high ( dout, oe, pin );
input dout, oe;
output pin;
    wire pin_tri_enable;
    tri pin_wire;
    assign pin = pin_wire;
    **TSGEN** pin_tri ( .\function (dout),
        .three_state(pin_tri_enable), .\output (pin_wire) );
    N1L U16 ( .Z(pin_tri_enable), .A(oe) );
endmodule
```

you need to change the line of code that contains the reference to a correct instantiation of a tri-state buffer. The next example corrects the previous instantiation to the LSI lcbg10p technology (shown in bold font):

```
module tri_enable_high ( dout, oe, pin );
input dout, oe;
output pin;
    wire pin_tri_enable;
    tri pin_wire;
    assign pin = pin_wire;
    BTS4A pin_tri ( .A (dout), .E (pin_tri_enable),
.Z
    (pin_wire) );
    N1A U16 ( .Z(pin_tri_enable), .A(oe) );
endmodule
```

Limiting Observable Xs for a Compact Pattern Set

EDT can handle Xs, but you may want to limit them in order to enhance compression. To achieve a compact pattern set (and decrease runtime as well), ensure the circuit has few, or no, X generators that are observable on the scan chains. For example, if you bypass a RAM that is tested by memory BIST, X sources are reduced because the RAM will no longer be an X generator in analysis mode.

If no Xs are captured on the scan chains, usually no fault effects are lost due to the compactors and the tool does not have to generate patterns that use scan chain output masking. For circuits with no Xs observable on the scan chains, the effective compression is usually much higher (everything else being equal) and the number of patterns is only slightly more than what ATPG generates without EDT. DRC's rule [E5](#) identifies sources of observable Xs.

One clue that you probably have many observable Xs is usually apparent in the transcript for an EDT pattern generation run. With few or no observable Xs, the number of effective patterns in each simulation pass without scan chain masking will (ideally) be 64. Numbers significantly lower can indicate that Xs are reducing test effectiveness. This is confirmed if the number of effective patterns rises significantly when the tool uses masking to block the observable Xs.

Applying Uncompressable Patterns Thru Bypass Mode

Occasionally, the tool will generate an effective pattern that cannot be compressed using EDT technology. Although it is a rare occurrence, if many faults generate such patterns, it can have an impact on test coverage. Decreasing the number of scan chains usually remedies the problem. Alternatively, you can bypass the EDT logic, which reconfigures the scan chains into fewer, longer scan chains. This requires an uncompressed ATPG run on the remaining faults.

Note

 You can use bypass mode to apply uncompressed patterns. You can also use bypass mode for system debugging purposes.

If Compression is Less Than Expected

If you find effective compression is much less than you targeted, taking steps to remedy or reduce the following should improve the compression:

- Many observable Xs—EDT can handle observable Xs but their occurrence requires the tool to use masking patterns. Masking patterns observe fewer faults than non-masking patterns, so more of them are required. More patterns lowers effective compression.

If the session transcript shows all patterns are non-masking, then observable Xs are not the cause of the lower than expected compression. If the tool generated both masking and non-masking patterns and the percentage of masking patterns exceeds 25% of the total, then there are probably many observable Xs. To find them, look for E5 DRC messages. You activate E5 messages by issuing a “set_drc_handling e5 note” command.

Note

 If there are many observable Xs, you will probably see a much higher runtime compared to uncompressed ATPG. You will probably also see a much lower number of effective patterns reported in the transcript when compressed ATPG is not using scan chain masking, compared to when the tool is using masking.

“[Resolving X Blocking with Scan Chain Masking](#)” on page 283 describes masking patterns. It also shows how the tool reports their use in the session transcript, and illustrates how masked patterns appear in an ASCII pattern file. See also “[Limiting Observable Xs for a Compact Pattern Set](#)” on page 365.

- EDT Aborted Faults—For information about these types of faults, refer to “[If there are EDT aborted faults](#)” on page 367 in the next section.
- If there are no EDT aborted faults, try a more aggressive compression configuration by increasing the number of scan chains.

If Test Coverage is Less Than Expected

If you find test coverage is much less than you expected, first compare it to the test coverage obtainable without EDT. If the test coverage with EDT is less than you obtain with

uncompressed ATPG, the following sections list steps you can take to raise it to the same level as uncompressed ATPG:

If there are EDT aborted faults

When the tool generates an effective fault test, but is unable to compress the pattern, the fault is classified as an EDT aborted fault.

See “[EDT Aborted Fault Analysis](#)” on page 288 for a method to perform analysis on these faults.

A warning is issued at the end of the run for EDT aborted faults and reports the resultant loss of coverage. You can also obtain this information by issuing the [report_aborted_faults](#) command and looking for the “edt” class of aborted faults. Each of the following increases the probability of EDT aborted faults:

- Relatively aggressive compression (large chain-to-channel ratio)
- Large number of ATPG constraints
- Relatively small design

If the number of undetected faults is large enough to cause a relevant decrease of test coverage, try re-inserting a fewer number of scan chains.

Internal Scan Chain Pins Incorrectly Shared with Functional Pins

Relatively low test coverage can indicate internal scan chain pins are shared with functional pins. These pins must not be shared because the internal scan chain pins are connected to the EDT logic and not to the top level. Also, the tool constrains internal scan chain input pins to X, and masks internal scan chain output pins. This has minimal impact on test coverage only if these are dedicated pins. By default, DRC issues a warning if scan chain pins are not dedicated pins.

Be sure none of the internal scan chain input or output pins are shared with functional pins. Only scan channel pins may be shared with functional pins. Refer to “[Scan Chain Pins](#)” on page 55 for additional information.

Masking Broken Scan Chains in the EDT Logic

You can set up the EDT logic to mask the load, capture, and/or unload values on specified scan chains by inserting custom logic between the scan chain outputs and the compactor. The custom logic allows you to either feed the desired circuit response (0/1) to the compactor or tie the scan chain output to an unknown value (X).

For more information, see the [add_chain_masks](#) command.

Appendix D

Dofile-Based Legacy IP Creation and Pattern Generation Flow

Prior to the introduction of the TCD-based EDT IP flow, dofiles created during the IP generation phase were used as the primary input into the EDT pattern generation phase.

The dofile-based legacy flow can still be used as an alternative method of transferring information from ETD IP to ATPG.

EDT IP Generation Dofiles	370
Test Pattern Generation Files	370
EDT Bypass Files	374
EDT Pattern Generation Dofiles	376
Generated Bypass Dofile and Procedure File	376
Creation of Test Patterns	377
Low Pin Count Test Controller Dofiles	379
Type 1 Controller Example	379
Type 2 Controller Example	383
Type 3 Controller Example	388

EDT IP Generation Dofiles

During the IP generation phase, the tool produces several dofiles for use during EDT pattern generation.

Test Pattern Generation Files	370
EDT Bypass Files	374

Test Pattern Generation Files

The tool automatically writes a dofile and a test procedure file containing EDT-specific commands and test procedure steps. As with the similar files produced by Tessent Scan after scan insertion, these files perform basic setups; however, you need to add commands for any pattern generation or pattern saving steps.

- **Dofile** — The dofile includes setup commands and/or switches required to generate test patterns. This is an example dofile *created_edt.dofile*, the EDT-specific parts of this file are in bold font:

```
// Define the instance names of the decompressor, compactor, and the
// container module which instantiates the decompressor and
// compactor. Locating those instances in the design allows DRC to
// provide more debug information in the event of a violation. If
// multiple instances exist with the same name, substitute the
// instance name of the container module with the instance's
// hierarchical path name.

set_edt_instances -edt_logic_top
cpu_edt_i
set_edt_instances -decompressor
cpu_edt_decompressor_i
set_edt_instances -compactor cpu_edt_compactor_i

add_scan_groups grp1 created_edt.testproc
add_scan_chains -internal chain1 grp1 /cpu_i/edt_si1 /cpu_i/edt_so1
add_scan_chains -internal chain2 grp1 /cpu_i/edt_si2 /cpu_i/edt_so2
add_scan_chains -internal chain3 grp1 /cpu_i/edt_si3 /cpu_i/edt_so3
add_scan_chains -internal chain4 grp1 /cpu_i/edt_si4 /cpu_i/edt_so4
add_scan_chains -internal chain5 grp1 /cpu_i/edt_si5 /cpu_i/edt_so5
add_scan_chains -internal chain6 grp1 /cpu_i/edt_si6 /cpu_i/edt_so6
add_scan_chains -internal chain7 grp1 /cpu_i/edt_si7 /cpu_i/edt_so7
add_scan_chains -internal chain8 grp1 /cpu_i/edt_si8 /cpu_i/edt_so8

add_clocks 0 clk
add_clocks 0 edt_clock

add_write_controls 0 ramclk

add_read_controls 0 ramclk

add_input_constraints edt_clock -C0

// EDT settings. Please do not modify.
// Inconsistency between the EDT settings and the EDT logic may
// lead to DRC violations and invalid patterns.

set_edt_options -channels 3 -ip_version 3 -decompressor_size 12 \
    -injectors_per_channel 2
```

Notice the **-internal** switch used with the [add_scan_chains](#) command. This switch must be used for all compressed scan chains (scan chains driven by and observed through the EDT logic) when setting up to generate compressed test patterns. The reason for this requirement is to define the compressed scan chains as internal, rather than external channels, as explained in “[Design Rule Checks](#)” on page 94.

Note

 Be sure the scan chain input and output pin pathnames specified with the `add_scan_chains -internal` command are kept during layout. If these pin pathnames are lost during the layout tool's design flattening process, the generated dofile will not work anymore. If that happens, you must manually generate the `add_scan_chains -internal` commands, substituting the original pin pathnames with new, logically equivalent, pin pathnames.

Note

 If your design includes uncompressed scan chains (chains whose scan inputs and outputs are primary inputs and outputs), you must define each such scan chain using the [add_scan_chains](#) command.

Other commands in this file add the EDT clock and constrain it to its off state, specify the number of scan channels, and specify the version of the EDT logic architecture.

- **Test Procedure File** — The tool also writes a test procedure file for test pattern generation. The tool takes the test procedure file used for EDT logic creation and adds the test procedures necessary to drive the EDT logic.

The following example is a test procedure file, `created_edt.testproc`. The EDT-specific parts of this file are shown in bold font.

```
//
set time scale 1.000000 ns ;
set strobe_window time 100 ;

timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 100 ;
    pulse clk 200 100;
    pulse edt_clock 200 100;
    pulse ramclk 200 100;
    period 400 ;
end;

procedure capture =
    timeplate gen_tp1 ;
    cycle =
        force_pi ;
        measure_po ;
        pulse_capture_clock ;
    end;
end;

procedure shift =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force_sci ;
        force edt_update 0 ;
        measure_sco ;
        pulse clk ;
        pulse edt_clock ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force clk 0 ;
        force edt_bypass 0 ;
        force edt_clock 0
    ;
    force edt_update 1 ;
    force ramclk 0 ;
    force scan_en 1 ;
    pulse edt_clock ;
    end ;
    apply shift 26;
end;

procedure test_setup =
    timeplate gen_tp1 ;
    cycle =
        force edt_clock 0 ;
    end;
end;
```

EDT Bypass Files

During IP creation, the tool creates files associated with EDT bypass.

- **Dofile**— This example dofile, `created_bypass.dofile`, enables you to run regular ATPG. The dofile specifies the scan channels as chains because in bypass mode, the channels connect directly to the input and output of the concatenated internal scan chains, bypassing the EDT circuitry.

```
//  
add_scan_groups grp1 created_bypass.testproc  
add_scan_chains edt_channel1 grp1 edt_channels_in1  
edt_channels_out1  
  
add_clocks 0 clk  
  
add_write_controls 0 ramclk  
  
add_read_controls 0 ramclk
```

- **Test Procedure File** — Notice the line (in bold font) near the end of this otherwise typical test procedure file, `created_bypass.testproc`. That line forces the EDT bypass signal, “`edt_bypass`” to a logic high in the `load_unload` procedure and activates bypass mode.

```
//
set time scale 1.000000 ns ;
set strobe_window time 100 ;

timeplate gen_tp1 =
    force_pi 0 ;
    measure_po 100 ;
    pulse clk 200 100;
    pulse ramclk 200 100;
    period 400 ;
end;

procedure capture =
    timeplate gen_tp1 ;
    cycle =
        force_pi ;
        measure_po ;
        pulse_capture_clock ;
    end;
end;

procedure shift =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force_sci ;
        measure_sco ;
        pulse clk ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tp1 ;
    cycle =
        force clk 0 ;
        force edt_bypass 1 ;
        force ramclk 0 ;
        force scan_en 1 ;
    end ;
    apply shift 125;
end;
```

EDT Pattern Generation Dofiles

The first two setups described in the preceding section are included in the dofile generated with the EDT logic.

For an example of this dofile, see “[Test Pattern Generation Files](#)” on page 370.

The test procedure file also needs modifications to ensure the EDT update signal is active in the load_unload procedure and the EDT clock is pulsed in the load_unload and shift procedures. These modifications are implemented automatically in the test procedure file output with the EDT logic as follows:

- The timeplate used by the shift procedure is updated to include the EDT clock.
- In this timeplate, there must be a delay between the trailing edge of the clock and the end of the period. Otherwise, a [P3](#) DRC violation will occur.
- The load_unload procedure is set up to initialize the EDT logic and apply shift a number of times corresponding to the longest “virtual” scan chain (longest scan chain plus additional shift cycles) seen by the tester. The number of additional shift cycles is reported by the [report_edt_configurations](#) command.

Note



Additional shift cycles refers to the sum of the initialization cycles, masking bits (when using Xpress), and low-power bits (when using a low-power decompressor).

- The shift procedure is updated to include pulsing of the EDT clock signal and deactivation of the EDT update signal.
- The EDT bypass signal is forced to a logic low if the EDT circuitry includes bypass logic.

Generated Bypass Dofile and Procedure File	376
Creation of Test Patterns	377

Generated Bypass Dofile and Procedure File

The tool generates a dofile and an test procedure file you can use with Tessent FastScan to activate bypass mode and run regular ATPG.

Examples of these files are shown in “[Bypass Mode Files](#)” on page 109. If your design includes boundary scan and you want to run in bypass mode, you must modify the bypass dofile and procedure file to work properly with the boundary scan circuitry.

Creation of Test Patterns

The compression technology supports all of the pattern functionality in uncompressed ATPG, with the exception of MacroTest and random patterns. This includes combinational, clock-sequential (including patterns with multiple scan loads), and RAM sequential patterns. It also includes all the fault types.

See “[EDT Aborted Fault Analysis](#)” on page 288 for additional considerations.

When you generate test patterns, you should use the dofile and test procedure files the tool generated during logic creation. If you added boundary scan, you will need to modify the files as explained in “[Modification of the Dofile and Procedure File for Boundary Scan](#)” on page 242.

To create the EDT logic, you invoked Tessent Shell with the core level of the design. To generate test patterns, you invoke Tessent Shell with the synthesized top level of the design that includes synthesized pads, boundary scan, if used, and the EDT logic. Here is an example invocation of Tessent Shell with a Verilog file named *created_edt_top.v*, assumed here to be the top-level file generated when the EDT logic was created:

Invoke Tessent Shell:

```
<Tessent_Tree_Path>/bin/tessent -shell
```

You are automatically placed in setup mode. Specify the context for generating test patterns and load the Verilog file and library:

```
set_context patterns -scan  
read_verilog created_edt_top.v  
read_cell_library my_atpg_lib  
set_current_design top
```

For a description of how the *created_edt_top.v* file is generated, refer to “[Creation of EDT Logic Files](#)” on page 96. Next, you need to set up for EDT pattern generation. To do this, execute the dofile. For example:

```
dofile created_edt.dofile
```

For information about the EDT-specific contents of this dofile, refer to “[Test Pattern Generation Files](#)” on page 370. Enter analysis mode and verify that no DRC violations occur. Pay special attention to the EDT DRC messages.

```
set_system_mode analysis
```

Now, you can enter the commands to generate the EDT patterns. If you ran uncompressed ATPG on just the core design prior to inserting the EDT logic, it is useful to add faults on just the core now to enable you to make valid comparisons of test performance using EDT versus not using EDT.

```
add_faults /my_core
// Only target faults in core
create_patterns
report_statistics
report_scan_volume
```

Another reason to add faults on the core is to avoid incorrectly reported low test coverage, as explained earlier in “[Adding Faults on the Core Only is Recommended](#)” on page 153.

The [report_scan_volume](#) command provides reference numbers when analyzing the achieved compression.

Low Pin Count Test Controller Dofiles

During IP creation, the tool creates files associated with LPCT controller.

This section provides an example for creating each of the three LPCT configuration types and examples of the dofile and test procedure files generated for each configuration.

Type 1 Controller Example.....	379
Type 2 Controller Example.....	383
Type 3 Controller Example.....	388

Type 1 Controller Example

This example for a Type 1 LPCT controller provides a sample tool-created pattern generation dofile and test procedure file.

Sample pattern generation dofile:

```
// Read the LPCT TCD file for EDT IP
read_core_description created_cpu_edt_lpct.tcd
set_edt_instances -edt_logic_top m8051_edt_i
set_edt_instances -decompressor m8051_edt_decompressor_i
set_edt_instances -compactor m8051_edt_compactor_i

add_scan_groups grp1 created_edt.testproc
add_scan_chains -internal chain1 grp1 /m8051_edt_i/edt_scan_in[0]
/m8051_edt_i/edt_scan_out[0]
add_scan_chains -internal chain2 grp1 /m8051_edt_i/edt_scan_in[1]
/m8051_edt_i/edt_scan_out[1]
add_scan_chains -internal chain3 grp1 /m8051_edt_i/edt_scan_in[2]
/m8051_edt_i/edt_scan_out[2]
add_scan_chains -internal chain4 grp1 /m8051_edt_i/edt_scan_in[3]
/m8051_edt_i/edt_scan_out[3]
add_scan_chains -internal chain5 grp1 /m8051_edt_i/edt_scan_in[4]
/m8051_edt_i/edt_scan_out[4]
add_scan_chains -internal chain6 grp1 /m8051_edt_i/edt_scan_in[5]
/m8051_edt_i/edt_scan_out[5]
add_scan_chains -internal chain7 grp1 /m8051_edt_i/edt_scan_in[6]
/m8051_edt_i/edt_scan_out[6]
add_scan_chains -internal chain8 grp1 /m8051_edt_i/edt_scan_in[7]
/m8051_edt_i/edt_scan_out[7]
add_scan_chains -internal chain9 grp1 /m8051_edt_i/edt_scan_in[8]
/m8051_edt_i/edt_scan_out[8]
add_scan_chains -internal chain10 grp1 /m8051_edt_i/edt_scan_in[9]
/m8051_edt_i/edt_scan_out[9]
add_scan_chains -internal chain11 grp1 /m8051_edt_i/edt_scan_in[10]
/m8051_edt_i/edt_scan_out[10]
add_scan_chains -internal chain12 grp1 /m8051_edt_i/edt_scan_in[11]
/m8051_edt_i/edt_scan_out[11]
add_scan_chains -internal chain13 grp1 /m8051_edt_i/edt_scan_in[12]
/m8051_edt_i/edt_scan_out[12]
add_scan_chains -internal chain14 grp1 /m8051_edt_i/edt_scan_in[13]
/m8051_edt_i/edt_scan_out[13]
add_scan_chains -internal chain15 grp1 /m8051_edt_i/edt_scan_in[14]
/m8051_edt_i/edt_scan_out[14]
add_scan_chains -internal chain16 grp1 /m8051_edt_i/edt_scan_in[15]
/m8051_edt_i/edt_scan_out[15]

add_primary_inputs /occ/NX2 -internal -pseudo_port_name NX2
add_primary_inputs /occ/NX1 -internal -pseudo_port_name NX1
add_clocks 0 refclk
add_clocks 0 NX1
add_clocks 0 NX2

add_input_constraints scan_en -C0

// EDT settings. Please do not modify.
// Inconsistency between the EDT settings and the EDT logic may
// lead to DRC violations and invalid patterns.

set_edt_options -channels 2 -longest_chain_range 2 32 -ip_version 5
-decompressor_size 12 -injectors_per_channel 3 -scan_chains 16
-compactor_type xpress
```

```

set_edt_pins update -
set_edt_pins clock -

set_mask_register -input_channel_mask_register_sizes    1 7    2 6

set_mask_decoder_connection -mode_bit                  1 7
set_mask_decoder_connection -lhot_decoder 1            1 6    1 5    1 4    1 3
set_mask_decoder_connection -xor_decoder chain1        1 6    1 5    1 4
set_mask_decoder_connection -xor_decoder chain2        1 6    1 5    1 3
set_mask_decoder_connection -xor_decoder chain3        1 6    1 5    1 2
set_mask_decoder_connection -xor_decoder chain4        1 6    1 5    1 1
set_mask_decoder_connection -xor_decoder chain5        1 6    1 4    1 3
set_mask_decoder_connection -xor_decoder chain6        1 5    1 4    1 3
set_mask_decoder_connection -xor_decoder chain7        1 4    1 2    1 1
set_mask_decoder_connection -xor_decoder chain8        1 3    1 2    1 1

set_mask_decoder_connection -lhot_decoder 2            2 6    2 5    2 4    2 3
set_mask_decoder_connection -xor_decoder chain9        2 6    2 5    2 4
set_mask_decoder_connection -xor_decoder chain10       2 6    2 5    2 3
set_mask_decoder_connection -xor_decoder chain11      2 6    2 5    2 2
set_mask_decoder_connection -xor_decoder chain12      2 6    2 5    2 1
set_mask_decoder_connection -xor_decoder chain13      2 6    2 4    2 3
set_mask_decoder_connection -xor_decoder chain14      2 5    2 4    2 3
set_mask_decoder_connection -xor_decoder chain15      2 4    2 2    2 1
set_mask_decoder_connection -xor_decoder chain16      2 3    2 2    2 1

// LPCT configuration settings. Please do not modify.
// Inconsistency between the LPCT configuration settings and the LPCT
// logic may lead to DRC violations and invalid patterns.

set_lpct_controller on -generate_scan_enable off
                        -tap_controller_interface off -shift_control clock
                        -load_unload_cycles 2 2

```

Sample pattern generation test procedure file:

```
set time scale 1.000000 ns ;
set strobe_window time 10 ;

timeplate gen_tpl =
    force_pi 0 ;
    measure_po 10 ;
    pulse /NX1 20 10;
    pulse /NX2 20 10;
    pulse refclk 20 10;
    period 40 ;
end;

procedure shift =
    scan_group grp1 ;
    timeplate gen_tpl ;
    // cycle 1 starts at time 0
    cycle =
        force_sci ;
        measure_sco ;
        pulse /NX1 ;
        pulse /NX2 ;
        pulse refclk ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tpl ;
    // cycle 1 starts at time 0
    cycle =
        force /NX1 0 ;
        force /NX2 0 ;
        force RST 0 ;
        force edt_bypass 0 ;
        force scan_en 1 ;
        pulse refclk ;
    end ;
    // cycle 2 starts at time 40
    cycle =
        force scan_en 1 ;
        pulse refclk ;
    end ;
    apply shift 45;
    // cycle 3 starts at time 120
    cycle =
        force scan_en 0 ;
        pulse refclk ;
    end ;
    // cycle 4 starts at time 160
    cycle =
        force scan_en 0 ;
        pulse refclk ;
    end;
end;

procedure test_setup =
```

```
timeplate gen_tpl ;  
// cycle 1 starts at time 0  
cycle =  
    force scan_en 0 ;  
    pulse refclk ;  
end ;  
// cycle 2 starts at time 40  
cycle =  
    force scan_en 0 ;  
    pulse refclk ;  
end;  
end;
```

Type 2 Controller Example

This example for a Type 2 LPCT controller provides a sample tool-created pattern generation dofile and test procedure file.

Sample pattern generation dofile:

```
set_edt_instances -edt_logic_top m8051_bscan_edt_i
set_edt_instances -decompressor m8051_bscan_edt_decompressor_i
set_edt_instances -compactor m8051_bscan_edt_compactor_i

add_scan_groups grp1 created_edt.testproc
add_scan_chains -internal chain1 grp1 /m8051_bscan_edt_i/edt_scan_in[0]
/m8051_bscan_edt_i/edt_scan_out[0]
add_scan_chains -internal chain2 grp1 /m8051_bscan_edt_i/edt_scan_in[1]
/m8051_bscan_edt_i/edt_scan_out[1]
add_scan_chains -internal chain3 grp1 /m8051_bscan_edt_i/edt_scan_in[2]
/m8051_bscan_edt_i/edt_scan_out[2]
add_scan_chains -internal chain4 grp1 /m8051_bscan_edt_i/edt_scan_in[3]
/m8051_bscan_edt_i/edt_scan_out[3]
add_scan_chains -internal chain5 grp1 /m8051_bscan_edt_i/edt_scan_in[4]
/m8051_bscan_edt_i/edt_scan_out[4]
add_scan_chains -internal chain6 grp1 /m8051_bscan_edt_i/edt_scan_in[5]
/m8051_bscan_edt_i/edt_scan_out[5]
add_scan_chains -internal chain7 grp1 /m8051_bscan_edt_i/edt_scan_in[6]
/m8051_bscan_edt_i/edt_scan_out[6]
add_scan_chains -internal chain8 grp1 /m8051_bscan_edt_i/edt_scan_in[7]
/m8051_bscan_edt_i/edt_scan_out[7]
add_scan_chains -internal chain9 grp1 /m8051_bscan_edt_i/edt_scan_in[8]
/m8051_bscan_edt_i/edt_scan_out[8]
add_scan_chains -internal chain10 grp1 /m8051_bscan_edt_i/edt_scan_in[9]
/m8051_bscan_edt_i/edt_scan_out[9]
add_scan_chains -internal chain11 grp1 /m8051_bscan_edt_i/edt_scan_in[10]
/m8051_bscan_edt_i/edt_scan_out[10]
add_scan_chains -internal chain12 grp1 /m8051_bscan_edt_i/edt_scan_in[11]
/m8051_bscan_edt_i/edt_scan_out[11]
add_scan_chains -internal chain13 grp1 /m8051_bscan_edt_i/edt_scan_in[12]
/m8051_bscan_edt_i/edt_scan_out[12]
add_scan_chains -internal chain14 grp1 /m8051_bscan_edt_i/edt_scan_in[13]
/m8051_bscan_edt_i/edt_scan_out[13]
add_scan_chains -internal chain15 grp1 /m8051_bscan_edt_i/edt_scan_in[14]
/m8051_bscan_edt_i/edt_scan_out[14]
add_scan_chains -internal chain16 grp1 /m8051_bscan_edt_i/edt_scan_in[15]
/m8051_bscan_edt_i/edt_scan_out[15]

add_primary_inputs /occ/NX2 -internal -pseudo_port_name NX2
add_primary_inputs /occ/NX1 -internal -pseudo_port_name NX1
add_clocks 0 tck -pulse_in_capture
add_clocks 0 NX1
add_clocks 0 NX2

add_input_constraints trst -C1
add_input_constraints tms -C0

// EDT settings. Please do not modify.
// Inconsistency between the EDT settings and the EDT logic may
// lead to DRC violations and invalid patterns.

set_edt_options -channels 1 -longest_chain_range 2 32 -ip_version 5
-decompressor_size 12 -injectors_per_channel 6 -scan_chains 16
-compactor_type xpress
```



```

set_edt_pins update -
set_edt_pins clock -
set_edt_pins input_channel 1 tdi
set_edt_pins output_channel 1 tdo

set_mask_register -input_channel_mask_register_sizes 1 8

set_mask_decoder_connection -mode_bit 1 8
set_mask_decoder_connection -lhot_decoder 1 1 7 1 6 1 5 1 4 1 3
set_mask_decoder_connection -xor_decoder chain1 1 7 1 6 1 5
set_mask_decoder_connection -xor_decoder chain2 1 7 1 6 1 4
set_mask_decoder_connection -xor_decoder chain3 1 7 1 6 1 3
set_mask_decoder_connection -xor_decoder chain4 1 7 1 6 1 2
set_mask_decoder_connection -xor_decoder chain5 1 7 1 6 1 1
set_mask_decoder_connection -xor_decoder chain6 1 7 1 5 1 4
set_mask_decoder_connection -xor_decoder chain7 1 6 1 5 1 4
set_mask_decoder_connection -xor_decoder chain8 1 3 1 2 1 1
set_mask_decoder_connection -xor_decoder chain9 1 5 1 4 1 3
set_mask_decoder_connection -xor_decoder chain10 1 6 1 5 1 2
set_mask_decoder_connection -xor_decoder chain11 1 7 1 2 1 1
set_mask_decoder_connection -xor_decoder chain12 1 6 1 5 1 1
set_mask_decoder_connection -xor_decoder chain13 1 6 1 3 1 1
set_mask_decoder_connection -xor_decoder chain14 1 6 1 4 1 2
set_mask_decoder_connection -xor_decoder chain15 1 6 1 3 1 2
set_mask_decoder_connection -xor_decoder chain16 1 4 1 3 1 2

// LPCT configuration settings. Please do not modify.
// Inconsistency between the LPCT configuration settings and the LPCT
// logic may lead to DRC violations and invalid patterns.

set_lpct_controller on -generate_scan_enable on -tap_controller_interface
on -shift_control clock -load_unload_cycles 3 2

```

Sample pattern generation test procedure file:

Note



The following test_setup procedure is not generated by the tool but copied from a user-provided test procedure file as an example.

```
set time scale 1.000000 ns ;
set strobe_window time 10 ;

timeplate gen_tpl =
    force_pi 0 ;
    measure_po 10 ;
    pulse /NX1 20 10;
    pulse /NX2 20 10;
    pulse tck 20 10;
    period 40 ;
end;

procedure shift lpct_tap_last_shift =
    scan_group grp1 ;
    timeplate gen_tpl ;
    // cycle 1 starts at time 0
    cycle =
        force_sci ;
        force tms 1 ;
        measure_sco ;
        pulse /NX1 ;
        pulse /NX2 ;
        pulse tck ;
    end;
end;

procedure test_setup =
    timeplate gen_tpl ;

    // cycle 1 starts at time 0
    cycle =
        force tck 0 ;
        force tms 1 ;
        force trst 0 ;
    end ;
    // cycle 2 starts at time 40
    cycle =
        force trst 1 ;
    end ;
    // cycle 3 starts at time 80
    cycle =
        force tms 0 ;
        pulse tck ;
    end ;
    // cycle 4 starts at time 120
    cycle =
        force tms 1 ;
        pulse tck ;
    end ;
    // cycle 5 starts at time 160
    cycle =
        force tms 1 ;
        pulse tck ;
    end ;
    // cycle 6 starts at time 200
    cycle =
        force tms 0 ;
        pulse tck ;
```

```

end ;
// cycle 7 starts at time 240
cycle =
    force tms 0 ;
    pulse tck ;
end ;
// cycle 8 starts at time 280
cycle =
    force tdi 0 ;
    force tms 0 ;
    pulse tck ;
end ;
// cycle 9 starts at time 320
cycle =
    force tdi 1 ;
    force tms 0 ;
    pulse tck ;
end ;
// cycle 10 starts at time 360
cycle =
    force tdi 0 ;
    force tms 0 ;
    pulse tck ;
end ;
// cycle 11 starts at time 400
cycle =
    force tdi 0 ;
    force tms 1 ;
    pulse tck ;
end ;
// cycle 12 starts at time 440
cycle =
    force tms 1 ;
    pulse tck ;
end ;
// cycle 13 starts at time 480
cycle =
    force tms 0 ;
    pulse tck ;
end;
end;
procedure shift =
    scan_group grp1 ;
    timeplate gen_tpl ;
    // cycle 1 starts at time 0
    cycle =
        force_sci ;
        force tms 0 ;
        measure_sco ;
        pulse /NX1 ;
        pulse /NX2 ;
        pulse tck ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tpl ;

```

```
// cycle 1 starts at time 0
cycle =
    force /NX1 0 ;
    force /NX2 0 ;
    force RST 0 ;
    force edt_bypass 0 ;
    force tck 0 ;
    force tdi 0 ;
    force tms 1 ;
    force trst 1 ;
    pulse tck ;
end ;
// cycle 2 starts at time 40
cycle =
    force tms 0 ;
    pulse tck ;
end ;
// cycle 3 starts at time 80
cycle =
    force tms 0 ;
    pulse tck ;
end ;
apply shift 51;
apply lpct_tap_last_shift 1;
// cycle 4 starts at time 200
cycle =
    force tms 1 ;
    pulse tck ;
end ;
// cycle 5 starts at time 240
cycle =
    force tms 0 ;
    pulse tck ;
end;
end;
```

Type 3 Controller Example

This example for a Type 3 LPCT controller provides a sample tool-created pattern generation dofile and test procedure file.

Sample pattern generation dofile:

```

set_edt_instances -edt_logic_top m8051_edt_i
set_edt_instances -decompressor m8051_edt_decompressor_i
set_edt_instances -compactor m8051_edt_compactor_i

add_scan_chains -internal chain1 grp1 /m8051_edt_i/edt_scan_in[0]
/m8051_edt_i/edt_scan_out[0]
add_scan_chains -internal chain2 grp1 /m8051_edt_i/edt_scan_in[1]
/m8051_edt_i/edt_scan_out[1]
add_scan_chains -internal chain3 grp1 /m8051_edt_i/edt_scan_in[2]
/m8051_edt_i/edt_scan_out[2]
add_scan_chains -internal chain4 grp1 /m8051_edt_i/edt_scan_in[3]
/m8051_edt_i/edt_scan_out[3]
add_scan_chains -internal chain5 grp1 /m8051_edt_i/edt_scan_in[4]
/m8051_edt_i/edt_scan_out[4]
add_scan_chains -internal chain6 grp1 /m8051_edt_i/edt_scan_in[5]
/m8051_edt_i/edt_scan_out[5]
add_scan_chains -internal chain7 grp1 /m8051_edt_i/edt_scan_in[6]
/m8051_edt_i/edt_scan_out[6]
add_scan_chains -internal chain8 grp1 /m8051_edt_i/edt_scan_in[7]
/m8051_edt_i/edt_scan_out[7]
add_scan_chains -internal chain9 grp1 /m8051_edt_i/edt_scan_in[8]
/m8051_edt_i/edt_scan_out[8]
add_scan_chains -internal chain10 grp1 /m8051_edt_i/edt_scan_in[9]
/m8051_edt_i/edt_scan_out[9]
add_scan_chains -internal chain11 grp1 /m8051_edt_i/edt_scan_in[10]
/m8051_edt_i/edt_scan_out[10]
add_scan_chains -internal chain12 grp1 /m8051_edt_i/edt_scan_in[11]
/m8051_edt_i/edt_scan_out[11]
add_scan_chains -internal chain13 grp1 /m8051_edt_i/edt_scan_in[12]
/m8051_edt_i/edt_scan_out[12]
add_scan_chains -internal chain14 grp1 /m8051_edt_i/edt_scan_in[13]
/m8051_edt_i/edt_scan_out[13]
add_scan_chains -internal chain15 grp1 /m8051_edt_i/edt_scan_in[14]
/m8051_edt_i/edt_scan_out[14]
add_scan_chains -internal chain16 grp1 /m8051_edt_i/edt_scan_in[15]
/m8051_edt_i/edt_scan_out[15]

add_primary_inputs /occ/NX2 -internal -pseudo_port_name NX2
add_primary_inputs /occ/NX1 -internal -pseudo_port_name NX1
add_primary_input -internal /m8051_lpct_clock_gater_i/
m8051_lpct_edt_clock_gater_i/clk_out
-pin_name edt_clock
add_primary_input -internal
/m8051_lpct_i/m8051_lpct_fsm_i/m8051_lpct_control_signal_generator_i/
edt_update -pin_name edt_update
add_primary_input -internal /m8051_lpct_i/m8051_lpct_interface_i/
edt_bypass -pin_name edt_bypass
add_primary_input -internal /m8051_lpct_i/m8051_lpct_fsm_i/
m8051_lpct_control_signal_generator_i/scan_en -pin_name lpct_scan_en
add_primary_input -internal /m8051_lpct_i/m8051_lpct_fsm_i/
m8051_lpct_control_signal_generator_i/lpct_capture_en -pin_name
lpct_capture_en
add_primary_input -internal /m8051_lpct_i/m8051_lpct_fsm_i/
m8051_lpct_control_signal_generator_i/lpct_clock_mux_select -pin_name
lpct_clock_mux_select
add_primary_input -internal /m8051_lpct_i/m8051_lpct_fsm_i/

```

```

m8051_lpct_control_signal_generator_i/lpct_shift_en -pin_name
lpct_shift_en
add_primary_input -internal /m8051_lpct_i/m8051_lpct_fsm_i/
m8051_lpct_control_signal_generator_i/lpct_test_active -pin_name
lpct_test_active
add_primary_input -internal /m8051_lpct_i/m8051_lpct_interface_i/
reset_control -pin_name reset_control
add_primary_input -internal
/m8051_lpct_i/m8051_lpct_interface_i/scan_en_control
-pin_name scan_en_control

add_clocks 0 refclk -pulse_always
add_clocks 0 NX1
add_clocks 0 NX2
add_clocks 0 edt_clock

add_input_constraints edt_clock -C0
add_input_constraints edt_update -C0
add_input_constraints edt_bypass -CX
add_input_constraints lpct_capture_en -C1
add_input_constraints lpct_clock_mux_select -C0
add_input_constraints lpct_shift_en -C0
add_input_constraints lpct_test_active -C1
add_input_constraints lpct_reset -C0
add_input_constraints reset_control -C0
add_input_constraints scan_en_control -C0

// EDT settings. Please do not modify.
// Inconsistency between the EDT settings and the EDT logic may
// lead to DRC violations and invalid patterns.

set_edt_options -channels 2 -longest_chain_range 2 32 -ip_version 5
-decompressor_size 12 -injectors_per_channel 3 -scan_chains 16
-compactor_type xpress

set_edt_pins update edt_update
set_edt_pins clock edt_clock
set_edt_pins bypass edt_bypass

set_mask_register -input_channel_mask_register_sizes 1 7 2 6

set_mask_decoder_connection -mode_bit 1 7
set_mask_decoder_connection -lhot_decoder 1 1 6 1 5 1 4 1 3
set_mask_decoder_connection -xor_decoder chain1 1 6 1 5 1 4
set_mask_decoder_connection -xor_decoder chain2 1 6 1 5 1 3
set_mask_decoder_connection -xor_decoder chain3 1 6 1 5 1 2
set_mask_decoder_connection -xor_decoder chain4 1 6 1 5 1 1
set_mask_decoder_connection -xor_decoder chain5 1 6 1 4 1 3
set_mask_decoder_connection -xor_decoder chain6 1 5 1 4 1 3
set_mask_decoder_connection -xor_decoder chain7 1 4 1 2 1 1
set_mask_decoder_connection -xor_decoder chain8 1 3 1 2 1 1

set_mask_decoder_connection -lhot_decoder 2 2 6 2 5 2 4 2 3
set_mask_decoder_connection -xor_decoder chain9 2 6 2 5 2 4
set_mask_decoder_connection -xor_decoder chain10 2 6 2 5 2 3
set_mask_decoder_connection -xor_decoder chain11 2 6 2 5 2 2
set_mask_decoder_connection -xor_decoder chain12 2 6 2 5 2 1
set_mask_decoder_connection -xor_decoder chain13 2 6 2 4 2 3

```

```

set_mask_decoder_connection -xor_decoder chain14 2 5 2 4 2 3
set_mask_decoder_connection -xor_decoder chain15 2 4 2 2 2 1
set_mask_decoder_connection -xor_decoder chain16 2 3 2 2 2 1

// LPCT configuration settings. Please do not modify.
// Inconsistency between the LPCT configuration settings and the LPCT
// logic may lead to DRC violations and invalid patterns.

set_lpct_controller on -generate_scan_enable on -tap_controller_interface
    off -shift_cycles_reg_width 10 -capture_cycles_reg_width 2
    -scan_patterns_reg_width 20 -chain_patterns_reg_width 10
    -test_mode_detect signal -shift_control clock -load_unload_cycles 0 2
    -bypass_controller off -reset_condition off

set_pattern_type -max_sequential 3

add_register_value lpct_config_edt_bypass 0
add_register_value lpct_config_reset_control 0
add_register_value lpct_config_scan_en_control 0
add_register_value lpct_config_chain_pattern_load_count -width 10
    -load_count chain_patterns -lsb_shifted_first
add_register_value lpct_config_scan_pattern_load_count -width 20
    -load_count scan_patterns -lsb_shifted_first
add_register_value lpct_config_capture_depth -width 2 -capture_cycles_max
    -lsb_shifted_first
add_register_value lpct_config_shift_length -width 10 -shift_length
    -lsb_shifted_first

set_chain_test -suppress_capture on

```

Sample pattern generation test procedure file:

```
set time scale 1.000000 ns ;
set strobe_window time 10 ;

timeplate gen_tpl =
    force_pi 0 ;
    measure_po 10 ;
    pulse /NX1 20 10;
    pulse /NX2 20 10;
    pulse edt_clock 20 10;
    pulse refclk 20 10;
    period 40 ;
end;

procedure load_unload_register lpct_shift_data =
    timeplate gen_tpl ;
    shift =
        // cycle 1 starts at time 0
        cycle =
            force lpct_data_in # ;
            pulse refclk ;
        end;
    end;
end;

procedure shift =
    scan_group grp1 ;
    timeplate gen_tpl ;
    // cycle 1 starts at time 0
    cycle =
        force_sci ;
        force edt_update 0 ;
        force lpct_shift_en 1 ;
        measure_sco ;
        pulse /NX1 ;
        pulse /NX2 ;
        pulse edt_clock ;
    end;
end;

procedure load_unload =
    scan_group grp1 ;
    timeplate gen_tpl ;
    // cycle 1 starts at time 0
    cycle =
        force /NX1 0 ;
        force /NX2 0 ;
        force RST 0 ;
        force edt_bypass 0 ;
        force edt_clock 0 ;
        force edt_update 1 ;
        force lpct_capture_en 0 ;
        force lpct_clock_mux_select 1 ;
        force lpct_scan_en 1 ;
        force lpct_shift_en 0 ;
        force lpct_test_active 1 ;
        pulse edt_clock ;
```



```

end ;
apply shift 45;
// cycle 2 starts at time 80
cycle =
    force lpct_clock_mux_select 1 ;
    force lpct_scan_en 0 ;
    force lpct_shift_en 0 ;
end ;
// cycle 3 starts at time 120
cycle =
    force lpct_clock_mux_select 0 ;
    force lpct_shift_en 0 ;
end;
end;

procedure test_setup =
    timeplate gen_tp1 ;
    // cycle 1 starts at time 0
    cycle =
        force edt_clock 0 ;
        force lpct_data_in 0 ;
        force lpct_reset 1 ;
        force lpct_test_mode 0 ;
    end ;
    // cycle 2 starts at time 40
    cycle =
        force lpct_reset 0 ;
    end ;
    // cycle 3 starts at time 80
    cycle =
        force lpct_test_mode 1 ;
    end ;
    apply lpct_shift_data  lpct_data_in = 1 ;
    apply lpct_shift_data  lpct_data_in = lpct_config_edt_bypass ;
    apply lpct_shift_data  lpct_data_in = lpct_config_reset_control ;
    apply lpct_shift_data  lpct_data_in = lpct_config_scan_en_control ;
    apply lpct_shift_data  lpct_data_in =
        lpct_config_chain_pattern_load_count ;
    apply lpct_shift_data  lpct_data_in =
        lpct_config_scan_pattern_load_count ;
    apply lpct_shift_data  lpct_data_in = lpct_config_capture_depth ;
    apply lpct_shift_data  lpct_data_in = lpct_config_shift_length ;
    apply lpct_shift_data  lpct_data_in = 0 ;
end;

procedure test_end =
    timeplate gen_tp1 ;
    // cycle 1 starts at time 0
    cycle =
        force lpct_test_active 1 ;
    end ;
    // cycle 2 starts at time 40

    cycle =
        force lpct_test_active 1 ;
    end ;
    // cycle 3 starts at time 80

```

```
cycle =  
    force lpct_test_active 1 ;  
end;  
end;
```

— A —

add_edt_blocks, 182
add_scan_chains -internal, 95, 135, 371, 372
Advanced topics, 185
Architecture, EDT, 21, 79

— B —

Batch mode, 35
Boundary scan
 circuitry, 123
 EDT and, 240
 EDT coexisting with, 240
 EDT signals driven by, 245
 inserting, 122
 modifying EDT dofile for, 52, 240
 modifying EDT test procedure file for, 52, 240
 pre-existing, 52
 synthesis, preparing for, 240
 top level wrapper for, 123
Boundaryscan
 inserting, 122
Bypass circuitry, 23, 103
 customizing, 78, 93
 diagram, 231
Bypass mode
 circuitry, 231
 generated files for, 101
 single chain, 329
Bypass patterns, EDT
 flow example, 235
 using, 235
Bypassing EDT logic, 230

— C —

Channel input pipeline stages
 defining, 247
Channel output pipeline stages
 defining, 247
Clocking in EDT, 25, 81

Commands

 running system, 35

Compressed ATPG

commands

 write_edt_files, 183

Compressed ATPG

commands

 add_edt_blocks, 182
 add_scan_chains, 95, 135, 372
 delete_edt_blocks, 183
 report_edt_blocks, 179, 183
 report_edt_configurations, 82, 83, 96, 182, 183, 376
 report_edt_instances, 183, 339
 report_edt_lockup_cells, 254
 report_edt_pins, 84, 88, 92
 report_environment, 35, 69
 report_scan_volume, 61
 set_bypass_chains, 93
 set_compactor_connections, 93
 set_current_edt_block, 179, 183
 set_dofile_abort, 36
 set_edt_instances, 183, 339, 371
 set_edt_mapping, 183
 set_edt_options, 68, 78, 79
 set_edt_options pins, 102
 set_edt_pins, 85, 88
 set_logfile_handling, 37
 write_edt_files, 98
emulating uncompressed ATPG with, 18
generating EDT patterns with, 46, 48
inputs and outputs, 48
 external flow, 47
 internal flow, 48
pre-synthesis flow, 292
tool flows, 48
 external logic, 24, 45

Compression

 see Effective Compression

compression

- baseline, [61](#)
- Control and channel pins
 - basic configuration, [84](#)
 - default configuration, [84](#), [85](#)
 - sharing with functional pins, [85](#)
 - channel input pin, [86](#)
 - channel output pin, [86](#)
 - EDT bypass pin, [87](#)
 - EDT clock pin, [86](#)
 - EDT configuration pin, [87](#)
 - EDT reset pin, [79](#), [86](#)
 - EDT scan enable pin, [87](#)
 - EDT update pin, [86](#)
 - example, [87](#)
 - reporting, [88](#)
 - requirements, [85](#)
- summary, [83](#)
- create_skeleton_design, [301](#)
 - flow, [301](#), [302](#)
 - input file, [305](#)
 - example, [307](#)
 - inputs, [305](#)
 - inputs and outputs, [304](#), [305](#)
 - interface file, [308](#), [313](#)
 - outputs, [305](#), [314](#)
 - skeleton design, [314](#)
 - skeleton design dofile, [314](#)
 - skeleton design Tessent cell library, [314](#)
 - skeleton design test procedure file, [314](#)

— D —

- Decompressor, [21](#), [23](#), [102](#)
- delete_edt_blocks command, [183](#)
- Design Compiler synthesis script, [104](#), [121](#), [124](#)
- Design flow, EDT
 - design requirements, [44](#)
 - tasks and products, [42](#)
- Design requirements, [44](#)
- Design rules checks
 - EDT-specific rules (K rules), [25](#)
 - introduction, [25](#)
 - TIE-X message, [95](#)
 - transcript messages, [96](#)
 - upon leaving setup mode, [94](#)

- verifying EDT logic operation with, [149](#)
- Dofile
 - for bypass mode (plain ATPG), [374](#), [376](#)
 - for generating EDT patterns, [80](#), [372](#)
 - for inserting scan chains, [54](#)

- Dofiles, [35](#)

— E —

- EDT
 - as extension of ATPG, [20](#)
 - clocking scheme, [25](#), [81](#)
 - compression
 - seeEffective compression
 - configuration, reporting, [81](#)
 - control and channel pins
 - seeControl and channel pins
 - definition of, [20](#)
 - diagnostics
 - flow example, [235](#)
 - with EDT bypass patterns, [235](#)
 - EDT bypass patterns, [235](#)
 - EDT internal patterns, [156](#), [157](#)
 - fundamentals, [17](#)
 - generating EDT test patterns
 - seePattern generation phase
 - I/O pads and, [44](#)
 - logic
 - conceptual diagram, [21](#), [162](#)
 - pattern generation
 - seePattern generation phase
 - pattern size, [61](#)
 - pattern types supported, [27](#)
 - scan channels, seeScan channels
 - signals
 - bypass
 - seePattern generation phase
 - clock
 - seePattern generation phase
 - internal control of, [25](#)
 - reset, [79](#)
 - update
 - seePattern generation phase
 - EDT internal patterns, [156](#), [157](#)
 - EDT logic
 - configuration, [79](#)
 - architecture, [79](#)

- pipeline stages, 78
- creating, 63
- multiple configurations
 - configuration pin, 87
- parameters, 72
- version of, specifying, 79
- EDT reset signal
 - specifying, 79
- Effective compression
 - chain-to-channel ratio and, 367
 - controlling, 25
- Embedded deterministic test
 - see* EDT
- EMPTY, 90, 136, 154, 156, 158, 240, 245, 246, 260, 286, 368, 379
- Enhanced procedure file
 - for bypass mode (plain ATPG), 376
- External logic location flow
 - definition of, 24
 - steps, 45
 - tasks and products, 42

— F —

- Fault aliasing, 285
- Fault sampling, 275
- Faults, supported, 27

— G —

- Generated EDT logic files
- Generated files
 - blackbox description of core, 103
 - described, 99
 - edt circuitry, 101, 102
 - for bypass mode (plain ATPG)
 - dofile, 374, 376
 - enhanced procedure file, 376
 - test procedure file, 376
 - for use in EDT pattern generation phase
 - dofile, 371
 - test procedure file, 372
 - synthesis script, 104, 105, 121, 124
 - top-level wrapper, 101
- Generating EDT test patterns
 - see* Pattern generation phase

— I —

- I/O pads
 - adding, 122
 - managing pre-existing, 51
 - requirements, 44
- I/O pins, usage, 23
- insert_test_logic -output new, 55, 57
- Intellectual property (IP)
 - blocks
 - detailed description of, 324
 - specification, 324
 - synthesizing
 - Design Compiler and, 121
 - verifying operation of, 149
 - design rules checks, 149

— L —

- Length of longest scan chain
 - specifying, 79
- Lockup cells
 - insertion, 254
 - reporting, 254
- Log files, 35
- Logic creation phase
 - in EDT design flow, 45
- Logic location
 - external, 24

— M —

- Masking, *see* Scan chains, masking
- Memories
 - handling of, 42, 365
 - X values and, 282, 365
- Modular Compressed ATPG
 - generating for a fully integrated design, 177
 - input channel sharing, 165

— O —

- Operating system commands, running within
 - tool, 35

— P —

- Pattern generation
 - see* Pattern generation phase
- Pattern generation phase, 133

- adding scan chains, [134](#)
- EDT signals, controlling
 - bypass, [134](#)
 - clock, [134](#)
 - update, [134](#)
- generating EDT patterns, [377](#)
- in EDT design flow, [46](#), [48](#), [133](#)
- optimizing compression, [155](#)
- pattern post-processing, [157](#)
- prerequisites, [134](#)
- reordering patterns, [287](#)
- setting up, [134](#)
- simulating EDT patterns, [157](#)
- test procedure waveforms, example, [134](#)
- verifying EDT patterns, [133](#), [157](#)
- Pattern verification, [61](#)
- Patterns
 - reordering, *see* Pattern generation phase, reordering patterns
 - types supported, [25](#), [27](#)
- Performance
 - evaluation flow, [274](#)
 - improving, [277](#)
 - measuring, [276](#)
- Pin sharing
 - allowed, [55](#), [85](#), [102](#)
 - not allowed, [102](#)
- Pipeline stages
 - description of, [245](#)
 - including, [78](#), [246](#)
- Pre-synthesis flow, [292](#)

— R —

- Reduced netlist
 - role in Internal logic location flow, [117](#)
 - using to improve synthesis run time, [117](#)
 - writing, [117](#)
- Reorder
 - patterns, *see* Pattern generation phase, reordering patterns
 - scan chains, *see* Scan chains, reordering
- Report
 - EDT configuration, [81](#)
 - report_edt_blocks command, [183](#)
 - report_edt_configurations command, [82](#), [83](#), [96](#), [167](#), [182](#), [183](#), [376](#)

- report_edt_instances command, [183](#), [339](#)
- report_edt_lockup_cells command, [254](#)
- report_edt_pins command, [84](#), [88](#), [92](#)
- report_scan_volume command, [61](#)
- Reset signal, [79](#)

— S —

- Scan chains
 - custom masking of, [367](#)
 - determining how many to use, [54](#)
 - length
 - longest, specifying range for, [79](#)
 - limitations on, [53](#), [102](#)
 - masking, [282](#)
 - pattern file example, [285](#)
 - transcript example, [285](#)
 - why needed, [282](#)
 - Xblocking and, [282](#)
 - prerequisites for inserting, [53](#)
 - reordering
 - impact on EDT logic, [101](#)
 - synthesizing, [51](#), [53](#)
 - uncompressed
 - defining for EDT pattern generation, [135](#), [163](#), [372](#)
 - effect on test coverage estimate, [54](#)
 - including, [54](#), [135](#), [372](#)
 - leaving undefined during IP creation, [54](#)
 - modular flow and, [163](#)
- Scan channels
 - conceptual diagram, [21](#)
 - controlling compression with, [21](#), [27](#)
 - definition of, [21](#)
 - introduction, [20](#)
 - pins, sharing with functional pins, [55](#), [85](#)
- Scripts, [35](#)
- set_bypass_chains command, [93](#)
- set_compactor_connections command, [93](#)
- set_current_edt_block command, [179](#), [183](#)
- set_edt_instances command, [183](#), [339](#), [371](#)
- set_edt_options command, [68](#), [78](#), [79](#)
- set_edt_pins command, [85](#), [88](#)
- Shell commands, running system commands, [35](#)
- Spacial compactor

connections, customizing, [93](#)

Spatial compactor, [23](#), [102](#)

Supported Functions, [17](#)

Supported pattern types, [25](#)

Synthesizing

scan chains, [51](#)

with a reduced netlist, [117](#)

— T —

Tessent FastScan

command-line mode, emulating with

Tessent TestKompress, [45](#)

creating bypass patterns, [238](#)

Tessent Scan

dofile for inserting scan chains, example,
[54](#)

insert_test_logic command, [55](#), [57](#)

Tessent TestKompress

creating logic with, [45](#)

emulating Tessent FastScan with, [45](#)

Test data volume, [275](#)

Test procedure file

for bypass mode (plain ATPG), [376](#)

for generating EDT patterns, [372](#)

Tools used in EDT flow, [42](#)

Troubleshooting, [338](#)

EDT aborted faults, [367](#)

incompressible patterns, [365](#)

K19 through K22 DRC violations, [339](#)

less than expected

compression, [366](#)

test coverage, [366](#)

lockup cells in EDT IP, reporting, [254](#)

masking broken scan chains, [367](#)

simulation mismatches, [337](#), [338](#)

too many observable Xs, [365](#)

TSGEN, incorrect references to, [122](#), [364](#)

— U —

User interface

dofiles, [35](#)

log files, [35](#)

running system commands, [35](#)

— V —

Verification of EDT IP, [149](#)

Verification of EDT patterns, [133](#), [157](#)

— W —

write_edt_files command, [98](#), [183](#)

— X —

X blocking, [283](#)

Xs, observable, [365](#)

Third-Party Information

Provides information on open source and third-party software that may be included in Tessent products.

For third-party information, refer to the [Third-Party Software for Tessent Products](#) document. Additional open source and third-party software information may be found in <your_Mentor_Graphics_documentation_directory>/legal

End-User License Agreement

The latest version of the End-User License Agreement is available on-line at:
www.mentor.com/eula

IMPORTANT INFORMATION

USE OF ALL SOFTWARE IS SUBJECT TO LICENSE RESTRICTIONS. CAREFULLY READ THIS LICENSE AGREEMENT BEFORE USING THE PRODUCTS. USE OF SOFTWARE INDICATES CUSTOMER'S COMPLETE AND UNCONDITIONAL ACCEPTANCE OF THE TERMS AND CONDITIONS SET FORTH IN THIS AGREEMENT. ANY ADDITIONAL OR DIFFERENT PURCHASE ORDER TERMS AND CONDITIONS SHALL NOT APPLY.

END-USER LICENSE AGREEMENT ("Agreement")

This is a legal agreement concerning the use of Software (as defined in Section 2) and hardware (collectively "Products") between the company acquiring the Products ("Customer"), and the Mentor Graphics entity that issued the corresponding quotation or, if no quotation was issued, the applicable local Mentor Graphics entity ("Mentor Graphics"). Except for license agreements related to the subject matter of this license agreement which are physically signed by Customer and an authorized representative of Mentor Graphics, this Agreement and the applicable quotation contain the parties' entire understanding relating to the subject matter and supersede all prior or contemporaneous agreements. If Customer does not agree to these terms and conditions, promptly return or, in the case of Software received electronically, certify destruction of Software and all accompanying items within five days after receipt of Software and receive a full refund of any license fee paid.

1. ORDERS, FEES AND PAYMENT.

- 1.1. To the extent Customer (or if agreed by Mentor Graphics, Customer's appointed third party buying agent) places and Mentor Graphics accepts purchase orders pursuant to this Agreement (each an "Order"), each Order will constitute a contract between Customer and Mentor Graphics, which shall be governed solely and exclusively by the terms and conditions of this Agreement, any applicable addenda and the applicable quotation, whether or not those documents are referenced on the Order. Any additional or conflicting terms and conditions appearing on an Order or presented in any electronic portal or automated order management system, whether or not required to be electronically accepted, will not be effective unless agreed in writing and physically signed by an authorized representative of Customer and Mentor Graphics.
- 1.2. Amounts invoiced will be paid, in the currency specified on the applicable invoice, within 30 days from the date of such invoice. Any past due invoices will be subject to the imposition of interest charges in the amount of one and one-half percent per month or the applicable legal rate currently in effect, whichever is lower. Prices do not include freight, insurance, customs duties, taxes or other similar charges, which Mentor Graphics will state separately in the applicable invoice. Unless timely provided with a valid certificate of exemption or other evidence that items are not taxable, Mentor Graphics will invoice Customer for all applicable taxes including, but not limited to, VAT, GST, sales tax, consumption tax and service tax. Customer will make all payments free and clear of, and without reduction for, any withholding or other taxes; any such taxes imposed on payments by Customer hereunder will be Customer's sole responsibility. If Customer appoints a third party to place purchase orders and/or make payments on Customer's behalf, Customer shall be liable for payment under Orders placed by such third party in the event of default.
- 1.3. All Products are delivered FCA factory (Incoterms 2010), freight prepaid and invoiced to Customer, except Software delivered electronically, which shall be deemed delivered when made available to Customer for download. Mentor Graphics retains a security interest in all Products delivered under this Agreement, to secure payment of the purchase price of such Products, and Customer agrees to sign any documents that Mentor Graphics determines to be necessary or convenient for use in filing or perfecting such security interest. Mentor Graphics' delivery of Software by electronic means is subject to Customer's provision of both a primary and an alternate e-mail address.

2. **GRANT OF LICENSE.** The software installed, downloaded, or otherwise acquired by Customer under this Agreement, including any updates, modifications, revisions, copies, documentation, setup files and design data ("Software") are copyrighted, trade secret and confidential information of Mentor Graphics or its licensors, who maintain exclusive title to all Software and retain all rights not expressly granted by this Agreement. Except for Software that is embeddable ("Embedded Software"), which is licensed pursuant to separate embedded software terms or an embedded software supplement, Mentor Graphics grants to Customer, subject to payment of applicable license fees, a nontransferable, nonexclusive license to use Software solely: (a) in machine-readable, object-code form (except as provided in Subsection 4.2); (b) for Customer's internal business purposes; (c) for the term of the license; and (d) on the computer hardware and at the site authorized by Mentor Graphics. A site is restricted to a one-half mile (800 meter) radius. Customer may have Software temporarily used by an employee for telecommuting purposes from locations other than a Customer office, such as the employee's residence, an airport or hotel, provided that such employee's primary place of employment is the site where the Software is authorized for use. Mentor Graphics' standard policies and programs, which vary depending on Software, license fees paid or services purchased, apply to the following: (a) relocation of Software; (b) use of Software, which may be limited, for example, to execution of a single session by a single user on the authorized hardware or for a restricted period of time (such limitations may be technically implemented through the use of authorization codes or similar devices); and (c) support services provided, including eligibility to receive telephone support, updates, modifications, and revisions. For the avoidance of doubt, if Customer provides any feedback or requests any change or enhancement to Products, whether in the course of receiving support or consulting services, evaluating Products, performing beta testing or otherwise, any inventions, product improvements, modifications or developments made by Mentor Graphics (at Mentor Graphics' sole discretion) will be the exclusive property of Mentor Graphics.

3. BETA CODE.

- 3.1. Portions or all of certain Software may contain code for experimental testing and evaluation (which may be either alpha or beta, collectively "Beta Code"), which may not be used without Mentor Graphics' explicit authorization. Upon Mentor Graphics' authorization, Mentor Graphics grants to Customer a temporary, nontransferable, nonexclusive license for experimental use to test and evaluate the Beta Code without charge for a limited period of time specified by Mentor Graphics. Mentor Graphics may choose, at its sole discretion, not to release Beta Code commercially in any form.
- 3.2. If Mentor Graphics authorizes Customer to use the Beta Code, Customer agrees to evaluate and test the Beta Code under normal conditions as directed by Mentor Graphics. Customer will contact Mentor Graphics periodically during Customer's use of the Beta Code to discuss any malfunctions or suggested improvements. Upon completion of Customer's evaluation and testing, Customer will send to Mentor Graphics a written evaluation of the Beta Code, including its strengths, weaknesses and recommended improvements.
- 3.3. Customer agrees to maintain Beta Code in confidence and shall restrict access to the Beta Code, including the methods and concepts utilized therein, solely to those employees and Customer location(s) authorized by Mentor Graphics to perform beta testing. Customer agrees that any written evaluations and all inventions, product improvements, modifications or developments that Mentor Graphics conceived or made during or subsequent to this Agreement, including those based partly or wholly on Customer's feedback, will be the exclusive property of Mentor Graphics. Mentor Graphics will have exclusive rights, title and interest in all such property. The provisions of this Subsection 3.3 shall survive termination of this Agreement.

4. RESTRICTIONS ON USE.

- 4.1. Customer may copy Software only as reasonably necessary to support the authorized use. Each copy must include all notices and legends embedded in Software and affixed to its medium and container as received from Mentor Graphics. All copies shall remain the property of Mentor Graphics or its licensors. Except for Embedded Software that has been embedded in executable code form in Customer's product(s), Customer shall maintain a record of the number and primary location of all copies of Software, including copies merged with other software, and shall make those records available to Mentor Graphics upon request. Customer shall not make Products available in any form to any person other than Customer's employees and on-site contractors, excluding Mentor Graphics competitors, whose job performance requires access and who are under obligations of confidentiality. Customer shall take appropriate action to protect the confidentiality of Products and ensure that any person permitted access does not disclose or use Products except as permitted by this Agreement. Customer shall give Mentor Graphics written notice of any unauthorized disclosure or use of the Products as soon as Customer becomes aware of such unauthorized disclosure or use. Customer acknowledges that Software provided hereunder may contain source code which is proprietary and its confidentiality is of the highest importance and value to Mentor Graphics. Customer acknowledges that Mentor Graphics may be seriously harmed if such source code is disclosed in violation of this Agreement. Except as otherwise permitted for purposes of interoperability as specified by applicable and mandatory local law, Customer shall not reverse-assemble, disassemble, reverse-compile, or reverse-engineer any Product, or in any way derive any source code from Software that is not provided to Customer in source code form. Log files, data files, rule files and script files generated by or for the Software (collectively "Files"), including without limitation files containing Standard Verification Rule Format ("SVRF") and Tcl Verification Format ("TVF") which are Mentor Graphics' trade secret and proprietary syntaxes for expressing process rules, constitute or include confidential information of Mentor Graphics. Customer may share Files with third parties, excluding Mentor Graphics competitors, provided that the confidentiality of such Files is protected by written agreement at least as well as Customer protects other information of a similar nature or importance, but in any case with at least reasonable care. Customer may use Files containing SVRF or TVF only with Mentor Graphics products. Under no circumstances shall Customer use Products or Files or allow their use for the purpose of developing, enhancing or marketing any product that is in any way competitive with Products, or disclose to any third party the results of, or information pertaining to, any benchmark.
 - 4.2. If any Software or portions thereof are provided in source code form, Customer will use the source code only to correct software errors and enhance or modify the Software for the authorized use, or as permitted for Embedded Software under separate embedded software terms or an embedded software supplement. Customer shall not disclose or permit disclosure of source code, in whole or in part, including any of its methods or concepts, to anyone except Customer's employees or on-site contractors, excluding Mentor Graphics competitors, with a need to know. Customer shall not copy or compile source code in any manner except to support this authorized use.
 - 4.3. Customer agrees that it will not subject any Product to any open source software ("OSS") license that conflicts with this Agreement or that does not otherwise apply to such Product.
 - 4.4. Customer may not assign this Agreement or the rights and duties under it, or relocate, sublicense, or otherwise transfer the Products, whether by operation of law or otherwise ("Attempted Transfer"), without Mentor Graphics' prior written consent and payment of Mentor Graphics' then-current applicable relocation and/or transfer fees. Any Attempted Transfer without Mentor Graphics' prior written consent shall be a material breach of this Agreement and may, at Mentor Graphics' option, result in the immediate termination of the Agreement and/or the licenses granted under this Agreement. The terms of this Agreement, including without limitation the licensing and assignment provisions, shall be binding upon Customer's permitted successors in interest and assigns.
 - 4.5. The provisions of this Section 4 shall survive the termination of this Agreement.
5. **SUPPORT SERVICES.** To the extent Customer purchases support services, Mentor Graphics will provide Customer with updates and technical support for the Products, at the Customer site(s) for which support is purchased, in accordance with Mentor Graphics' then current End-User Support Terms located at <http://supportnet.mentor.com/supportterms>.
6. **OPEN SOURCE SOFTWARE.** Products may contain OSS or code distributed under a proprietary third party license agreement, to which additional rights or obligations ("Third Party Terms") may apply. Please see the applicable Product documentation (including license files, header files, read-me files or source code) for details. In the event of conflict between the terms of this Agreement

(including any addenda) and the Third Party Terms, the Third Party Terms will control solely with respect to the OSS or third party code. The provisions of this Section 6 shall survive the termination of this Agreement.

7. LIMITED WARRANTY.

- 7.1. Mentor Graphics warrants that during the warranty period its standard, generally supported Products, when properly installed, will substantially conform to the functional specifications set forth in the applicable user manual. Mentor Graphics does not warrant that Products will meet Customer's requirements or that operation of Products will be uninterrupted or error free. The warranty period is 90 days starting on the 15th day after delivery or upon installation, whichever first occurs. Customer must notify Mentor Graphics in writing of any nonconformity within the warranty period. For the avoidance of doubt, this warranty applies only to the initial shipment of Software under an Order and does not renew or reset, for example, with the delivery of (a) Software updates or (b) authorization codes or alternate Software under a transaction involving Software re-mix. This warranty shall not be valid if Products have been subject to misuse, unauthorized modification, improper installation or Customer is not in compliance with this Agreement. MENTOR GRAPHICS' ENTIRE LIABILITY AND CUSTOMER'S EXCLUSIVE REMEDY SHALL BE, AT MENTOR GRAPHICS' OPTION, EITHER (A) REFUND OF THE PRICE PAID UPON RETURN OF THE PRODUCTS TO MENTOR GRAPHICS OR (B) MODIFICATION OR REPLACEMENT OF THE PRODUCTS THAT DO NOT MEET THIS LIMITED WARRANTY. MENTOR GRAPHICS MAKES NO WARRANTIES WITH RESPECT TO: (A) SERVICES; (B) PRODUCTS PROVIDED AT NO CHARGE; OR (C) BETA CODE; ALL OF WHICH ARE PROVIDED "AS IS."
- 7.2. THE WARRANTIES SET FORTH IN THIS SECTION 7 ARE EXCLUSIVE. NEITHER MENTOR GRAPHICS NOR ITS LICENSORS MAKE ANY OTHER WARRANTIES EXPRESS, IMPLIED OR STATUTORY, WITH RESPECT TO PRODUCTS PROVIDED UNDER THIS AGREEMENT. MENTOR GRAPHICS AND ITS LICENSORS SPECIFICALLY DISCLAIM ALL IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OF INTELLECTUAL PROPERTY.

8. LIMITATION OF LIABILITY. TO THE EXTENT PERMITTED UNDER APPLICABLE LAW, IN NO EVENT SHALL MENTOR GRAPHICS OR ITS LICENSORS BE LIABLE FOR INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES (INCLUDING LOST PROFITS OR SAVINGS) WHETHER BASED ON CONTRACT, TORT OR ANY OTHER LEGAL THEORY, EVEN IF MENTOR GRAPHICS OR ITS LICENSORS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES. IN NO EVENT SHALL MENTOR GRAPHICS' OR ITS LICENSORS' LIABILITY UNDER THIS AGREEMENT EXCEED THE AMOUNT RECEIVED FROM CUSTOMER FOR THE HARDWARE, SOFTWARE LICENSE OR SERVICE GIVING RISE TO THE CLAIM. IN THE CASE WHERE NO AMOUNT WAS PAID, MENTOR GRAPHICS AND ITS LICENSORS SHALL HAVE NO LIABILITY FOR ANY DAMAGES WHATSOEVER. THE PROVISIONS OF THIS SECTION 8 SHALL SURVIVE THE TERMINATION OF THIS AGREEMENT.

9. THIRD PARTY CLAIMS.

- 9.1. Customer acknowledges that Mentor Graphics has no control over the testing of Customer's products, or the specific applications and use of Products. Mentor Graphics and its licensors shall not be liable for any claim or demand made against Customer by any third party, except to the extent such claim is covered under Section 10.
- 9.2. In the event that a third party makes a claim against Mentor Graphics arising out of the use of Customer's products, Mentor Graphics will give Customer prompt notice of such claim. At Customer's option and expense, Customer may take sole control of the defense and any settlement of such claim. Customer WILL reimburse and hold harmless Mentor Graphics for any LIABILITY, damages, settlement amounts, costs and expenses, including reasonable attorney's fees, incurred by or awarded against Mentor Graphics or its licensors in connection with such claims.
- 9.3. The provisions of this Section 9 shall survive any expiration or termination of this Agreement.

10. INFRINGEMENT.

- 10.1. Mentor Graphics will defend or settle, at its option and expense, any action brought against Customer in the United States, Canada, Japan, or member state of the European Union which alleges that any standard, generally supported Product acquired by Customer hereunder infringes a patent or copyright or misappropriates a trade secret in such jurisdiction. Mentor Graphics will pay costs and damages finally awarded against Customer that are attributable to such action. Customer understands and agrees that as conditions to Mentor Graphics' obligations under this section Customer must: (a) notify Mentor Graphics promptly in writing of the action; (b) provide Mentor Graphics all reasonable information and assistance to settle or defend the action; and (c) grant Mentor Graphics sole authority and control of the defense or settlement of the action.
- 10.2. If a claim is made under Subsection 10.1 Mentor Graphics may, at its option and expense: (a) replace or modify the Product so that it becomes noninfringing; (b) procure for Customer the right to continue using the Product; or (c) require the return of the Product and refund to Customer any purchase price or license fee paid, less a reasonable allowance for use.
- 10.3. Mentor Graphics has no liability to Customer if the action is based upon: (a) the combination of Software or hardware with any product not furnished by Mentor Graphics; (b) the modification of the Product other than by Mentor Graphics; (c) the use of other than a current unaltered release of Software; (d) the use of the Product as part of an infringing process; (e) a product that Customer makes, uses, or sells; (f) any Beta Code or Product provided at no charge; (g) any software provided by Mentor Graphics' licensors who do not provide such indemnification to Mentor Graphics' customers; (h) OSS, except to the extent that the infringement is directly caused by Mentor Graphics' modifications to such OSS; or (i) infringement by Customer that is deemed willful. In the case of (i), Customer shall reimburse Mentor Graphics for its reasonable attorney fees and other costs related to the action.
- 10.4. THIS SECTION 10 IS SUBJECT TO SECTION 8 ABOVE AND STATES THE ENTIRE LIABILITY OF MENTOR GRAPHICS AND ITS LICENSORS, AND CUSTOMER'S SOLE AND EXCLUSIVE REMEDY, FOR DEFENSE,

SETTLEMENT AND DAMAGES, WITH RESPECT TO ANY ALLEGED PATENT OR COPYRIGHT INFRINGEMENT OR TRADE SECRET MISAPPROPRIATION BY ANY PRODUCT PROVIDED UNDER THIS AGREEMENT.

11. TERMINATION AND EFFECT OF TERMINATION.

- 11.1. If a Software license was provided for limited term use, such license will automatically terminate at the end of the authorized term. Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement immediately upon written notice if Customer: (a) exceeds the scope of the license or otherwise fails to comply with the licensing or confidentiality provisions of this Agreement, or (b) becomes insolvent, files a bankruptcy petition, institutes proceedings for liquidation or winding up or enters into an agreement to assign its assets for the benefit of creditors. For any other material breach of any provision of this Agreement, Mentor Graphics may terminate this Agreement and/or any license granted under this Agreement upon 30 days written notice if Customer fails to cure the breach within the 30 day notice period. Termination of this Agreement or any license granted hereunder will not affect Customer's obligation to pay for Products shipped or licenses granted prior to the termination, which amounts shall be payable immediately upon the date of termination.
- 11.2. Upon termination of this Agreement, the rights and obligations of the parties shall cease except as expressly set forth in this Agreement. Upon termination of this Agreement and/or any license granted under this Agreement, Customer shall ensure that all use of the affected Products ceases, and shall return hardware and either return to Mentor Graphics or destroy Software in Customer's possession, including all copies and documentation, and certify in writing to Mentor Graphics within ten business days of the termination date that Customer no longer possesses any of the affected Products or copies of Software in any form.
12. **EXPORT.** The Products provided hereunder are subject to regulation by local laws and European Union ("E.U.") and United States ("U.S.") government agencies, which prohibit export, re-export or diversion of certain products, information about the products, and direct or indirect products thereof, to certain countries and certain persons. Customer agrees that it will not export or re-export Products in any manner without first obtaining all necessary approval from appropriate local, E.U. and U.S. government agencies. If Customer wishes to disclose any information to Mentor Graphics that is subject to any E.U., U.S. or other applicable export restrictions, including without limitation the U.S. International Traffic in Arms Regulations (ITAR) or special controls under the Export Administration Regulations (EAR), Customer will notify Mentor Graphics personnel, in advance of each instance of disclosure, that such information is subject to such export restrictions.
13. **U.S. GOVERNMENT LICENSE RIGHTS.** Software was developed entirely at private expense. The parties agree that all Software is commercial computer software within the meaning of the applicable acquisition regulations. Accordingly, pursuant to U.S. FAR 48 CFR 12.212 and DFAR 48 CFR 227.7202, use, duplication and disclosure of the Software by or for the U.S. government or a U.S. government subcontractor is subject solely to the terms and conditions set forth in this Agreement, which shall supersede any conflicting terms or conditions in any government order document, except for provisions which are contrary to applicable mandatory federal laws.
14. **THIRD PARTY BENEFICIARY.** Mentor Graphics Corporation, Mentor Graphics (Ireland) Limited, Microsoft Corporation and other licensors may be third party beneficiaries of this Agreement with the right to enforce the obligations set forth herein.
15. **REVIEW OF LICENSE USAGE.** Customer will monitor the access to and use of Software. With prior written notice and during Customer's normal business hours, Mentor Graphics may engage an internationally recognized accounting firm to review Customer's software monitoring system and records deemed relevant by the internationally recognized accounting firm to confirm Customer's compliance with the terms of this Agreement or U.S. or other local export laws. Such review may include FlexNet (or successor product) report log files that Customer shall capture and provide at Mentor Graphics' request. Customer shall make records available in electronic format and shall fully cooperate with data gathering to support the license review. Mentor Graphics shall bear the expense of any such review unless a material non-compliance is revealed. Mentor Graphics shall treat as confidential information all information gained as a result of any request or review and shall only use or disclose such information as required by law or to enforce its rights under this Agreement. The provisions of this Section 15 shall survive the termination of this Agreement.
16. **CONTROLLING LAW, JURISDICTION AND DISPUTE RESOLUTION.** The owners of certain Mentor Graphics intellectual property licensed under this Agreement are located in Ireland and the U.S. To promote consistency around the world, disputes shall be resolved as follows: excluding conflict of laws rules, this Agreement shall be governed by and construed under the laws of the State of Oregon, U.S., if Customer is located in North or South America, and the laws of Ireland if Customer is located outside of North or South America or Japan, and the laws of Japan if Customer is located in Japan. All disputes arising out of or in relation to this Agreement shall be submitted to the exclusive jurisdiction of the courts of Portland, Oregon when the laws of Oregon apply, or Dublin, Ireland when the laws of Ireland apply, or the Tokyo District Court when the laws of Japan apply. Notwithstanding the foregoing, all disputes in Asia (excluding Japan) arising out of or in relation to this Agreement shall be resolved by arbitration in Singapore before a single arbitrator to be appointed by the chairman of the Singapore International Arbitration Centre ("SIAC") to be conducted in the English language, in accordance with the Arbitration Rules of the SIAC in effect at the time of the dispute, which rules are deemed to be incorporated by reference in this section. Nothing in this section shall restrict Mentor Graphics' right to bring an action (including for example a motion for injunctive relief) against Customer in the jurisdiction where Customer's place of business is located. The United Nations Convention on Contracts for the International Sale of Goods does not apply to this Agreement.
17. **SEVERABILITY.** If any provision of this Agreement is held by a court of competent jurisdiction to be void, invalid, unenforceable or illegal, such provision shall be severed from this Agreement and the remaining provisions will remain in full force and effect.
18. **MISCELLANEOUS.** This Agreement contains the parties' entire understanding relating to its subject matter and supersedes all prior or contemporaneous agreements. Any translation of this Agreement is provided to comply with local legal requirements only. In the event of a dispute between the English and any non-English versions, the English version of this Agreement shall govern to the extent not prohibited by local law in the applicable jurisdiction. This Agreement may only be modified in writing, signed by an authorized representative of each party. Waiver of terms or excuse of breach must be in writing and shall not constitute subsequent consent, waiver or excuse.